
Майкл Коллинз



Защита сетей. Подход на основе анализа данных





Network Security Through Data Analysis

Michael Collins



O'REILLY®

Защита сетей. Подход на основе ЛАНЬ® анализа данных

Майкл Коллинз



Москва, 2020

УДК 004.058
ББК 32.973
К60



Майкл Коллинз

К60 Защита сетей. Подход на основе анализа данных / пер. с англ. А.В. Добровольская. – М.: ДМК Пресс, 2020. – 308 с.: ил.

ISBN 978-5-97060-649-0

Эта книга – подробное пошаговое руководство по эффективному использованию доступных инструментов обеспечения безопасности сетей. Её оценят как опытные специалисты по безопасности, так и новички.

Подробно рассматриваются процессы сбора и организации данных, инструменты для их анализа, а также различные аналитические сценарии и методики.

Издание идеально подходит для системных администраторов и специалистов по операционной безопасности, владеющих навыками написания скриптов.

УДК 004.458
ББК 32.973

All rights reserved. This work may not be translated or copied in whole or in part without the written permission of the author, except for brief excerpts in connection with reviews or scholarly analysis. Use in connection with any form of information storage and retrieval, electronic adaptation, or computer software is forbidden

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN 978-1-449-35790-0 (анг.)
ISBN 978-5-97060-649-0 (рус.)

© 2016 Michael Collins
© Оформление, издание, перевод, ДМК Пресс, 2020

Об авторе

Майкл Коллинз является руководителем исследовательских работ для RedJack, LLC, компании по сетевой безопасности и анализу данных, расположенной в Вашингтоне, округ Колумбия. До работы в RedJack доктор Коллинз был членом технического штата в CERT-сети, ситуативная группа осведомленности в университете Карнеги-Меллон. Его основное внимание направлено на инструментарий сети и анализ трафика, особенно на анализ больших наборов данных трафика. Коллинз получил степень доктора по электротехнике в университете Карнеги-Меллон в 2008 г. Он имеет степени магистра и бакалавра этого университета.



Оглавление



Об авторе	5
Предисловие	11
Целевая аудитория	13
Содержание книги	14
Принятые обозначения	16
Использование примеров кода	17
Safari® Books Online (Сафари Букс Онлайн)	17
Контактная информация	18
Благодарственное слово	18
Предисловие от издательства	19
Отзывы и пожелания	19
Список опечаток	19
Нарушение авторских прав	19
ЧАСТЬ I. ДАННЫЕ	21
Глава 1. Сенсоры и детекторы: введение	23
Область обзора сенсора: зависимость сбора данных от расположения сенсора	24
Уровни расположения сенсоров: какие данные можно собрать	27
Действия сенсора: как сенсор обрабатывает данные	30
Заключение	32
Глава 2. Сетевые сенсоры	33
Влияние уровней сети на ее оснащение	34
Уровни сети и область обзора сенсоров	36
Уровни сети и адресация	40
Пакетные данные	41
Форматы пакетов и фреймов	42
Циклический (кольцевой) буфер	42
Лимитирование захваченных пакетных данных	42
Фильтрация специфических типов пакетов	43
Если вы не используете Ethernet	46
NetFlow	47
Форматы и поля NetFlow v5	47
«Поток и наполнение». NetFlow v9 и стандарт IPFIX	48
Генерация и сбор данных в NetFlow	49
Дополнительные материалы для чтения	50
Глава 3. Датчики хостов и сервисов: журналирование трафика в источнике данных	51
Доступ и управление файлами журнала	52
Содержание файлов журнала	54

Характеристики хорошего сообщения журнала	54
Существующие файлы журнала, и как ими управлять	57
Представительные форматы файла журнала	58
HTTP: CLF и ELF	58
SMTP	62
Microsoft Exchange: журналы, отслеживающие сообщения	64
Транспорт файла журнала: передачи, системы и очереди сообщений	65
Передача и ротация файла журнала	65
Системный журнал	66
Дополнительные материалы для чтения	67

Глава 4. Хранение данных для анализа: реляционные базы данных, большие данные и другие опции

Данные журналов и парадигма CRUD	69
Создание хорошо организованной плоской файловой системы: уроки от SiLK	70
Краткое введение в системы NoSQL	72
Какой подход к хранению данных использовать	75
Иерархия устройств хранения данных, время выполнения запроса и старение	77

Часть II. Инструменты

Глава 5. Комплект SiLK

Что такое SiLK, и как он работает?	81
Получение и установка SiLK	82
Файлы данных	82
Выбор и форматирование выходного управления полем: rwcut	83
Основное управление полем: rwfiler	87
Порты и протоколы	88
Размер	89
IP-адреса	89
Время	91
Опции TCP	91
Вспомогательные опции	93
Разные опции фильтрации и некоторые взломы	94
rwfileinfo и источник	94
Объединение информационных потоков: rwcoun	96
rwset и IP Sets	98
rwuniq	101
rwbag	103
Усовершенствованные средства SiLK	103
pmaps	104
Сбор данных SiLK	105
YAF	106
rwptoflow	108
rwtuc	108
Дополнительные материалы для чтения	109

Глава 6. Введение в R для аналитиков по вопросам безопасности

Установка и настройка	111
Основы языка	111

Подсказка R	111
R-переменные	113
Запись функций	118
Условные выражения и итерация	119
Использование рабочей области R	121
Фреймы данных	122
Визуализация	125
Команды визуализации	126
Параметры визуализации	126
Аннотация технологии визуализации	128
Экспорт визуализации	129
Анализ: проверка статистических гипотез	129
Проверка гипотез	130
Тестирование данных	132
Дополнительные материалы для чтения	134

Глава 7. Классификация и инструменты события:

IDS, AV и SEM	135
Как работает IDS	135
Базовый словарь	136
Частота отказов классификатора: понимание ошибки базовой ставки	140
Применение классификации	142
Улучшение производительности IDS	143
Улучшение обнаружения IDS	144
Улучшение ответа IDS	148
Упреждающая выборка данных	149
Дополнительные материалы для чтения	150

Глава 8. Ссылка и поиск: инструменты

для выяснения, кто есть кто	151
MAC и аппаратные адреса	151
IP-адресация	153
Адреса IPv4, их структура и важные адреса	154
Адреса IPv6, их структура и важные адреса	155
Проверка возможности соединения: используйте ping для соединения с адресом	157
Tracerouting	158
Интеллект IP: геолокация и демография	160
DNS	161
Структура имени DNS	161
Направление DNS-запроса с использованием dig	163
Обратный поиск DNS	169
Использование whois для нахождения владельца	171
Дополнительные ссылочные инструменты	173
DNSBLs	174

Глава 9. Больше инструментов

Визуализация	176
Graphviz	176
Связь и зондирование	179
netcat	179
nmap	181
Scapy	182

Проверка пакетов и ссылка.....	184
Wireshark.....	185
GeoIP	185
NVD, вредоносные сайты и C*Es	186
Поисковые системы, списки рассылки и люди	187
Дополнительные материалы для чтения.....	188

ЧАСТЬ III. АНАЛИТИКА.....189

Глава 10. Разведочный анализ данных и визуализация191

Цель разведочного анализа: проведение анализа	193
Порядок выполнения разведочного анализа	194
Переменные и визуализация.....	196
Визуализация одномерных данных:	
гистограммы, графики квантилей и коробчатые диаграммы.....	197
Гистограммы	197
Столбиковые диаграммы.....	199
Графики квантилей	200
Пятичисловая сводка и коробчатая диаграмма	202
Создание коробчатой диаграммы	203
Визуализация двумерных данных.....	206
Диаграммы рассеяния	206
Таблицы сопряженности	208
Визуализация многомерных данных.....	209
Оперативная визуализация.....	211
Дополнительные материалы для чтения.....	217

Глава 11. О «прощупывании»218

Модели нападения.....	218
Прощупывание: неверная конфигурация, автоматизация и сканирование	221
Ошибки в процессе поиска.....	221
Автоматизация.....	222
Сканирование.....	222
Определение попытокощупывания.....	223
Прощупывание TCP: диаграмма состояний.....	223
Сообщения ICMP иощупывание	226
Определениеощупывания в UDP	227
Прощупывание на уровне сервисов.....	228
Прощупывание HTTP.....	228
Прощупывание SMTP.....	230
Анализ попытокощупывания	230
Создание предупреждений оощупывании	230
Расследование попытокощупывания	231
Проектирование сети для извлечения пользы отощупывания	232
Дополнительные материалы для чтения.....	233

Глава 12. Анализ объема и времени.....234

Влияние рабочих часов на объем трафика	234
Тревожные сигналы.....	237
Рейдерство – несанкционированное копирование файлов	239
Локальность	243

Отказ в обслуживании, флешмобы и исчерпание ресурсов.....	245
DDoS и инфраструктура маршрутизации.....	246
Применение анализа объема и локальности.....	251
Сбор данных	252
Создание тревог на основе объема	254
Создание тревог из тревожных сигналов.....	254
Создание тревог по признакам локальности	255
Инженерные решения	256
Дополнительные материалы для чтения.....	256
Глава 13. Анализ графа	257
Атрибуты графа: что такое граф?	257
Метки, вес и пути.....	261
Компоненты и возможность соединения	266
Коэффициент кластеризации	267
Анализ графов.....	268
Создание тревог с использованием анализа компонентов	268
Использование оценок центральности при расследовании	270
Использование поиска в ширину при расследовании	270
Использование анализа центральности для проектирования	272
Дополнительные материалы для чтения.....	272
Глава 14. Идентификация приложения.....	273
Механизмы идентификации приложений	273
Номер порта	274
Идентификация приложений по баннерам	277
Идентификация приложений по поведению.....	280
Идентификация приложений по обращениям к сайтам	284
Баннеры приложений: идентификация и классификация.....	285
Баннеры, не принадлежащие браузерам.....	285
Баннеры веб-клиентов: заголовок User-Agent	286
Дополнительные материалы для чтения.....	287
Глава 15. Сетевое картирование.....	288
Создание начальной описи и карты сети	288
Создание описи: данные, охват и файлы	289
Этап I: три первых вопроса	290
Этап II: исследование пространства IP-адресов	293
Этап III: выявление слепого и странного трафика	297
Этап IV: идентификация клиентов и серверов.....	301
Идентификация инфраструктуры контроля и блокирования	303
Обновление описи: к непрерывному аудиту.....	303
Дополнительные материалы для чтения.....	304
Предметный указатель.....	305

Предисловие

<https://t.me/portalToIT>



Эта книга – обо всем, что связано с сетями: их мониторинге, изучении и использовании результатов этого изучения с целью улучшения. «Улучшение» в данном контексте означает повышение безопасности сети, но я не думаю, что мы владеем достаточным количеством терминов и знаний, чтобы сказать наверняка. Во всяком случае, пока. В попытке обеспечить безопасность мы пытаемся достичь чего-то более конкретного и осязаемого – *ситуационной осведомленности*.

Термин «ситуационная осведомленность» часто используется в вооруженных силах и буквально означает понимание среды, в которой вы работаете.

В нашем случае ситуационная осведомленность включает также понимание компонентов сети и их работы. Зачастую мы *сильно* далеки от понимания настроек сети и первоначальных принципов ее построения.

Для понимания важности ситуационной осведомленности представьте свой дом и посчитайте количество веб-серверов в нем. Вы посчитали ваш беспроводной роутер? А кабельный модем? Принтер? А веб-интерфейс сервера печати? Не забыли ли включить сюда свой телевизор?

Не каждый специалист по информационным технологиям отнесет вышеперечисленные устройства в разряд веб-серверов. Тем не менее встроенные веб-серверы используют протокол HTTP, у них есть уязвимости, количество которых растет, поскольку на смену специализированным протоколам управления приходит веб-интерфейс. Взломщики будут атаковать встроенные системы, не раздумывая, чем они фактически являются: SCADA-система – не что иное, как Windows-сервер с парой интересных дополнительных каталогов, а аппарат MPT – готовый к эксплуатации бот для рассылки спама.

Эта книга о том, как собирать данные и анализировать сети с целью понимания принципов их использования. Особое внимание уделяется анализу – процессу сбора данных о безопасности и принятия решительных мер на их основе. Подчеркиваю, что *решительные меры* в данном контексте – ключевое слово, поскольку эффективные меры по обеспечению безопасности – это запрет на определенные действия. Политика обеспечения безопасности обязывает говорить людям, чего делать не стоит (или, в более требовательном варианте, что они делать *должны*): не использовать Dropbox в качестве хранилища для корпоративных данных, осуществлять вход в систему при помощи пароля и аутентификатора RSA и не копировать весь сервер проекта целиком и не продавать его конкурентам. Когда мы принимаем решения по обеспечению безопасности, мы вторгаемся в рабочий процесс сотрудников, и должны иметь для этого очень веские основания.

Все системы безопасности целиком и полностью зависят от пользователей, которые осознают необходимость безопасности и воспринимают меры по ее обеспечению как вынужденное зло. Безопасность зиждется на людях, на пользователях системы, которые соблюдают определенные правила, а также на анализах и программах мониторинга, помогающих выявлять случаи их нарушения. Безопасность – лишь в небольшой степени техническая задача. Информационная

безопасность предполагает борьбу с невероятно творческими людьми, постоянно ищущими новые способы завладеть вашими технологиями. И в борьбе с этой постоянно изменяющейся угрозой вам необходимо добиться сотрудничества как со стороны защитников, так и со стороны пользователей. Неверно выстроенная политика безопасности вынудит сотрудников обходить меры безопасности, чтобы выполнить свою работу, или попросту нервничать, а это добавит работы специалистам по безопасности.

Акцент на решительности мер и цель достичь безопасности – это факторы, отличающие данную книгу от более общих текстов по анализу и обработке данных. Раздел, посвященный анализу, включает в себя методы статистического анализа и анализа данных из различных дисциплин, но самое пристальное внимание уделяется пониманию структуры сети и решениям, которые помогут защитить ее. В этой связи я сократил теоретическую часть до минимума и сконцентрировался на механизмах обнаружения вторжений. Проблема анализа безопасности состоит в том, что объекты наблюдения не только знают, что за ними следят, но и делают все возможное, чтобы этому воспрепятствовать.

МРТ и ноутбук генерального

Несколько лет назад я общался со специалистом по безопасности, работающим в основном для университетской больницы. Он рассказал, что самым загруженным устройством в его сети был томограф. В ретроспективе это легко объяснить. «Только подумайте, – сказал он мне, – МРТ – это медицинское оборудование, а это значит, что на нем может использоваться лицензионная версия Windows. Поэтому каждую неделю кто-то взламывал его и устанавливал на него спам-бот. Спам начинал идти приблизительно в среду». Когда я спросил его, почему он просто не отключил томограф от интернета, он сказал, пожав плечами, что докторам были нужны их снимки. Он был первым специалистом с такой проблемой, которого я встретил, но не был последним. Мы сталкиваемся с подобной проблемой в любой организации, иерархия которой включает в себя высокие должности: доктора, старшие партнеры, генеральные директора. Вы можете создать сколь угодно много рубежей защиты, но если генеральный директор хочет взять рабочий ноутбук, чтобы его внучка поиграла в Neopets (Новые питомцы) в выходные, то в понедельник вы получите зараженный ноутбук, требующий ремонта.

Чтобы развить свою мысль, я продолжу. Я твердо уверен в том, что самый эффективный способ защитить сети – сохранять и защищать *только* то, что вам действительно нужно сохранить и защитить. Я так считаю, потому что информационная безопасность всегда будет требовать участия людей в мониторинге и расследовании. Модели атак постоянно меняются, поэтому, когда мы используем автоматизированные средства защиты, мы обнаруживаем, что взломщики теперь могут использовать их для атаки на нас самих¹.

Как специалист по безопасности я твердо уверен в том, что безопасность должна доставлять неудобство, быть хорошо организованной и вводить жесткие огра-

¹ Рассмотрим автоматическую блокировку аккаунтов после некоторого числа неудачных попыток ввода пароля, когда логин – это адрес электронной почты. Представьте, сколько аккаунтов можно заблокировать таким способом.

ничения. Безопасность должна быть искусственным поведением, распространяющимся на активы, которые необходимо сохранить. Поведение должно быть искусственным, потому что последняя линия защиты в любой защищенной системе – это *люди*. А люди, полностью вовлеченные в вопросы безопасности, должны быть недоверчивыми, выискивающими подозрительные явления с упорством параноика. Это не самый лучший способ прожить жизнь, поэтому, чтобы сделать ее сносной, мы должны обеспечить безопасность лишь того, что необходимо. Пытаясь уследить за всем, вы теряете ту грань, которая помогает вам защищать только то, что действительно имеет значение.

Поскольку безопасность доставляет неудобство, эффективные специалисты по безопасности должны *уметь убедить* пользователей в необходимости изменить свой привычный режим работы и плясать под их дудку, а в противном случае ограничить деятельность пользователей с целью предотвратить гипотетическую атаку в будущем. В этой связи специалисту необходимо определить решение, подкрепить его информативно и продемонстрировать риски своей аудитории.

Процесс анализа данных, описанный в этой книге, направлен на развитие знаний в области безопасности с целью принятия эффективных решений в этой сфере. Это могут быть экспертные решения: реконструкция событий постфактум с целью определить, почему произошла атака и что способствовало ее осуществлению, или оценить причиненный ущерб. Также можно прибегнуть к профилактическим мерам: установка ограничителей скорости передачи, установка систем обнаружения вторжений или разработка стратегий, которые могут ограничить воздействие взломщика на сеть.

ЦЕЛЕВАЯ АУДИТОРИЯ

Анализ информационной безопасности – это молодая дисциплина, поэтому не существует четко определенной совокупности знаний, которыми нужно обязательно владеть. Данная книга предлагает те аналитические методы, которые я или другие специалисты по безопасности использовали за последние 10 лет и видели отличный результат.

Целевая аудитория этой книги – сетевые администраторы и специалисты по операционной безопасности, персонал Центров управления сетями (NOC) и все те, кто регулярно использует консоль СОВ. Я надеюсь, что вы уже знакомы с инструментами TCP/IP, такими как netstat, а также владеете базовыми статистическими и математическими навыками.

Кроме того, я надеюсь, что вы имеете представление о языках программирования. В этой книге я использую излюбленный мной Python для объединения инструментов. Код в Python показателен и может быть понятен людям без опыта работы на нем. Тем не менее вам необходимо владеть навыками создания фильтров или других инструментов на вашем языке программирования.

В данной книге я собрал методы из различных дисциплин, включив ссылки на оригинал там, где это было возможно. Таким образом, вы можете просмотреть эти материалы и найти другие подходы к решению проблемы. Многие из этих методов имеют математическое или статистическое обоснование, которое я намеренно оставил на функциональном уровне, не углубляясь в разновидности рассматриваемого подхода. Тем не менее базовое понимание статистики пригодится.

СОДЕРЖАНИЕ КНИГИ

Книга состоит из трех частей: «Данные», «Инструменты» и «Аналитика». Часть I «Данные» описывает процесс сбора и организации данных. В части II «Инструменты» рассказывается об инструментах поддержания аналитического процесса. В части III «Аналитика» предлагаются различные аналитические сценарии и методы.

Часть I посвящена сбору, хранению и организации данных. Хранение данных и логистика являются насущными проблемами анализа безопасности: собрать данные не сложно, гораздо сложнее осуществлять в них поиск конкретного явления. Данные занимают определенный объем, и можно собрать такое количество данных, в котором будет невозможно что-либо найти. Эта часть содержит следующие главы:

Глава 1

Описывает процесс сбора данных в целом. Она предлагает концепцию для понимания того, как сенсоры собирают информацию, формируют отчет и как они взаимодействуют друг с другом.

Глава 2

Продолжает тему предыдущей главы, уделяя особое внимание сенсорам, которые собирают данные о сетевом трафике. Эти сенсоры, включая `tcpdump` и NetFlow, представляют понятную модель активности сети, но зачастую их сложно толковать из-за трудностей, связанных с реконструкцией сетевого трафика.

Глава 3

В этой главе описываются сенсоры, расположенные в определенной системе, например в хостовой системе определения вторжений или журналах сервисов, таких как HTTP. Хотя вышеупомянутые сенсоры покрывают гораздо меньше трафика, чем сетевые, информация, поступающая с них, гораздо проще для понимания и требует меньше времени для толкования и построения догадок.

Глава 4

В главе 4 вы найдете различные инструменты для хранения данных трафика, в том числе традиционно используемые базы данных, системы больших данных, такие как Hadoop, а также специализированные инструменты, такие как графовые базы данных и сетевые журналируемые хранилища данных, например REDIS.

В *части II* собраны различные инструменты для анализа, визуализации и отчетности. Инструменты из этой части подробно разбираются в последующих разделах в контексте проведения различных видов анализа.

Глава 5

SiLK (System for Internet-Level Knowledge) – это набор инструментов для анализа потока данных, разработанный Университетом Карнеги-Меллон (Carnegie Mellon's CERT). В данной главе описываются возможности SiLK и то, каким образом использовать его инструменты для анализа данных, передаваемых протоколом NetFlow.

Глава 6

Данная глава посвящена языку программирования R – среде для проведения статистического анализа и визуализации, в которой можно качественно исследовать практически все возможные источники данных. Данная глава дает базовое представление об R и предлагает способы его использования для углубленного статистического анализа.

Глава 7

Система обнаружения вторжений, сокр. COB (Intrusion Detection System – IDS) – это автоматизированная система анализа трафика, подающая сигналы опасности при обнаружении подозрительных явлений. В данной главе особое внимание уделяется принципам работы COB, влиянию ошибок обнаружения на подаваемые COB сигналы опасности и построению эффективных систем обнаружения с применением инструментов для COB типа SiLK или конфигурации уже существующей COB типа Snort.

Глава 8

Одной из наиболее частых и трудоемких задач анализа является выявление происхождения IP-адреса или определение сигнатуры. В данной главе речь идет об инструментах и методах расследования, которые можно использовать для определения владельца адреса и его происхождения, имени, а также других элементов.

Глава 9

Глава вкратце рассказывает о некоторых специализированных инструментах анализа, не вошедших в предыдущие главы. Речь пойдет об инструментах для визуализации, создания пакетов и обработки данных, а также некоторых других наборах инструментов, которые необходимо знать специалисту по безопасности. В *части III*, заключительном разделе книги, заключена цель всего процесса сбора данных – анализ. В следующих главах описаны различные явления трафика и математические модели для изучения данных.

Глава 10

Глава посвящена *разведочному анализу данных*, сокр. РАД (*Exploratory Data Analysis – EDA*), процессу изучения данных с целью определения их структуры или выявления необычных явлений. Поскольку данные о безопасности быстро меняются, каждому специалисту необходимо владеть РАД. Данная глава дает основы визуализации и описывает математические методы, используемые для исследования данных.

Глава 11

Глава посвящена ошибкам в ходе обмена данными и тому, как можно использовать их для обнаружения таких явлений, как сканирование.

Глава 12

В этой главе приводятся виды анализа, которые можно осуществить путем исследования объема и поведения трафика в динамике. Речь пойдет о DDoS-атаках, атаках на базы данных, а также об изменениях объемов трафика в течение рабочего дня и механизмах фильтрации объемов трафика для более эффективного анализа.

Глава 13

Данная глава посвящена преобразованию сетевого трафика в данные графов и использованию графов с целью определения значимых структур сетей. Такие атрибуты графов, как центрированность, могут быть использованы для определения значимых хостов или отклонений в работе.

Глава 14

В этой главе речь пойдет о методах определения вида трафика, проходящего через сервисные порты сети. Среди этих методов можно выделить обыкновенный поиск, например по номеру порта, а также баннер-граббинг и анализ ожидаемых размеров пакетов.

Глава 15

В главе описывается поэтапный процесс инвентаризации сети и определения важных хостов внутри нее. Составление карты сети и инвентаризация являются важными аспектами обеспечения информационной безопасности, которые необходимо применять на регулярной основе.

ПРИНЯТЫЕ ОБОЗНАЧЕНИЯ

В книге использованы следующие типографические обозначения

Курсивом

выделены новые термины, адреса URL, электронные адреса, названия и расширения файлов.

Моноширинный шрифт

используется в листингах, а также внутри параграфов для ссылки на программные элементы, такие как названия функций, баз данных, типов данных, переменные окружения, комментарии и ключевые слова.

Моноширинным жирным шрифтом

выделяются команды или любой другой текст, вводимый пользователем.

Моноширинным курсивом

выделяется текст, который должен быть заменен пользовательскими значениями или значениями, предписанными контекстом.



– этим символом обозначаются подсказки, предложения или общие примечания.



– этим символом обозначаются предостережения или предупреждения.

ИСПОЛЬЗОВАНИЕ ПРИМЕРОВ КОДА

Дополнительные материалы (примеры кода, упражнения и т. д.) доступны для скачивания по ссылке https://github.com/mpcollins/nsda_examples.

Эта книга написана для того, чтобы помочь вам сделать вашу работу. Если пример кода приведен в данной книге, вы можете использовать его в своих программах и документации. Вам не нужно запрашивать у нас разрешения на использование небольших частей кода. Например, написание программы с использованием нескольких фрагментов кода из этой книги не требует особого разрешения. Продажа и дистрибуция CD-дисков с примерами от издательства O'Reilly требует получения особого разрешения. Ответ на вопрос цитатой с примером кода из этой книги не требует особого разрешения. Внесение крупного фрагмента кода из этой книги в документацию по вашему продукту требует получения особого разрешения.

Мы приветствуем, но не требуем атрибуцию. Атрибуция, как правило, включает в себя название книги, имя автора, название издательства и международный стандартный книжный номер (ISBN). Пример атрибуции: «*Network Security Through Data Analysis by Michael Collins* (O'Reilly). Copyright 2014 Michael Collins, 978-1-449-3579-0».

Если использование вами фрагментов кода не подпадает под условия свободного использования или использования с разрешения издательства, свяжитесь с нами по электронной почте: permissions@oreilly.com.

SAFARI® BOOKS ONLINE (САФАРИ БУКС ОНЛАЙН)



Safari Books Online – это цифровая библиотека по запросу, предоставляющая материалы экспертного уровня от ведущих мировых авторов книг в сфере технологий и бизнеса.

Профессионалы в области технологий, разработчики ПО, веб-дизайнеры, деловые и креативные люди используют Safari Books Online в качестве основного источника информации для исследований, решения задач, обучения и сертификации.

Safari Books Online предлагает продукты и ценовые программы для организаций, государственных органов и частных лиц. Подписчики имеют доступ к тысячам книг, обучающих видео и рукописей до публикации в виде удобной базы данных от таких издательств, как O'Reilly Media, Prentice Hall Professional, Addison-Wesley Professional, Microsoft Press, Sams, Que, Peachpit Press, Focal Press, Cisco Press, John Wiley & Sons, Syngress, Morgan Kaufmann, IBM Redbooks, Packt, Adobe Press, FT Press, Apress, Manning, New Riders, McGraw-Hill, Jones & Bartlett, Course Technology, и десятков других. Для получения более подробной информации о Safari Books Online посетите наш сайт.



КОНТАКТНАЯ ИНФОРМАЦИЯ

Просим отправлять комментарии и вопросы, касающиеся данной книги, в издательство по адресу:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international or local)
707-829-0104 (fax)

Эта книга имеет собственную веб-страницу, доступную по ссылке <http://oreil.ly/nstda>, где публикуется список опечаток, примеры и дополнительная информация.

Комментарии и вопросы технического характера просим отправлять по адресу bookquestions@oreilly.com.

Для получения более подробной информации о наших книгах, курсах, конференциях и новостях посетите наш веб-сайт <http://www.oreilly.com>.

Подпишитесь на нас в Facebook: <http://facebook.com/oreilly>, в Twitter: <http://twitter.com/oreillymedia>.

Подпишитесь на наш канал на YouTube: <http://www.youtube.com/oreillymedia>.

БЛАГОДАРСТВЕННОЕ СЛОВО

Выражаю благодарность моему редактору Энди Ораму (Andy Oram) за его исключительную поддержку и обратную связь, без которых я бы сотый раз переписывал комментарий к точкам установки сенсоров сети. Также выражаю признательность ассистентам редактора Элисон МакДональд (Allyson MacDonald) и Марии Гулик (Maria Gulick) за то, что заставили поднажать и закончить книгу. Благодарю технических редакторов Риэннона Уивера (Rhiannon Weaver), Марка Томаса (Mark Thomas), Роба Томаса (Rob Thomas), Андре ДиМино (André DiMino) и Генри Стерна (Henry Stern). Их комментарии помогли мне избежать пустой болтовни и сконцентрироваться на действительно важных аспектах.

Эта книга – попытка донести самые полезные знания до отделов по эксплуатации и исследовательских центров, и я благодарю всех причастных по обе стороны, а именно (в произвольном порядке): Тома Лонгстафа (Tom Longstaff), Джея Кадейна (Jay Kadane), Майка Рейтера (Mike Reiter), Джона МакХью (John McHugh), Кэрри Гейтс (Carrie Gates), Тима Шимилла (Tim Shimeall), Маркуса ДеШона (Markus DeShon), Джима Дауни (Jim Downey), Уилла Франклина (Will Franklin), Сэнди Пэррис (Sandy Parris), Шона МакАллистера (Sean McAllister), Грегга Верджина (Greg Virgin), Скотта Каула (Scott Coull), Джеффа Джэниса (Jeff Janies) и Майка Уитта (Mike Witt).

И наконец, я хочу поблагодарить моих родителей Джеймса и Кэтрин Коллинз (James and Catherine Collins). Отец скончался в процессе написания этой книги, но он задавал так много вопросов. И поскольку ответов он не понимал, то были вопросы о вопросах, вновь и вновь, до самого конца.

Предисловие от издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв прямо на нашем сайте www.dmkpress.com, зайдя на страницу книги, и оставить комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com, при этом напишите название книги в теме письма.

Если есть тема, в которой вы квалифицированы, и вы заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы удостовериться в качестве наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг – возможно, ошибку в тексте или в коде, – мы будем очень благодарны, если вы сообщите нам о ней. Сделав это, вы избавите других читателей от расстройств и поможете нам улучшить последующие версии этой книги.

Если вы найдете какие-либо ошибки в коде, пожалуйста, сообщите о них главному редактору по адресу dmkpress@gmail.com, и мы исправим это в следующих тиражах.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательства «ДМК Пресс» и O'Reilly очень серьезно относятся к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконно выполненной копией любой нашей книги, пожалуйста, сообщите нам адрес копии или веб-сайта, чтобы мы могли применить санкции.

Пожалуйста, свяжитесь с нами по адресу электронной почты dmkpress@gmail.com со ссылкой на подозрительные материалы.

Мы высоко ценим любую помощь по защите наших авторов, помогающую нам предоставлять вам качественные материалы.





ДАННЫЕ

Эта часть посвящена сбору и хранению данных для последующего анализа и принятия мер. Эффективный анализ безопасности требует сбора данных из множества разных источников, каждый из которых лишь частично отражает положение дел в сети.

Чтобы понять важность гибридных источников данных, примите во внимание тот факт, что большинство современных ботов – это системы общего назначения. Один бот может использовать несколько методов для вторжения в другие хосты сети. Перечень этих атак может включать переполнение буфера, распространение через общие сетевые ресурсы или простое взламывание пароля. Попытка бота атаковать SSH-сервер путем ввода пароля может быть зафиксирована в SSH-журнале данного хоста, подтверждая факт атаки, но не предоставляя информацию о других действиях бота. Процесса сбора сетевого трафика может быть и недостаточно для реконструкции сессии, но он может рассказать вам о других действиях взломщика, допустим, о долгом, успешном сеансе взаимодействия с хостом, который ранее не был замечен в таком взаимодействии.

Самая сложная задача в проведении анализа на основе данных – это сбор достаточного количества данных для воссоздания редких событий. Достаточного, но не избыточного, в противном случае будет невозможно выполнить поисковый запрос. Сбор данных удивительно прост, но осмысление полученных данных гораздо сложнее. В сфере безопасности эта проблема осложняется редким возникновением *реальных* угроз. Большая часть сетевого трафика не несет никакой угрозы и часто повторяется: массовая рассылка писем или одновременный просмотр видео на YouTube большим числом пользователей, доступ к файлам. Многие из небольшого количества фактических атак будут *действительно* безобидными, например слепое сканирование пустых IP-адресов. Но эта небольшая часть таит в себе крошечное число атак, которые представляют собой реальную угрозу, например утечку файлов или обмен данными между ботнетами.

Все виды анализа данных, которые мы рассматриваем в этой книге, ограничены по вводу-выводу. Это означает, что процесс анализа данных предполагает точное определение нужных данных и последующую выборку. Поиск нужных данных требует времени, и эти данные имеют определенный объем: лишь один ОС-3 может давать 5 терабайт сырых данных в день. Для сравнения, интерфейс eSATA может считывать около 0,3 гигабайта в секунду, таким образом расходуя несколько часов для *одного* поиска по всему массиву данных, учитывая, что в это

время вы считываете или записываете новые данные при работе с различными дисками. Необходимость сбора данных из множественных источников предполагает их избыточность, что требует дополнительного места на диске и увеличивает время запросов.

Правильно организованное хранилище и система запросов помогают специалистам по безопасности произвольно выполнять запросы данных и ожидать ответа в относительно короткий срок. При слабой организации системы на выполнение запроса требуется большее количество времени, нежели на сбор данных. Разработка правильной структуры требует понимания того, каким образом различные сенсоры осуществляют сбор данных, как они дополняют, дублируют и взаимодействуют друг с другом, а также понимания принципов эффективного хранения данных, дабы обеспечить возможность проведения анализа. Именно на этих проблемах и сделан акцент в данной главе.

Эта часть включает четыре главы. В *главе 1* содержится введение в общий процесс распознавания данных сенсором и их сбора, а также термины для описания взаимодействия сенсоров между собой. В *главе 2* приведены сенсоры, такие как tcpdump и NetFlow, которые осуществляют сбор данных из сетевых интерфейсов. *Глава 3* посвящена хост-сенсорам и сервисным сенсорам, осуществляющим сбор данных о различных процессах, происходящих, например, в серверах и операционных системах. *Глава 4* рассказывает о различных опциях применения систем сбора данных, начиная с баз данных и заканчивая современной технологией больших данных.



Глава 1



<https://t.me/portalToIT>

Сенсоры и детекторы: введение

Эффективный мониторинг информации строится на данных, собранных из многочисленных сенсоров, которые генерируют различные виды данных и создаются различными людьми для различных целей. Сенсором может быть все, что угодно, от сетевого отвода до журнала файрвола – тем, что осуществляет сбор информации о вашей сети и может быть использовано для оценки информационной безопасности. Построение эффективной системы сенсоров требует достижения баланса между ее укомплектованностью и избыточностью. Идеальная система сенсоров укомплектована, но не избыточна. Под укомплектованностью понимается то, что каждое событие тщательно описано, а под отсутствием избыточности – то, что сенсоры не дублируют информацию о событиях. Эти, возможно, недостижимые цели являются идеальной моделью для построения решения по мониторингу.

Ни один из сенсоров не может выполнять все функции в одиночку. Сетевые сенсоры действительно выполняют много работы, но их легко сбить с толку в процессе управления потоками трафика, они неэффективны в отношении зашифрованного трафика и могут лишь предположить наличие активности в хосте. Хост-сенсоры предоставляют более исчерпывающую и точную информацию относительно явлений, для описания которых они имеют достаточный инструментарий. С целью эффективного комбинирования сенсоров я классифицирую их в трех плоскостях:

Область обзора (Vantage).

Расположение сенсоров внутри сети. Сенсоры, расположенные в разных точках, будут видеть разные стороны одного события.

Уровень (Domain).

Информация, предоставляемая сенсором, вне зависимости от его местонахождения (хост, сервис хоста или сеть). Сенсоры с одинаковой областью обзора, но разного уровня дополняют друг друга в процессе предоставления данных об одном и том же событии. Информацию о некоторых событиях можно получить лишь на одном из уровней. Например, мониторинг хоста – это единственный способ определить, имел ли место физический доступ к этому хосту.

Действие сенсора (Action).

Как сенсор принимает решение о создании информационного отчета. Он может просто записывать данные, предоставлять информацию о событиях или же обрабатывать трафик, который предоставляет данные. Сенсоры различного действия могут, потенциально, мешать работе друг друга.

ОБЛАСТЬ ОБЗОРА СЕНСОРА: ЗАВИСИМОСТЬ СБОРА ДАННЫХ ОТ РАСПОЛОЖЕНИЯ СЕНСОРА

Область обзора сенсора дает представление о том, какие пакеты сенсор сможет изучать. Область обзора определяется взаимозависимостью между расположением сенсора и инфраструктурой маршрутизации сети. Чтобы понять, как процессы влияют на область обзора, взгляните на *рис. 1-1*. На данном рисунке показаны уникальные потенциальные сенсоры, обозначенные заглавными буквами. В порядке очередности эти сенсоры имеют следующее расположение:

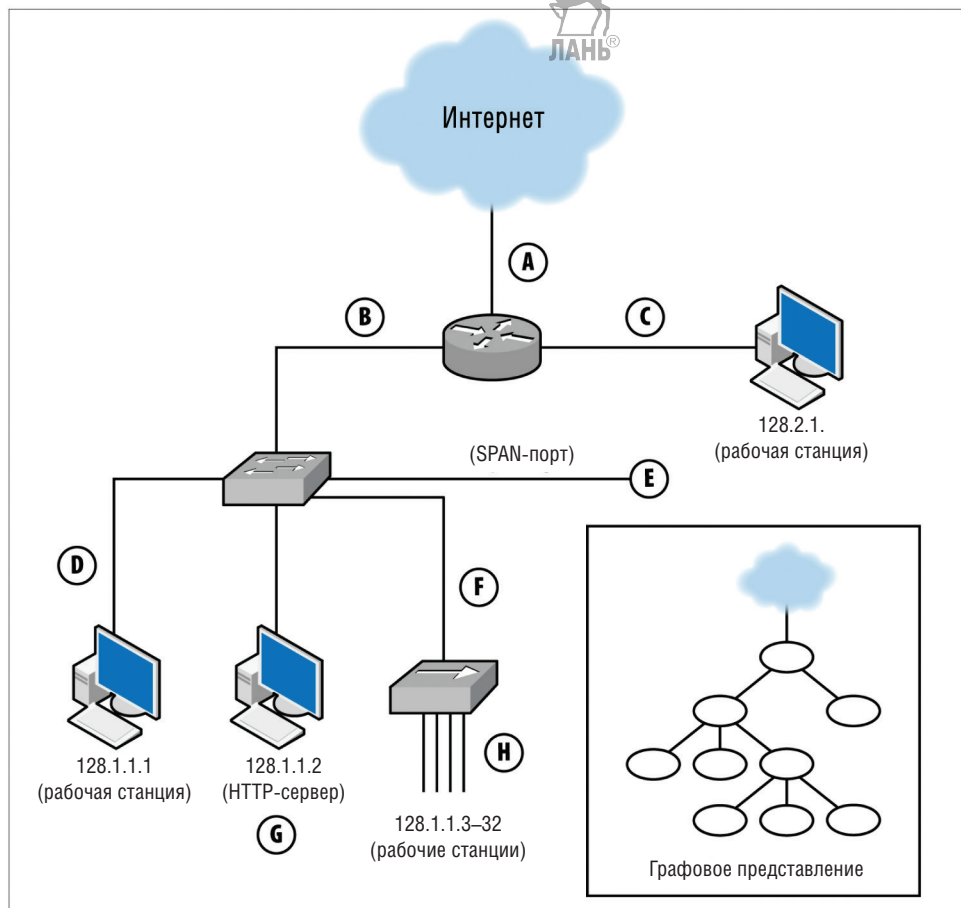


Рис. 1-1. Позиционирование сенсоров в простой сети и графовое представление

- A Проверяет интерфейс, соединяющий роутер с интернетом.
- B Проверяет интерфейс, соединяющий роутер с сетевым коммутатором.
- C Проверяет интерфейс, соединяющий роутер и хост с IP-адресом 128.2.1.1.

D

Проверяет хост с адресом 128.1.1.1.

E

Проверяет SPAN-порт сетевого коммутатора. Этот порт записывает весь трафик, проходящий через коммутатор (см. раздел «Зеркалирование» главы 2 для получения более подробной информации о SPAN-портах).

F

Проверяет интерфейс, соединяющий сетевой коммутатор и сетевой концентратор (хаб).

G

Осуществляет сбор данных журнала НТТР в хосте с адресом 128.1.1.2.

H

Анализирует весь трафик протокола TCP в сетевом концентраторе.

Каждый из этих сенсоров имеет разную область обзора, поэтому будет видеть разные участки трафика. Вы можете приблизительно рассчитать область обзора сенсоров внутри сети при помощи простого графа, состоящего из вершин и ребер, как показано в правом нижнем углу *рис. 1-1*, а затем проследить, какие из ребер пересекаются между вершинами. Сенсор, обозначенный ребром, будет регистрировать весь трафик, пересекающий это ребро по пути к точке назначения. Например, согласно *рис. 1-1*:

- сенсор в точке А будет видеть только трафик между сетью и интернетом, но не будет видеть, к примеру, трафик между адресами 128.1.1.1 и 128.2.1.1;
- сенсор в точке В видит весь трафик между одним из адресов, расположенных ниже его на схеме, и адресом 128.2.1.1 или интернетом;
- сенсор С видит только исходящий и входящий трафики 128.2.1.1;
- сенсор D, как и С, видит только трафик, исходящий от адреса 128.1.1.1 или передаваемый им;
- сенсор Е видит весь трафик, циркулирующий между портами коммутатора: трафик от адреса 128.1.1.1 куда-то еще, трафик от адреса 128.1.1.2 куда-то еще, а также трафик из 128.1.1.3 в 128.1.1.32, взаимодействующий с чем-то еще *за пределами* данного концентратора;
- сенсор F видит часть трафика, видимого сенсором Е, а именно ту его часть, которая передается от 128.1.1.3 к 128.1.1.32, взаимодействующему с чем-то еще *за пределами* данного концентратора;
- сенсор G – особый случай, поскольку является журналом НТТР. Он видит только трафик протокола НТТР (порты 80 и 443), где 128.1.1.2 – это адрес сервера;
- и наконец, сенсор H видит любой трафик, отправляемый или получаемый любым из адресов диапазона 128.1.1.3–128.1.1.32, а также трафик между этими хостами.

Обратите внимание на то, что ни один из сенсоров не охватывает всю сеть целиком. Кроме того, в процессе работы придется столкнуться с избыточным трафиком. Например, если я задействую сенсоры H и E, то увижу трафик от 128.1.1.3 к 128.1.1.1 дважды. При выборе места установки сенсора необходимо стремиться охватить сеть целиком, не погрязнув при этом в избыточных данных.

Оснащая сеть, необходимо определять правильные места установки сенсоров в три этапа: создание карты сети, определение потенциальных точек установки сенсоров и определение оптимального охвата сети.

Первый этап предполагает разработку карты сети, понимание того, как ее элементы соединены друг с другом, а также определение потенциальных точек установки сенсоров. *Рисунок 1-1* представляет собой упрощенную схему такой сети.

На втором этапе, при оценке области обзора, необходимо найти потенциально приемлемые точки установки сенсоров сети и определить область, видимую из этих точек. Это значение может быть выражено в виде перечня комбинаций IP-адрес/порт. *Таблица 1-1* показывает пример отчета для *рис. 1-1*. Построения графа достаточно, чтобы предположить, какой охват сети будет обеспечиваться с точек установки сенсоров, но построение более точной модели требует больше информации о маршрутизаторах и сетевом оборудовании. Например, при работе с роутерами мы можем обнаружить, что обзор с точки установки сенсора асимметричен (обратите внимание, что трафик, показанный на *рис. 1-1*, всегда симметричен). Обратитесь к разделу «Уровни сети и область обзора сенсоров» на стр. 36 для получения более подробной информации.

Таблица 1-1. Область видимости с точек установки сенсоров на рис. 1-1

Точка установки	IP-адрес источника	IP-адрес пункта назначения
A	интернет	128.1, 2.1.1-32
	128.1, 2.1.1-32	интернет
B	128.1.1.1-32	128.2.1.1, интернет
	128.2.1.1, интернет	128.1.1.1-32
C	128.2.1.1	128.1.1.1-32, интернет
	128.1.1.1-32, интернет	128.2.1.1
D	128.1.1.1	128.1.1.2-32, 128.2.1.1, интернет
	128.1.1.2-32, 128.2.1.1, интернет	128.1.1.1
E	128.1.1.1	128.1.1.2-32, 128.2.1.1, интернет
	128.1.1.2	128.1.1.1, 128.1.1.3-32, 128.2.1.1, интернет
	128.1.1.3-32	128.1.1.1-2, 128.2.1.1, интернет
F	128.1.1.3-32	128.1.1.1-2, 128.2.1.1, интернет
	128.1.1.1-32, 128.2.1.1, интернет	128.1.1.3-32
G	128.1,2.1.1-32, интернет	128.1.1.2:tcp/80
	128.1.1.2:tcp/80	128.1,2.1.1-32
H	128.1.1.3-32	128.1.1.1-32, 128.2.1.1, интернет
	128.1.1.1-32, 128.2.1.1, интернет	128.1.1.3-32

Последний этап предполагает выбор оптимальных точек установки, показанных в данной таблице. Цель – выбрать точки, которые обеспечивают мониторинг сети при наименьшей избыточности трафика. Например, сенсор E, помимо прочих, видит все данные сенсора F, поэтому нет смысла выбирать обе точки. При выборе точек установки практически всегда приходится иметь дело с избыточно-

стью трафика. В этой ситуации поможет применение правил фильтрации. Например, чтобы обработать трафик между хостами 128.1.1.3–32, в точке Н необходимо установить сенсор, и этот трафик будет всплывать снова и снова в точках Е, F, В и А. Если настроить сенсоры в этих точках таким образом, чтобы они не отчитывались о трафике, поступающем с адресов 128.1.1.3–32, проблема дублирования становится неактуальной.



УРОВНИ РАСПОЛОЖЕНИЯ СЕНСОРОВ: КАКИЕ ДАННЫЕ МОЖНО СОБРАТЬ

Сенсор G сильно отличается от других сенсоров, показанных на *рис. 1-1*. Пока другие сенсоры фиксируют весь трафик сети, G фиксирует только трафик протокола HTTP (tcp/80). Пока другие сенсоры осуществляют сбор данных трафика в пределах сети, G собирает данные с другого *уровня*. Уровень сенсора дает представление об информации, которую он собирает. Сенсор может осуществлять сбор данных на одном из трех уровней:

Сеть

Сенсоры этого уровня собирают информацию о сетевом трафике. Примеры таких сенсоров включают в себя VPN, большинство систем обнаружения вторжений (IDSes), программы сбора данных протокола NetFlow, такие как YAF (см. раздел "YAF" главы 5 на стр. 106), а также программы сбора данных протокола TCP, такие как Snort, и сырые данные tcpdump.

Хост

Хост-сенсоры следят за происходящими процессами на хосте, такими как вход в систему, выход из системы, доступ к файлам и т. д. Хост-сенсор может предоставить информацию, которую не дает сетевой сенсор, например данные о фактическом доступе к определенному хосту или об использовании внешних периферийных USB-устройств. Хост-сенсоры включают в себя инструменты систем предотвращения вторжений (IPS), такие как Tripwire или приложение HIPS от McAfee, а также журналы системы и безопасности. Хост-сенсоры предоставляют информацию о низкоуровневых операциях, но не расскажут многого о запущенных сервисах. Очевидно, что вы можете использовать такие сенсоры только на хостах, о которых вам известно. Неавторизованные хосты необходимо обнаружить до того, как вы сможете их проверить.

Сервис

Сервисные сенсоры генерируются определенными процессами, такими как журналы серверов HTTP и SMTP. Сервисные сенсоры ведут мониторинг правильно сформированных, если не сказать легитимных, событий внутри сервиса (например, HTTP-сенсор зафиксирует неудачную попытку обращения по URL, но не запишет сеанс 80 порта, в ходе которого не произошла отправка совместимых с HTTP команд. В отличие от журналов хоста и сенсора, относящихся к обыкновенным сенсорам, сервисные сенсоры фиксируют в большей степени взаимодействия с определенным сервисом: отправку электронных писем, выполнение запросов HTTP и т. д. Как и в случае с хост-сенсором, необходимо знать о существовании сервиса до использования сенсора на его уровне.

Восстановление потока и разбивка пакетов

Существуют различные инструменты, которые могут принимать трафик и формировать служебный журнал путем извлечения релевантной информации из пакетов. Например, содержание записи CLF (см. раздел HTTP: CLF и ELF" на стр. 58 для получения более подробной информации) передается между клиентом и сервером HTTP.

Инструменты сетевого анализа предоставляют возможность разбивки пакетов или средства восстановления сеанса для глубокого анализа пакетов. Они создают модель сеанса на основе пакетных данных. Данные инструменты очень полезны для создания примерного представления процессов, протекающих во время сеанса при отсутствии журнала сервиса, но довольно стандартны в части восстановления сеанса сети: они не работают с зашифрованными данными, давая лишь приближенные данные о сеансе, и могут упустить из виду детали реализации, при этом восстановление процесса довольно затратно. В то же время эти программы сбора данных работают с любыми данными сетевого трафика и не требуют логистически сложного процесса определения и установки отдельного сервиса.

Обратите внимание, что уровни расположения сенсора дают представление об информации, которую *использует* сенсор, а не о той, которую он *включает в отчет*. Например, NetFlow, *tcpdump* и сетевые сенсоры IDS – все работают на уровне сети, но выводимые данные у всех разные.

Для понимания разницы между этими уровнями рассмотрим взаимодействия протокола HTTP с точки зрения сенсоров трех различных уровней: сенсора, анализирующего пакеты и установленного на уровне сети; сенсора, расположенного на уровне хоста, который следит за производительностью и контролирует доступ к файлам; и наконец, журнала HTTP-сервера. Сетевой сенсор видит пакеты, которые были отправлены, но не связывает их вместе в структуры протокола HTTP, такие как сеансы, куки-файлы или страницы. Хост-сенсор может определить время последнего доступа к файлу, но не связывает этот файл с URL или запросом. Сервисный сенсор может показать наличие сеанса HTTP и обработавшую страницу, но не определяет незаконченное сканирование порта 80.

Из всех вышеупомянутых сенсоров лишь тот, что расположен на уровне сервиса, – единственный, который может определить, что *имело место* конкретное взаимодействие (предотвратив вторжение в диспетчер протоколирования), остальные могут лишь предоставить информацию специалисту по безопасности для выдвижения гипотез. При прочих равных условиях, лучше иметь в распоряжении сенсор, расположенный максимально близко к цели.

Уровни сенсоров и их области обзора определяют избыточность при работе комбинаций сенсоров. Если два сенсора расположены на одном уровне и область обзора одного из них шире, чем у другого, то сенсор с меньшей областью обзора избыточен и, возможно, не должен использоваться. И наоборот, если два сенсора имеют одинаковую область обзора, но расположены на разных уровнях, то они должны дополнять друг друга.

Рассмотрим пример сети, изображенный на рис. 1-2, где 128.2.1.1 – это адрес HTTPS-сервера, 128.2.1.2 – неизвестный HTTP-сервер и 128.2.1.3 – клиент.

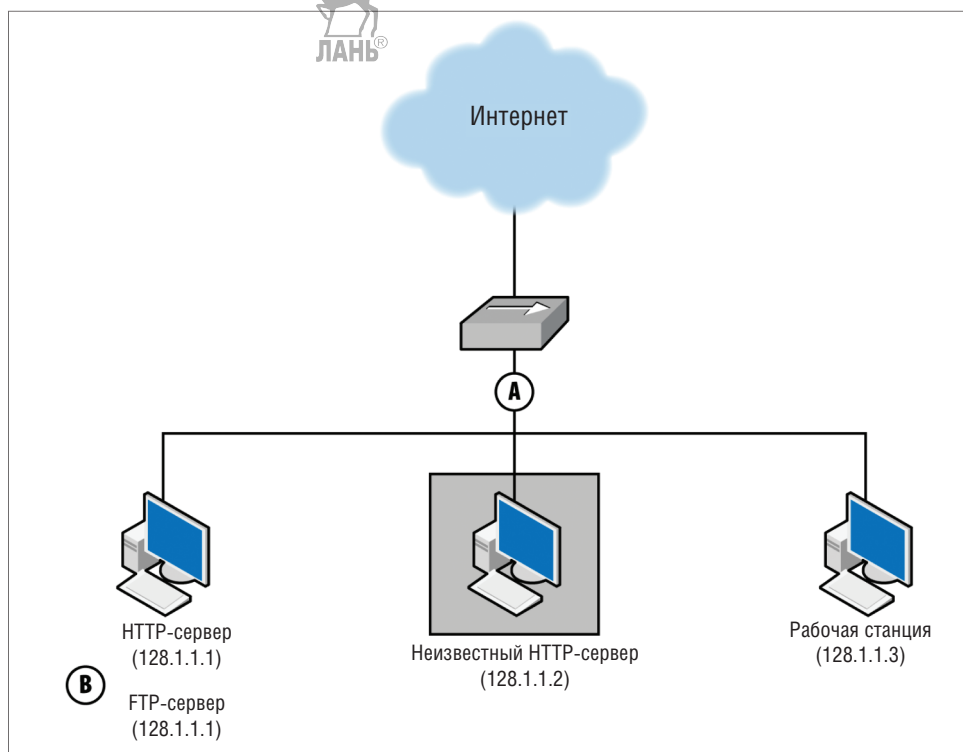


Рис. 1-2. Пример совместной работы сенсоров, расположенных на уровнях хоста и сети

HTTPS-сервер доступен через протокол FTP, не имеющий журнала. Мы приходим к такому выводу, расширяя табличный формат *табл. 1-1* и добавляя уровни, показанные в *табл. 1-2*.

Таблица 1-2 для рис. 1-2. Область обзора и уровень сенсора

Точка установки	IP-адрес источника	IP-адрес назначения	Уровень
A	128.1.1.1-3	интернет	сеть
	128.1.1.1-3	128.1.1.1-3	сеть
	интернет	128.1.1.1-3	сеть
B	128.1.1.2-3, интернет	128.1.1.1:tcp/443	сервис/HTTPS
	128.1.1.1:tcp/443	128.1.1.2-3, интернет	сервис/HTTPS

Теперь давайте рассмотрим некоторые виды атак и реакцию сенсоров на них:

- взломщик сканирует сеть с целью выявления FTP-серверов. Сканирование и ответ увидит сенсор A. Сенсор B не увидит сканирование, поскольку это не FTP-сенсор;
- взломщик сканирует сеть с целью обнаружения HTTPS-сервера путем отправки GET-запроса или запроса на порт 443. Сенсор A видит наличие сеанса с участием 128.1.1.1, но сенсор B предоставляет конкретную информацию о сеансе;

- взломщик ищет HTTP-серверы. Сенсор А видит сканирование, но сенсор В регистрирует события протокола HTTPS, а не HTTP, поэтому не видит сканирование. Сенсор А также видит ответ от 128.1.1.2, идентифицируя незапомненный ранее HTTP-сервер.

Сенсоры, установленные на разных уровнях, предоставляют более полную информацию, чем единичные сенсоры, даже если они имеют равную область обзора. Хост-сенсоры дают больше информации и могут предоставить, например, незашифрованные данные о полезной нагрузке порта, что не всегда доступно сетевому сенсору. Тем не менее специалист по безопасности *должен знать* о существовании хост-сенсора до фактического его использования.

Сетевые сенсоры дают больше информации, чем хост-сенсоры, не только потому, что они видят множество хостов, но и потому, что хост может не реагировать на трафик, отправляемый по всей сети. В то же время если принимать во внимание объем сетевых данных, то их ценность невелика: приходится анализировать большее количество записей для понимания происшествия, и зачастую сложно определить, *отреагировал* ли хост на сетевой трафик. Сетевые сенсоры могут помочь в расследовании и служить подспорьем для хост-сенсоров, когда данная информация недоступна.



ДЕЙСТВИЯ СЕНСОРА: КАК СЕНСОР ОБРАБАТЫВАЕТ ДАННЫЕ

Действие сенсора показывает, как сенсор взаимодействует с собранными данными. Сенсор может применить одно из следующих действий:

Отчет

Данное действие сводится к предоставлению информации по всем явлениям, которые видит данный сенсор. Сенсоры отчетов просты и важны для получения базовой информации. Они также важны для создания сигнатур и сигналов тревоги для явлений, в отношении которых сенсоры тревоги и блокирования пока неэффективны из-за особенностей конфигурации. Сенсоры отчетов включают в себя программы сбора данных NetFlow, *tcpdump* и журналы серверов.

Событие

Сенсоры событий отличаются от сенсоров отчета тем, что они собирают множественные данные для создания *события* с целью формирования некой совокупности этих данных. Например, IDS хоста может анализировать образ данных, обнаружить вредоносную сигнатуру в памяти и отправить событие, оповещая о том, что ее хост столкнулся с вредоносной программой. В крайнем случае, сенсоры событий выполняют роль черных ящиков, которые создают события в ответ на внутренние процессы, запускаемые экспертами. IDS и антивирусы являются сенсорами события.

Контроль

Сенсор контроля, как и сенсор событий, собирает множественные данные и изучает их, а затем реагирует. В отличие от сенсора событий, сенсор контроля модифицирует или блокирует трафик при отправке события. Сенсоры контроля включают в себя систему управления пакетами IPS, системы сетевой защиты, системы борьбы со спамом и некоторые антивирусы.

Действие сенсора влияет не только на формирование отчета, но и на то, как он взаимодействует с анализируемыми данными. Сенсоры контроля могут модифицировать или блокировать трафик. Рисунок 1-3 показывает, как сенсоры различного действия взаимодействуют с данными. Рисунок показывает работу трех сенсоров: R – сенсор отчета, E – сенсор события, C – сенсор контроля. Сенсоры события и контроля – это системы сопоставления сигнатур, реагирующие на строку «АТАКА». Каждый сенсор расположен между интернетом и единственной целью.

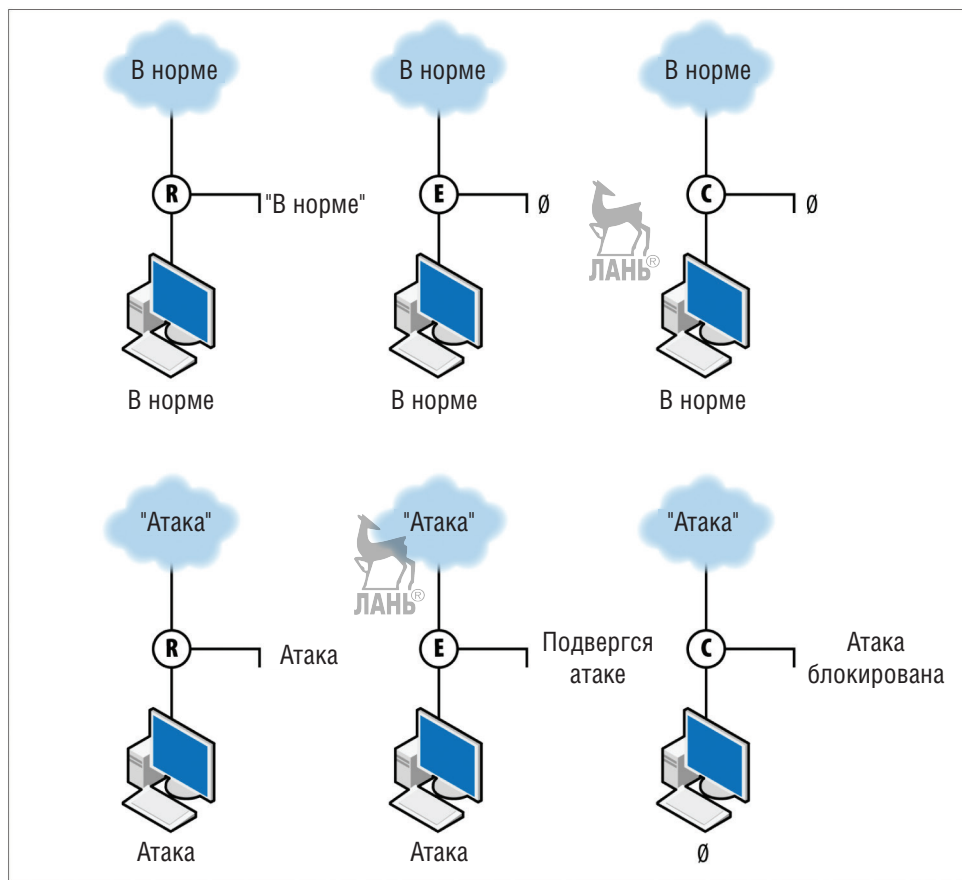


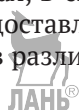
Рис. 1-3. Три разновидности действия сенсоров

R, сенсор отчетов, просто составляет отчеты об анализируемом трафике. В данном случае он включает в отчет как нормальный трафик, так и вредоносный, не влияя на него, а также эффективно резюмирует анализируемые данные. Сенсор событий E создает событие лишь в ответ на вредоносный трафик, оставляя без внимания нормальный. E не останавливает трафик, он просто отправляет событие. Сенсор контроля C отправляет событие, когда видит вредоносный трафик, оставляя без внимания нормальный. Тем не менее C *блокирует* аномальный трафик и не дает ему достичь цели. Если за сенсором C установлен сенсор другого действия, он никогда не распознает трафик, который C блокирует.

Инструменты для агрегации и передачи данных

При оценке пакета логирования убедитесь в том, что он предоставляет программное обеспечение, которое агрегирует и передает записи. Эти возможности не добавят данных при реакции на явление, но они могут изменять формат и содержание записей.

Некоторые примеры предполагают агрегацию в Cisco NetFlow и использование различных инструментов переадресации и передачи данных *flow-tools* (флоу-тулз). Раньше записи NetFlow в базовом формате (необработанный поток) отправлялись в *программу сбора данных*, которая, в свою очередь, агрегировала их в различные отчеты. А пакет *flow-tools* предоставляет инструменты, которые могут брать данные потока и отправлять их в различные сенсоры в случае необходимости.



Заключение

Систематическая классификация, приведенная в данной главе, дает подробную информацию обо всех сенсорах, задействованных в мониторинге безопасности, и об их потенциальном взаимодействии. Данного описания должно быть достаточно, для того чтобы специалист мог классифицировать сенсоры, не углубляясь в детали. В *главах 2 и 3* мы обсуждаем понятия «область обзора», «уровень расположения сенсора» и «действие сенсора» для более глубокого понимания их связи с реальными системами¹.



<https://t.me/portalToIT>

¹ Список рассылки и репозиторий пакета *flow-tools* доступны для бесплатного скачивания.

Глава 2

<https://t.me/portalToIT>

Сетевые сенсоры

Сетевой сенсор собирает данные напрямую из сетевого трафика без помощи посреднического приложения, что и отличает его от хост-сенсоров, описанных в главе 3. В пример можно привести NetFlow-сенсоры на роутере и сенсоры, использующие для сбора данных трафика такие инструменты мониторинга, как *tcpdump*.

При работе с сетевым трафиком мы сталкиваемся с проблемой, аналогичной той, которая возникает при работе со всеми данными журналов: фактические события безопасности происходят редко, в то время как работа с данными требует времени и места для хранения. Предпочтение отдается, по возможности, данным журналов, поскольку они достоверны (в данных журнала фиксируются события высокого уровня) и компактны. Одно и то же явление в сетевом трафике пришлось бы извлекать из множества пакетов, которые зачастую дублируются, шифруются или попросту нечитаемы. В то же время злоумышленнику не составляет труда манипулировать сетевым трафиком и запускать в сети безобидные на первый взгляд, но достаточно вредоносные сессии. Событие, отображаемое в 300-байтном журнале, может запросто представлять из себя мегабайты пакетных данных, из которых лишь первые 10 пакетов имеют аналитическую ценность.

Это были плохие новости. Хорошая новость — в том, что «протокольный агностицизм» сетевого трафика, назовем его так за неимением лучшего термина, характеризует его как лучший источник информации для выявления пробелов в ходе проверки. Системы сбора данных на уровне хоста требуют в первую очередь уверенности в существовании самого хоста, и во многих случаях, с большой долей вероятности, вы не знаете о работе какого-либо сервиса, пока не увидите его трафик в сети. Сетевой трафик дает полное представление о сети, оставляя небольшой простор для предположений: указывает на хосты, о существовании которых вы не знали, инструменты обхода систем защиты и маршруты в вашей сети, которые вы никогда не принимали во внимание. В то же время, когда вы сталкиваетесь с уязвимостями нулевого дня или вредоносным ПО, пакеты могут оказаться единственным доступным источником данных.

Оставшаяся часть данной главы поделена на разделы. Следующий раздел посвящен *области обзора сети*: тому, как пакеты перемещаются по сети и как это с выгодой учесть в процессе ее оснащения. Далее раздел посвящен *tcpdump*, основному протоколу определения сетевого трафика, а также инструментам отбора пакетов, их фильтрации и изменению их размера. Потом следует раздел, посвященный NetFlow, мощному инструменту резюмирования данных

трафика, который компактно агрегирует важную информацию о сетевом трафике. В конце данной главы мы рассмотрим пример сети и обсудим, как воспользоваться преимуществами различных стратегий сбора данных.

Влияние уровней сети на ее оснащение

Компьютерные сети спроектированы по уровням. Уровень – это абстрактное представление совокупности функций сети, целью которого является сокрытие механики и деталей реализации. В идеале каждый уровень – это дискретная сущность. Замена одной реализации на другую в пределах одного уровня не отразится на более высоких уровнях. Например, интернет-протокол IP находится на третьем уровне сетевой модели OSI, реализация протоколов 2-го уровня, таких как Ethernet или FDDI, может осуществляться аналогично реализации протокола IP.

Существует целый ряд сетевых моделей. Самые распространенные из них – семиуровневая сетевая модель OSI и четырехуровневая модель TCP/IP. На рис. 2-1 показаны эти две модели, задействованные в них протоколы и их отношение к уровням сенсоров, описанных в главе 1. Согласно рис. 2-1, модели OSI и TCP/IP имеют грубое сходство. Модель OSI имеет семь уровней:

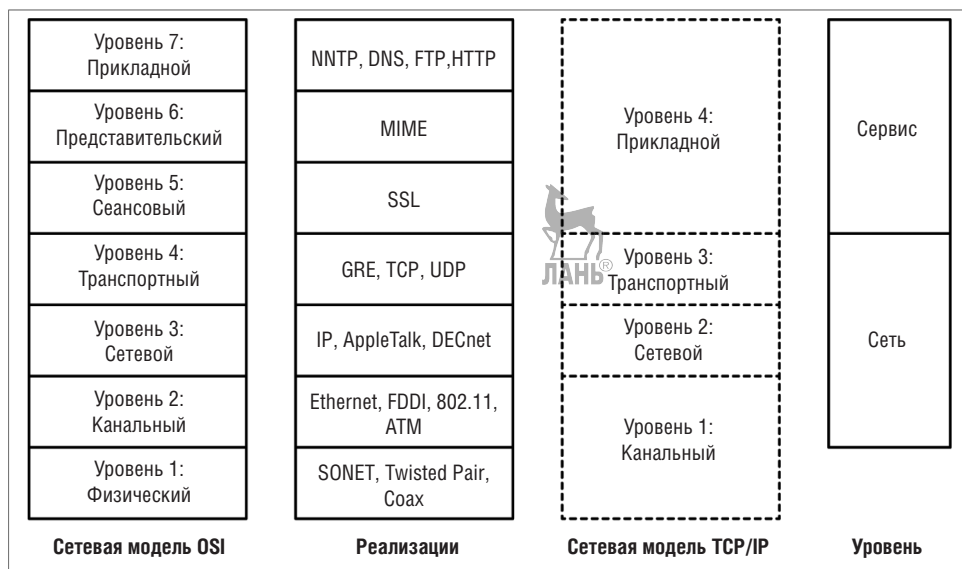


Рис. 2-1. Сетевые модели

- 1) физический. *Физический уровень* состоит из механических компонентов, соединяющих сеть воедино: провода, кабели, радиоволны и прочие механизмы, используемые для передачи данных из одного местоположения в другое;
- 2) канальный. На *канальном уровне* обеспечивается контроль за информацией, передаваемой на физическом уровне. Канальные протоколы, такие как Ethernet, обеспечивают корректную передачу асинхронных сообщений. В IP-модели канальный и физический уровни образуют единый *канальный уровень*;

- 3) сетевой. На *сетевом уровне* происходит маршрутизация трафика из одного канала в другой. В IP-модели данный уровень эквивалентен уровню 2 и также называется *сетевым*;
- 4) транспортный. На *транспортном уровне* осуществляется контроль за информацией, передаваемой на сетевом уровне. Его задачи схожи с задачами канального уровня и включают в себя управление потоками и обеспечение корректной передачи данных, хоть и в других масштабах. В IP-модели транспортный уровень – третий по счету;
- 5) сеансовый. На *сеансовом уровне* происходят создание и поддержание сеансов. Особое внимание уделяется проблемным моментам, таким как аутентификация. Типичным примером протокола сеансового уровня на сегодняшний день является SSL. Этот уровень, предоставляющий возможности кодирования и аутентификации, используется HTTP, SMTP и многими другими сервисами для обеспечения безопасной передачи данных;
- 6) представительский. На данном уровне происходит кодирование информации для отображения на более высоком уровне. Типичный пример протокола представительского уровня – MIME, протокол кодирования сообщений, используемый в электронной почте;
- 7) прикладной. К *прикладному уровню* относятся сервисы, такие как HTTP, DNS и SSH. Уровни 5–7 модели OSI соответствуют прикладному уровню (уровню 4) IP-модели.

Вот что представляют из себя сетевые модели. Именно модели, а не спецификации, а модели всегда несовершенны. Например, в TCP/IP – модель не такая подробная, как модель OSI, и в ряде случаев протоколы в данной модели могут функционировать сразу на нескольких уровнях. Контроллеры сетевого интерфейса (NIC) относятся к уровням 1 и 2 модели. Уровни взаимодействуют друг с другом в основном в части передачи данных и наблюдения за ними, а также посредством наложения ограничений производительности на более высоких уровнях.

Чаще всего мы сталкиваемся с воздействием уровней на сетевой трафик при передаче максимального модуля данных (MTU). MTU представляет собой верхнюю границу размера фрейма данных и влияет на максимальный размер пакета, который можно передать через данную среду. MTU протокола Ethernet равен 1500 байтам, и данное ограничение означает, что размер IP-пакетов, за редким исключением, не превысит этого значения.

Сетевая модель также дает четкое представление о разнице между сенсорами, расположенными на уровнях сети и сервиса. Как показано на рис. 2-1, сетевые сенсоры относятся к уровням 2–4 модели OSI, тогда как сервис-сенсоры – к уровню 5 и выше.

Сетевые модели и роль сетевых сенсоров

Возникает логичный вопрос: почему сетевые сенсоры не могут видеть всей картины? Ведь мы говорим о сетевых атаках. Кроме того, сетевые сенсоры невозможно видоизменить или удалить, как журналы хоста, и они могут видеть такие явления, как сканирование или неудавшиеся попытки установить соединение, которых не заметят журналы хоста.

Сетевые сенсоры обеспечивают широкое покрытие, но воссоздание ситуации, произошедшей в зоне этого покрытия, тем сложнее, чем выше уровень сетевой модели OSI. Начиная с уровня 5 проблема интерпретации протокола и пакетов данных встает все более остро. На уровнях 6 и 7 для извлечения значимой информации необходимо знать нюансы используемого протокола.

Реконструкция протокола на основании пакетных данных сложна и неоднозначна. Протокол TCP/IP построен по сквозному принципу, что означает, что для создания сеанса на основании пакетных данных достаточно лишь сервера и клиента. Такие инструменты, как Wireshark (глава 9) или NetWitness, способны воссоздать содержание сеанса, но лишь приблизительно.

Сетевые, сервисные и хост-сенсоры эффективны, когда дополняют друг друга. Сетевые сенсоры предоставляют информацию, которую другие сенсоры не способны зафиксировать, тогда как сервисные и хост-сенсоры фиксируют фактическое событие.

Как уже упоминалось в главе 1, область обзора сенсора – это трафик, который фиксирует данный сенсор. В компьютерных сетях область обзора включает в себя пакеты, которые видит сенсор, непосредственно благодаря способности самостоятельно передавать пакеты (посредством коммутатора или роутера) или путем их перехвата (внутри домена коллизий). Поскольку моделирование области обзора имеет огромную важность для эффективного оснащения сети, нам необходимо подробнее рассмотреть принципы работы сети.

Уровни сети и область обзора сенсоров

Наилучший способ определить область обзора сети – проанализировать движение трафика на трех различных уровнях сетевой модели OSI. Эти уровни проходят через общую шину или домен коллизий (уровень 1), сетевые коммутаторы (уровень 2) и оборудование для маршрутизации (уровень 3). На каждом уровне реализуются различные области обзора и механизмы применения последней.

Самая простая форма организации сети – через *домен коллизий*. Домен коллизий – это ресурс, используемый одним или несколькими сетевыми интерфейсами для передачи данных. Примеры доменов коллизий – сетевой концентратор (хаб) или канал, применяемый беспроводным роутером. Домен коллизий назван так потому, что отдельные элементы могут потенциально отправлять данные в одно и то же время, что заканчивается коллизией. Протоколы второго уровня включают в себя механизмы для компенсации или предотвращения коллизий.

В конечном итоге дейтаграммы уровня 2 передаются через общий источник, как показано на рис. 2-2. Сетевые интерфейсы одного домена коллизий видят одни и те же дейтаграммы, интерпретируя только те дейтаграммы, которые адресованы непосредственно им. Инструменты для сбора сетевых данных, такие как *tcpdump*, можно использовать в режиме приема всех сетевых пакетов для последующей записи всех дейтаграмм, анализируемых в рамках домена коллизий.

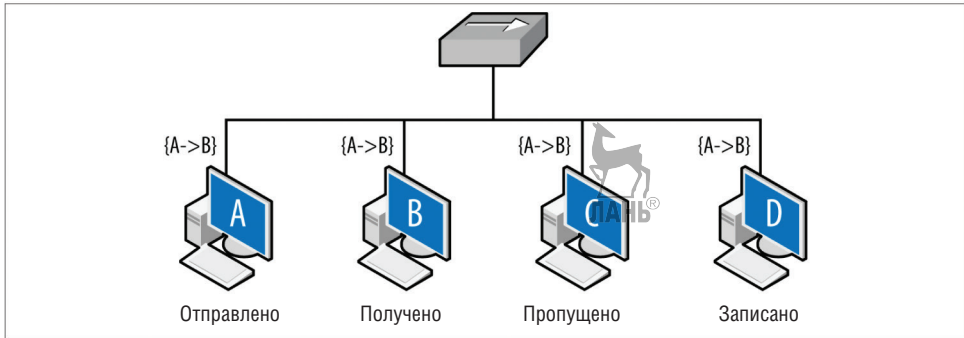


Рис. 2-2. Область обзора и домены коллизий

Рисунок 2-2 показывает область обзора в рамках широковещательного домена. Как показано на данном рисунке, начальный фрейм (из пункта А в пункт В) передается через концентратор, который выполняет роль общей шины. Каждый хост, соединенный с концентратором, может принимать фреймы и реагировать на них, но это условие обязательно только для хоста В. Хост С пропускает фрейм и сбрасывает его. Хост D, работающий в режиме приема всех сетевых пакетов, записывает фрейм. Следовательно, область обзора концентратора составляют все соединенные с ним адреса.

Общие домены коллизий неэффективны, особенно в отношении асинхронных протоколов, таких как Ethernet. Следовательно, оборудование уровня 2, например коммутаторы Ethernet, используется, как правило, для того, чтобы каждый хост, соединенный с сетью, имел собственный порт Ethernet. Это показано на рис. 2-3.

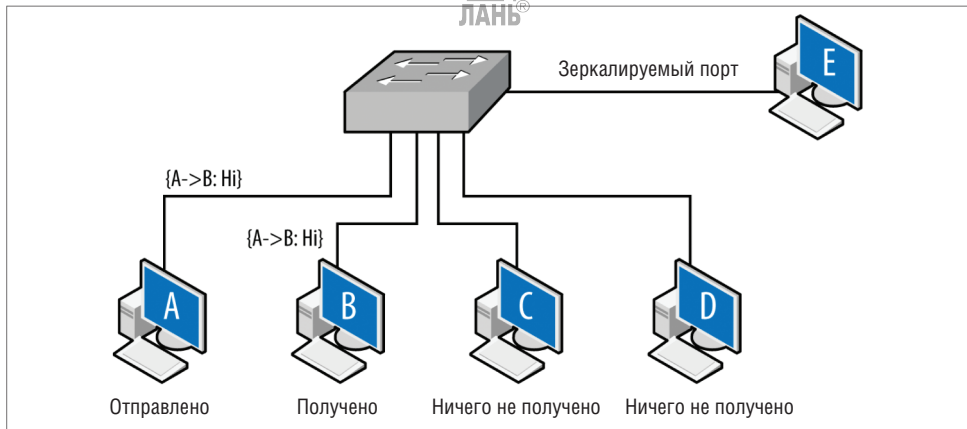


Рис. 2-3. Область обзора и коммутатор

Инструмент захвата, работающий в режиме приема всех сетевых пакетов, будет копировать каждый фрейм, получаемый интерфейсом, но коммутатор уровня 2 обеспечивает получение интерфейсом только тех фреймов, которые адресованы непосредственно ему. Следовательно, как показано на рис. 2-3, фрейм, передаваемый от А к В, был получен В, тогда как С и D не получили ничего.

Для данной проблемы существует решение на основе аппаратных средств. Большинство коммутаторов использует какой-либо вариант зеркалирования портов. Благодаря конфигурациям зеркалирования фреймы, передаваемые между различными портами, дублируются в общие зеркалируемые порты дополнительно к пункту своего первоначального назначения. Зеркалирование помогает настроить коммутатор для отправки копии каждого полученного данным коммутатором фрейма в общий интерфейс. Тем не менее зеркалирование портов может быть затратным, а большинство коммутаторов ограничивает количество интерфейсов или контролируется виртуальными локальными компьютерными сетями (VLAN).

Область обзора коммутатора является функцией порта и конфигурацией коммутатора. По умолчанию область обзора каждого отдельно взятого порта будет содержать исключительно трафик, отправляемый или принимаемый интерфейсом, соединенным с данным портом. Зеркальный порт будет видеть порты, которые он должен зеркалировать, согласно конфигурации.

На уровне 3, при возникновении проблем с маршрутизацией, возникает неразбериха с областью обзора. Маршрутизация – это полуавтономный процесс, который настраивается администратором, но он создан для обеспечения определенного уровня локальной автоматизации, чтобы повысить надежность. Кроме того, маршрутизация обладает определенными характеристиками производительности и надежности, такими как TTL (время жизни пакета данных), которые могут повлиять на мониторинг.

Область обзора на уровне 3 в самом простом выражении схожа с областью обзора уровня 2. Подобно коммутаторам, роутеры отправляют трафик через определенные порты. Для роутеров можно создать конфигурацию с функциональностью, подобной зеркалированию, но используемые термины разнятся в зависимости от производителя роутера. Основное различие состоит в том, что на уровне 3 интерфейсы работают с блоками IP-адресов, тогда как на уровне 2 используются индивидуальные IP-адреса, поскольку интерфейсы роутеров, как правило, соединены с десятками хостов посредством коммутаторов или концентраторов.

С областью обзора уровня 3 сложности возникают во время работы с несколькими интерфейсами, как показано на рис. 2-4. До настоящего момента все описываемые в данной книге области обзора были симметричными: при установке оснащения в заданной точке возможно увидеть трафик, передаваемый как из А в В, так и из В обратно в А. Многоинтерфейсный хост, подобно роутеру, имеет множество интерфейсов, способных принимать и отправлять трафик.

На рис. 2-4 показаны пример множества интерфейсов и их потенциальное влияние на область обзора уровня 3. В данном примере А и В взаимодействуют друг с другом. А отправляет пакет {А > В} в В, В отправляет пакет {В > А} в А. С и D осуществляют мониторинг в роутерах: роутер 1 настроен таким образом, что кратчайший путь из А в В лежит через него. Роутер 2 настроен таким образом, что кратчайший путь из В в А лежит через него. Совокупный эффект подобной конфигурации состоит в том, что области обзора с точек С и D асимметричны. С будет видеть трафик, передаваемый из А в В, D увидит трафик, передаваемый из В в А, но ни в одной из этих точек не будут видны оба направления движения трафика. Несмотря на то что это придуманный пример, такой вариант конфигурации может встретиться в связи с особенностями деловых отношений или нестабильностью сети. Особую сложность представляют сети, имеющие множественные интерфейсы подключения к интернету.

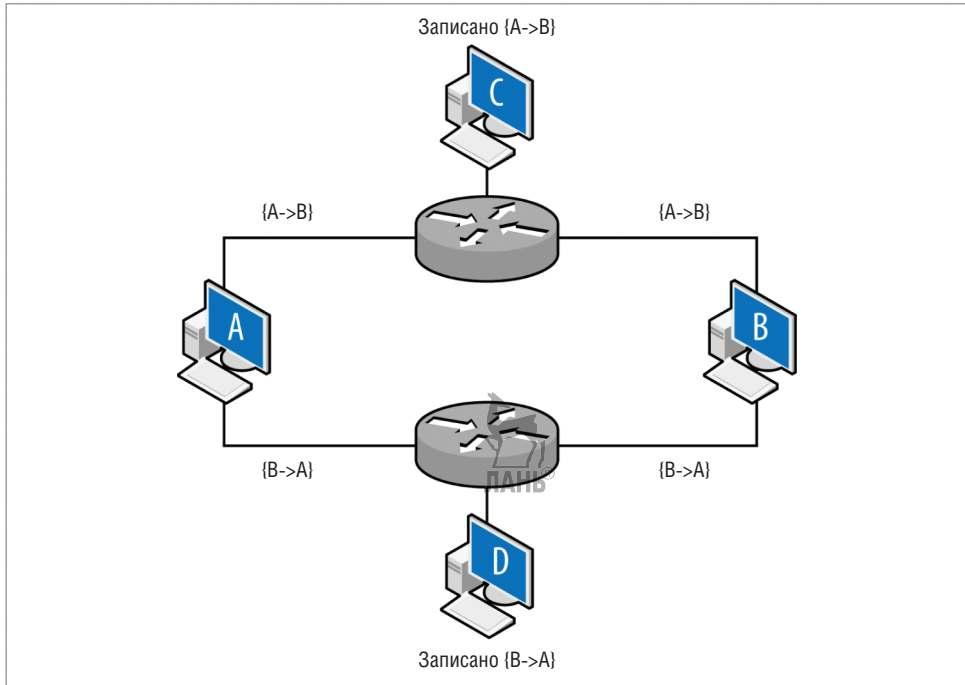


Рис. 2-4. Область обзора при работе со множественными интерфейсами

IP-пакеты имеют встроенную функцию истечения срока действия: поле, содержащее значение «*время жизни пакета данных*» (TTL). Данное значение уменьшается всякий раз, когда пакет проходит через роутер (а не устройство уровня 2, например коммутатор), пока не достигнет нуля. В большинстве случаев время жизни пакета не представляет большой проблемы, так как многие современные стеки имеют время жизни, равное минимум 64, что значительно превышает значение, необходимое для того, чтобы пересечь весь интернет. Тем не менее время жизни можно менять вручную, и существуют виды атак, при которых время жизни используется для обхода защиты. В табл. 2-1 приведены TTL операционной системы по умолчанию.

Таблица 2-1. Значения TTL по умолчанию в операционных системах

Операционная система	Значение TTL
Linux (2.4, 2.6)	64
FreeBSD	64
Mac OS X	64
Windows XP	128
Windows 7, Vista	128
Solaris	255

Рисунок 2-5 показывает работу TTL. Представим, что hosts C и D функционируют на портах контроля, а пакет данных передается из A в B. Кроме того, первоначальное значение TTL равно 2. Первый роутер получает пакет и передает его вто-

рому роутеру. Второй роутер сбрасывает пакет, иначе значение TTL будет равно 0. TTL не влияет на область обзора напрямую, но формирует своего рода слепую зону: пакеты видимы одному сенсору, но не видимы следующим за ним нескольким роутерам по мере уменьшения значения TTL.

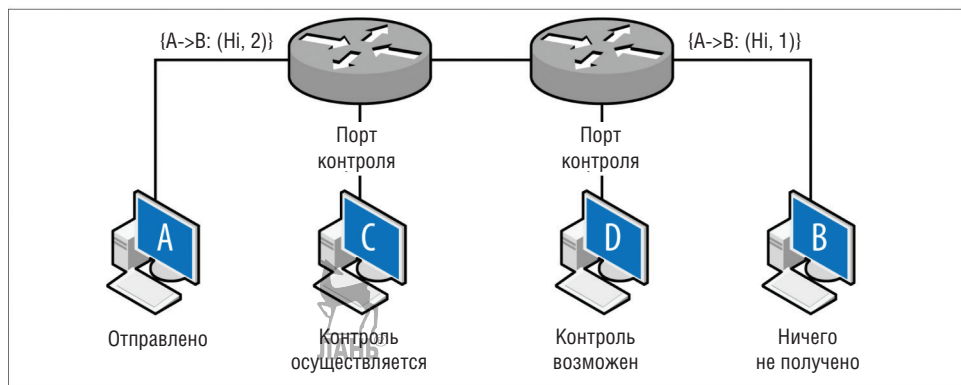


Рис. 2-5. Маршрут пакета и область обзора роутера

Результатом всего вышеупомянутого является то, что пакет попадает в область обзора C, при этом никогда не попадает в B и, возможно (в зависимости от конфигурации роутера), попадает в область обзора D.

Сетевые отводы

Вместо того чтобы конфигурировать сетевое оборудование на формирование отчетов на определенном интерфейсе, можно проверять непосредственно кабели. Это можно сделать при помощи сетевых отводов, устройств, физически присоединенных к кабелям и дублирующих трафик с целью мониторинга. Преимущество сетевых отводов состоит в том, что они освобождают сетевое оборудование от процесса сбора и копирования данных, при этом контролируя трафик в кабелях, к которым подключены.

Уровни сети и адресация

Для связи с сетевыми объектами используются различные адреса. Например, хост `www.mysite.com` может иметь IP-адрес `196.168.1.1` и Ethernet-адрес `0F:2A:32:AA:2B:14`. Эти адреса используются для распознавания объектов хоста на различных абстрактных уровнях сети. В большинстве сетей хост имеет MAC(Ethernet)-адрес, а также адреса IPv4 и IPv6.

Эти адреса модерируются в динамике посредством различных протоколов, а различные типы сетевого оборудования видоизменяют отношения между адресами. Самыми типичными примерами являются модификации системы доменных имен (DNS), которые соотносят одно имя со множественными адресами, и наоборот. Данный процесс описывается более подробно в главе 8. Как правило, сети используют следующие виды адресов:

MAC-адреса

Идентификатор размером 48 байт, используемый большинством протоколов уровня 2, в том числе и Ethernet, FDDI, Token Ring, Bluetooth и ATM. MAC-

адреса, как правило, записываются в виде 6 пар чисел в шестнадцатеричной системе исчисления (например, 12:34:56:78:9A:BC). MAC-адреса присваиваются оборудованию производителем, и первые 24 бита интерфейса резервируются для идентификационного номера производителя. Являясь адресами уровня 2, MAC-адреса не маршрутизируются. Когда фрейм передается через роутер, адресная информация заменяется адресной информацией интерфейса роутера. При помощи адресов IPv4 и IPv6 можно узнать MAC-адрес устройства, используя протокол определения адреса (ARP).

IPv4-адреса

IPv4-адрес – это 32-битное целочисленное значение, присвоенное каждому маршрутизируемому хосту, кроме той памяти, которая зарезервирована для динамических адресов (с более подробной информацией об этих адресах можно ознакомиться в главе 8). IPv4-адреса обычно представлены в виде десятичной записи через точку: 4 целых числа от 0 до 25, разделенных точками (например, 128.1.11.3).

IPv6-адреса

IPv6 – это постоянно совершенствующаяся замена для IPv4, которая помогает исправить ряд дефектов в ходе создания оригинального протокола, в частности при распределении IP-адресов. В IPv6 используются 128-битные адреса для определения хоста. По умолчанию эти адреса представляют собой набор 16-битных шестнадцатеричных значений, разделенных двоеточием (например, AAAA:2134:0918:F23A:A13F:2199:FABE:FAAF). Принимая во внимание их длину, IPv6-адреса используют ряд условных обозначений, чтобы сократить запись: нули в первоначальной записи отсекаются, и длинная последовательность 16-битных нулевых значений заменяется двойным двоеточием (например, 0019:0000:0000:0000:0000:0000:0000:0182 становится 19::182). Все эти взаимодействия динамичны, и целый ряд адресов одного уровня может быть ассоциирован с одним адресом на другом уровне. Как говорилось ранее, одно имя из системы доменных имен (DNS) может ассоциироваться с целым рядом IP-адресов посредством DNS-сервиса. Похожим образом один MAC-адрес может поддерживать несколько IP-адресов посредством протокола ARP. Такой вид динамики может использоваться как в конструктивных целях (например, для туннелирования), так и в деструктивных (например, для получения несанкционированного доступа к ресурсам сети).

ПАКЕТНЫЕ ДАННЫЕ

В контексте данной книги термин **пакетные данные** обозначает выходные данные библиотек *libpcap* через либо *COB*, либо *tcpdump*. *libpcap* была разработана компанией LBNL's Network Research Group (ЛБНЛ Нетворк Ресерч Груп) и представляет собой основную инструмент захвата трафика, а также функционирует в качестве сборщика данных для таких инструментов, как *Snort*, *bro* и *tcpdump*.

Данные о захвате пакетов – это своего рода стог, в котором вам придется искать иголки – данные, представляющие ценность. Захват этих данных – это балансирование между огромным объемом данных, которые можно собрать, и теми данными, которые действительно необходимо собрать.

Форматы пакетов и фреймов

В любой современной системе *tcpdump* будет в первую очередь захватывать скорее данные IP, нежели Ethernet. Это означает, что данные, захваченные *libpcap*, состоят из Ethernet-фреймов, содержащих IP-пакеты. Хотя IP содержит более 80 уникальных протоколов, в любой операционной системе большая часть трафика будет поступать от трех протоколов: TCP (протокол 6), UDP (протокол 17) и ICMP (протокол 1).

Хотя протоколы TCP, UDP и ICMP – источники большей части IP-трафика, целый ряд других протоколов может появиться в сетях, в частности при использовании VPN. Администрация адресного пространства интернет (IANA) содержит полный перечень протоколов IP. Среди наиболее важных – IPv6 (протокол 41), GRE (протокол 47) и ESP (протокол 50). GRE и ESP используются при работе с VPN-трафиком.

Захват всего трафика при помощи *pcap* зачастую непрактичен. Огромный размер и избыточность данных осложняет хранение значимой части сетевого трафика в течение необходимого времени. Существуют три основных механизма фильтрации и лимитирования пакетных данных: использование циклических буферов для хранения *timed*-выборки с управлением длиной захвата таким образом, чтобы захватывать только пакеты определенного размера (например, заголовки), и фильтрацией трафика с применением пакетного фильтра Беркли или другого способа фильтрации. Каждый из этих подходов является аналитическим компромиссом и имеет как свои достоинства, так и недостатки.

Кольцевой (циклический) буфер

Циклический буфер – это пространство в памяти, где данные выводятся циклически. Информация поступает последовательно, а когда буфер заполнен, первыми выводятся данные, которые поступили первыми, и процесс повторяется. Пример 2-1 показывает использование циклического буфера с *tcpdump*. В данном примере приблизительно 128 Мб записывается на диск в ходе процесса, а затем наступает очередь другого файла. После того как 32 файла буфера заполнено (наполнение определяется параметром *-W*), процесс перезапускается.

Пример 2-1. Применение циклического буфера в *tcpdump*

```
host$ tcpdump -i en1 -s 0 -w result -C 128 -W 32
```

В основе работы циклического буфера при анализе трафика лежит временной интервал, то есть данные доступны только тогда, когда они в буфере. По этой причине рекомендуется работать с файлами небольшого размера, поскольку в случае обнаружения аномалий необходимо быстро извлечь данные из буфера.

Лимитирование захваченных пакетных данных

Альтернативой захвату целого пакета данных является захват ограниченного количества полезных данных, за которое в *tcpdump* отвечает аргумент *snaplen* (*-s*). *Snaplen* ограничивает пакеты размером фрейма, заданным аргументом. Если размер фрейма, по меньшей мере, 68 байт, вы сможете записать заголовки TCP или UDP¹. Таким образом, данное решение – довольно слабая альтернатива NetFlow, речь о котором пойдет далее в этой главе.

¹ В основе *snaplen* лежит размер фрейма Ethernet, таким образом, еще 20 байт необходимо добавить к размеру соответствующих IP-заголовков.

Фильтрация специфических типов пакетов

Альтернативой фильтрации данных в роутере является фильтрация после сбора трафика в SPAN-порте. При работе с `tcpdump` и другими инструментами это можно легко сделать, используя *пакетный фильтр Беркли (BPF)*, позволяющий пользователю задавать фильтры произвольной сложности, таким образом расширяя его возможности. На рис. 2-6 представлены схемы заголовков фреймов протоколов Ethernet, IP, UDP, ICMP и TCP.

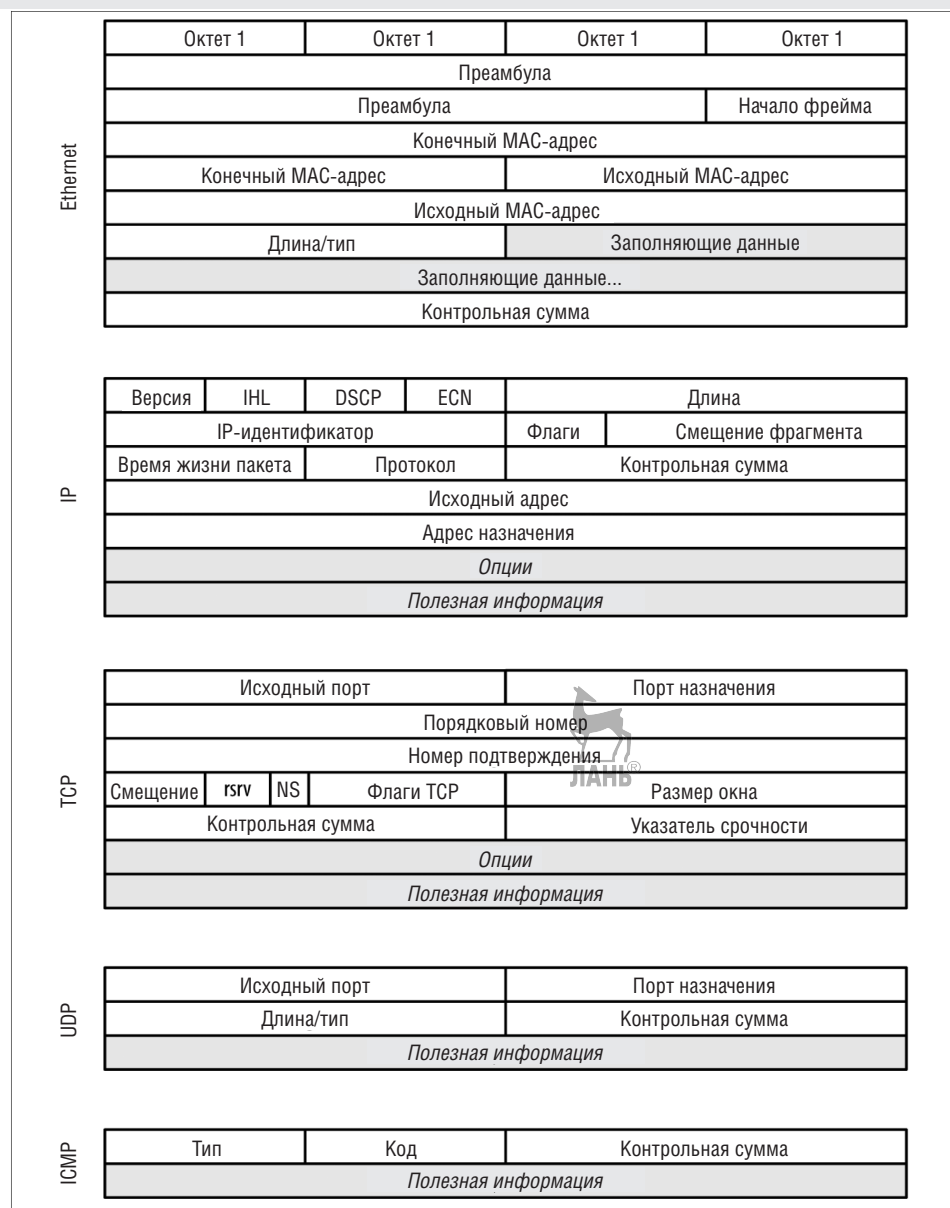


Рис. 2-6. Форматы фреймов и пакетов для протоколов Ethernet, IP, TCP, UDP и ICMP

Анализируя основные поля, мы будем определять макросы пакетного фильтра Беркли (BPF), которые можно использовать для фильтрации в данных полях. В большинстве систем типа Unix справочная страница *pcap-фильтра* содержит краткий обзор синтаксиса BPF. Перечень доступных команд также изложен на справочной странице операционной системы FreeBSD для BPF.

В Ethernet-фрейме самые важные поля – это поля с обоими MAC-адресами. Эти поля размером 48 байт каждое используются для определения аппаратного адреса интерфейсов, отправивших и принимающих трафик. MAC-адреса ограничены одним доменом коллизий и будут модифицированы при прохождении пакетом нескольких сетей (см. пример на рис. 2-5). Для доступа к MAC-адресам обычно используются предикаты *ether src* или *ether dst* при использовании BPF.

tcpdump и MAC-адреса

Большинство реализаций tcpdump требует введения параметра командной строки перед отображением информации канального уровня (в данном случае информацию протокола Ethernet). В операционной системе MAC OS X параметр *-e* покажет MAC-адреса.

Самые важные для вас поля в IP-заголовке – это IP-адреса, длина, TTL (время жизни пакета) и протокол. Поля IP-идентификатор, флаги и смещение фрагмента используются при атаках, включающих в себя повторную сборку пакетов, однако они представляют собой, скорее, исторический артефакт со времен, когда Ethernet еще не был универсальным транспортным протоколом. Доступ к IP-адресам осуществляется при помощи предикатов *src host* или *dst host*, которые также позволяют фильтровать с использованием маски подсети.

Фильтрация адресов в BPF

В BPF адреса можно фильтровать, используя различные хост- и сетевые предикаты. Для понимания принципов работы рассмотрим простой пример выводимых данных tcpdump.

```
host$ tcpdump -n -r sample.pcap | head -5
reading from file sample.pcap, link-type EN10MB (Ethernet)
20:01:12.094915 IP 192.168.1.3.56305 > 208.78.7.2.389: Flags [S],
  seq 265488449, win 65535, options [mss 1460,nop,wscale 3,nop,
  nop,TS val 1111716334 ecr 0,sackOK,eol], length 0
20:01:12.094981 IP 192.168.1.3.56302 > 192.168.144.18.389: Flags [S],
  seq 1490713463, win 65535, options [mss 1460,nop,wscale 3,nop,
  nop,TS val 1111716334 ecr 0,sackOK,eol], length 0
20:01:12.471014 IP 192.168.1.102.7600 > 192.168.1.255.7600: UDP, length 36
20:01:12.861101 IP 192.168.1.6.17784 > 255.255.255.255.17784: UDP, length 27
20:01:12.862487 IP 192.168.1.6.51949 > 255.255.255.255.3483: UDP, length 37
```

Предикаты *src host* или *dst host* помогут отфильтровать по конкретным IP-адресам. Отфильтровать только трафик, отправленный или полученный с помощью 192.168.1.3, можно следующим образом:

```
host$ tcpdump -n -r sample.pcap src host 192.168.1.3 | head -1
reading from file sample.pcap, link-type EN10MB (Ethernet)
20:01:12.094915 IP 192.168.1.3.56305 > 208.78.7.2.389: Flags [S],
```



```
seq 265488449, win 65535, options [mss 1460,nop,wscale 3,nop,
  nop,TS val 1111716334 ecr 0,sackOK,eol], length 0
host$ tcpdump -n -r sample.pcap dst host 192.168.1.3 | head -1
reading from file sample.pcap, link-type EN10MB (Ethernet)
20:01:13.898712 IP 192.168.1.6.48991 > 192.168.1.3.9000: Flags [S],
  seq 2975851986, win 5840, options [mss 1460,sackOK,TS val 911030 ecr 0,
  nop,wscale 1], length 0
```

src net и dst net позволяют фильтровать по блокам адресов. Ниже приведен пример того, как фильтровать адреса диапазона 192.168.1, используя лишь адрес или запись бесклассовой адресации (CIDR).

```
# use src net to filter just by matching octets
host$ tcpdump -n -r sample.pcap src net 192.168.1 | head -3
reading from file sample.pcap, link-type EN10MB (Ethernet)
20:01:12.094915 IP 192.168.1.3.56305 > 208.78.7.2.389: Flags [S],
  seq 265488449, win 65535, options [mss 1460,nop,wscale 3,nop,nop,
  TS val 1111716334 ecr 0,sackOK,eol], length 0
20:01:12.094981 IP 192.168.1.3.56302 > 192.168.144.18.389: Flags [S],
  seq 1490713463, win 65535, options [mss 1460,nop,wscale 3,nop,
  nop,TS val 1111716334 ecr 0,sackOK,eol], length 0
# Match an address
host$ tcpdump -n -r sample.pcap src net 192.168.1.5 | head -1
reading from file sample.pcap, link-type EN10MB (Ethernet)
20:01:13.244094 IP 192.168.1.5.50919 > 208.111.133.84.27017: UDP, length 84
# Match using a CIDR block
host$ tcpdump -n -r sample.pcap src net 192.168.1.64/26 | head -1
reading from file sample.pcap, link-type EN10MB (Ethernet)
20:01:12.471014 IP 192.168.1.102.7600 > 192.168.1.255.7600: UDP, length 36
```

Для фильтрации по протоколам используйте предикат ip proto. Пакетный фильтр Беркли предоставляет целый ряд предикатов для различных протоколов, таких как tcp, udp и icmp. Фильтровать по длине пакета также возможно, используя предикаты less и greater, тогда как фильтрация по времени жизни требует более сложных манипуляций с битами, как описано далее. Следующий фрагмент демонстрирует фильтрацию всего трафика, за исключением того, который поступает в рамках данного блока (хосты с маской подсети /24):

```
host$ tcpdump -i en1 -s 0 -w result src net 192.168.2.0/24
```

Пример 2-2 демонстрирует фильтрацию в tcpdump.

Пример 2-2. Примеры фильтрации с использованием tcpdump

```
host$ # Filtering out everything but internal traffic
host$ tcpdump -i en1 -s 0 -w result src net 192.168.2.0/24 && dst net \
  192.168.0.0/16
host$ # Filtering out everything but web traffic, identified by port
host$ tcpdump -i en1 -s 0 -w result ((src port 80 || src port 443) && \
  (src net 192.168.2.0))
```

При работе с протоколом TCP важную роль играют номер порта и флаги. Флаги TCP используются для обслуживания машины состояний TCP, а порты используются для того, чтобы разделять сеансы, и для сервисной индикации. Используя параметры src port и dst port, можно применить фильтрацию по номерам портов.

`src portrange` и `dst portrange` позволяют фильтровать по целому ряду значений порта. Пакетный фильтр Беркли поддерживает предикаты для флагов TCP, такие как `tcp-fin`, `tcp-syn`, `tcp-rst`, `tcp-push`, `tcp-ack` и `tcp-urg`.

Адресные классы и блоки CIDR

IPv4-адрес – это 32-битное целое число. Для удобства отображения адреса записываются в четырехкомпонентной системе обозначения адресов с точками, например 01.02.03.04. Таким образом, IP-адреса, представленные в виде 0x000010FF, имеют запись 0.0.16.255. Маршрутизация уровня 3 почти всегда задействует группы адресов, а не индивидуальные адреса. Ранее эти группы назывались классами, сейчас – блоками.

Ранее адреса класса A (0.0.0.0–127.255.255.255) имели старший бит, равный нулю, следующие 7 бит составляли адрес сети, и оставшиеся 24 бита были под контролем владельца. Данная схема позволяла владельцу работать с 224 адресами. Адрес класса B (128.0.0.0–191.255.255.255) предоставлял 16 бит владельцу, а адрес класса C (192.0.0.0–223.255.255.255) – 8 бит. Данный подход быстро привел к исчерпанию адресов, и в 1993 г. была разработана *бесклассовая адресация* (Classless Inter-Domain Routing – CIDR), чтобы заменить примитивную классовую систему. Согласно бесклассовой системе, блок задается адресом и маской подсети. Маска подсети определяет, какие биты адреса пользователь может изменять, и, согласно правилу, эти биты имеют нулевое значение. Например, пользователь, владеющий адресами 192.28.3.0–192.28.3.255, получит блок 192.28.3.0/24.

Как и в случае с TCP, номера портов протокола UDP имеют наибольшее значение. Доступ к ним осуществляется посредством тех же параметров `port` и `portrange`.

Поскольку протокол ICMP является интернет-протоколом передачи сообщений об ошибках, сообщения ICMP могут содержать большой объем данных. Тип и код ICMP очень важны, так как они определяют синтаксис для всего, что содержит полезную информацию. Пакетный фильтр Беркли содержит ряд фильтров по типу и коду, таких как `icmp-echo-reply`, `icmp-unreach`, `icmp-tstamp` и `icmp-redirect`.

Если вы не используете Ethernet

Для краткости в данной книге большее внимание уделяется протоколу IP и сети Ethernet, но вы также можете найти здесь информацию о других транспортных протоколах и протоколах передачи данных. Многие из этих протоколов узкоспециализированы и могут потребовать дополнительное ПО захвата, помимо инструментов, построенных на `libcap`.

Асинхронный способ передачи данных (ATM)

ATM, главный конкурент IP, получивший широкое распространение в 1990-х гг., сегодня широко применяется для передачи данных с цифровой сети с интеграцией служб (ISDN), а также в телефонных сетях общего пользования и некоторых устаревших версиях сетей.

Оптический канал

Волоконно-оптический канал в основном применяется в работе с быстродействующими ЗУ и является основой целого ряда реализаций сети хранения данных.

Сеть контроллеров (CAN)

CAN (controller area network) – *сеть контроллеров*, в основном ориентированная на встроенные системы, такие как децентрализованные транспортные сети. CAN – это протокол общей шины, используемый для отправки сообщений в небольших изолированных сетях.

Любая форма фильтрации предполагает потери в производительности. Использование SPAN-порта на коммутаторе или роутере влечет за собой потери в производительности, которая могла бы быть использована при работе с трафиком. Чем сложнее фильтр, тем ниже производительность. Для нетривиальных полос частот это может стать проблемой.

NetFlow

NetFlow – это стандарт резюмирования трафика, разработанный компанией Cisco Systems и первоначально использовавшийся для биллинга сетевых служб. Хотя NetFlow не был разработан для решения вопросов безопасности, он невероятно полезен в данной области, т. к. предоставляет быстродоступные, краткие сведения о сеансах трафика сети и содержит ценнейшую информацию, которую можно относительно компактно хранить. NetFlow широко используется в целях обеспечения информационной безопасности со времен публикации первого *пакета инструментов* в 1999 г. Кроме того, был разработан ряд инструментов NetFlow с дополнительными полями, такими как избранные фрагменты кода, содержащие полезную информацию.

В основе работы NetFlow лежит *поток* – концепция, сопоставимая с сеансом TCP. Мы знаем, что TCP-сеансы создаются в конечной точке (рабочей станции) путем сравнения порядковых номеров. Жонглирование всеми порядковыми номерами, задействованными в многочисленных TCP-сеансах, невозможно в роутере, но зато можно достичь достаточно точной аппроксимации, используя время ожидания. Поток – это совокупность идентично адресованных пакетов, тесно сгруппированных по времени.

Форматы и поля NetFlow v5

NetFlow v5 – это самый ранний общий стандарт NetFlow. Перед тем как обсуждать альтернативы, необходимо обратить внимание на значение полей в NFv5. Поля NetFlow v5, описанные в табл. 2-2, можно разделить на три большие категории: поля, копируемые напрямую из IP-пакетов, поля, обобщающие результаты IP-пакетов, и поля, относящиеся к маршрутизации.

Таблица 2-2. Поля NetFlow v5

Байты	Наименование	Описание
0–3	srcaddr	Source IP address
4–7	dstaddr	Destination IP address
8–11	nexthop	Address of the next hop on the router
12–13	input	SNMP-индекс входного интерфейса
14–15	output	SNMP-индекс выходного интерфейса
16–19	packets	Пакеты в потоке

Байты	Наименование	Описание
20–23	d0ctets	Число бит 3 уровня в потоке
24–27	first	sysuptime в начале потока ¹
28–31	last	sysuptime в момент получения пакета последнего потока
32–33	srcport	Порт источника TCP/UDP
34–35	dstport	Порт назначения TCP/UDP, код и тип ICMP
36	pad1	Холостое заполнение
37	tcp_flags	OR всех TCP-флагов потока
38	prot	Протокол IP
39	tos	Тип сервиса IP
40–41	src_as	ASN-номер источника
42–43	dst_as	ASN-номер пункта назначения
44	src_mask	Маска префикса адреса источника
45	dst_mask	Маска префикса адреса назначения
46–47	pad2	Заполняющие байты

¹ Это значение относительно системного времени работы маршрутизатора.

Поля srcaddr, dstaddr, srcport, dstport, prot и tos записи NetFlow копируются напрямую из соответствующих полей IP-пакетов. Тем не менее потоки генерируются для каждого протокола IP-семейства, а это значит, что поля srcport и dstport, которые, строго говоря, представляют собой TCP/UDP-явления, не обязательно имеют какое-то значение. В случае с протоколом ICMP NetFlow записывает тип и код поля dstport. Игнорируйте данное значение при работе с другими протоколами.

Поля packets, d0ctets, first, last и tcp_flags содержат основные данные о трафике из одного или нескольких пакетов. Поля packets и d0ctets содержат просто суммарные величины, с оговоркой, что значение поля d0ctets – это суммарное количество октетов уровня 3. Это означает добавление IP-заголовков и заголовков протокола (например, однопакетный поток TCP, не содержащий полезной информации, будет иметь размер 40 бит, а однопакетный UDP-поток, не содержащий полезной информации, – 28 бит). Значения полей first и last содержат информацию о первом и последнем появлениях пакета в потоке.

Поле tcp_flags – это особый случай. В NetFlow v5 поле tcp_flags является результатом выполнения логической операции OR над всеми флагами, появившимися в этом потоке. В хорошо организованных потоках это означает, что флаги SYN, FIN и ACK будут всегда установлены. И наконец, последний набор полей, nexthop, input, output, src_as, dst_as, src_mask и dst_mask, имеет отношение к маршрутизации. Эти значения могут быть собраны только в роутере.

«Поток и наполнение». NetFlow v9 и стандарт IPFIX

За все время существования NetFlow компания Cisco разработала несколько его версий, в результате чего версия NetFlow v5 стала наиболее надежной реализацией стандарта. Но v5 является устаревшим и ограниченным стандартом, фокусирующимся в основном на IPv4 и разработанным еще до массового использования потоков. Решением Cisco этого был NetFlow v9, основанный на шаблоне стандарт

создания отчетов потока, который позволил администраторам маршрутизатора определить, какие поля были включены в поток.

Основанный на шаблоне NetFlow был с тех пор стандартизирован IETF как IPFIX¹. IPFIX обеспечивает несколько сотен потенциальных полей для потоков, которые описаны в RFC 5102.

Приоритетом данного стандарта является скорее мониторинг сети и анализ трафика, нежели информационная безопасность. Для обращения к опциональным полям IPFIX использует концепцию «вендорного пространства». В процессе создания инструментария SiLK группа специалистов из Университета Карнеги-Меллон, ответственная за ситуационную осведомленность при работе с сетями, разработала и предоставила ряд полей, находящихся в своем вендорном пространстве IPFIX и представляющих большую важность для анализа безопасности.

Генерация и сбор данных в NetFlow

Записи NetFlow генерируются непосредственно сетевым оборудованием, таким как роутер, или сетевым коммутатором, или при помощи программного обеспечения с целью преобразования пакетов в потоки. Каждый подход имеет свои плюсы и минусы.

Генерирование при помощи оборудования предполагает использование любых устройств для работы с NetFlow, предлагаемых производителями оборудования. Различные производители предлагают свои устройства под разными именами, но со схожими с продуктом Cisco звучаниями, например Jflow от Juniper Networks или NetStream от Huawei. Поскольку NetFlow предлагается целым рядом производителей с набором различных характеристик, провести технический обзор необходимых конфигураций в рамках данной книги не представляется возможным. Тем не менее необходимо рассмотреть следующие основные правила:

- генерация в NetFlow может негативно сказаться на продуктивности роутеров, особенно это касается старых моделей. Производители решают данную проблему различными способами, начиная с уменьшения приоритета процесса (и удаления записей) и заканчивая делегированием задач по генерации NetFlow в опциональное (и дорогое) оборудование;
- большинство конфигураций NetFlow по умолчанию с некоторой формой выборки для сокращения загрузки производительности. Для анализа безопасности NetFlow должен быть сконфигурирован к просмотру невыбранных записей;
- многие конфигурации NetFlow предлагают несколько форматов агрегации и отчетов. Необходимо осуществлять сбор сырых данных NetFlow, а не агрегаций.

Альтернативой сбору данных в роутере является использование приложения, которое генерирует NetFlow из данных рсар, например инструмент YAF от CERT, softflowd или Argus – набор инструментов для проведения масштабного мониторинга потока от QoSient. Эти приложения берут данные рсар в виде файлов или напрямую от сетевого интерфейса и агрегируют пакеты в потоки. Данные сенсоры лишены области обзора роутера, но в то же время располагают большими

¹ RFC 5101, 5102 и 5103.

ресурсами для обработки и анализа пакетов, обеспечивая тем самым лучшие результаты работы NetFlow.

ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ ДЛЯ ЧТЕНИЯ

1. *Ричард Беджтлик (Richard Bejtlich)*. Дао контроля сетевой безопасности: вне обнаружения проникновения (Addison–Wesley, 2004).
2. *Кевин Фол, Ричард Стивенс (Kevin Fall, Richard Stevens)*. Иллюстрированный TCP/IP. Том 1: Протоколы (Вып. 2) (Addison–Wesley, 2011).
3. *Майкл Лукас (Michael Lucas)*. Анализ сетевых потоков (No Starch Press, 2010).
4. *Радия Перлман (Radia Perlman)*. Взаимосвязи: мосты, маршрутизаторы, коммутаторы и протоколы межсетевого взаимодействия (Вып. 2) (Addison–Wesley, 1999).
5. *Крис Сандерс (Chris Sanders)*. Практический пакетный анализ: использование Wireshark для решения реальных проблем (No Starch Press, 2011).



Глава 3

.....

Датчики хостов и сервисов: журналирование трафика в источнике данных

В этой главе мы рассматриваем датчики, работающие в сервисном домене или хосте. Датчики хоста включают системные журналы, а также средства обеспечения безопасности хоста, такие как антивирусное программное обеспечение (AV) и система предотвращения проникновений хоста (HIPS) McAfee. Датчики хоста контролируют состояние хоста и его операционной системы, отслеживая функции, такие как использование локального диска и периферийный доступ. Сервисные датчики, включая журналы сервера HTTP и журналы передачи почты, описывают действие конкретного сервиса: кто и кому отправил почту, к каким URL был получен доступ за последние пять минут и другие действия этого сервиса. Далее «журнал» будет использован для обозначения как журналов хоста, так и журналов сервисов.

При возможности журналы зачастую более предпочтительны, чем сетевые данные, потому что они сгенерированы самим процессом и исключают процесс интерпретации и догадок, часто необходимых с сетевыми данными. Хост и сервисные журналы предоставляют конкретную информацию о событиях, которые, используя сетевые данные, трудно восстановить.

Журналы имеют ряд проблем. Самая важная, являющаяся головной болью для управления, – чтобы воспользоваться журналом, нужно знать, что он существует, и иметь к нему доступ. Кроме того, журналы хоста имеют большое количество форматов, многие из них плохо документированы. Рискую широким обобщением, подавляющее большинство журналов разработано для отладки, а также поиска и устранения неисправностей отдельных хостов, а не для оценки безопасности в сетях. По возможности, вам будет необходимо реконфигурировать их для включения большей информации, важной для безопасности, возможно, даже путем написания собственных программ агрегации. Наконец, журналы являются целью; атакующие изменяют или отключают журналирование, если это возможно.

Журнал дополняет сетевые данные. Сетевые данные способны находить мертвые зоны, подтверждая результаты журналирования и идентифицируя вещи, которые журналы отобразить не смогут. Система эффективной безопасности комбинирует сетевые журналы для широкого обзора и журналы для детализации.

И наконец, сфокусируемся на данных ряда журналов хоста, включая системные файлы журнала. Начнем с обсуждения нескольких вариантов данных журналов и предпочтительных форматов сообщений. А затем обсудим определенные хост и сервисные журналы: системные журналы Unix, форматы журнала сервера HTTP и почтовые форматы журнала.

Доступ и управление файлами журнала

Операционные системы имеют десятки процессов, генерирующих данные журналов в любое время. В системах Unix эти файлы журнала обычно сохранены как текстовые файлы в каталоге `/var/log`. Пример 3-1 отображает этот каталог для Mac OS X (замещающие знаки указывают, где строки были удалены для большей ясности).

Пример 3-1. Каталог `/var/log` от системы Mac OS X



drwxr-xr-x	2 uucp	wheel	68	Jun	20	2012	uucp
...							
drwxr-xr-x	2 root	wheel	68	Dec	9	2012	apache2
drwxr-xr-x	2 root	wheel	68	Jan	7	1:47	ppp
drwxr-xr-x	3 root	wheel	102	Mar	12	12:43	performance
...							
-rw-r--r--	1 root	wheel	332	Jun	1	5:30	monthly.out
-rw-r-----	1 root	admin	6957	Jun	5	0:30	system.log.7.bz2
-rw-r-----	1 root	admin	5959	Jun	6	0:30	system.log.6.bz2
-rw-r-----	1 root	admin	5757	Jun	7	0:30	system.log.5.bz2
-rw-r-----	1 root	admin	5059	Jun	8	0:30	system.log.4.bz2
-rw-r--r--	1 root	wheel	870	Jun	8	3:15	weekly.out
-rw-r-----	1 root	admin	10539	Jun	9	0:30	system.log.3.bz2
-rw-r-----	1 root	admin	8476	Jun	10	0:30	system.log.2.bz2
-rw-r-----	1 root	admin	5345	Jun	11	0:31	system.log.1.bz2
-rw-r--r--	1 root	root	131984	Jun	11	18:57	vnetlib
drwxrwx---	33 root	admin	1122	Jun	12	0:23	DiagnosticMessages
-rw-r-----	1 root	admin	8546	Jun	12	0:30	system.log.0.bz2
-rw-r--r--	1 root	wheel	108840	Jun	12	3:15	daily.out
-rw-r--r--	1 root	wheel	22289	Jun	12	4:51	fsck_hfs.log
-rw-r-----	1 root	admin	899464	Jun	12	20:11	install.log

Отметим несколько функций этого каталога. Файлы `system.log` ежедневно запускаются в 00:30 и дифференцируются численно. Существует ряд подкаталогов для обработки различных сервисов. Проверьте конфигурацию каждого отдельного сервиса, для которого хотите получить файлы журнала. Однако системам Unix весьма свойственно вывести их к подкаталогу `/var/log` по умолчанию.

Файлы журнала Unix являются почти всегда простым текстом. Например, краткий отрывок системы регистрирует чтение следующим образом:

```
$ cat system.log
Jun 19 07:24:49 local-imac.home loginwindow[58]: in pam_sm_setcred(): Done getpwnam ()
Jun 19 07:24:49 local-imac.home loginwindow[58]: in pam_sm_setcred(): Done
```



```
setegid() & seteuid()
```

```
Jun 19 07:24:49 local-imac.home loginwindow[58]: in pam_sm_setcred (): pam_sm_setcred: krb5  
user admin doesn't have a principal
```

```
Jun 19 07:24:49 local-imac.home loginwindow[58]: in pam_sm_setcred (): Done cleanup3
```

Большинство системных журналов Unix является текстовыми сообщениями, созданными путем заполнения шаблонов с информацией об определенном событии. Этот вид *шаблонного текста* легко считать, но он не очень хорошо масштабируется.

Как и Vista, Windows экстенсивно обновил их структуру журналирования. Windows распознает два класса файлов журнала: журналы Windows и журналы приложения/сервиса. Далее журналы Windows подразделяются на пять классов.

Журнал приложения (Application log)

Журнал приложения содержит сообщения из отдельных приложений. Обратите внимание на то, что сервисы, такие как IIS, могут использовать вспомогательные журналы для дополнительной информации.

Журнал безопасности (Security log)

Содержит события безопасности, такие как попытки входа в систему и изменения политики аудита.

Системный журнал (System log)

Сообщения о состоянии системы, такие как отказы драйвера.

Журнал переадресации (Forwardedevents)

Хранит события от удаленных хостов.

Эти журналы зарегистрированы в %SystemRoot%\System32\Config по умолчанию на большей части установки Windows; однако более эффективный механизм для доступа к файлам и их чтения – это использование Windows Event Viewer, как видно на рис. 3-1.

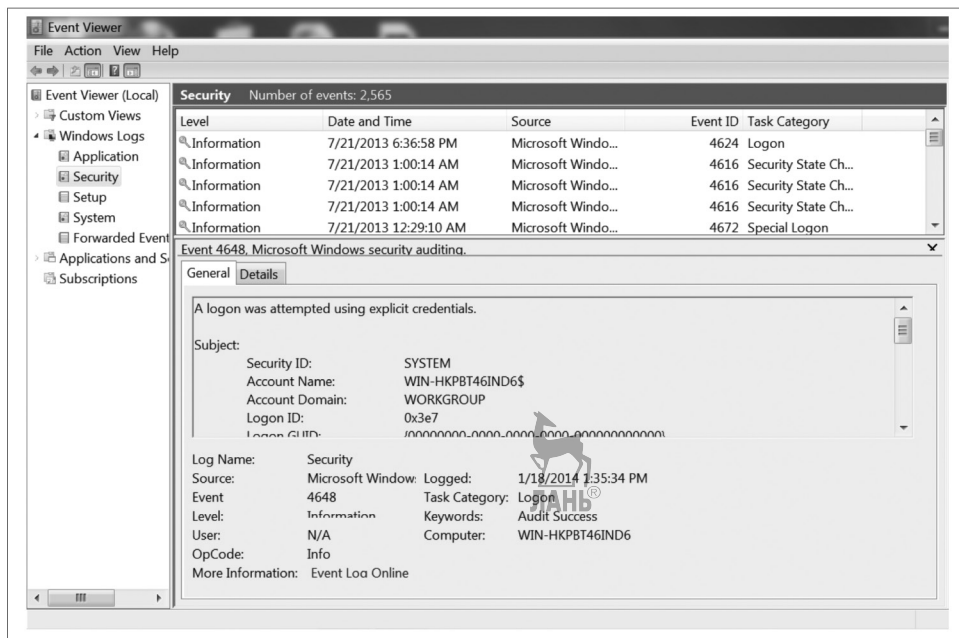


Рис. 3-1. Журнал событий Windows

Обратите внимание на использование идентификатора события (Event ID) на рис. 3-1. Как и в системах Unix, сообщения о событиях Windows являются шаблонным текстом, хотя Windows явно идентифицирует тип события с помощью уникального цифрового кода. Эти сообщения доступны с веб-сайта Microsoft.

Файлы журнала приложения расположены значительно менее последовательно. Как видно в `/var/log`-каталоге, административная структура может быть настроена для записи файла журнала в определенном месте, но почти каждое приложение может переместить файлы журнала при необходимости. При работе с конкретным приложением ознакомьтесь с его документацией для определения места нахождения журналов.

СОДЕРЖАНИЕ ФАЙЛОВ ЖУРНАЛА

Как правило, журналы разрабатываются для проведения отладки, а также поиска и устранения неисправностей администратором хоста. Поэтому часто можно обнаружить, что журналы хоста требуют определенной степени обработки (parsing) и определенной степени переупорядочивания, чтобы сделать их пригодными для журналов безопасности. В этом разделе мы обсуждаем механизмы для интерпретации, поиска и устранения неисправностей, а также преобразования данных журналов хоста.

Характеристики хорошего сообщения журнала

Прежде чем обсудить, как преобразовать сообщение журнала, и жаловаться на то, как плохо большинство из сообщений журнала, нам надлежит описать, каким должно быть хорошее сообщение безопасности. Хороший журнал безопасности должен быть *описательным*, он должен быть *связанным* с другими данными и должен быть *полным*.

Описательным является сообщение, которое содержит достаточно для анализа информации для идентификации всех необходимых доступных ресурсов для события, описанного сообщением. Например, записи журнала хоста о том, что пользователь предпринял попытку нелегального доступа к файлу, должны содержать ID пользователя и файл доступа. Журнал хоста, записывающий изменение в полномочиях группы для пользователя, должен содержать запись о пользователе и группе. Журнал, записывающий неудавшуюся попытку удаленного входа в систему, должен включать ID, который делал попытку входа в систему, и адрес, который делал попытку входа в систему.

Например, рассмотрим сообщение журнала о неудавшейся попытке входа в систему на хосте 192.168.2.2, локальное имя *myhost*. Неописательное сообщение было бы похоже на это:

```
Mar 29 11:22:45.221 myhost sshd[213]: Failed login attempt
```

Это сообщение ничего не говорит о том, почему произошел отказ, и не содержит никакой информации для определения разницы между этой и любыми другими неудачными попытками входа в систему. Информации о цели нападения тоже нет: направлено ли это против учетной записи *администратора* или некоего пользователя? Аналитики, имея только эту информацию, должны будут восстановить попытку лишь по синхронизации данных и не смогут даже быть уверены,

с тем ли хостом связались, потому что название хоста является неописательным и нет никакой адресной информации.

Более описательное сообщение может быть таким:

```
Mar 29 11:22:45.221 myhost (192.168.2.2) sshd[213]: Failed
login attempt from host 192.168.3.1 as 'admin',
incorrect password
```

Хорошее упражнение для ума при создании описательного сообщения – это обратиться к подходу следователей и журналистов «пяти Ws и одному H»: кто, что, когда, где, почему и как. Неописательный журнал отвечает на вопросы «что» (неудачная попытка доступа) и «когда», а также предоставляет частичный ответ на вопрос «где» (*myhost*). Описательный журнал отвечает еще и на вопросы «кто» (192.168.3.1 как *администратор*), «почему» и «как» (неправильный пароль) и обеспечивает лучшее «где».

Узнаваемое сообщение – это такое сообщение, где событие легко связано с информацией из других источников. Для событий хоста это требует IP-адреса и информации синхронизации, включая сведения, было событие удаленным или физически локальным. Если событие было удаленным – IP-адрес и порт удаленного события и IP-адрес и порт хоста. Узнаваемость является конкретной головной болью при работе с сервисными журналами, поскольку эти типы журналов часто представляют дополнительные схемы адресации поверх IP. Например, это неузнаваемое сообщение журнала почты:

```
Mar 29 11:22:45.221 myhost (192.168.2.2) myspamapp[213]:
Message <21394.283845@spam.com> title 'Herbal Remedies and Tiny Cars'
'from 'spammer@spam.com' rejected due to unsolicited commercial content
```

Сообщение содержит много информации, но не позволяет связать сообщение, отправленное обратно, с конкретным IP-адресом, который отправил сообщение. При рассмотрении сообщений журнала решите, как можно соотнести эту информацию с другими источниками, особенно с сетевым трафиком. Более узнаваемое сообщение может быть таким:

```
Mar 29 11:22:45.221 myhost (192.168.2.2) myspamapp[213]:
Message <21394.283845@spam.com> title 'Herbal Remedies and Tiny Cars' from
'spammer@spam.com' at SMTP host 192.168.3.1:2034 rejected due to unsolicited commercial
content
```

Этот пример включает клиентский порт и адресную информацию, таким образом, теперь можно связать его с сетевым трафиком.

Полное сообщение журнала – это сообщение, которое содержит всю информацию о конкретном событии в рамках данного сообщения. Полнота сокращает количество записей, которые аналитик должен перерывать, и предоставляет аналитику ясный индикатор того, что нет никакой дополнительной информации об этом процессе, которую еще можно получить. Неполные сообщения обычно являются функцией сложного процесса. Например, инструмент против спама использует несколько фильтров для сообщения, и для каждого фильтра и окончательного решения будет отдельная строка в журнале. Например:

```
Mar 29 11:22:45.221 myhost (192.168.2.2) myspamapp[213]:
Received Message <21394.283845@spam.com> title
'Herbal Remedies and Tiny Cars' from 'spammer@spam.com' at
SMTP host 192.168.3.1:2034
```

```
Mar 29 11:22:45.321 myhost (192.168.2.2) myspamapp[213]:  
Message <21394.283845@spam.com> passed reputation filter  
Mar 29 11:22:45.421 myhost (192.168.2.2) myspamapp[213]:  
Message <21394.283845@spam.com> FAILED Bayesian filter  
Mar 29 11:22:45.521 myhost (192.168.2.2) myspamapp[213]:  
Message <21394.283845@spam.com> Dropped
```

С неполными сообщениями необходимо отслеживать состояние через многокомпонентные сообщения, каждое из которых дает только фрагмент информации и которые необходимо группировать, чтобы сделать какой-либо полезный анализ. Следовательно, предпочтительно, чтобы сообщение было изначально агрегировано, как это:

```
Mar 29 11:22:45.521 myhost (192.168.2.2) myspamapp[213]:  
Received Message <21394.283845@spam.com> title  
'Herbal Remedies and Tiny Cars' from 'spammer@spam.com' at  
SMTP host 192.168.3.1:2034 reputation=pass Bayesian=FAIL decision=DROP
```

Сообщения журнала часто могут быть лишь минимально модифицируемы. Поэтому для создания эффективного сообщения, возможно, придется написать некоторый алгоритм журналирования. Например, сообщения системного журнала можно получить, разобрать и преобразовать в более дружественный формат, а затем передать их. Если решено преобразовывать файлы журнала, в дополнение к правилам, описанным выше, необходимо учесть следующее.

Преобразуйте время во время эпохи

Почти вся корреляция записей включает идентификацию одного и того же явления от разных датчиков, это значит, что необходимо искать записи, которые близки по времени. Преобразование всех временных интервалов во время эпохи уменьшает сложность разбора, избавляет от кошмара часовых поясов и летнего времени и гарантирует последовательную обработку для последовательных записей.

Удостоверьтесь, что датчики синхронизированы

В дополнение к первому примечанию удостоверьтесь, что когда датчики сообщают об одном и том же событии, они сообщают одно и то же время. Исправить что-то постфактум ужасно трудно, поэтому удостоверьтесь, что все датчики скоординированы, что они все сообщают об одном и том же времени, а также что часы исправны и регулярно синхронизируются.

Включайте адресную информацию

Везде, где возможно, включайте поток из пяти данных (источник IP, целевой IP, исходный порт, целевой порт, протокол). Если некоторые данные могут быть получены из самой записи (например, HTTP-серверы работают поверх протокола TCP), они могут быть пропущены.

Убедитесь, что разделители поняты регистратором

Иногда встречаются услужливые администраторы, реконфигурирующие журналы HTTP для использования знаков пайп «|», а не пробелов в качестве разделителей. Достойное желание, кроме тех случаев, когда регистрирующий модуль не знает, что эти знаки нужно убирать, если они появляются в тексте. Если регистратор может изменить свой разделитель и понимает, что изменение требует ограничивающего символа, позвольте регистратору сделать это.

Используйте коды ошибок, а не текст, если это возможно

Текст плохо масштабируется – он громоздкий, трудно анализируемый и часто повторяющийся. Журналирующие системы, которые генерируют шаблонные сообщения, могут включать код ошибки определенного вида как компактное представление сообщения. Используйте это, а не текст, для экономии места.

Существующие файлы журнала, и как ими управлять

Можно разделить файлы журнала на три главные категории: *колоночный*, *шаблонный* и *аннотационный*. Колоночные журналы записывают информацию в дискретных столбцах, которые обозначены разделителями или шириной фиксированного текста. Шаботонные файлы журнала похожи на английский текст, но текст поступает из ряда шаблонов документов и является перечисляемым. Аннотационные файлы журнала используют несколько текстовых записей для описания единственного события.

Колоночные данные, такие как формат CLF HTTP, записывают одно сообщение на событие. Это сообщение является сводкой всего события и состоит из фиксированного набора полей в колоночном формате. С колоночными журналами относительно легко иметь дело, поскольку поля четко разделены и формат жесткий; каждое сообщение имеет те же столбцы и ту же информацию.

При работе с колоночными данными необходимо помнить следующее:

- данные разграничены или используется фиксированная ширина? Если это фиксированная ширина, есть ли поля, которые очевидно превышают эту ширину? И если так, являются ли усеченными результаты или расширен столбец?
- если данные разграничены, остаются ли разделители в полях? Пользовательские форматы (такие как журналы HTTP) могут применять разделитель по умолчанию и автоматически выйти из него; если вы решите использовать свой собственный разделитель, возможно, автоматически выйти из него не получится;
- существует ли максимальная длина записи? Если да, то возможно наличие усеченных сообщений с недостающими полями.

Файлы журнала ELF и CLF, о которых пойдет речь далее в этой главе, являются хорошими примерами колоночных форматов.

Шаблонные текстовые сообщения записывают одно сообщение на событие, но события зарегистрированы как неформатированный английский текст. Сообщения являются *шаботонными* в том смысле, что они происходят из фиксированного и счетного множеств шаблонов. Где возможно, лучше преобразовывать шаботонные текстовые сообщения в некоторый индексируемый числовой формат. В лучшем случае это, по крайней мере частично, сделано. Например, Windows Event Log, показанный на рис. 3-1, имеет идентификатор события, который описывает тип события и может использоваться для определения других параметров, которые будут обеспечены.

При работе с шаботонным текстом необходимо помнить следующее:

- можно ли получить полный список сообщений журнала? Как пример рассмотрим файл журнала Windows на [рис. 3-1](#). Каждое из этих сообщений является текстом, но при этом имеет уникальный целочисленный ID. Проверьте документацию на список всех потенциальных сообщений журнала.

Преобразование текста в столбцы

Шаблонный текст может быть проанализирован; сообщения принадлежат счетному множеству и могут быть преобразованными в колоночный формат. Создание такой системы, однако, требует разработки посреднического приложения, которое может считать текст, проанализировать каждое отдельное сообщение и внести результат в схему. Такая разработка является нетривиальной задачей (и потребует обновления при разработке новых сообщений), но она также может уменьшить объем требуемого пространства и увеличить удобочитаемость данных.

1. В любой документации можно найти в текстовом формате, идентифицировать и выбрать сообщения, имеющие отношение к безопасности. Любой сценарий преобразования должен состоять из набора регулярных выражений, и чем меньше выражений, которые необходимо поддерживать, тем лучше.
2. Для каждого сообщения идентифицируйте параметры, которые оно содержит. Например, рассмотрим следующие искусственные шаблонные сообщения: «Инструмент SPAMKILLER против спама идентифицирует электронную почту <12938@yahoo.com> как спам», «Инструмент SPAMKILLER против спама идентифицирует электронную почту <12938@yahoo.com> как коммерческую», «Инструмент SPAMKILLER против спама идентифицирует электронную почту <12938@yahoo.com> как законную». В данном случае имеется три потенциальных параметра: название инструмента против спама (счетное), идентификатор сообщения (строка) и вывод (счетное).
3. Как только вы идентифицировали параметры для каждого потенциального сообщения, объедините их для создания подмножества. Цель этого этапа состоит в том, чтобы создать представление схемы всех параметров, которые может потенциально иметь сообщение; конкретное сообщение не может иметь всех параметров.
4. Попытайтесь генерировать, по крайней мере, одну запись события для каждого шаблонного сообщения. Документация может быть неточной.

В аннотационных журналах единственное событие разделено на несколько записей, объединенных через общий ID. Журналы событий, системные журналы и против спама – все они могут потенциально использовать этот формат. Аннотационные журналы распространяют событие через множество записей, и эффективный их анализ требует распознавания общего идентификатора, получения по запросу всех этих сообщений и контакта с сообщениями, которые могут быть потеряны.

Представительные форматы файла журнала

В этом разделе обсудим несколько общих форматов журнала, включая ELF и CLF, стандартные форматы журнала для сообщений HTML. Эти форматы настраиваемые, и мы предоставляем инструкции по улучшению сообщений журнала с целью предоставления большей информации, важной для безопасности.

HTTP: CLF и ELF

HTTP является основой существования современного интернета, и начиная с его разработки в 1991 г. он превратился из простого протокола библиотеки

в связующее звено интернета. Приложения, для которых 10 лет назад разработчик реализовал бы новый протокол, теперь штатно подгружаются к HTTP и веб-серверам.

HTTP является сложным протоколом для закрепления. Протокол ядра невероятно прост, но любой современный сеанс просмотра веб-страниц включает комбинацию HTTP, HTML и JavaScript для создания оперативных клиентов повышенной сложности. В этом разделе мы кратко обсудим базовые компоненты http, фокусируясь на аналитических аспектах.

HTTP является чрезвычайно простым протоколом доступа к файлам. Чтобы понять, насколько он прост сегодня, выполните упражнение в примере 3-2, используя netcat. netcat (который может также быть назван nc, возможно, потому, что администраторы считают его столь полезным, что они хотят облегчить название), гибкий инструмент доступа сетевого порта, который может использоваться для прямой отправки информации к портам. Это удобно для сценариев и применимо ко множеству задач с минимальной автоматизацией.

Пример 3-2. Доступ к серверу HTTP с помощью командной строки

```
host$ echo 'GET /' | nc www.oreilly.com 80> oreilly.html host$ kill%1
```

Выполнение команды в предыдущем примере должен произвести действующий файл HTML. В самой простой, по большей части непреукрашенной форме сеансы HTTP состоят из открытия связи, передачи метода и URI и получения файла в ответ.

HTTP достаточно прост для исполнения в командной строке вручную в случае необходимости. Однако следует понимать, что значительная часть функциональности передана дополнительным заголовкам. При работе с журналами HTTP основная проблема состоит в том, чтобы решить, какой заголовок включать, а какой проигнорировать.

HTTP-заголовки, которые стоит отметить

Существует более ста уникальных HTTP-заголовков, отслеживаемых в RFC 4229. Из них ограниченное количество особенно важно для отслеживания. А именно:

Cookie (куки)

Заголовок Cookie описывает содержание cookies HTTP, отправленных клиентом на сервер.

Host (хост)

Заголовок хоста определяет название хоста, с которым связывается клиент. Это очень важно при контакте с виртуальными http-серверами хоста – то есть множеством серверов с одним IP-адресом и отличающимися их доменными именами.

Referer (ссылка)

Referer (sic) заголовок включает URL веб-страницы, содержащей ссылку, которая инициировала этот запрос.

User-Agent (агент пользователя)

Заголовок User-Agent предоставляет информацию о клиенте HTTP, обычно тип клиента и сборки.

Существует два стандарта для данных HTTP журналов: общий формат журнала (CLF) и расширенный формат журнала (ELF). Большинство генераторов журнала HTTP (таких как *mod_log* Apache) обеспечивает обширные параметры конфигурации.

CLF является однострочным форматом журналирования, разработанным NCSA для исходного сервера HTTP; W3C предоставляет минимальное определение стандарта. Событие CLF определено как однострочная запись с семью значениями в следующем формате:

```
remotehost rfc931 authuser [date] "request" status bytes
```

где *remotehost* – это IP-имя или адрес удаленного хоста, *rfc931* является именем учетной записи удаленного входа в систему пользователя, *authuser* – аутентифицируемое имя пользователя, *date* – дата и время запроса, “*request*” – запрос, *status* – код состояния HTTP и *bytes* – число байтов.

Чистый CLF имеет несколько особенностей, которые могут сделать распознавание проблематичным. Поля *rfc931* и *authuser* фактически являются артефактами; в подавляющем большинстве записей CLF в этих полях будут установлены “–”. Фактический формат значения даты не определен и может варьироваться между различными реализациями сервера HTTP.

Общая модификация CLF является *объединенным форматом журнала*. Объединенный формат журнала содержит два дополнительных поля: поле «ссылка HTTP» и «строка агента пользователя».

ELF является расширяемым колоночным форматом, который был в основном ограничен IIS, несмотря на то что инструменты, такие как Bluescoat, также используют его для журналирования. Как и с CLF, W3C поддерживает стандарт на своем веб-сайте.

Файл ELF состоит из последовательности директив, сопровождаемых последовательностью записей. Директивы используются для определения атрибутов, характерных для записей, таких как дата всех записей (директива даты) и поля в записи (полевая директива). Каждая запись в ELF – это один запрос HTTP и поля, которые определены директивой, включенной в эту запись.

Поля ELF пребывают в одной из трех форм: *идентификатор*, *префикс-идентификатор* или *префикс (заголовок)*. Префикс – это одна или две символьные строки, которые определяют направление, откуда взята информация (с для клиента, s для сервера, r для удаленного). Идентификатор описывает содержание поля, и значение *префикс (заголовок)* включает соответствующий HTTP-заголовок. Например, *cs-метод* находится в формате *префиксного идентификатора* и описывает метод, отправленный от клиента к серверу, а *время* является простым *идентификатором*, обозначающим время окончания сеанса.

Пример 3-3 показывает простые выводы от CLF, объединенного формата журнала и ELF. Как видно из примера, каждое событие является одной строкой.

Пример 3-3. Примеры CLF и ELF

#CLF

```
192.168.1.1 - [2012/Oct/11 12:03:45 - 0700] "GET/index.html" 200 1294
```

Combined Log Format

```
192.168.1.1 - [2012/Oct/11 12:03:45 - 0700] "GET /index.html" 200 1204 "http://www.example.com/link.html" "Mozilla/4.08 [en] (Win98; I; Nav)"
```



```
#ELF #Version: 1.0
#Date: 2012/Oct/11 00:00:00
#Fields: time c-ip cs-method cs-uri
12:03:45 192.168.1.1 GET/index.html
```

Большинство журналов HTTP является некоторой формой вывода CLF. Несмотря на то что ELF является расширяемым форматом, я нахожу проблематичной потребность сохранять все HTTP-заголовки, т. к. не ожидаю такого сильного изменения формата и предпочитаю, чтобы отдельные записи журнала оставались воспринимаемыми и без этой информации. На основе принципов, обсужденных ранее, я изменяю записи CLF следующим образом:

1. Удалите поля rfc931 и authuser. Эти поля являются артефактами и только зря занимают место.
2. Преобразуйте дату во время эпохи и представляйте его как числовую строку. В дополнение к моему общему презрению к тексту, в отличие от числового представления, представления времени никогда не стандартизировались в файлах журнала HTTP. Лучше перейти к числовому формату для игнорирования прихотей сервера.
3. Добавьте IP-адрес сервера, исходный порт и порт назначения. Я собираюсь перемещать файлы журнала в центральное расположение для анализа, таким образом, мне будут нужны адреса серверов для их дифференциации. Это делает меня ближе к пяти данным, которые я могу коррелировать с другими данными.
4. Добавьте продолжительность события, снова для помощи с корреляцией по времени.
5. Добавьте заголовок хоста. В случае если я имею дело с виртуальными хостами, это также помогает мне идентифицировать системы, которые связываются с сервером, не используя DNS в качестве модератора.

Поваренная книга: создание файлов журнала

Конфигурация журнала в Apache обработана через модуль `mod_log_config`, который обеспечивает способность отобразить журналы с помощью последовательности строковых макросов. Например, по умолчанию формат CLF выглядит так:

```
LogFormat "%h%l%u%t \"%r\"%>s%b"
```

Combined Log Format выражен как

```
LogFormat "%h%l%u%t \"%r\"%> s%b \"%{Referer}i\" \"%{User-agent}i\""
```

В то время как мой расширенный формат содержит имя узла, локальный IP-адрес, порт сервера, время эпохи, строку запроса, состояние запроса, размер ответа, время отклика, ссылающийся домен, строку агента пользователя и хост от запроса:

```
LogFormat "%h%A%p{%msec}t \"%r\"%> s%bT \"%{Referer}i\" \"%$ {User-agent}i\" \"%${Host}i\""
```

Журналированием в `nginx` управляют с `ngx_http_log_module`, который использует подобную директиву `log_format`. Для конфигурирования CLF определите его следующим образом:

```
log_format clf $remote_addr - $remote_user [$time_local] "$request" $status$body_bytes_sent';
```

Объединенный формат журнала определен следующим образом:

```
log_format combined $remote_addr - $remote_user [$time_local] "$request" $status$body_
bytes_sent "$http_referer" "$http_user_agent";
```

Мой расширенный формат определен как

```
log_format extended $server_addr $remote_addr $remote_port $msec
"$request$" $status $body_bytes_sent $request_time $http_referer
$http_user_agent $http_host;
```

SMTP

Сообщения журнала SMTP варьируются путем использования MTA (Metropolitan Transportation Authority), и они очень хорошо настраиваются. В этом разделе мы обсудим два формата журнала, которые показательны для таких систем, как Unix и Windows: *sendmail* и Microsoft Exchange.

Здесь мы сфокусируемся на журналировании передачи сообщений электронной почты. Инструменты журналирования для этих приложений обеспечивают огромный объем информации о внутреннем состоянии сервера, попытках подключения и других данных, который, будучи чрезвычайно ценным, заслуживает сбора в отдельную книгу.

Sendmail модерирует почтовый обмен через *системный журнал* и, следовательно, способен к отправке огромного количества информационных сообщений, помимо фактической почтовой транзакции. Для наших целей мы разберемся с двумя классами сообщений журнала: сообщениями, описывающими связи с и от почтового сервера, и сообщениями, описывающими фактическую доставку почты.

По умолчанию *sendmail* отправит сообщения в */var/maillog*, несмотря на то что журналирование информации, которую он отправляет, управляется внутренним уровнем журналирования *sendmail*. *Sendmail* использует свои собственные внутренние уровни журналирования от 1–96, ранжируя сообщения по уровню важности от 1 до *n*. Важные для нас (значимые) уровни логирования включают 9 (все залогированные входящие соединения), 10 (залогированные входящие соединения), 12 (залогированные исходящие соединения) и 14 (залогированные отклонения соединений). При этом то, что выше уровня 8, считают информационным в *системном журнале*, а все, что выше 11, – сообщениями журнала отладки.

Строка журнала *sendmail* состоит из пяти фиксированных значений, сопровождаемых списком одного или более *equates*:

```
<date> <host> sendmail [<pid>]: <qid>: <equates>
```

где <date> является датой, <host> – название хоста, *sendmail* является литеральной строкой, <pid> – это ID процесса передачи почты и <qid> является ID внутренней очереди для однозначного определения сообщения. *Sendmail* отправляет, по крайней мере, два сообщения журнала при отправке электронного письма, и единственный способ собрать в группу эти сообщения – через *qid*. *Equates* – это описательные параметры, представленные в форме <ключ> = <значение>. *Sendmail* может отправлять ряд возможных *equates* для сообщений, которые перечислены в табл. 3-1.

Для каждого полученного электронного письма *sendmail* генерирует, по крайней мере, две строки журнала. Первая строка является строкой *получателя* и описывает место происхождения сообщения. Заключительная строка – строка *отправителя*, описывает расположение почты: отправлена, изолирована и куда была доставлена.

Таблица 3-1. Соответствующие Sendmail equates

Параметр	Описание
arg1	Текущие sendmail-реализации включают внутреннюю фильтрацию с помощью наборов правил; arg1 – это параметр передачи к набору правил
from	Адрес отправителя
msgid	Идентификатор сообщения электронной почты
quarantine	Если sendmail изолирует почту, указана причина для удержания
reject	Если sendmail отклоняет почту, указана причина для отклонения
relay	Это имя и адрес хоста, который отправил сообщение; в строках получателя это хост, который отправил его; в строках отправителя – хост, который получил его
ruleset	Это ruleset, который обработал сообщение и обеспечивает обоснование для отклонения, изоляции или отправки сообщения
stat	Статус доставки сообщения
to	Адрес электронной почты получателя; в строке может быть несколько токенов to=

Sendmail выполнит одно из четырех основных действий с сообщением: отклонит его, изолирует, возвратит или отправит. Отклонение реализовано фильтрацией сообщения и используется для фильтрации спама; отклоненное сообщение сбрасывается. Изолированные сообщения отправляются в очередь к отдельной области для дальнейшего рассмотрения. Возврат означает, что почта не была отправлена по адресу, и отправителю будет направлен отчет о недоставке сообщения.

Почтовые правила управления и фильтрация

Анализ почтового трафика является сложным, в основном потому, что электронная почта ежесекундно подвергается нападению (через спам), и существует постоянно возрастающая война между спаммерами и защитниками. Даже на относительно малом предприятии легко создать сложную защитную инфраструктуру с относительно небольшой работой. В дополнение к спаму и проблемам защиты электронная почта работает в ее собственном небольшом мире – IP-адреса, зарегистрированные по электронной почте, в значительной степени используются исключительно почтовой инфраструктурой.

Как обычно, первый шаг в почтовом инструментарии – это выяснить, как электронная почта направлена. Есть ли какие-нибудь специализированные аппаратные средства против спама в шлюзе, такие как Barracuda или IronPort box? Сколько там есть серверов SMTP, как они соединяются с фактическими почтовыми серверами (POP, IMAP, Eudora, Exchange)? Выяснить, куда сообщение электронной почты будет отправлено, если оно будет правильно маршрутизировано, изолировано, отклонено или возвращено. Если доступна веб-почта, выяснить, где она находится на самом деле; где сервер веб-почты, какой маршрут к SMTP и т. д. Как только вы идентифицировали аппаратные средства, выясните, на что идет блокирование. Блокирующие методы включают ресурсы черного ящика (такие как AV или сервис репутации IronPort), общедоступные черные списки, такие как SBL SpamHaus и внутренние правила. Каждый требует немного разной обработки.

Так как системы обнаружения черного ящика в основном непрозрачны, важно отследить, какая версия базы знаний системы используется и когда система обновлена; проверка обновлений с сетевым контролем является хорошей идеей. Если у вас есть многократные экземпляры одного и того же детектора, удостоверьтесь, что их обновления скоординированы.

Большинство сервисов черного списка публично доступно. При этом хорошо бы знать, какая организация выполняет черный список, частоту его обновлений и механизмы доставки. Как и с AV, проверка коммуникации (особенно если это DNSBL) – также вещь хорошая.

Внутренний контроль должен идентифицироваться, контролироваться и сохраняться при управлении версиями. Поскольку данные правила следует строго контролировать, то хорошо было бы сравнить их с остальной частью вашей инфраструктуры блокирования, а также понять, что может быть продвинуто из почтовой системы. Например, блокировать конкретный адрес лучше в маршрутизаторе или брандмауэре.

Электронная почта работает в ее собственной вселенной, и подавляющее большинство IP-адресов, зарегистрированных в почтовых журналах, являются адресами других почтовых серверов. Поэтому, когда надо понять, что случилось с общением, важно отслеживать не только SMTP, но и IMAP- или POP3-серверы.

Microsoft Exchange: журналы, отслеживающие сообщения

Exchange имеет один основной формат журнала для обработки сообщений – журнал, отслеживающий сообщения (MTL).

Таблица 3-2. Поля MTL

Название поля	Описание
date-time	Представление ISO 8601 формата даты и времени
client-ip	IP-адрес хоста, который отправил сообщение серверу
client-hostname	IP-адрес клиента, FQDN (полное доменное имя)
server-ip	IP-адрес сервера
server-hostname	IP-адрес сервера. FQDN (полное доменное имя)
source-context	Это дополнительная информация об источнике, таком как идентификатор для транспортного агента
connector-id	Название коннектора
source	Exchange перечисляет много исходных идентификационных данных для определения источника сообщения, такого как правило ящика входящих сообщений, транспортный агент или DNS. Поле-источник будет содержать эти идентификационные данные
event-id	Тип события. Это также численные значения и включает количество сообщений состояния о том, как сообщение было обработано
internal-message-id	Это внутренний целочисленный идентификатор, используемый Exchange для дифференциации сообщения. ID не используется совместно серверами Exchange, поэтому если сообщение отдано, это значение изменится
message-id	Это стандартный идентификатор сообщения SMTP. Exchange создаст ID, если у сообщения его еще нет
network-message-id	Это идентификатор сообщения, такой как _internal-message-id +, за исключением того, что он совместно используется копиями сообщения и создается, когда сообщение клонируется или дублируется, как при передаче в список рассылки



Окончание табл. 3-2

Название поля	Описание
recipient-address	Адреса получателей; это разграниченный точкой с запятой список имен
recipient-status	Это код статуса получателя, указывающий, как каждый получатель был обработан
total bytes	Общий размер сообщения в байтах
recipient-count	Размер адреса получателя с учетом числа получателей
related-recipient-address	Некоторые события Exchange (например, переадресация) приводят к тому, что создаются дополнительные получатели – они добавляются в это поле.
reference	Это специфичная для сообщения информация, содержание которой является функцией типа сообщения (определенный в event-id)
message-subject	То, что находится в Subject: заголовок
sender-address	Отправитель, как определено в Sender: заголовок; если Sender, отсутствует; From – используется вместо этого
return-path	Адрес электронной почты возврата, как определено в почте From
message-info	Информация о сообщении, зависящая от типа события
directionality	Направление сообщения; счетное количество
tenant-id	Больше не используется
original-client-ip	Исходный IP-адрес клиента
original-server-ip	Исходный IP-адрес сервера
custom-data	Дополнительные данные, зависящие от типа события

ТРАНСПОРТ ФАЙЛА ЖУРНАЛА: ПЕРЕДАЧИ, СИСТЕМЫ И ОЧЕРЕДИ СООБЩЕНИЙ

Журналы хоста могут быть переданы со своих хостов различными способами в зависимости от того, как эти журналы сгенерированы, и от возможностей операционной системы. Наиболее распространенные подходы включают использование обычной передачи файлов или протокол *системного журнала*. Более новый подход применяет *очереди сообщений* для переноса информации журнала.

Передача и ротация файла журнала

Большинство регистрируемых приложений пишет в циклический файл журнала (см., например, циклические системные журналы в «Доступ и управление файлами журнала»). В этих случаях файл журнала будет закрыт и заархивирован после установленного срока и нового запущенного файла. Как только файл закрыт, он может быть скопирован в необходимое место для поддержки аналитики.

Передача файлов проста. Это может быть реализовано с помощью ssh или любого другого протокола копирования. Самая большая проблема – это убедиться в том, что файлы на самом деле завершены после копирования; период вращения для файла эффективно диктует ваше время отклика. Например, если файл поворачивается каждые 24 часа, то необходимо будет в среднем ожидать день для получения доступа к последним событиям.

Системный журнал

Дедушкой систематических системных утилит журналирования является *системный журнал*, стандартный подход к журналированию, первоначально разработанный для систем Unix, который теперь включает стандарт, протокол и общие рамки для обсуждения регистрирующихся сообщений. Системный журнал определяет фиксированный формат сообщения и способность отправить это сообщение демонам регистратора, которые могут находиться на хосте или быть удаленно расположены.

Все сообщения системного журнала содержат время, *процесс*, *важность* и текстовое сообщение. Таблицы 3-3 и 3-4 описывают процессы и приоритеты, закодированные в протоколе системного журнала. Как показывает табл. 3-3, процессы, упомянутые системным журналом, включают множество фундаментальных систем (некоторые из них сильно устаревшие). Больше всего беспокоит то, какие процессы *не* покрыты, – DNS и HTTP, например. Приоритеты (в табл. 3-4) в целом более релевантны, поскольку перечень их значений вошел в обычный обиход.

Таблица 3-3. Средства системного журнала

Значение	Значение
0	Kernel
1	User-level
2	Mail
3	System daemons
4	Security/Authorization
5	Syslogd
6	Line printer
7	Network news
8	UUCP
9	Clock daemon
10	Security/Authorization
11	ftpd
12	ntpd
13	Log audit
14	Log alert
15	Clock daemon
16–23	Reserved for local use

Таблица 3-4. Приоритеты системного журнала

Значение	Значение
0	Чрезвычайная ситуация: система не может быть использована
1	Предупреждение: меры должны быть приняты немедленно
2	Очень важно: критические состояния
3	Ошибка: состояние ошибки
4	Предупреждение: есть опасность
5	Уведомление: нормальное, но важное состояние

Окончание табл. 3-4

Значение	Значение
6	Информационный: информационные сообщения
7	Отладка: отладочная информация

Ссылочные реализации системного журнала основаны на UDP, что вносит несколько ограничений. Самое важное – длина датаграммы UDP ограничена MTU протоколом уровня 2, несущего датаграмму, с жестким ограничением – 1450 символов на любое сообщение системного журнала. Сам протокол системного журнала определяет, что сообщения должны быть меньше, чем 1024 символа, но это редко наблюдается, в то время как обрезание UDP повлияет на длинные сообщения. Кроме того, системный журнал работает поверх UDP, это означает, что когда сообщения пропущены, они потеряны навсегда.

Самый легкий способ решить эту проблему состоит в том, чтобы использовать основанный на TCP системный журнал, который реализован в домене с открытым исходным кодом такими инструментами, как системные журналы `syslog-ng` и `rsyslog`. Оба этих инструмента обеспечивают транспорт TCP, а также много других возможностей, таких как интерфейс базы данных, способность переписать сообщения в пути и выборочный транспорт сообщений системного журнала к различным получателям. Windows не поддерживает системный журнал исходно, но там существует много коммерческих приложений, которые обеспечивают схожую функциональность.

CEF: общий формат описания события безопасности

Системный журнал является транспортным протоколом – он ничего не определяет в фактическом содержании сообщения. Много различных организаций пыталось разработать функционально совместимые стандарты для приложений защиты, таких как Единый стандарт обнаружения уязвимостей (CIDF) и формат обмена сообщениями обнаружения уязвимостей (IDMEF). Ни один из них не достиг серьезного промышленного внедрения. Что было принято широко – это CEF. Первоначально разработанный ArcSight (теперь часть Hewlett-Packard) для предоставления разработчикам датчика стандартный формат, в котором можно отправить сообщения в их SIEM. CEF является форматом записи, который определяет события безопасности с помощью числового заголовка и ряда пар ключ/значение. Например, сообщение CEF для нападения от хоста 192.168.1.1 могло бы быть похожим на это:

```
CEF:0|My Attack Detector|Test|1.0|1000|Attack|5|src=192.168.1.1
```

CEF является транспортно-независимым, но большинство реализаций CEF использует системный журнал в качестве преимущественного транспорта. Фактическая спецификация и ключи и соответствующие им значения доступны от HP.

ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ ДЛЯ ЧТЕНИЯ

1. Ричард Беджтлик (*Richard Bejtlich*). Практика контроля сетевой безопасности: понимание обнаружения события и ответ (No Starch Press, 2013).
2. Антон Чувакин (*Anton Chuvakin*). Журналирование и управление журналом: Авторитетное руководство по работе с системным журналом, журналами аудита, предупреждениями и другим IT-«шумом» (Syngress, 2012).

Глава 4

.....

Хранение данных для анализа: реляционные базы данных, большие данные и другие опции



Эта глава фокусируется на технике хранения данных для анализа трафика. Хранение данных относится к основной проблеме в анализе информационной безопасности: события информационной безопасности рассеяны в огромном количестве безвредных файлов журнала, а эффективный анализ безопасности требует способности быстрой обработки больших объемов данных.

Существует много разных подходов, доступных для упрощения быстрого доступа к данным, основные – это плоские файлы, традиционные базы данных и находящая на стадии становления парадигма NoSQL. Каждый из этих вариантов имеет свои достоинства и недостатки с точки зрения структуры накопленных данных и навыков использующих их аналитиков.

Плоские файловые системы записывают данные на диск и доступны непосредственно аналитикам, обычно с помощью простых инструментов парсинга. Большинство систем журнала создает данные плоского файла по умолчанию: после создания некоторого постоянного числа записей они закрывают файл и открывают новый файл. Плоские файлы просто считывать и анализировать, но нет никаких конкретных инструментов для обеспечения оптимизированного доступа.

Системы баз данных, такие как Oracle и Postgres, являются основой обработки данных предприятия. Они используют четко определенные интерфейсные языки, легко можно найти системных администраторов и специалистов по обслуживанию, и они могут быть сконфигурированы для предоставления чрезвычайно стабильных и масштабируемых решений. В то же время они *не* разработаны для работы с данными журналов; данные, которые мы обсуждаем в этой книге, име-

ют много функций, которые гарантируют, что большая часть реляционной базы данных останется неиспользованной.

Наконец, существуют новые технологии, свободно сгруппированные под «NoSQL» и «большими данными». Они включают распределенные платформы, такие как Hadoop, базы данных как MongoDB и Monet и специализированные инструменты, как Redis и Apache SOLR. Эти инструменты, с правильной аппаратной инфраструктурой, способны обеспечить чрезвычайно мощные и надежные инструменты распределенного запроса. Однако они требуют усиленного программирования и навыков системного администрирования, а также значительных аппаратных средств.

Анализ включает возврат к хорошему многократно — при работе над проблемой аналитики будут возвращаться к основному хранилищу данных и связанным с полученными по запросу данным. Данные, которые они выбирают, будут функцией данных, которые они уже выбрали, поскольку шаблоны становятся очевидными и вопросы начинают обретать форму (см. главу 10). Поэтому эффективный доступ к данным является критичным техническим усилием; время доступа к данным непосредственно влияет на количество запросов, которые может сделать аналитик, и это конкретно влияет на тип исследований, которые они сделают.

Выбор правильной системы передачи и обработки данных является функцией объема хранящихся данных, типа этих данных и населения, которое собирается их проанализировать. Нет никакого единственно правильного выбора, и в зависимости от комбинации ожидаемых запросов и хранящихся данных каждая из этих стратегий может быть лучшей.

ДАННЫЕ ЖУРНАЛОВ И ПАРАДИГМА CRUD

Парадигма CRUD (создает, считывает, обновляет и удаляет) описывает основные операции, ожидаемые от системы долговременного хранения (персистентной). Системы управления реляционными базами данных (RDBMS), самая распространенная форма долговременного устройства хранения данных, ожидают, что пользователи будут регулярно и асинхронно обновлять существующее содержание. Реляционные базы данных прежде всего предназначены для обеспечения целостности данных, а не производительности.

Обеспечение целостности данных требует существенного количества ресурсов системы. Базы данных используют много различных механизмов для осуществления целостности, включая дополнительную обработку и метаданные по каждой строке. Эти функции необходимы для типа данных, для которых были разработаны RDBMS. Эти данные не являются данными журналов.

Это различие показано в рис. 4-1. В RDBMS пользователи постоянно добавляют и запрашивают данные от системы, и система тратит ресурсы на отслеживание этих взаимодействий. Однако данные журналов не изменяются; событие, случившееся однажды, никогда не обновляется. Это изменяет поток данных, как показано на рисунке справа. В системах сбора журнала на диск записывают только датчики; пользователи лишь *считывают* с диска.

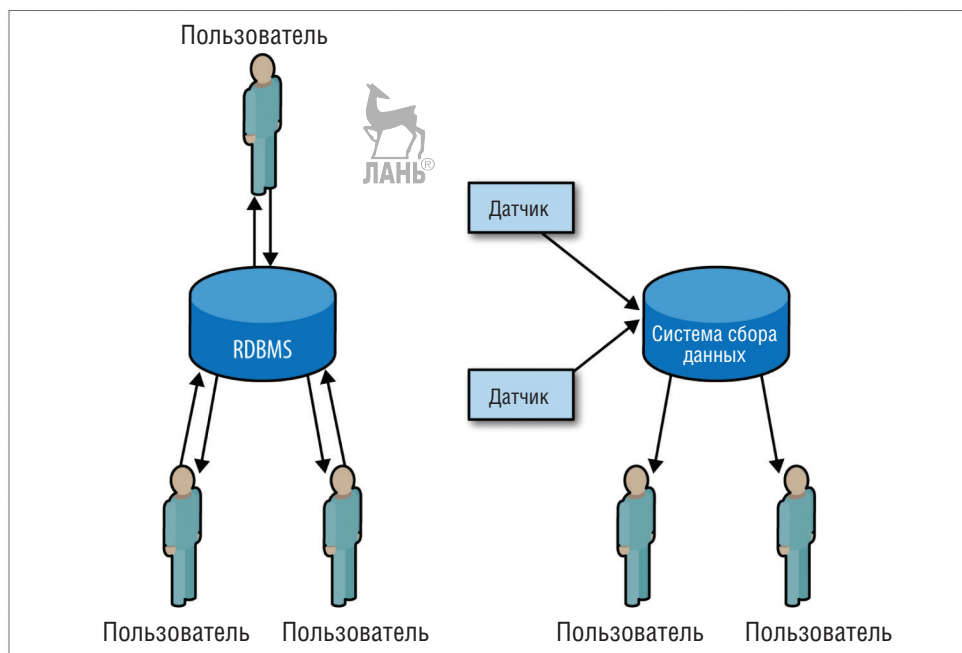


Рисунок 4-1. Сравнение RDBMS и систем сбора журнала

Такое разделение обязанностей между пользователями и датчиками означает, что при работе с данными журналов механизмы целостности, используемые базами данных, потрачены впустую. Для данных журналов правильно разработанная система сбора плоского файла часто будет так же быстра, как реляционная база данных.

Создание хорошо организованной плоской файловой системы: уроки от SiLK

В главе 5 мы обсуждаем SiLK, систему анализа, разработанную CERT для управления большими Netflows. SiLK – это очень ранняя система больших данных. Хотя эта система не использует текущие технологии больших данных, она была разработана на основе подобных принципов, и понимание того, как эти принципы работают, может помочь при разработке более актуальных систем.

Анализ журналов ограничивается прежде всего вводом-выводом (I/O), то есть основным ограничением производительности является количество прочитанных записей, а не сложность алгоритмов обработки записей. Например, в первоначальном проекте SiLK мы поняли, что было значительно быстрее сохранить сжатые файлы на диске – результат считывания отчетов с диска был гораздо лучше, чем при распаковке файла в памяти.

Поскольку производительность является связанной с I/O, хорошая система запросов считает минимальное из возможных количество соответствующих записей. В системах сбора журнала самый эффективный способ уменьшить количество считанных записей состоит в том, чтобы индексировать их по времени и *всегда* требовать, чтобы пользователь определил требуемое время. В SiLK записи журнала сохранены в почасовых файлах в ежедневной иерархии. Например: от

/data/2013/03/14/sensor1_20130314.00 до /data/2013/03/14/sensor1_20130314.23. Команды SiLK включают функцию globbing, которая скрывает фактические имена файлов от пользователя; запросы определяют дату начала и дата окончания, которая, в свою очередь, используется для получения файлов.

Этот процесс разделения не должен останавливаться со временем. Поскольку сетевой трафик (и данные журналов) обычно во власти нескольких главных протоколов, эти отдельные протоколы могут быть отделены в их собственные файлы. В установках SiLK весьма обычно отделить интернет-трафик от всего другого трафика, потому что интернет-трафик составляет 40–80 % трафика в большинстве сетей.

Как с большинством схем выделения разделов данных, решение, когда прекратить подразделять данные, – это больше искусство, чем наука. Как показывает опыт, приемлемо наличие не больше, чем три–пять дальнейших разделов после времени, поскольку добавление дополнительных разделов увеличивает сложность для пользователей и разработчиков. Кроме того, определение точной схемы обычно требует некоторого знания трафика в сети, таким образом, вы не можете сделать этого до тех пор, пока не получите лучшего понимания структуры сети, состава и типа имеющихся данных.

Форматы данных и оптимизация данных

Вы решаете хранить данные в плоских файлах и создать систему, которая принимает миллиард записей в день. Вы решаете использовать текст ASCII и записываете упакованные нулем IP-адреса источника и назначения. Это означает, что ваши адреса IPv4 займут 15 байтов устройства хранения данных каждый, по сравнению с 4-байтовым двоичным представлением. Это означает, что каждый день вы будете использовать 22 ГБ пространства для этого текстового представления. Если у вас будет единственный интерфейс GigE для передачи этих данных, понадобится три минуты только для передачи потраченного впустую пространства.

Как только вы начнете работать с наборами данных большого объема, пространственные зависимости становятся проблемами дисков (влияющими на время выполнения запроса и продолжительность хранения), а также сети (влияющими на время выполнения запроса и производительность). Поскольку ваши операции являются связанными с I/O, преобразование представления данных в двоичном формате сохранит свободное место, увеличит производительность и, очень часто, действительно сделает дизайн реализуемым.

Проблема фактической разработки компактного двоичного представления данных была в основном решена через многие различные схемы представления, разработанные Google и другими компаниями. Все эти инструменты работают примерно тем же способом: вы определяете схему с помощью языка определения интерфейсов (IDL) и затем запускаете инструмент по схеме для создания связываемой библиотеки, которая может считать и записать данные в компактном формате. Существует свободное подобие XML и JSON, но с акцентом на очень компактное, двоичное представление.

Google разработал первую из этих систем в форме буферов протокола. Теперь доступно множество инструментов, включая, но не ограничиваясь:

Protocol Buffers (PB)

Google описывает их как «меньшую, более быструю, более простую» версию XML. Привязки к языку доступны в Java, C++ и Python. Буферы протокола (PB) являются старейшей реализацией и, хотя и не такие многофункциональные, как другие реализации, очень стабильны.

Thrift

Первоначально пришел из Facebook и теперь поддерживается Apache. В дополнение к обеспечению сериализации и возможностей десериализации Thrift включает транспорт данных и механизмы RPC.

Avro

Разработанный в тандеме с Hadoop и даже более динамичный, чем PB или Thrift, Avro определяет схемы с помощью объектной нотации JavaScript (JSON) и передает схему как часть содержания сообщения. Соответственно, Avro более гибок к изменениям схемы.

Существуют и другие стандарты сериализации, включая [MessagePack](#), [ICE](#) и [Etch](#). С публикации этой книги, однако, PB, Thrift и Avro считают большой тройкой. Получение записи и преобразование ее в строковый двоичный формат all-ASCII являются пустой тратой пространства. Цель любого процесса преобразования должна состоять в том, чтобы уменьшить сумму бесплатных данных в записи; читайте раздел «Характеристики хорошего сообщения журнала» в главе 3 для дальнейшего обсуждения того, как уменьшить размеры записи.

КРАТКОЕ ВВЕДЕНИЕ В СИСТЕМЫ NoSQL

Популяризация систем больших данных NoSQL, особенно парадигма MapReduce, представленная Google, стала большим шагом вперед в больших данных в прошлое десятилетие. MapReduce базируется на двух понятиях от функционального программирования: сопоставление (mapping), которое является независимым приложением функции ко всем элементам в списке, и сокращение (reducing), которое является комбинированием последовательных элементов в списке в единственный элемент. Пример 4-1 ясно показывает, как эти элементы работают.

Пример 4-1. Функции сопоставления и сокращения в Python

```
>>> # Map works by applying a function to every element in an array, for example, we
... # create a sample array of 1 to 10
>>> sample = range(1,11)
>>> # We now define a doubling function
...
>>> def double(x):
...     return x * 2
...
>>> # We now apply the doubling function to the sample data
... # This results in a list whose elements are double the
... # original's
...
>>> map(double, sample)
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

```

>>> # Now we create a 2-parameter function which adds two elements
...
>>> def add(a, b):
...     return a + b
...
>>> # We now run reduce with add and the sample, add is applied
... # to every element in turn, so we get add(1,2) which produces
... # 3, the list now looks like [3,3,...] as opposed to
... # [1,2,3,...], and the process is repeated, 3 is added to 3
... # and the list now looks like [6,4,...] until everything is
... # added
...
>>> reduce(add, sample)
55

```

MapReduce является удобной парадигмой для распараллеливания. Операции сопоставления неявно параллельны, потому что отображенная функция применена для листинга элемента индивидуально, и восстановление предоставляет ясное описание того, как результаты объединены. Такое легкое распараллеливание позволяет реализовать любой из множества подходов к большим данным.

В наших целях система больших данных является распределенной архитектурой хранения данных, которая полагается на крупное распараллеливание. Вспомните дискуссию выше о том, как плоские файловые системы могут улучшить производительность путем интеллектуальной индексации данных. Но теперь вместо того, чтобы просто хранить почасовой файл на диске, разделите его на несколько хостов и выполняйте тот же запрос на этих хостах параллельно. Более прекрасные детали зависят от типа устройства хранения данных, для которого мы можем определить три главные категории:

Базы ключей

Включая MongoDB, Accumulo, Cassandra, Hypertable и LevelDB. Эти системы эффективно действуют в качестве гигантской хеш-таблицы, полный документ или структура данных связаны с ключом для последующего извлечения. В отличие от других двух опций, системы базы ключей не используют схемы; структура и интерпретация зависят от конструктора.

Колоночные базы данных

Включая MonetDB, Sensage и Paracel. Колоночные базы данных разделяют каждую запись через многократные файлы столбца с тем же индексом.

Реляционные базы данных

Включая MySQL, Postgres, Oracle и SQL Server Microsoft. RDBMS хранят полные записи как индивидуально различимые строки.

Рисунок 4-2 объясняет эти отношения графически. В базе ключей запись сохранена ее ключом, в то время как отношение зарегистрированных данных и любой схемы оставляют пользователю. В колоночной базе данных строки анализируются в их отдельные поля и затем сохранены, одно поле на файл, в файлах отдельного столбца. В RDBMS каждая строка является уникальным и различимым объектом. Схема определяет содержание каждой строки, и строки сохранены последовательно в файле.

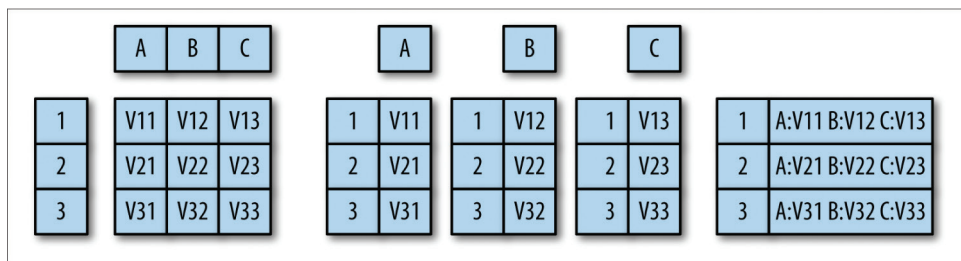


Рисунок 4-2. Сравнение систем хранения данных

Базы ключей являются хорошим выбором, когда вы понятия не имеете, какова структура данных, необходимо реализовать собственные запросы низкого уровня (например, обработка изображений и что-либо, не легко выраженное в SQL) или даже если данные имеют структуру. Это отражает их исходную цель – поддерживать неструктурированные текстовые поисковые запросы через веб-страницы. Базы ключей будут работать хорошо с веб-страницами, *tcpdump*-записями, содержащими полезную нагрузку, изображениями и другими наборами данных, где отдельные записи являются относительно большими (порядка 60 Кбит или больше около размера HTML на современной веб-странице). Однако если данные обладают некоторой структурой, такой как способность, разделения на столбцы или обширные и повторные ссылки на те же данные, то колоночная или реляционная модель может быть предпочтительной.

Колоночные базы данных предпочтительны, когда данные легко разделить на отдельные записи журнала, которые перекрестно не ссылаются друг на друга и когда размер является относительно маленьким, таким как форматы записи CLF и ELF, обсужденные в главе 3. Колоночные базы данных могут оптимизировать запросы путем выбора и обработки данных из подмножества столбцов в каждой записи; их производительность улучшается, когда они запрашивают из меньшего количества столбцов или возвращают меньше столбцов. Если ваша схема будет иметь ограниченное количество столбцов (например, база данных изображения, содержащая небольшое поле даты, небольшое поле ID и большое поле изображений), то колоночный подход не обеспечит повышения производительности.

RDBMSes были первоначально разработаны для часто повторяющейся информации, которая тиражируется через многократные записи, такие как тарификационная база данных, где у единственного человека могут быть многокомпонентные счета. RDBMSes работают лучше всего с данными, которые могут быть подразделены через многократные таблицы. В средах безопасности они обычно подходят лучше всего для поддержания записей персонала, отчетов события и другого знания – вещи, которые произведены после обработки данных или которые отражают структуру организации. RDBMSes способны поддерживать целостность и параллелизм; если необходимо обновить строку, они – выбор по умолчанию. Подход RDBMS является, вероятно, неоправданным, если ваши данные не изменяются после их создания, отдельные записи не имеют перекрестных ссылок или ваши схемы хранят большиеbloбы (большие бинарные объекты).

Другие инструменты для хранения данных

В дополнение к трем главным системам хранения, обсужденным ранее, существует несколько других инструментов и методов для улучшения скорости доступа. Эти системы хранения менее распространены, чем большая тройка, но обычно оптимизируются для определенных данных или типов запроса.

Базы данных графа включают Neo4j, ArangoDB и Titan. Базы данных графа обеспечивают масштабируемые, очень эффективные запросы при работе с данными графа (см. главу 13). Традиционные системы баз данных, включая три, упомянутые ранее, известны плохи в управлении графами, поскольку любое представление включает выполнение многократных запросов для генерации графа во времени. Базы данных графа обеспечивают запросы и инструменты для анализа структуры графа.

Библиотека Lucene и ее сопутствующая поисковая система, Solr, составляют текстовый инструмент поисковой системы с открытым исходным кодом.

Redis является основанной на памяти системой хранения значения ключа. Если необходимо быстро получить доступ к данным, которые могут уместиться в памяти (например, таблицы поиска), Redis является очень хорошим выбором для обработки поиска и модификаций.

Наконец, если финансы позволяют, необходимо рассмотреть преимущества твердотельного устройства хранения данных (SSD). Решения SSD могут быть дорогими, но у них есть огромное преимущество быть функционально прозрачными как часть файловой системы. На верхнем уровне такие компании, как Violin memory, Fusion-I/O и STEC, предоставляют в стойке мульти-TB смонтированные модули (юниты), которые могут быть сконфигурированы так, чтобы получить и обработать данные на текущих скоростях передачи данных.

Какой подход к хранению данных использовать

При выборе архитектуры хранения решите, какой тип данных будете собирать и какой тип отчетов будете создавать, с их использованием. Ожидаете ли вы в основном генерации фиксированных отчетов, или ваши аналитики будут проводить большое количество исследовательских запросов?

Таблица 4-1 предоставляет сводку типов решений, которые входят в выбор подхода устройства хранения данных. Решения перечислены в порядке предпочтения: 1 является лучшим, 3 – хуже, X означает – не беспокоятся вообще. Мы обсудим каждую опцию подробно, чтобы объяснить, как они влияют на варианты хранения.

Таблица 4-1. Принятие решений о системах передачи и обработки данных

Ситуация	Реляционный	Колоночный	Ключ–значение
Наличие доступа к многократным дискам и хостам	2	1	1
Наличие доступа к единственному хосту	1	X	X
Данные меньше терабайта	1	2	3
Данные мультитерабайтные	2	1	1
Необходимость обновлять строки	1	X	X
Нет необходимости обновлять строки	2	1	1
Данные являются неструктурированным текстом	2	3	1

Окончание табл. 4.1

Ситуация	Реляционный	Колоночный	Ключ--значение
Данные имеют структуру	2	1	3
Отдельные записи являются маленькими	2	1	3
Отдельные записи являются большими	3	2	1
У аналитиков есть некоторые навыки разработки	1	1	1
У аналитиков нет навыков разработки	1	1	2

Первое решение является аппаратным. Системы больших данных, такие как колоночные базы данных и базы ключей, *только* предоставят вам преимущество производительности, если есть параллельные узлы, и чем больше, тем лучше. Если у вас есть единственный хост, или даже меньше, чем четыре доступных хоста, вы, вероятно, более преуспеете с традиционной архитектурой базы данных благодаря использованию ее средств более зрелого администрирования и разработки.

Следующая пара вопросов реально связана с первым вопросом об аппаратных средствах: являются ли действительно настолько большими ваши данные? Я использую терабайт в качестве произвольного предела для больших данных, потому что действительно могу купить SSD на 1 TB. Если ваши данные не являются настолько большими, снова по умолчанию обращаемся к реляционным базам данных или системе хранения в оперативной памяти как Redis.

Следующий вопрос связан с потоком данных и парадигмой CRUD. Если вы ожидаете регулярно обновлять содержание строки, то лучшим выбором является реляционная база данных. Колоночные и другие распределенные архитектуры разработаны вокруг идеи, что их содержание относительно статично. Возможно обновить данные и в них, но это обычно включает некоторый процесс пакетной обработки, когда исходные данные удаляют и на их место помещают обновленные.

Потоковая аналитика в сравнении с хранением в одном месте

Классическая аналитическая система является централизованным хранилищем. Данные из многократных датчиков попадают в огромную базу данных, и затем аналитики достают данные из огромной базы данных. Это не единственный подход, и горячая альтернатива использует аналитику потоковой передачи. Пока писалась эта книга, набирали популярность распределенные потоковые аналитические системы, такие как [Storm](#) и [Websphere IBM](#).

Потоковые подходы обеспечивают сложный анализ в реальном времени путем переработки данных в видеопоток информации. В потоке данные один раз обрабатываются процессом и минимально связаны с прошлым состоянием.

Потоковая передача обработки чрезвычайно полезна в областях, где процесс четко определен и существует потребность в анализе в режиме реального времени. По сути, это не особенно полезно для исследовательского анализа (см. главу 10). Однако, работая с четко определенными предупреждениями и процессами, потоковая аналитика уменьшает издержки, требуемые центральным хранилищем, которые в больших системах передачи и обработки данных могут быть довольно затратны.

После вопросов, связанных с обновлениями, следующий ряд вопросов связан со структурой и размером данных. Колоночные и реляционные базы данных предпочтительны, когда вы имеете дело с хорошо структурированными, маленькими записями (такими как оптимизированные файлы журнала). Эти подходы могут иметь преимущество перед схемой – например, если колоночная база данных использует лишь два столбца, она может вернуть только их для последующей обработки, тогда как база ключей должна вернуть полную запись. Если записи являются маленькими, или структурированными, колоночные базы данных предпочтительны, затем идут реляционные базы данных. Если записи являются большими или не структурированы, то подход с использованием ключа более гибок.

Заключительный вопрос в списке является, возможно, более социальным, чем техническим, но также и важным при рассмотрении дизайна системы анализа. Если вы собираетесь позволить аналитикам относительно открытый, неструктурированный анализ данных, то у вас должна быть некоторая четко определенная и безопасная платформа, чтобы позволить им это делать. Если ваши аналитики могут работать с записями функций MapReduce, то можно использовать любую систему без особых трудностей. Однако если вы ожидаете, что у аналитиков будут минимальные навыки, тогда предпочтительно можно найти колоночные или реляционные системы, которые имеют интерфейсы SQL. Существуют относительно недавние усилия разработать подобные SQL интерфейсы для баз ключей, особенно Hive и Pig проектов от Apache.

Там, где это возможно, предпочтительно ограничить прямой доступ аналитиков к хранилищу данных, вместо этого позволяя им извлечь образцы, которые могут быть обработаны в инструментах EDA, таких как SiLK или R.

Иерархия устройств хранения данных, время выполнения запроса и старение

Любая система сбора должна будет иметь дело с непрерывным притоком новых данных, вынуждая более старые данные со временем переместиться в более медленные, менее дорогостоящие системы хранения. Для целей аналитической системы мы можем разделить иерархию устройства хранения данных, которую мы имеем, на четыре уровня:

- RAM;
- SSDs и флеш-память;
- жесткие диски и магнитные устройства хранения данных;
- лентопротяжные устройства и долгосрочные архивы.

Путем подготовки системы мониторинга потока можно оценить объем входящего трафика и использовать эти данные для определения требований к исходной системе хранения. Ключевой вопрос состоит в том, в каком количестве данных нуждаются аналитики.

Хорошее эмпирическое правило в деловой среде – это то, что аналитикам нужен быстрый доступ к недельным данным, разумный доступ к 90-дневным данным, прочие же данные могут быть депонированы в ленточном архиве. 90-дневное правило означает, что аналитики могут получить данные, по крайней мере, за предыдущий квартал. Конечно, если ваш бюджет позволяет, больше данных на диске лучше, но 90 дней являются хорошим минимальным требованием. Если вы дей-

ствительно архивируете для записи на ленту, удостоверьтесь, что данные ленты доступны в пределах разумного – роботы в большинстве сетей хранят данные в течение приблизительно года, если не дольше, и трекинг их действий включит рассмотрение этого архива.

Множество внешних ограничений также оказывает влияние на хранение данных, особенно требования хранения данных для вашего сегмента рынка и индустрии. Например, директива EU о хранении данных (директива 2006/24/ЕС) устанавливает требования хранения для телекоммуникационных провайдеров¹.

Перемещаясь вниз по иерархии, данные могут быть переформатированы в более обобщенный – или благоприятный для устройства хранения данных – формат. Например, для получения быстрого ответа я хотел бы сохранить прокручивающийся архив пакетов на высокоскоростном устройстве хранения данных. Когда данные переходят на более медленные устройства (от RAM на SSD, от SSD на диск, с диска для записи на ленту), я начну полагаться больше на уже обобщенные данные, такие как NetFlow.

В дополнение к простому обобщению, такому как NetFlow, может быть облегчено долгосрочное хранение путем выявления и обобщения наиболее очевидных моделей поведения. Например, сканирование и обратное рассеяние (см. главу 11 для получения дополнительной информации) занимают огромную часть дискового пространства в больших сетях; трафик не имеет никакой полезной нагрузки, и хранение полного пакета не имеет смысла. Идентификация, обобщение и затем сжатие или удаление сканирования сокращают место необработанных данных, особенно в больших сетях, где этот тип фоновой трафика может принять диспропорциональное количество записей.

Соединение данных – удаляя идентичные записи или объединяя их – является другим жизнеспособным методом, когда сбор данных из многочисленных источников, объединение записей, которые описывают одно и то же явление (путем проверки IP-адресов, портов и времени), может преобразовать полезную нагрузку этих отдельных записей.



¹ В РФ действуют собственные законы, регулирующие хранение и обращение с данными. – Прим. ред.

Часть II

.....

ИНСТРУМЕНТЫ

Эта часть о ряде инструментов, используемых при анализе данных. Основное внимание этой части уделяется двум конкретным инструментам: SiLK и R. Система знания интернет-уровня (SiLK) является аналитическим инструментарием NetFlow, разработанным CERT в Университете Карнеги-Меллон, который позволяет аналитикам быстро и эффективно производить сложные системы анализа потоков. R – это пакет статистического анализа, разработанный в Оклендском университете, обеспечивает исследовательский анализ данных и визуализацию.

В настоящее время нет единого убойного приложения для сетевого анализа. Анализ требует использования многих инструментов, часто способами, для которых они не были действительно разработаны. Надеюсь, что инструменты, описанные в данной части, станут основным функциональным инструментарием для аналитика. Объединение их с легким языком сценариев, таким как Python, позволит аналитикам исследовать данные и разрабатывать оперативно полезные продукты.

Эта часть разделена на пять глав. Глава 5 описывает комплект SiLK, глава 6 разбирает R. Глава 7 обсуждает IDS; в то время как IDSes были кратко рассмотрены в главе 1, эта глава анализирует конструкцию и обслуживание данных инструментов – аналитики будут часто производить оперативные IDSes, чтобы идентифицировать или иметь дело с нападениями. В главе 8 обсуждаются инструменты для идентификации способов, которыми хосты подключены к интернету, включая обратный DNS lookup, зеркала и инструменты, такие как *traceroute* и *ping*. Наконец, глава 9 обсуждает дополнительные инструменты, которые полезны для конкретных аналитических задач.





Глава 5



Комплект SiLK

SiLK, система для знания интернет-уровня, является инструментарием, первоначально разработанным CERT Carnegie Mellon для проведения крупномасштабного netflow-анализа. В настоящее время SiLK широко используется министерством обороны, академическими учреждениями и промышленностью как основной аналитический инструментарий.

Эта глава фокусируется прежде всего на использовании SiLK как аналитического инструмента. CERT-команда осведомленности CERT опубликовала многочисленные ссылки на использование SiLK, описания установки и настройки.

Что такое SiLK, и как он работает?

SiLK является комплектом инструментов для запроса и анализа данных NetFlow. Комплект SiLK позволяет аналитику быстро и эффективно запросить очень большие объемы сетевого трафика, чтобы идентифицировать сложные совокупные явления или извлечь одиночные события.

SiLK является фактически базой данных в командной строке. Каждый инструмент выполняет определенный запрос, управление или агрегацию данных, и команды объединены в цепочку для получения результатов. Путем объединения многих записей в цепочку обработки SiLK позволяет аналитику создать сложные команды, которые одновременно выставляют данные вдоль мультиплексных каналов. Например, следующая последовательность запросов SiLK вытягивает HTTP (порт 80) трафик из данных потока, производя временной ряд и список действия самым занятым адресом. Посмотрите пример 5-1 для основ эксплуатации SiLK: команды переданы через ряд каналов, которые могут быть stdin, stdout или fifos (именованные каналы).

Пример 5-1. Некоторые чрезмерно сложные `rwfilter` `vooodoo`

```
$ mkfifo out2
$ rwfilter --proto=6 --aport=80 data.rwf --pass=stdout |
  rwfilter --input=stdin --proto=6 --pass=stdout
  --all=out2 | rwstats --top --count=10 --fields=1 &
  rwcoun out2 --bin-size=300
```

Данные сохраняются в эффективном двоичном представлении вплоть до последнего момента, пока команды, которые производят текст или данные не вызовут, чтобы выполнить вывод этих данных.

SiLK является в значительной степени пакетом приложений старой школы Unix: семейство инструментов связано вместе с каналами и использует много дополнительных аргументов. При помощи этого подхода возможно создать мощные аналитические сценарии с SiLK, потому что инструменты имеют четко определенные интерфейсы, которые эффективно обработают двоичные данные. Эффективное использование SiLK включает соединение вместе соответствующих инструментов, чтобы обработать двоичные данные и выпустить текст только в самом конце процесса.

Эта глава также использует некоторые основные команды оболочки Unix, такие как `ls`, `cat` и `head`. Я не требую, чтобы вы знали оболочку на экспертном уровне.

Получение и установка SiLK



Домашняя страница SiLK сохраняется на веб-странице NetSA CERT. Пакет SiLK находится в свободном доступе для загрузки и может быть установлен в большинстве систем Unix без особых проблем. CERT также обеспечивает живой образ CD, который может использоваться самостоятельно.

Установочный CD SiLK появился с обучающим набором данных под названием LBNL-05 (<https://tools.netsa.cert.org/silk/referencedata.html>), с анонимизированными заголовками от Lawrence Berkeley National Labs в 2005 г. При установке CD данные будут сразу доступны. В противном случае можно выбрать данные со страницы справочных данных LBNL-05¹.

В дополнение к живому CD SiLK доступен в нескольких диспетчерах пакетов, включая `homebrew`.

Файлы данных

Файлы данных LBNL сохранены в файловой иерархии; пример 5-2 показывает результаты их загрузки и разархивирования.

Пример 5-2. Загрузка архивов SiLK

```
$ gunzip-c SiLK-LBNL-05-noscan.tar
$ gunzip-c SiLK-LBNL-05-scanners.tar
$ cd SiLK-LBNL-05
$ ls
README-S0.txt inout silk.conf
README-S1.txt inweb outweb
$ ls in/2005/01/07/*.01
in/2005/01/07/in-S0_20050107.01 in/2005/01/07/in-S1_20050107.01
```



При сборе данных SiLK разделяет данные в подкаталоги, которые делят трафик на тип трафика и время, в которое событие имело место. Это обеспечивает масштабируемость и ускоряет анализ. Однако обычно это также черный ящик, и один мы ломаем прямо сейчас, просто чтобы получить несколько файлов для работы. В целях демонстрации и обучения мы собираемся работать с четырьмя определенными файлами:

¹ Вы заметите, что существует два набора данных, один со сканированием и один без. Для понимания, почему, читайте: *Pang et al. The Devil and Packet Trace Anonymization*, ACM CCR 36(1), январь 2006.

- `inweb/2005/01/06/iw-S0_20050106.20;`
- `inweb/2005/01/06/iw-S0_20050106.21;`
- `in/2005/01/07/in-S0_20050107.01;`
- `in/2005/01/07/in-S1_20050107.01.`

Эти файлы не являются особенными ни в каком случае. Я выбрал их только для обеспечения примеров трафика сканирования и несканирования. Далее обсудим, как разделить данные и что означает имя файла.

ВЫБОР И ФОРМАТИРОВАНИЕ ВЫХОДНОГО УПРАВЛЕНИЯ ПОЛЕМ: RWCUT

Записи SiLK сохранены в компактном двоичном формате. Они не могут быть считаны напрямую, и поэтому получен доступ с помощью инструмента `rwcut` (см. пример 5-3). В следующем примере и любых других примерах с выводом больше, чем 80 символов, строки разорваны вручную для ясности.

Пример 5-3. Простой доступ к файлу с `rwcut`

```
$ rwcut inweb/2005/01/06/iw-S0_20050106.20 | more
      sIP|          dIP|sPort|dPort|proto|packets|bytes|\ | \
      flags|          sTime|    dur|    eTime|sen|
148.19.251.179| 128.3.148.48| 2497| 80|    6|    16|    2 631|\
FS PA |2005/01/06T20:01:54.119| 0.246|2005/01/06T20:01:54.365|    ?|
148.19.251.179| 128.3.148.48| 2498| 80|    6|    14|    2 159|\
S PA |2005/01/06T20:01:54.160| 0.260|2005/01/06T20:01:54.420|    ?|
...
```

В его вызове по умолчанию `rwcut` выводит 12 полей: входящие и исходящие IP-адреса и порты, протокол, количество пакетов, число байтов, флаги TCP, время начала, продолжительность, время окончания и датчик потока. Эти значения были обсуждены ранее в главе 2, за исключением поля датчика. SiLK может быть сконфигурирован для идентификации отдельных датчиков, что может быть полезно, когда вы пытаетесь выяснить, откуда трафик прибыл или куда он идет. Датчиком поля является любой ID, присвоенный во время конфигурации. В данных по умолчанию нет никаких датчиков, таким образом, значение установлено как вопросительный знак (?).

Все команды SiLK имеют встроенную документацию. При вводе `rwcut --help` появляется огромная страница справки. Мы покроем основные опции. Более полное описание опций может быть найдено в [документации SiLK для rwcut](#).

Наиболее часто используемые `rwcut` команды выбирают поля, выведенные на экран во время вызова. `rwcut` может на самом деле распечатать 29 различных полей в произвольном порядке. Список этих полей находится в табл. 5-1.

Поля `rwcut` определены с помощью `--fields=option` (поля = опция), которая принимает числовые значения в табл. 5-1 или строковых значениях и распечатывает требуемые поля в определенном порядке, как в примере 5-4.

Таблица 5-1. Поля rwcut

Поле	Числовой ID	Описание
sIP	1	Исходный IP-адрес
dIP	2	Целевой IP-адрес
sPort	3	Исходный порт
dPort	4	Целевой порт: если ICMP, тип ICMP и код закодированы. Здесь так же
protocol	5	Протокол уровня 3
packets	6	Пакеты в потоке
bytes	7	Байты в потоке
flags	8	OR флагов TCP
sTime	9	Время начала в секундах
eTime	10	Время окончания в секундах
dur	11	Продолжительность (eTime-sTime)
sensor	12	ID датчика
in	13	ID SNMP входящего интерфейса на маршрутизаторе
out	14	ID SNMP исходящего интерфейса на маршрутизаторе
nhIP	15	Адрес следующего транзитного участка
sType	16	Классификация исходного адреса (внутренний, внешний)
dType	17	Классификация целевого адреса (внутренний, внешний)
scc	18	Код страны источника IP
dcc	19	Код страны целевого IP
class	20	Класс потока
type	21	Тип потока
sTime+msec	22	Время начала в миллисекундах
eTime+msec	23	Время окончания в миллисекундах
dur+msec	24	Продолжительность в миллисекундах
IcmpTypeCode	25	Тип и код ICMP

Пример 5-4. Некоторые примеры упорядочивания поля

```

$# Show a limited set of fields
$ rwcut --field=1-5 inweb/2005/01/06/iw-S0_20050106.20 | head -2
    sIP|                dIP|sPort|dPort|pro|
148.19.251.179| 128.3.148.48| 2497|   80| 6|
$#Note the -, now explicitly enumerate
$ rwcut --field=1,2,3,4,5 inweb/2005/01/06/iw-S0_20050106.20 | head -2
    sIP|                dIP|sPort|dPort|pro|
148.19.251.179|   128.3.148.48| 2497|   80| 6|
$#Field order is based      on what you enter in --field
$ rwcut --field=5,1,2,3,4    inweb/2005/01/06/iw-S0_20050106.2 | head -2
pro|                sIP|                dIP|sPort|dPort|
6|   148.19.251.179|   128.3.148.48| 2497|   80|
$# We can use text instead of      numbers
$ rwcut --field=sIP,dIP,proto inweb/2005/01/06/iw-S0_20050106.20 | head -2
    sIP|                dIP|proto|
148.19.251.179| 128.3.148.48| 6|

```


rwcut поддерживает много других выходных инструментов форматирования и управления. Особенно полезны те, которые позволяют вам контролировать, какие строки появляются на выходе, и включают:

--no-title (без заголовка)

Обычно используемый с командами SiLK, которые производят табличный вывод. Исключает заголовок из выходной таблицы.

--num-recs (количество записей)

Выводит определенное количество записей, избавляя от необходимости использовать утилиту head(1), как в предыдущем примере. Значение по умолчанию является нулем, который заставляет **rwcut** вывести все содержание любого файла, который он читает.

--start-rec-num --end-rec-num(начальный и конечный номера записи)

Может использоваться для выборки диапазона записей в файле.

Пример 5-5 показывает несколько способов управления количеством записей и заголовками.

Пример 5-5. Управление количеством записей и заголовками

```
$# Drop the title
$ rwcut --field=1-9 --no-title inweb/2005/01/06/iw-S0_20050106.20 | head - 5
148.19.251.179| 128.3.148.48| 2497| 80| 6| 16| 2631|FS PA
|2005/01/06T20:01:54.119|
148.19.251.179| 128.3.148.48| 2498| 80| 6| 14| 2159| S PA
|2005/01/06T20:01:54.160|
148.19.251.179| 128.3.148.48| 2498| 80| 6| 2| 80|F A
|2005/01/06T20:07:07.845|
56.71.233.157| 128.3.148.48|48906| 80| 6| 5| 300| S
|2005/01/06T20:01:50.011|
56.96.13.225| 128.3.148.48|50722| 80| 6| 6| 360| S
|2005/01/06T20:02:57.132|
$# Drop the head statement
$ rwcut --field=1-9 inweb/2005/01/06/iw-S0_20050106.20 --num-recs=5
sIP| dIP|sPort|dPort|pro| packets | bytes| flags
| sTime|
148.19.251.179| 128.3.148.48| 2497| 80| 6| 16| 2631|FS PA
|2005/01/06T20:01:54.119|
148.19.251.179| 128.3.148.48 | 2498| 80| 6| 14| 2159| S PA
|2005/01/06T20:01:54.160|
148.19.251.179| 128.3.148.48 | 2498| 80| 6| 2| 80|F A
|2005/01/06T20:07:07.845|
56.71.233.157| 128.3.148.48|48906| 80| 6| 5| 300| S
|2005/01/06T20:01:50.011|
56.96.13.225| 128.3.148.48|50722| 80| 6| 6| 360| S
|2005/01/06T20:02:57.132|
$# Print only the third through fifth record
$ rwcut --field=1-9 inweb/2005/01/06/iw-S0_20050106.20 --start-rec-num=3
--end-rec-num=5
sIP| dIP|sPort|dPort| | packets | bytes| flags
| sTime|
148.19.251.179| 128.3.148.48 | 2498| 80| 6| 2| 80|F A
|2005/01/06T20:07:07.845 |
```

56.71.233.157	128.3.148.48 48906	80	6	5	300	S
2005/01/06T20:01:50.011						
56.96.13.225	128.3.148.48 50722	80	6	6	360	S
2005/01/06T20:02:57.132						

Ряд опций управляют выходным форматом. Табулирование управляемо с опциями `column-separator` (разделитель столбцов), `no-final-column` (без последних столбцов) и переключателем `no-columns` (без столбцов); `column-separator` изменяет символ, разделяющий столбцы, в то время как `no-final-column` отбрасывает разделитель в конце строки; `no-columns` удаляет любое дополнение пространства между столбцами; (разграничительный) переключатель комбинирует все три: он берет символ как параметр, использует этот символ в качестве разделителя столбцов, удаляет все дополнения в столбцах и пропускает разделитель последнего столбца.

Кроме того, существует множество переключателей для изменения содержимого столбца:

- `integer-ips` (целочисленный `ips`)

Преобразовывает IP-адреса в целые числа, а не отмеченные точкой четверки. Этот переключатель является устаревшим, как из SiLK v3, и пользователи должны теперь применять - `ip-format=decimal`.

- `ip-format`

Обновленная версия; `integer-ips`, `ip-format` определяют, как представлены адреса. Опции включают канонический (отмеченная точкой четверка для IPv4, канонический IPv6 для IPv6), дополненный нулем (канонический, кроме нулей, расширяющих до максимального значения для каждого формата, таким образом, 127.0.0.1 127.000.000.001), десятичное число (печать как соответствующее 32-разрядное или 128-разрядное целое число), шестнадцатеричный (распечатывает целое число в шестнадцатеричном формате) и `force-ipv6` (распечатывает все адреса в каноническом формате IPv6, включая адреса IPv4, отображенные как `ffff:0:0/96 netblock`).

- `epoch-time`

Печатает метки времени как значения времени Unix с плавающей точкой с точностью до миллисекунды.

- `integer-tcp-flags`

Преобразовывает флаги TCP в их целочисленные эквиваленты.

- `zero-pad-ips`

Дополняет точечный квадратический формат IP-адреса нулями, так чтобы 128.2.11.12 был распечатан как 128.002.011.012. Удержанный от использования в пользу `ip-format` в SiLK v3.

- `icmp-type-and-code`

Помещает тип ICMP в исходный порт и код ICMP в целевой порт.

- `pager`

Определяет программу использования для страничного вывода.

Пример 5-6 показывает некоторые предыдущие опции.

Пример 5-6. Другие примеры форматирования

```

$# Change from fixed with columns to delims
$ rwcut -field=1-5 inweb/2005/01/06/iw-S0_20050106.20 -no-columns-num-recs=2
sIP|dIP|sPort|dPort|protocol|
148.19.251.179|128.3.148.48|2497|80|6|
148.19.251.179|128.3.148.48|2498|80|6| $# Change the column separator
$ rwcut -field=1-5 inweb/2005/01/06/iw-S0_20050106.20 -column-sep=:
-num-recs=2
      sIP:                dIP:sPort:dPort:pro:
148.19.251.179: 128.3.148.48: 2497: 80: 6:
148.19.251.179: 128.3.148.48: 2498: 80: 6:
$# Use delim to change everything at once
$ rwcut -field=1-5 inweb/2005/01/06/iw-S0_20050106.20 -delim =: -num-recs=2
sIP:dIP:sPort:dPort:protocol
148.19.251.179:128.3.148.48:2497:80:6
148.19.251.179:128.3.148.48:2498:80:6
$# Convert IP-addresses to integers
$ rwcut -field=1-5 inweb/2005/01/06/iw-S0_20050106.20 -integer-ip -num-recs=2
      sIP |      dIP|sPort|dPort|pro |
2484337587|2147718192| 2497| 80| 6|
2484337587|2147718192| 2498| 80| 6|
$# Use epoch time
$ rwcut -field=1-5,9 inweb/2005/01/06/iw-S0_20050106.20 -epoch -num-recs=2
      sIP |      dIP|sPort|dPort|pro|      sTime|
148.19.251.179 | 128.3.148.48 | 2497 | 80| 6|1105041714.119|
148.19.251.179 | 128.3.148.48 | 2498 | 80| 6|1105041714.160|
$# Zero Pad IP addresses
$ rwcut -field=1-5,9 inweb/2005/01/06/iw-S0_20050106.20 -zero-pad -num-recs=2
      sIP |      dIP|sPort|dPort|pro |      sTime|
148.019.251.179| 128.003.148.048| 2497| 80| 6|2005/01/06T20:01:54.119|
148.019.251.179| 128.003.148.048| 2498| 80| 6|2005/01/06T20:01:54.160|

```

Можно заметить, что поскольку командные строки становятся более сложными, я сократил более длинные опции. SiLK использует стиль GNU длинные опции универсально, таким образом, единственное требование для определения опции – это необходимость ввести достаточно символов, чтобы сделать имя одно-значным. Ожидайте все большего сокращения, поскольку мы создаем все более и более сложные команды.

ОСНОВНОЕ УПРАВЛЕНИЕ ПОЛЕМ: RWFILTER

Самая основная команда SiLK с аналитическими значениями является `rwcut` в паре с `rfilter` через канал. Пример 5-7 отображает простую `rfilter`-команду.

Пример 5-7. Простая `rfilter`-команда

```

$ rfilter -dport=80 inweb/2005/01/06/iw-S0_20050106.20 -pass=stdout
| rwcut -field=1-9 -num-recs=5
      sIP |      dIP|sPort|dPort|pro | packets|      bytes|      flags
      |      sTime|
148.19.251.179| 128.3.148.48| 2497| 80| 6| 16| 2631|FS      PA
|2005/01/06T20:01:54.119|
148.19.251.179| 128.3.148.48| 2498| 80| 6| 14| 2159| S PA

```

```
|2005/01/06T20:01:54.160|
148.19.251.179| 128.3.148.48| 2498| 80| 6| 2| 80|F A
|2005/01/06T20:07:07.845|
56.71.233.157| 128.3.148.48| 48906| 80| 6| 5| 300| S
|2005/01/06T20:01:50.011|
56.96.13.225| 128.3.148.48|50722| 80| 6| 6| 360| S
|2005/01/06T20:02:57.132|
```

rwfilter с единственным фильтром (-dport опция в этом случае) и единственным перенаправлением (-pass=stdout) так прост, как может быть; rwfilter является рабочей лошадкой комплекта SiLK: он читает ввод (непосредственно из файла, с помощью ряда globbing-спецификаций, или через канал), применяет один или несколько фильтров к каждой записи в данных, затем перенаправляет записи на основе того, соответствует запись фильтру (передает) или не соответствует (сбрасывает).

rwfilter-документация SiLK огромна, но прежде всего состоит из повторяющегося описания спецификаций фильтра для каждого поля, поэтому не пугайтесь; rwfilter-опции в основном делают одну из трех вещей: они определяют, как отфильтровать данные, как считать данные или как направить результат этих фильтров.

Порты и протоколы

Самые легкие фильтры для запуска -sport, -dport и -protocol. Имена подразумевают, что они соответственно фильтруют на исходном, целевом портах и протоколе (см. пример 5-8). Эти значения могут отфильтровать на определенном значении (например, -sport=80 передаст любой трафик, где исходный порт равняется 80) или в диапазоне, определенном с тире или запятыми (таким образом, -sport=79-83 передаст что-либо, где исходный порт находится между 79 и 83 и мог быть выражен как -sport=79,80,81,82,83).

Пример 5-8. Фильтрация в качестве примера на sPort

```
$ rwfilter - dport=4350-4360 inweb/2005/01/06/iw-S0_20050106.20
- pass=stdout | rwcut - field=1-9 - num-recs=5
      sIP |          sTime| dIP|sPort|dPort|pro| packets| bytes| flags
|
218.131.115.42| 131.243.105.35| 80| 4360| 6| 2| 80|F A
|2005/01/06T20:24:21.879|
148.19.96.160|131.243.107.239| 80| 4350| 6| 27| 35445|FS PA
|2005/01/06T20:59:42.451|
148.19.96.160|131.243.107.239| 80| 4352| 6| 4| 709|FS PA
|2005/01/06T20:59:42.507|
148.19.96.160|131.243.107.239| 80| 4351| 6| 15| 16938|FS PA
|2005/01/06T20:59:42.501|
148.19.96.160|131.243.107.239| 80| 4353| 6| 4| 704|FS PA
|2005/01/06T20:59:42.544|

$ rwfilter - sport=4000-inweb/2005/01/06/iw-S0_20050106.20
-pass=stdout | rwcut -field=1-9 -num-recs=5
      sIP |          sTime| dIP|sPort|dPort|pro | packets| bytes| flags
|
56.71.233.157| 128.3.148.48|48906| 80| 6| 5| 300| S
|2005/01/06T20:01:50.011|
56.96.13.225| 128.3.148.48|50722| 80| 6| 6| 360| S
|2005/01/06T20:02:57.132|
```

```

56.96.13.225| 128.3.148.48|50726| 80| 6| 6| 360| S
|2005/01/06T20:02:57.432|
58.236.56.129| 128.3.148.48|32621| 80| 6| 3| 144| S
|2005/01/06T20:12:10.747|
56.96.13.225| 128.3.148.48|54497| 443| 6| 6| 360| S
|2005/01/06T20:09:30.124|
$ rwfilter - dport=4350,4352 inweb/2005/01/06/iw-S0_20050106.20
-pass=stdout | rwcut -field=1-9 -num-recs=5
      sIP | dIP|sPort|dPort|pro | packets| bytes| flags
|
| sTime|
148.19.96.160|131.243.107.239| 80| 4350| 6| 27| 35445|FS PA
|2005/01/06T20:59:42.451|
148.19.96.160|131.243.107.239| 80| 4352| 6| 4| 709|FS PA
|2005/01/06T20:59:42.507|
148.19.96.160|131.243.107.239| 80| 4352| 6| 1| 40| A
|2005/01/06T20:59:42.516|
$ rwfilter -proto=1 in/2005/01/07/in-S0_20050107.01 -pass=stdout
| rwcut -field=1-6 -num-recs=2
      sIP | dIP|sPort|dPort|pro | packets|
35.223.112.236| 128.3.23.93| 0| 2048| 1| 1|
62.198.182.170| 128.3.23.81| 0| 2048| 1| 1|
$ rwfilter - proto=1,6,17 in/2005/01/07/in-S0_20050107.01 -pass=stdout
| rwcut - num-recs=2 - fields=1-6
      sIP | dIP|sPort|dPort|pro | packets|
116.66.41.147|131.243.163.201| 4283| 1026| 17| 1|
116.66.41.147|131.243.163.201| 3131| 1027| 17| 1|
$ rwfilter -proto=1,6,17 in/2005/01/07/in-S0_20050107.01 -fail=stdout
| rwcut -num-recs=2 -fields=1-6
      sIP | dIP|sPort|dPort|pro | packets|
57.120.186.177| 128.3.26.171| 0| 0| 50| 70|
57.120.186.177| 128.3.26.171| 0| 0| 50| 81|

```

Обратите внимание на использование `-fail` в последнем примере. Поскольку существует 255 потенциальных протоколов, указание «все, кроме TCP, ICMP и UDP» можно выразить двумя способами: либо путем определения того, что вы хотите (`-proto=0,2-5,7-16,18-`), либо при помощи опции `-fail`. В следующей главе обсудим более усовершенствованное управление, используя `-pass` и `-fail`¹.

Размер

Объем (размер) опций (байты и пакеты) соответствует протоколу и опциям порта в численном выражении. В отличие от перечислений (порты и протокол), эти числовые значения могут быть выражены только как единственные цифры или диапазоны, но не как разделенные запятой значения. Таким образом, `-packets=70-81` приемлемо, а `-bytes=1,2,3,4` – нет.

IP-адреса

Самая простая форма фильтрующего IP-адреса, непосредственно выражает просто IP-адрес (см. пример 5-9). Следующие примеры показывают строгую фильтра-

¹ `--pass-destination=` # file to get records that pass разрешает запись.

`--fail-destination=` # file to get records that fail запрещает запись. – Прим. ред.

цию по адресу источника (--saddress) и по адресу назначения (-daddress) и опцию --any-address., которая будет матчиться (увязываться) или с адресом источника, или адресом назначения.

Пример 5-9. Фильтрация по IP-адресам

```
$ rfilter - saddress=197.142.156.83 -pass=stdout
in/2005/01/07/in-S0_20050107.01 | rwcut -num-recs=2
```

sIP	dIP	sPort	dPort	pro	packets	bytes	flags
sTime	dur	eTime	sen				
197.142.156.83	224.2.127.254	44510	9875	171	12	7163	
2005/01/07T01:24:44.359	16.756	2005/01/07T01:25:01.115	?				
197.142.156.83	224.2.127.254	44512	9875	171	4	2590	
2005/01/07T01:25:02.375	5.742	2005/01/07T01:25:08.117	?				

```
$ rfilter -daddress=128.3.26.249 -pass=stdout
in/2005/01/07/in-S0_20050107.01 | rwcut -num-recs=2
```

sIP	dIP	sPort	dPort	pro	packets	bytes	flags
sTime	dur	eTime	sen				
211.210.215.142	128.3.26.249	4068	25	6	7	388	FS PA
2005/01/07T01:27:06.789	5.052	2005/01/07T01:27:11.841	?				
203.126.20.182	128.3.26.249	51981	4587	6	56	2240	F A
2005/01/07T01:27:04.812	18.530	2005/01/07T01:27:23.342	?				

```
$ rfilter -any-address=128.3.26.249
-pass=stdout in/2005/01/07/in-S0_20050107.01 | rwcut -num-recs=2
```

sIP	dIP	sPort	dPort	pro	packets	bytes	flags
sTime	dur	eTime	sen				
211.210.215.142	128.3.26.249	4068	25	6	7	388	FS PA
2005/01/07T01:27:06.789	5.052	2005/01/07T01:27:11.841	?				
203.126.20.182	128.3.26.249	51981	4587	6	56	2240	F A
2005/01/07T01:27:04.812	18.530	2005/01/07T01:27:23.342	?				

Опции адреса могут быть описаны различными способами. Каждая четверка в IP-адресе может быть выражена с помощью того же формата «запятая – тире», который используется для портов и протоколов. IP-адреса могут также применять значение символа *x* от 0 до 255. Это выражение может использоваться в каждой четверке; SiLK будет сравнивать каждую четверку отдельно. В дополнение к этому формату «запятая – тире» SiLK может сравнивать блоки CIDR.

SiLK поддерживает IPv6 при помощи основанной на двоеточии нотации IPv6. Следующие ниже примеры все являются допустимыми для IPv6, фильтруемыми SiLK; и пример 5-10 отображает, как их отфильтровать:

```
::-ffff:x
::-ffff:0:aaaa,0-5
::-ffff:0.0.5-130,1,255.x
```



Пример 5-10. Фильтрация диапазонов IP

```
$#Filtering on the last quad
$ rfilter -daddress=131.243.104.x inweb/2005/01/06/iw-S0_20050106.20
--pass=stdout | rwcut --field=1-5 -num-recs=5
```

sIP	dIP	sPort	dPort	pro	packets	bytes	flags
150.52.105.212	131.243.104.181	80	1262	6			
150.52.105.212	131.243.104.181	80	1263	6			
59.100.39.174	131.243.104.27	80	3188	6			
59.100.39.174	131.243.104.27	80	3191	6			

```

59.100.39.174| 131.243.104.27| 80| 3193| 6|
#, Filtering a range of specific values in the third quad
$ rfilter -daddress=131.243.104,107,219.x inweb/2005/01/06/iw-S0_20050106.20
--pass=stdout | rwcut -field=1-5 -num-recs=5
sIP| dIP|sPort|dPort|pro|
208.122.23.36| 131.243.219.201| 80| 2473| 6|
205.233.167.250| 131.243.219.201| 80| 2471| 6|
58.68.205.40| 131.243.219.37| 80| 3433| 6|
208.233.181.122| 131.243.219.37| 80| 3434| 6|
58.68.205.40| 131.243.219.37| 80| 3435| 6|
# Using CIDR blocks
$ rfilter -saddress=56.81.0.0/16 inweb/2005/01/06/iw-S0_20050106.20 -pass=stdout | rwcut
-field=1-5 -num-recs=5
sIP| dIP|sPort|dPort|pro| 56.81.19.218|131.243.219.201| 80| 2480| 6| 56.81.16.73
|131.243.219.201| 80| 2484| 6| 56.81.16.73 |131.243.219.201| 80| 2486| 6| 56.81.30.48
|131.243.219.201|443| 2490| 6| 56.81.31.159|131.243.219.201|443| 2489| 6|

```

Время

Существует три опции времени: `-stime`, `-etime` и `-active-time`. Эти поля требуют диапазона времени, который в SiLK записан в формате:

```
YYYY/MM/DDTHH:MM:SS-YYYY/MM/DDTHH:MM:SS
```

Отметьте T-разделение дня и часа; поля `-stime` и `-etime` фильтруют точно, хотя то, что они отображают, может показаться немного нелогичным.

```
-stime=2012/11/08T00:00:00-2012/11/08T00:02:00
```

фильтрует любую запись, *время начала* которой между полночью и двумя минутами первого ночи 8 ноября 2012 г. Записи, которые запустились *до* полночи и все еще передаются во время этого диапазона, не пройдут. Для нахождения записей, которые произошли в рамках конкретного периода, используйте фильтр `-active-time`.

Опции TSP

Потоки агрегируют пакеты, и в большинстве случаев эту агрегацию относительно легко понять. Например, число байтов в потоке является суммой числа байтов всех пакетов, которые включает поток. Флаги TSP, однако, немного более проблематичны. В v5 флаги потока являются результатом битовой операции «ИЛИ» над флагами в составляющих его пакетах – означающие, что поток указывает на то, что флаг присутствовал или отсутствовал во *всем* потоке, но неизвестно *где*. Поток мог, очевидно, состоять из последовательности тарабарщины флагов, таких как FIN, затем ACK и SYN. Управляющее программное обеспечение, такое как YAF, расширяет NetFlow для включения дополнительных полей флага, которые может использовать в своих интересах SiLK.

Основные фильтры флагов `-flags-initial`, `flags-all` и `flags-session`. Эти параметры принимают флаги в форме *high flags / mask flags* (*сигнальный флаг/маска-флаг*). Если флаг перечислен в *mask*, SiLK всегда анализирует его. Если флаг перечислен в *high*-флагах, SiLK передает его, *только если* значение *high*. Сами флаги выражены с помощью символов в табл. 5-2.

Таблица 5-2. Выражение TCP-флагов в `rwfilter`

Символ	Флаг
F	FIN
S	SYN
R	RST
P	PSH
A	ACK
U	URG
E	ECE
C	CWR

Комбинация сигнальных флагов и флагов маски имеет тенденцию смущать людей, поэтому давайте рассмотрим некоторые примеры. Помните, что основное правило состоит в том, что для оценки флага он *должен* быть в *mask*. Флаг, определенный как *high*, но не определенный как в *mask*, будет проигнорирован.

- Установка значения S/S передаст любую запись, где флаг SYN установлен *high*.
- Установка значения S/SA передаст любую запись, где флаг SYN *high* и флаг ACK установлены как *low*.
- Определение значения SA/SA передаст любую запись, где *и* SYN, и флаги ACK установлены *high*.
- Комбинация SAF/SAFR возвратит любую запись, где SYN, ACK, флаги FIN *high* и флаг RST *low* – это минимум, которого можно ожидать от обычного протокола TCP-соединения.

В дополнение к этим опциям SiLK обеспечивает ряд специфичных для флага опций в форме `-syn-flag`, `-fin-flag` и т. д. для каждого потенциального флага. Эти опции берут 1 или 0 как параметр: установка значения 1 передаст записи, где флаг *high*, 0 передаст записи, где флаг *low*, и использование без параметра передаст все записи.

Как должны выглядеть флаги TCP?

Комбинация флагов TCP в каком-то конкретном потоке может быть полезным индикатором поведения потока, и существуют определенные комбинации флагов, которые вызывают подозрение.

Почти все потоки TCP должны передать или SAF/SAFR, или SAR/SAFR, не передавая SAFR/SAFR. Это вызвано тем, что большинство сеансов закончится в FIN, с ошибками – заканчиваются в RST. Если замечены и FIN, и RST, это подозрительно.

Сеанс TCP без флага ACK – это любопытно, особенно если этот сеанс имеет четыре пакета или больше. Стеки обычно жестко закодированы на отказ после *n* пакетов, где *n* обычно около трех.

Для клиента начальный флаг должен быть SYN, в то время как сервер должен иметь SYN+ACK. Вы никогда не должны видеть SYN после начального флага.

Ресинхронизация будет означать, что новый сеанс начался с использования одного и того же динамического порта, что странно для TCP.

По-моему, флаги PSH и URG являются универсальным индикатором скучных сессий. Если я вижу сессию без PSH, особенно если она длинная, это кажется мне

любопытным. На мой взгляд, у «нормальной» сессии TCP-значение FSPA будет высоким. Поток только с высоким RA обычно говорит о повреждении данных в потоке – посмотрите в репозитории для той же комбинации адреса, и вы, вероятно, найдете поток SAP, проходящий перед ним.

Фальшивые/ответные сообщения (Backscatter) включают потоки A, SA и RA. Большое количество пакетов RA придет в любую большую сеть из-за имитированных DDos-атак. Совершенно ничего невозможно сделать с этими пакетами; они даже не нацелены непосредственно на вашу сеть¹.

Вспомогательные опции

При сравнении основанной на опции фильтрации rfilter с фильтрацией BPF tcpdump сразу становится очевидно, что подход rfilter намного более примитивен. Это было намеренным решением: rfilter фокусируется на обработке больших объемов так быстро, как это возможно, и издержки на разработку некоторого способствующего анализу языка посчитали слишком большими.

Местом, где это обычно сбивает людей с толку, является отсутствие очевидных операторов not и or. Например, если вы хотите отфильтровать все веб-сеансы, можно попытаться отфильтровать трафик, где один порт – 80, а другой динамический. Начальная попытка могла бы быть:

```
rfilter -sport=80,1024-65535 -dport=80,1024-65535 -pass=stdout
```

Проблема состоит в том, что это также передаст любые потоки, где исходный и целевой порт – 80, и потоки, где исходный и целевой порт оба эфемерны. Чтобы иметь дело с подобными проблемами, rfilter имеет набор функций помощника, который совместно с -fail option и различными фильтрами должен быть в состоянии решить любую из этих проблем.

В случае портов -aport опция относится или к источнику, или к целевому порту. Используя -aport и два фильтра, можно идентифицировать соответствующие сеансы следующим образом:

```
rfilter -aport=80 -pass=stdout | rfilter -input-pipe=stdin -aport=1024-65535 -pass=stdout
```

Первый фильтр идентифицирует какой-либо трафик, привлеченный на порт 80, а второй использует эти данные и идентифицирует что-то, что использовало динамический порт.

Доступно несколько вариантов помощника фильтрации IP-адресов. -anyaddress фильтрует через исходные адреса и адреса назначения одновременно.

¹ Термин «Backscatter» относится к письмам, которые получают пользователи якобы в ответ на сообщения, которые они никогда не отправляли. Письма содержат фальшивый адрес в поле «Return-Path». В результате когда одно из таких сообщений отклоняется сервером получателя, либо когда у получателя включен автоответчик, либо когда для учетной записи включено специальное сообщение о том, что сотрудник находится вне офиса или в отпуске, ответ на это ложное сообщение будет направлен на фальшивый адрес. Это может привести к появлению в почтовых ящиках ваших пользователей огромного числа ложных сообщений о статусе доставки DSN (Delivery Status Notifications) и автоответов. После этого спамеры и вирусописатели часто могут использовать это явление в своих интересах и запустить атаку на отказ в обслуживании (DoS – Denial of Service) против почтовых серверов, вызывая целую лавину ложных писем с различных серверов по всему миру.

`-not-saddress` и `-not-daddress` передают записи с адресами, которые *не* соответствуют спецификации опции.



Разные опции фильтрации и некоторые взломы

`rwfilter` имеет несколько вариантов прямого вывода текста: `-print-stat` (см. пример 5-11). Они могут быть использованы для печати сводки по трафику без необходимости прибегать к его вырезанию, подсчету или другим инструментам. Они также напечатают все записи, которые не прошли фильтр.

Пример 5-11. Использование `-print-stat`

```
$ rwfilter -print-volume-stat in/2005/01/07/in-S0_20050107.01 -proto=0-255
|          Recs|          Packets|          Bytes|          Files|
Total|          2019|          2730488|          402105501|          1|
Pass|          2019|          2730488|          402105501|          |
Fail|           0|              0|              0|          |
$ rwfilter --print-stat in/2005/01/07/in-S0_20050107.01 --proto=0-255
Files  1. Read      2019. Pass      2019. Fail  0.
```

Отметьте в примере 5-11 использование опции `-proto=0-255`. Почти во всех вызовах `rwfilter` ожидает *некоторую* форму фильтрации, относящуюся к нему, поэтому при необходимости в фильтре, который передает все, самый легкий подход – это только определить все протоколы; `-print-stat` и `-print -volume-stat` выводят к `stderr`, таким образом, можно все еще использовать `stdout` для передачи, сбоя и всех каналов.

Как и `gwcut`, `rwfilter` имеет команду лимита записи; `-max-pass-records`, `-max-fail-records` могут использоваться для ограничения количества записей, прошедших через канал передачи или сбоя.

RWFILEINFO И ИСТОЧНИК

Файлы фильтра SiLK содержат изрядное количество метаданных, к которым можно получить доступ с помощью команды `rwfileinfo` (см. пример 5-12). Как видно в примере ниже, `rwfileinfo` может работать с файлами или непосредственно на `stdin` при помощи `stdin`, или как параметр.

Пример 5-12. Использование `rwfileinfo`

```
$ rwfileinfo in/2005/01/07/in-S0_20050107.01
in/2005/01/07/in-S0_20050107.01:
format (id)          FT_RWAUGMENTED (0x14)
version              2
byte-order            littleEndian
compression(id)      none (0)
header-length        28
record-length        28
record-version       2
silk-version         0
count-records        2019
file-size            56560
packed-file-info     2005/01/07T01:00:00??
$ rwfilter -print-stat in/2005/01/07/in-S0_20050107.01 -proto=6
-pass=example.rwf
```

```

Files      1.      Read   2019. Pass                               1353. Fail           666.
$ rwfileinfo
example.rwf:
  format (id)          FT_RWGENERIC (0x16)
  version              16
  byte-order           littleEndian
  compression(id)      none(0)
  header-length        156
  record-length        52
  record-version       5
  silk-version         2.1.0
  count-records        1353
  file-size            70512
  command-lines
1  rwfilter -print-stat -proto=6 -pass=example.rwf
in/2005/01/07/in-S0_20050107.01
$ rwfilter -aport=25 example.rwf -pass=example2.rwf -fail=example2_fail.rwf
$ rwfileinfo example2.rwf
example2.rwf:
  format (id)          FT_RWGENERIC (0x16)
  version              16
  byte-order           littleEndian
  compression(id)      none(0)
  header-length        208
  record-length        52
  record-version       5
  silk-version         2.1.0
  count-records        95
  file-size            5148
  command-lines
1  rwfilter -print-stat -proto=6 -pass=example.rwf
in/2005/01/07/in-S0_20050107.01
2  rwfilter -aport=25 -pass=example2.rwf
-fail=example2_fail.rwf example.rwf

```

Поля, о которых сообщает `rwfileinfo`, следующие:

`example2.rwf`

Первая строка каждого `rwfileinfo dump` является названием файла.

`format (id)`

Файлы SiLK сохраняются в ряде различных оптимизированных форматов; значение формата является макросом C, описывающим тип файла, сопровождаемого шестнадцатеричным ID этого типа.

`version`

Версия формата файла.

`byte-order`

Порядок, в котором байты сохранены на диске; SiLK поддерживает отличные форматы с прямым и обратным порядками байтов для более быстрого чтения.

`compression(id)`

Сжат ли файл исходно, также для более быстрого чтения.

`header-length`

Размер заголовка файла; файл SiLK без записей будет просто размером длины заголовка.

`record-length`

Размер отдельных записей файла. Это значение будет 1, если записи будут с переменной длиной.

`record-version`

Версия записей (заметим, что версии записей отличны от версий файла и версий SiLK).

`silk-version`

Версия комплекта SiLK, использованная при создании файла.

`count-records`

Количество записей в файле.

`file-size`

Общий размер файла; если файл является несжатым, это значение должно быть эквивалентно длине заголовка, добавленной к продукту длины записи и количества записей.

`command-lines`

Запись команд SiLK, использованных при создании файла.

Пример 5-13 показывает, как использовать команду `-note-add`.

Пример 5-13. Использование `-note-add`

```
$ rfilter -aport=22 example.rwf -note-add='Filtering ssh' -pass=ex2.rwf
```

```
$ rfileinfo ex2.rwf
```

```
ex2.rwf:
```

```
format (id)      FT_RWGENERIC (0x16)
version          16
byte-order       littleEndian
compression(id)  none(0)
header-length    260
record-length    52
record-version   5
silk-version     2.1.0
count-records    10
file-size        780
command-lines
```



```
1 rfilter -print-stat -proto=6 -pass=example.rwf
```

```
in/2005/01/07/in-S0_20050107.01
```

```
2 rfilter -aport=22 -note-add=Filtering ssh
```

```
-pass=ex2.rwf example.rwf
```

```
annotatios
```

```
1 Filtering ssh
```

Объединение информационных потоков: `rwcount`

`rwcount` может произвести данные временного ряда из вывода команды `rfilter`. Это работает путем помещения количества байтов, пакетов и записей потока в *корзины* фиксированной длины, которые являются одинаково измеренными периодами времени, определенными пользователем. `rwcount` является относительно прямым

приложением. Наибольшая его сложность возникает из-за связи потоков, которые сами имеют соответствующую продолжительность с корзинами.

Самый простой вызов gwscount показан в примере 5-14. Первое, что можно заметить, – это использование опции. В этом примере корзины являются получасовыми, или 1800-секундными. Если `-bin-size` не будет определен, то по умолчанию gwscount примет значение корзины 30-секундным. Размеры корзины не обязательно должны быть целыми числами; спецификации с плавающей точкой с разрешением вниз к миллисекунде приемлемы для людей, которым нравится *много* корзин на выходе.

Пример 5-14. Простой вызов gwscount

```
$ rwfilter in/2005/01/07/in-S0_20050107.01 -all=stdout |
gwscount -bin-size=1800
```

Date	Records	Bytes	Packets
2005/01/07T01:00:00	257.58	42827381.72	248724.14
2005/01/07T01:30:00	1589.61	211453506.60	1438751.93
2005/01/07T02:00:00	171.81	147824612.67	1043011.93

Как показывает пример 5-14, gwscount выводит четыре столбца: столбец даты в стандартном формате даты SiLK (YYYY/MM/DDTHH:MM:SS) и далее столбцы записей, байтов и пакетов. Значения с плавающей точкой являются функцией gwscount, интерполирующей, какое количество трафика должно быть в каждой корзине; gwscount называет это *схемой загрузки*.

Схема загрузки является попыткой gwscount приблизительно оценить, сколько из потока прошло за период, определенный корзинами. В схеме загрузки по умолчанию gwscount разделяет каждый поток пропорционально через все корзины, во время которых происходил поток. Например, если период потока с 0:04:00 до 0:11:00 и корзины пять минут длиной, то 1/7 потока будет добавлена к первой (0:00:00-0:04:59) корзине, 5/7 – ко второй корзине (0:05:00-0:09:59) и 1/7 – к третьей (0:10:00-0:14:59) корзине; gwscount выбирает целочисленный параметр в опции `-load-schem` со следующими результатами:

- 0 Разделить трафик равномерно через все покрытые интерполяции. В предыдущем примере поток был бы разделен на трети и треть добавлена к каждой корзине.
- 1 Добавить весь поток к первой корзине, покрытой потоком. В предыдущем примере 0:00:00-0:04:59.
- 2 Добавить весь поток к последней корзине, покрытой потоком. В предыдущем примере 0:10:00-0:14:59.
- 3 Добавить весь поток к средней корзине, покрытой потоком. В предыдущем примере 0:05:00-0:09:59.
- 4 Схема загрузки по умолчанию.

gwscount использует данные потока, обеспеченные для предположения, какие корзины времени требуются, но иногда необходимо явно определить время, особенно при координировании многократных файлов. Это может быть сделано с помощью опций `-start-epoch` и `-end-epoch`, чтобы определить время запуска и окончания корзины. Обратите внимание на то, что эти параметры могут использовать время эпохи или формат уууу/мм/дд:HH:MM:SS; gwscount также имеет опцию распечатать даты, использующие время эпохи: `-epoch-slots`.

Опция `-skip-zero` (см. пример 5-15) является одной из нескольких опций выходного формата. Обычно `rwcount` распечатывает каждый пустой контейнер, который он выделил, но `-skip-zero` заставляет пропускать пустые контейнеры при выводе. Кроме того, `rwcount` поддерживает многие выходные опции, упомянутые для `rwcut`: `-no-titles`, `-no-columns`, `-column-separator`, `-no-final-delimiter` и `-delimited`.

Пример 5-15. Использование разных таймслотов и опции `-skip-zero`

```
rwfilter in/2005/01/07/in-S0_20050107.01 -all=stdout |
rwcount -bin-size=1800.00 -epoch
```

Date	Records	Bytes	Packets
1105059600	257.58	42827381.72	248724.14
1105061400	1589.61	211453506.60	1438751.93
1105063200	171.81	147824612.67	1043011.93

```
$ rwfilter in/2005/01/07/in-S0_20050107.01 -all=stdout |
rwcount -bin-size=1800.00
      -epoch -start-epoch=1105057800
```

Date	Records	Bytes	Packets
1105057800	0.00	0.00	0.00
1105059600	257.58	42827381.72	248724.14
1105061400	1589.61	211453506.60	1438751.93
1105063200	171.81	147824612.67	1043011.93

```
$ rwfilter in/2005/01/07/in-S0_20050107.01 -all=stdout |
rwcount -bin-size=1800.00
      -epoch -start-epoch=1105056000
```

Date	Records	Bytes	Packets
1105056000	0.00	0.00	0.00
1105057800	0.00	0.00	0.00
1105059600	257.58	42827381.72	248724.14
1105061400	1589.61	211453506.60	1438751.93
1105063200	171.81	147824612.67	1043011.93

```
$ rwfilter in/2005/01/07/in-S0_20050107.01 -all=stdout |
rwcount -bin-size=1800.00
      -epoch -start-epoch=1105056000 -skip zero
```

Date	Records	Bytes	Packets
1105059600	257.58	42827381.72	248724.14
1105061400	1589.61	211453506.60	1438751.93
1105063200	171.81	147824612.67	1043011.93

RWSET и IP SETS

IP Sets (наборы IP) являются самой мощной возможностью SiLK и чем-то, что отличает инструментарий от большинства других аналитических инструментов. Набор IP является двоичным представлением произвольного набора IP-адресов. Наборы IP могут быть созданы из текстовых файлов, из данных SiLK или при помощи других двоичных структур SiLK.

Самый легкий способ запуститься с наборах IP состоит в том, чтобы создать один, как в примере 5-16.

Пример 5-16. Создание наборов IP с `rwset`

```
$ rwfilter in/2005/01/07/in-S0_20050107.01 -all=stdout |
  rwset -sip-file=sip.set -dip-file=dip.set
$ ls-l *.set
```

```

-rw-r--r- 1 mcollins staff 580 10 Jan 10 01:06 dip.set
-rw-r--r- 1 mcollins staff 15088 10 Jan 10 01:06 sip.set
$ rwsetcat sip.set |head-5
0.0.0.0
32.16.40.178
32.24.41.181
32.24.215.49
32.30.13.177
$ rwfileinfo sip.set
sip.set:
  format (id)      FT_IPSET (0x1d)
  version          16
  byte-order       littleEndian
  compression(id)  none(0)
  header-length    76
  record-length    1
  record-version   2
  silk-version     2.1.0
  count-records    15012
  file-size        15088
  command-lines
1  rwset -sip-file=sip.set -dip-file=dip.set

```



rwset берет записи потока и производит до четырех выходных файлов. Файл, определенный при помощи `-sip-file`, будет содержать исходные IP-адреса из потока, `-dip-file` – адреса назначения, `-any-file` будет содержать исходные и целевые IP-адреса, `-nhip-file` – адреса следующего транзитного участка. Вывод является двоичным и читается с `rwset cat`, как и все файлы SiLK, файл может быть исследован с помощью `rwfileinfo`.

Сила наборов IP увеличивается, когда они объединены с `rwfilter`. `rwfilter` имеет восемь команд, которые принимают наборы IP (`-sipset`, `-dipset`, `-nhipset`, `-anyset` и их отрицание). Наборы явно разработаны, и, таким образом, `rwfilter` может быстро запросить их использование, позволяя множество полезных запросов, как видно в примере 5-17.

Пример 5-17. Управление набором и ответом

```

$ # Сначала мы создаем IP; я использую апорт=123 (NTP на UDP) для
$ # отфильтровывания к разумному набору адресов. Клиенты NTP и серверы
$ # используют один и тот же порт.
$ rwfilter in/2005/01/07/in-S0_20050107.01 - pass=stdout - апорт=123 | rwset -sip-file=sip.set
-dip-file=dip.set
$ # Теперь давайте посмотрим, сколько создано IP-адресов
$ rwsetcat -count-ip sip.set
15
$ # Генерируем вывод с помощью rwfilter; отметьте использование файла
$ # -dipset как sip set; это означает, что я теперь ищу сообщения, которые
$ # ответили на эти адреса. Это значит, что я видел ntp, идущие в и от
$ # адреса, что означает вероятность законного пользователя, в отличие от
$ # скана на порте 123.
$ rwfilter out/2005/01/07/out-S0_20050107.01 - dipset=sip.set - pass=stdout
-апорт=123 | rwcut |head-5

```

flags	sIP	dIP sPort dPort pro	packets	bytes	\
	sTime	dur	eTime sen		

```

128.3.23.152| 56.7.90.229| 123| 123| 17| 1| 76| \
| 2005/01/07T01:10:00.520| 0.083|2005/01/07T01:10:00.603| ?|
128.3.23.152 | 192.41.221.11| 123| 123| 17| 1| 76| \
| 2005/01/07T01:10:15.519| 0.000|2005/01/07T01:10:15.519| ?|
128.3.23.231 | 87.221.134.185| 123| 123| 17| 1| 76| \
| 2005/01/07T01:24:46.251 | 0.005|2005/01/07T01:24:46.256| ?|
128.3.26.152 | 58.243.214.183| 123|10123| 17| 1| 76| \
| 2005/01/07T01:27:08.854| 0.000|2005/01/07T01:27:08.854| ?|
$ # давайте посмотрим на статистику; используя тот же файл, я смотрю на хосты,
$ # которые ответили
$ gwfilter out/2005/01/07/out-S0_20050107.01 -dipset=sip.set -aport=123
-print-stat
Files 1. Read 12393. Pass 21. Fail 12372.
$ # Теперь я смотрю на всех остальных;-not-dipset означает, что я смотрю на все
$ # на порте 123, что не идет в эти адреса.
$ gwfilter out/2005/01/07/out-S0_20050107.01 -not-dipset=sip.set -aport=123
-print-stat
Files 1. Read 12393. Pass 337. Fail 12056.

```

Наборы могут быть также сгенерированы вручную с помощью `gwsetbuild`, который берет текст на входе и производит набор файла как вывод; `gwsetbuild-спецификация` берет любую из спецификаций IP-адреса, используемых опцией `-saddress` в `gwfilter`: литеральные адреса, целые числа располагаются в точечных четверках и сетевых масках. Пример 5-18 демонстрирует это.

Пример 5-18. Создание набора с помощью `gwsetbuild`

```

$ cat > setsample.txt
Комментарии в файлах набора снабжены предисловием со знаком "диз"
Литеральный адрес
255.230.1.1
Обратите внимание на то, что я помещаю адреса в некоторый
полупроизвольный порядок; вывод будет заказан.
111.2.3-4.1-2
# Сетевая маска
22.11.1.128/30
^D
$ gwsetbuild setsample.txt setsample.set
$ gwsetcat -print-ip setsample.set 22.11.1.128
22.11.1.129
22.11.1.130
22.11.1.132
111.2.3.1
111.2.3.2
111.2.4.1
111.2.4.2
255.230.1.1

```



Наборами можно также управлять с помощью команды `gwsettool`, которая обеспечивает множество механизмов для добавления и удаления наборов; `gwsettool` поддерживает четыре манипуляции:

- union (объединение)

Создает набор, который включает любой адрес, появляющийся в любом из наборов.

- intersect (пересечься)

Создает набор, который включает только адреса, появляющиеся во всех определенных наборах.

-difference (различие)

Удаляет адреса в последних наборах из первого набора.

-sample (образец)

Случайным образом выбирает набор для создания подмножества.

rwsettool обычно вызывается с помощью выходного пути (-output=_file_), но если ничто не будет определено, это выведет к stdout. Как с rfilter, rwsettool вывод является двоичным, таким образом, чистый дамп терминала иницирует ошибку. [Пример 5-19](#) отображает управление с rwsettool.

Пример 5-19. Управление набором с rwsettool



```
$ rm setsample2.set
$ cat > setsample2.txt
Создайте набор, который покрывает наш исходный файл setsample, чтобы
увидеть, что происходит с различными функциями
22.11.1.128/29
$ rwsetbuild setsample2.txt setsample.set
$ rwsettool -union setsample.set setsample2.set | rwsetcat
22.11.1.128
22.11.1.129
22.11.1.130
22.11.1.131
22.11.1.132
22.11.1.133
22.11.1.134
22.11.1.135
111.2.3.1
111.2.3.2
111.2.4.1
111.2.4.2
255.230.1.1
$ rwsettool -intersect setsample.set setsample2.set | rwsetcat
22.11.1.128
22.11.1.129
22.11.1.130
22.11.1.131
$ rwsettool -difference setsample.set setsample2.set | rwsetcat
111.2.3.1
111.2.3.2
111.2.4.1
111.2.4.2
255.230.1.1
```



RWUNIQ

rwuniq – это нож для подсчета инструментов. Она позволяет аналитику определять ключ, содержащий одно или несколько полей, и затем будет считать много различных значений, включая общее количество байтов, пакетов, записей потока или уникальных IP-адресов, соответствующих ключу.

По умолчанию конфигурация rwuniq считает количество потоков, которые произошли для конкретного ключа. Сам ключ должен быть определен с помощью

опции `-field`, которая принимает спецификаторы поля в табл. 5-1; `gwuniq` может принять многократные поля, и ключ будет сгенерирован в порядке, определенном в командной строке. Пример 5-20 демонстрирует главные особенности опции `-field`. Как видно, порядок поля в опции влияет на упорядочивание поля на выходе.

Пример 5-20. Различные спецификаторы поля, использующие `gwuniq`

```
$ rwfilter out/2005/01/07/out-S0_20050107.01 -all=stdout |
  gwuniq -field=sip,proto | head-4
      sip|proto| Records|
131.243.142.85| 17|      1|
131.243.141.187| 17|      6|
128.3.23.41| 17|      4|

$ rwfilter out/2005/01/07/out-S0_20050107.01 -all=stdout |
  gwuniq -field=1,2 | head-4
      sip|      dIP| Records|
128.3.174.158| 128.3.23.44|      2|
128.3.191.1|239.255.255.253|      8|
128.3.161.98|131.243.163.206 |      1|

$ rwfilter out/2005/01/07/out-S0_20050107.01 -all=stdout |
  gwuniq -field=sip,sport | head-4
      sip|sport|      Records|
131.243.63.143|53504|      1|
131.243.219.52|61506|      1|
131.243.163.206| 1032|      1|

$ rwfilter out/2005/01/07/out-S0_20050107.01 -all=stdout |
  gwuniq -field=sport, sip | head-4
sport|      sip|      Records|
55876|      131.243.61.70|      1|
51864|      131.243.103.106|      1|
50955|      131.243.103.13|      1|
```

Кроме того, обратите внимание, что когда заказы полей изменены, порядок, в котором записи выведены, также изменится; `gwuniq` не гарантирует запись, заказанную по умолчанию; сортировка может быть заказана при помощи опции `-sort-output`.

`gwuniq` обеспечивает много переключателей количества, которые дают ему команду считать дополнительные значения (см. пример 5-21). Переключатели подсчета: `-bytes`, `-packets`, `-flows`, `-sip-distinct` и `-dip-distinct`. Каждое из этих полей может использоваться самостоятельно или путем определения порога (например, `-bytes`, `-bytes=10` или `-bytes=10-100`). Порог единственного значения (`-bytes=10`) обеспечивает минимум, в то время как порог с двумя значениями (`-bytes=10-100`) предоставляет диапазону минимум и максимум. Если параметр не определен, то переключатель возвращает все значения.

Пример 5-21. Спецификация поля с `gwuniq`

```
$ rwfilter out/2005/01/07/out-S0_20050107.01 -all=stdout |
  gwuniq -field=sport,sip -bytes -packets | head-5
sport|      sip| Bytes| Packets|
55876|      131.243.61.70| 308|      4|
51864|      131.243.103.106| 308|      4|
50955|      131.243.103.13| 308|      4|
56568|      128.3.212.145| 360|      5|
```

```
$ rwfilter out/2005/01/07/out-S0_20050107.01 -all=stdout |
rwuniq      -field=sport,sip      -bytes      -packets=8 | head -5
sPort|      sIP|                  Bytes|      Packets|
0|          131.243.30.224|        2520|      30|
959|         128.3.215.60|         876|      19|
2315| 131.243.124.237|          608|       8|
56838| 131.243.61.187|          616|       8|
$ rwfilter out/2005/01/07/out-S0_20050107.01 -all=stdout |
rwuniq      -field=sport,sip      -bytes      -packets=8-20 | head -5
sPort|      sIP|                  -Bytes|      Packets|
959|         128.3.215.60|         876|      19|
2315| 131.243.124.237|          608|       8|
56838| 131.243.61.187|          616|       8|
514|          128.3.97.166|        2233|      20|
```

RWBAG

Последний набор инструментов для обсуждения в этой главе – *bag tools*. *Bag* является формой структуры устройства хранения данных. Они содержат ключ (который может быть IP-адресом, портом, протоколом или интерфейсным индексом) и количество значений для этого ключа. Bag (упаковки) могут быть созданы с нуля или из данных потока с помощью команды `gwbag` (см. пример 5-22).

Пример 5-22. Вызов `gwbag`, создание сумки IP-адреса

```
$rwfilter out/2005/01/07/out-S0_20050107.01 -all=stdout |
  gwbag -sip-bytes=sip_bytes.bag
$gwbagcat sip_bytes.bag | head -5
128.3.2.16|          10026403|
128.3.2.46|           27946|
128.3.2.96|          218605|
128.3.2.98|            636|
128.3.2.102|          1568|
```

Как и наборы, упаковки являются двоичной структурой второго порядка для SiLK, это означает, что у них есть свой собственный инструментарий (`gwbagcat`, `gwbagtool` и `gwbagbuild`), данные являются двоичными (таким образом, это не может быть считано с помощью `cat` или текстового редактора), и они могут быть получены из данных потока или созданы из файла данных.

Основным инструментом генерации упаковки является `gwbag`, который, как замечено в примере 5-22, берет данные потока и производит из них файл упаковки; `gwbag` может генерировать 27 типов упаковок одновременно, если угодно. Эти 27 типов включают три типа подсчета (`bytes`, `packets`, `flows`) и девять типов ключа (`sip`, `dip`, `sport`, `dport`, `proto`, `sensor`, `input`, `output`, `nhip`). Объедините ключ и тип подсчета, и получится переключатель, который создаст сумку. Например, для подсчета всех пакетов из источника и целевых IP-адресов наберите `gwbag -sip-packets=b1.bag -dip-packets=b2.bag`.

Усовершенствованные средства SiLK

В этом разделе мы обсуждаем более усовершенствованные средства SiLK, в частности использование PMAPs, а также сбора и преобразования данных SiLK.

pmaps

Карта префикса SiLK (PMAP) является двоичным файлом, который связывает определенные подсети (префиксы) с тегами. PMAPs используются для записи различных отображений сети, таких как принадлежит ли сеть конкретной организации, или ASN, или поиску по коду страны. Используя источник, такой как [GeoIP](#), можно создать PMAP, которая связывает IP-адреса с их страной происхождения.

Комплект инструментов SiLK ожидает некоторых основных PMAPs:

`address_types.pmap`

Описывает тип адреса, традиционно указывая, расположен ли адрес внутри или снаружи сети, которую вы контролируете. Определите местоположение файловой системы по умолчанию для этой PMAP, используя переменную окружения `SILK_ADDRESS_TYPES`.

`country_codes.pmap`

Эта PMAP описывает код страны для адреса. Определите местоположение по умолчанию для этой PMAP, используя переменную окружения `SILK_COUNTRY_CODES`.

PMAPs как файлы набора могут быть созданы из текста. Пример 5-23 показывает простой файл PMAP. Отметьте следующие атрибуты:

- набор меток в начале. PMAPs не хранят строку, но перечисляемые типы идентифицированы целым числом. Это перечисление определено с помощью меток. Вы видите, что PMAP в примере 5-23, например, хранит 3 для маркировки обычного трафика;
- ключ по умолчанию. Любому значению, которое не соответствует одному из сетевых блоков, перечисленных в карте, дают значение по умолчанию;
- фактические объявления. Каждое объявление состоит из сетевой спецификации, такой как `192.168.0.0/16`, сопровождаемой меткой.

Пример 5-23. PMAP на входе

Это простой файл PMAP, который отслеживает часть стандартного RFC 1918 зарезервированных адресов

```
#
Сначала мы создаем некоторые метки
label 0 1918-reserved
label 1 multicast
label 2 future
label 3 normal
```

Определяем режим; это должно быть или `ip`, или первичным портом; `ip` в этом случае относится к адресам `v4`

```
#
режим ip
#
```

Все неуказанное имеет значение `normal`

`normal` – значение по умолчанию

Теперь распределение

```
192.168.0.0/16      1918- reserved
10.0.0.0/8          1918- reserved
```

172.16.0.0/12	1918- reserved
224.0.0.0/4	многоадресная передача
240.0.0.0/4	будущее



Как только вы создали текст представления РМАР, можно скомпилировать двоичный файл РМАР с помощью команды `gwrmapbuild`; `gwrmapbuild` имеет два обязательных аргумента: входное имя файла, с файлом в текстовом формате, описанным выше, и имя для выходного файла. Как с большинством команд SiLK, `gwrmapbuild` не перезапишет существующий выходной файл. Например:

```
$ gwrmapbuild-i reserve.txt-o reserve.pmap
$ ls-l reserve.*
-rw-r--r-- 1 mcollins staff 406 27 may 17:16 reserve.pmap
-rw-r--r-- 1 mcollins staff 526 27 may 17:00 reserve.txt
```

Как только файл РМАР создан, он может быть добавлен к `rwfilter` и `rwcut`, используя аргумент `pmap-file`. Определение использования файла РМАР эффективно создает новый набор полей в фильтре и командах сокращения; так как файлы РМАР явно связаны с IP-адресами, эти новые поля привязаны к IP-адресам.

Рассмотрим пример 5-24, который использует `rwcut`. В этом примере аргумент `-pmap-file` разграничен двоеточием; значение перед двоеточием (`reserve` в примере) является меткой, а значение после него – имя файла; `rwcut` связывает термин `reserve` с `pmap` для источника и целевого IP-адреса, создавая два новых поля: `src-reserve` (для отображения исходного адреса РМАР) и `dst-reserve` (для отображения адреса назначения РМАР).

Пример 5-24. Создание `src-резервных` и `dst-резервных` полей

```
$ rwcut -pmap-file=reserve:reserve.pmap -fields=1-4,src-reserve,dst-reserve
traceroute.rwf | head-5
      sIP|          dIP|sPort|dPort|          src-reserve|          dst-reserve|
192.168.1.12| 192.168.1.1|65428| 53| 1918-reserved| 1918-reserved|
192.168.1.12| 192.168.1.1|56126| 53| 1918-reserved| 1918-reserved|
192.168.1.12| 192.168.1.1|52055| 53| 1918-reserved| 1918-reserved|
192.168.1.1| 192.168.1.12| 53| 56126| 1918-reserved| 1918-reserved|

$ # Using the pmap in filter; note, that rwcut is not using pmap
$ rwfilter -pmap-file=reserve:reserve.pmap -pass=stdout traceroute.rwf
- pmap-src-reserve=1918-reserved | rwcut -field=1-5
  | head-5)
sIP | dIP|sPort|dPort|pro|
192.168.1.12|192.168.1.1|65428|53|17|
192.168.1.12|192.168.1.1|56126|53|17|
192.168.1.12|192.168.1.1|52055|53|17|
192.168.1.1|192.168.1.12|53|56126|17|
```



СБОР ДАННЫХ SiLK

Существует много различных инструментов для сбора данных и продвижения его в SiLK. Главными выступают YAF, который является коллектором потока, и `gwptoflow` и `rwcut`, которые преобразовывают другие данные в формат SiLK.

YAF

Yet Another Flowmeter (YAF) является эталонной реализацией для стандарта IETF IPFIX и стандартным программным обеспечением захвата пакетов потока для инструментария SiLK. YAF может считать pcap данные из файлов или получить пакеты непосредственно, которые он затем собирает в записи потока и экспортирует на диск. Имеется онлайн-документация <http://bit.ly/yaf-docu>. Сам инструмент может быть полностью конфигурирован при помощи опций командной строки, но число опций является довольно пугающим. В самом простом случае команда YAF похожа на это:

```
$ sudo yaf -i en1 --live=pcap -out /tmp/yaf/yaf
```

Он считывает данные из интерфейса `en1` и сохраняет в файле во временном каталоге. Дополнительные опции контролируют, как считываются данные и как они преобразуются в потоке обработки и в выходной формат.

Вывод `yaf` определен через переключатель `-out` в тандеме с переключателями `-ipfix` и `-rotate`. По умолчанию `-out` выводит к файлу; в примере выше этот файл `/tmp/yaf/yaf`, но любое допустимое имя файла тоже сработает (если `-out` будет установлен в `to-`, то `yaf` выведет к `stdout`).

Когда `-out` определен с `-rotate`, `yaf` записывает выходные данные в файлы, которые обновляются с задержкой, определенной параметром `-rotate` (например, `-rotate 3600` будет обновлять файлы каждый час). В этом режиме `yaf` использует имя, определенное `-out` как основное имя файла, и присоединяет префикс, определенный в формате `YYYYMMDDhhmmss`, наряду с десятичным порядковым номером и расширением файла `.yaf`.

Когда `yaf` определен с переключателем `-ipfix`, он передает данные IPFIX демону, расположенному в произвольном месте в сети. В этом случае (самая сложная опция) `-ipfix` берет транспортный протокол в качестве параметра, в то время как `-out` берет IP-адрес хоста. Дополнительно параметр `ipfix-port` берет номер порта, если необходимо. Для получения более подробной информации обращайтесь к документации.

Самые важные опции:

`-live`

Определяет тип считанных данных; возможные форматы значений `-pcap`, `dag` или `paratech`. `dag` и `paratech` – обращаются к собственным системам захвата пакетов, поэтому если вы пока не имеете этих аппаратных средств, просто установите `-live to pcap`.

`-filter`

Применяет фильтр BPF к данным pcap.

`-out`

Выходной спецификатор, проанализированный выше. Выходной спецификатор будет файлом, префиксом файла или IP-адресом – в зависимости от того, что используют другие параметры.

`-ipfix`

Берет транспортный протокол (`tcp`, `udp`, `sctp`, или `spread`) как параметр и определяет, что выводит IPFIX, транспортируемый по сети. Консультируйтесь с `yaf`-документацией для получения дополнительной информации.

-ipfix-port

Используется, только если `-ipfix` определен. Определяет порт, в который отправлены данные IPFIX.

-rotate

Используется только с файлами. Если присутствует, то имя файла `in -out` используется в качестве префикса, а файлы пишутся с меткой времени, приложенной к ним. Параметр `-rotate` принимает значение аргумента и количество секунд до перехода к новому файлу.

-silk

Указывает выходные данные, которые могут быть проанализированы средствами Rflowpack SiLK.

-idle-timeout

Задаёт время ожидания простоя для потоков в секундах. Если поток присутствует в кеше потоков и неактивен, он сбрасывается, если он был неактивен в течение всего времени простоя. По умолчанию используется значение 300 секунд (пять минут).

-active-timeout

Определяет активный тайм-аут для потоков; активный тайм-аут является максимальным количеством времени, в течение которого активный поток хранится в кеше, прежде чем будет сброшен. Значение по умолчанию 30 минут (1800 секунд). Обратите внимание на то, что активный тайм-аут определяет максимум продолжительности наблюдаемых потоков.

YAF имеет еще много опций, но это основные для рассмотрения при конфигурировании потоков. Консультируйтесь со страницей справочника YAF для получения дополнительной информации.

Поваренная книга: YAF

YAF имеет тонну опций, и описание, как они работают вместе, может немного сбить с толку. Вот некоторые примеры вызовов YAF:

Считать `yaf` из интерфейса (`en1`) и записать в файл на диске:

```
$sudo yaf -i en1 -live=pcap-o/tmp/yaf/yaf
```

Обновлять файлы каждые пять минут:

```
$sudo yaf -i en1 --rotate 300 --live=pcap-o/tmp/yaf/yaf
```

Считать файл с диска и преобразовать его:

```
$yaf < example.pcap > yafout
```

Запустить BPF-фильтр на данных, в этом случае только для данных TCP:

```
$ sudo yaf -i en1 --rotate 300 --live=pcap-o/tmp/yaf/yaf -фильтр="tcp"
```

Экспортировать данные YAF по IPFIX для обращения 128.2.14.11:3059:

```
$ sudo yaf -live pcap -in eth1 -out 128.2.14.11 -ipfix-port=3059  
-ipfix tcp
```

rwptoflow

SiLK использует свои собственные компактные двоичные форматы для представления данных NetFlow, так что такие инструменты, как `gwcut` и `gwcount`, представляют в человекочитаемой форме. Бывают случаи, когда аналитику необходимо преобразовать другие данные в формат SiLK, например захват пакетов из идентификаторов оповещения и преобразование его в формат, где IP-фильтры представлены данными.

Дежурный инструмент для этой задачи `rwptoflow`; `rwptoflow` – это инструмент преобразования пакета данных в поток. Он *не* агрегирует потоки; вместо этого каждый поток, сгенерированный `rwptoflow`, преобразован в запись потока с одним пакетом. В результате комплект SiLK сможет управлять получающимся файлом, как и любым другим файлом потока.

`rwptoflow` вызывается относительно просто: с входным именем файла в качестве параметра. В примере 5-25 данные `pcap` от маршрутизатора преобразованы в данные потока с помощью `rwptoflow`. Получающийся необработанный файл затем считывается с помощью `gwcut`, и вы видите соответствие между записями маршрутизатора и получившимися записями потока.

Пример 5-25. Преобразование данных `pcap` с помощью `rwptoflow`

```
$ tcpdump -v -n -r traceroute.pcap | head -6
reading from file traceroute.pcap, link-type EN10MB (Ethernet)
21:06:50.559146 IP (tos 0x0, ttl 255, id 8010, offset 0, flags [none],
    proto UDP (17), length 64)
    192.168.1.12.65428 > 192.168.1.1.53: 63077+ A? jaws.oscar.aol.com. (36)
21:06:50.559157 IP (tos 0x0, ttl 255, id 37467, offset 0, flags [none],
    proto UDP (17), length 86)
    192.168.1.12.56126 > 192.168.1.1.53: 30980 + PTR?
    dr._dns-sd._udp.0.1.168.192.in-addr.arpa. (58)
21:06:50.559158 IP (tos 0x0, ttl 255, id 2942, offset 0, flags [none],
    proto UDP (17), length 66),
    192.168.1.12.52055 > 192.168.1.1.53: 990+ PTR? db._dns-sd._udp.home. (38)
$ rwptoflow traceroute.pcap> traceroute.rwf
$ gwcut -num-recs=3 -fields=1-5 traceroute.rwf
sIP| dIP|sPort|dPort|proto|
192.168.1.12| 192.168.1.1|65428| 53|17|
192.168.1.12| 192.168.1.1|56126| 53|17|
192.168.1.12| 192.168.1.1|52055| 53|17|
```

rwttuc

При сопоставлении данных между разными источниками может возникнуть желание преобразовать их в формат SiLK; по умолчанию `rwttuc` является инструментом для преобразования данных в формат SiLK, поскольку он работает с колоночными текстовыми файлами. Используя `rwttuc`, можно преобразовать предупреждения IDS и другие данные в данные SiLK для дальнейших манипуляций.

Самый легкий способ вызвать `rwttuc` состоит в том, чтобы использовать его в качестве обратного к `gwcut`. Создайте файл с записями в колонки и удостоверьтесь, что заголовки соответствуют используемым `gwcut`:

```
$ cat rwttuc_sample.txt
sIP      |dIP      |proto
128.2.11.4 | 29.3.11.4 | 6
11.8.3.15  | 9.12.1.4  | 17
```



```
$ rwtuc <rwttuc_sample.txt> rwtuc_sample.rwf
$ rwcut rwtuc_sample.rwf - field=1-6
  sIP| dIP|sPort|dPort|pro|   packets|
128.2.11.4|   29.3.11.4|   0|   0|   6|           1|
11.8.3.15|    9.12.1.4|   0|   0|  17|           1|
```

Как видно из следующего фрагмента, `rwtuc` считает столбцы, использует заголовки для определения содержимого столбца и заполняет все неопределенные поля значением по умолчанию, если столбец отсутствует; `rwtuc` может также взять спецификации столбца в командной строке с помощью параметров `-fields` и `-column-separator`:

```
$cat rwtuc_sample2.txt
128.2.11.4 x 29.3.11.4 x 6 x 5
7.3.1.1 x 128.2.11.4 x 17 x 3
$ rwtuc - fields=sip, dip, proto, packets -column-sep=x <rwtuc_sample2.txt
> rwtuc_sample2.rwf
$ rwcut - fields=1-7 rwtuc_sample2.rwf
  sIP|   dIP|sPort|dPort|pro|   packets|   bytes|
128.2.11.4 |   29.3.11.4|   0|   0|   6|           5|           5|
  7.3.1.1 |   128.2.11.4|   0|   0|  17|           3|           3|
```

Двоичный формат SiLK требует значений для каждого поля, это означает, что `rwtuc` лучше всего угадывает значения полей, которых нет. Например, предыдущий пример определяет пакеты как поле, но не байты, таким образом, `rwtuc` просто определяет пакетное значение, идентичное значению байта.

Если существует общее значение по умолчанию (например, весь трафик имеет один и тот же протокол), это значение можно определить, используя один из нескольких вариантов заполнения полей в `rwtuc`. Эта опция идентична параметрам фильтрации полей в `rfilter`, за исключением того, что они принимают только отдельные значения. Например, `-proto=17` устанавливает протокол каждой записи равным 17.

Во фрагменте ниже мы используем команду заполнения поля `-bytes=300`, чтобы установить значение 300 байт для каждой записи в `rwttuc_sample2.txt`:

```
$ rwtuc -fields=sip, dip, proto, packets -column-sep=x -bytes=300 <
rwttuc_sample2.txt> rwtuc_sample2.rwf
$ rwcut -fields=1-7 rwtuc_sample2.rwf
  sIP|   dIP|sPort|dPort|pro|   packets|   bytes|
128.2.11.4|   29.3.11.4|   0|   0|   6|           5|        300|
  7.3.1.1|   128.2.11.4|   0|   0|  17|           3|        300|
```

Полученный в результате файл RWF будет содержать значение 300 байт, хотя такое значение не находится в оригинальном файле. Пакетные значения, указанные в файле, имеют значение, что бы там ни было указано.

ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ ДЛЯ ЧТЕНИЯ

1. *Тайм Шаймил, Сид Фабер, Маркус Дешон, Дрю Компанек (Time Shimeall, Sid Faber, Markus DeShon, Drew Kompanek). Использование SiLK для анализа сетевого трафика. Институт программной инженерии.*

Глава 6

.....

Введение в R для аналитиков по вопросам безопасности



R является пакетом статистического анализа с открытым исходным кодом, разработанным первоначально Ross Ihaka и Robert Gentleman из Оклендского университета. R был разработан прежде всего статистиками и аналитиками данных и связан с коммерческими статистическими пакетами, такими как S и SPSS. R является инструментарием для исследовательского анализа данных; он обеспечивает возможности статистического моделирования и манипулирования данными, визуализацию и является полнофункциональным языком программирования.

R выполняет исключительно полезную роль для анализа – он подобен ножу. Аналитическая работа требует некоторого инструмента для создания и управления небольшими специальными (оперативными) базами данных, которые суммируют необработанные данные. Например, часовые сводки объема трафика с конкретного хоста с разбивкой по сервисам. Эти таблицы более сложны, чем необработанные данные, но не предназначены для заключительной публикации – они все еще требуют дополнительного анализа. Исторически Microsoft Excel был приложением для этого типа анализа. Он обеспечивает числовой анализ, построение графика и простое колоночное представление данных, которые могут быть отфильтрованы, отсортированы и упорядочены. Я вижу, что аналитики торгуют файлами Excel, как будто бы это клочки бумаги.

Я переключился с Excel на R, потому что обнаружил, что это превосходный продукт для крупномасштабного числового анализа. Графическая природа Excel делает его неуклюжим, когда вы имеете дело с огромными объемами данных. Я нахожу возможности работы с таблицами R превосходными, они обеспечивают создание в форме сохраняемых и совместно используемых рабочих областей, мощные возможности визуализации, а наличие полнофункционального языка сценариев включает быструю автоматизацию. Большая часть того, что обсуждено в этой главе, может быть сделано в Excel, но если есть возможность потратить время для изучения R, я уверен – вы поймете, что оно потрачено не напрасно.

Первая половина этой главы сфокусирована на доступе к данным и управлении ими с использованием среды программирования R. Вторая половина внимания направлена на процесс статистического тестирования с помощью R.

УСТАНОВКА И НАСТРОЙКА

R является хорошо поддерживаемым проектом с открытым исходным кодом. Всесторонняя сеть архива R (CRAN) поддерживает текущие двоичные файлы для Windows, Mac OS X и систем Linux, хранилища пакетов R и обширную документацию.

Самый легкий способ установить R состоит в том, чтобы захватить соответствующий двоичный файл (наверху домашней страницы). R также доступен для всех основных менеджеров пакетов. Для остальной части этой главы я собираюсь предположить, что вы используете R в его собственном графическом интерфейсе.

Существует много других инструментов, доступных для работы с R, в зависимости от инструментов и сред, с которыми вам комфортно. [RStudio](#) является интегрированной средой разработки обеспечения данных, проекта и инструментов управления задачами на более традиционной платформе IDE. Для пользователей Emacs, Emacs Speaks Statistics или режим ESS обеспечивает интерактивную среду.

ОСНОВЫ ЯЗЫКА

Этот раздел является интенсивным курсом на языке R. R – богатый язык, который я едва затрону поверхностно. Однако в конце этого раздела вы будете в состоянии написать простую программу R, выполнить ее в командной строке и сохранить как библиотеку.

Подсказка R

Запуск R представит вам окно и командную строку. В качестве примера на рис. 6-1 показана консоль R. Как можно увидеть, консоль по большей части состоит из большого текстового окна и серии кнопок наверху, которые обеспечивают дополнительные функции. Отметьте еще два текстовых поля под строкой кнопок. Первые отображают текущий рабочий каталог, а вторые являются функцией помощи. R очень хорошо задокументирован, поэтому привыкайте к его использованию.

На рис. 6-1 отображено несколько простых команд, воссозданных здесь:

```
s <- 'Hi There'
x <- -3 + 11 + (3 * log(exp(2)))
print(s)
[1] "Hi There" > print(x)
[1] 20
```



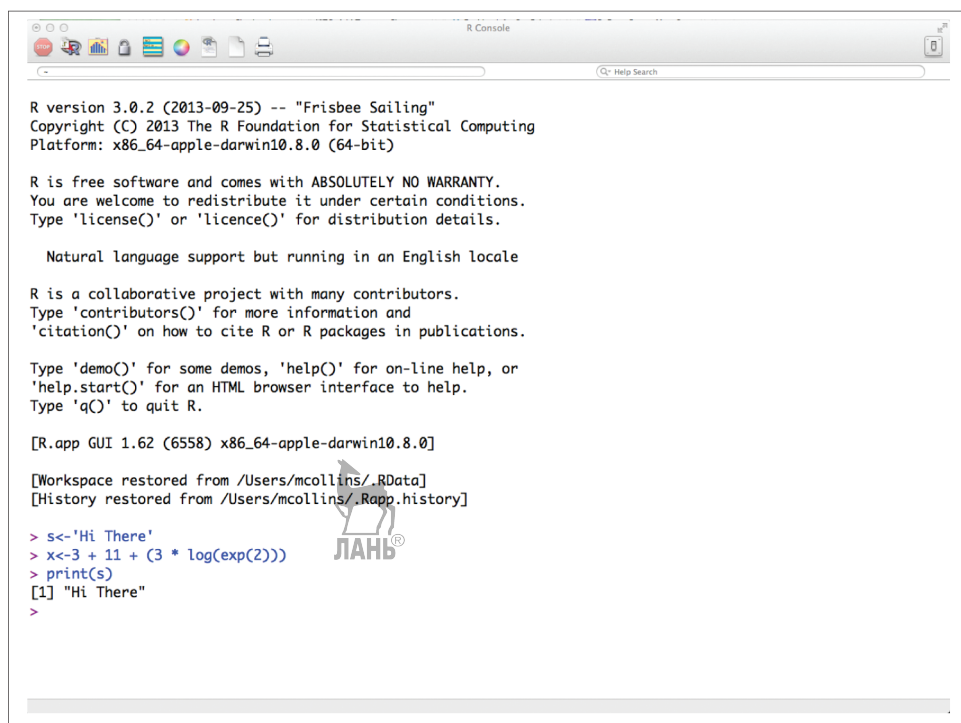


Рисунок 6-1. Консоль R

Приглашение командной строки для R – это `>`; после этого можно ввести команды вручную. Если команда будет частично завершена (например, путем открытия, но не закрытия скобки), следующая подсказка будет знаком и продолжится до закрытия.

```
> s <-3 *(
+ 5 + 11
+ 2
+ )
> s
[1] 54
```

Обратите внимание на то, что когда R возвращает значение (например, вывод `s` в предыдущем примере), то распечатывает `[1]` в квадратных скобках. Значение в скобках является индексом массива; если массив распространяется по нескольким строкам, соответствующий индекс будет распечатан в начале каждой строки.

```
> s <-seq (1,20)
> s
[1] 1 2 3 4 5 6 7 8 9 10 11 12
[13] 13 14 15 16 17 18 19 20
```

Помощь можно получить, используя `help(term)` или `?term`. Ищите справку через `help.search()` или `??`.

Для выхода из R используйте значок переключателя или соответствующую команду выхода (`Command-Q` или `Ctrl-Q`) для операционной системы. При исполь-

зовании чистой командной строки R (т. е. без графического интерфейса) можно закончить сеанс с помощью Ctrl-D или введя *q* () в подсказке.

Когда сеанс R завершается, он спрашивает, хотите ли вы сохранить рабочую область. Файлы рабочей области могут быть перезагружены после сеанса для продолжения работы, независимо от того, что сама работа выполнялась во время завершения сеанса.

R-переменные

R поддерживает много различных типов данных, включая скалярные целые числа, символьные данные (strings), булевы переменные и значения с плавающей точкой, векторы, матрицы и списки. Скалярные типы, как показано в следующем примере, могут быть присвоены с помощью операторов: $\leftarrow$ (“gets”), =, и $\rightarrow$. R перегружает некоторые сложные определения в свои операторы, для наших целей (и почти для всего R-программирования) руководства по R рекомендуют использовать оператор $\leftarrow$ вместо =.

```
># Assign some value directly
>a <-1
>b <-1.0
>c <- 'A String'
>d <-T
># Мы присвоим e d
>e <-d
>e
[1] TRUE > d
[1] TRUE
># Теперь мы переопределим d, и увидим, что d изменяется, но e остается тем же самым.
>d <-2
>d
[1] 2 > e
[1] TRUE
```

Вектор R – это упорядоченный набор одного или нескольких значений одного типа: символьное, логическое или строчное. Векторы могут быть созданы с помощью функции с или любого из ряда других функций. Векторы являются наиболее часто используемыми элементами в R: скалярные значения, которые мы использовали ранее, были технически векторами длины 1¹.

```
># пример целочисленного вектора
>int.vec <-c (1,2,3,4,5)
>int.vec
[1] 1 2 3 4 5
># Числа с плавающей точкой будут приведены к целому числу,
># или целые числа к числам с плавающей точкой при необходимости
>float.vec <-c (1,2.0,3)
>float.vec
[1] 1 2 3
```

¹ Обратите внимание на использование периодов, а не подчеркиваний; предшественники R (S и S-Plus) установили это соглашение, и хотя использование подчеркивания не является синтаксической ошибкой, большинство R-кодов будет использовать точки так же, как и другие языки используют подчеркивания.

```

>float.vec <-c (1,2.45,3)
>float.vec
>[1] 1.00 2.45 3.00
># Векторы могут также быть логическими
>logical.vec <-c (T, F, F, T)
>logical.vec
TRUE FALSE FALSE TRUE
# Они будут отнесены к целым числам, если помещены в числовой вектор
mixed.vec <-c (1,2, FALSE, TRUE)
>mixed.vec
[1] 1 2 0 1
># Векторы символа состоят из одной или нескольких строк; отметьте,
># что строка является отдельным элементом
>char.vec <-c ("One", "Two", "Three")
>char.vec
[1] "One", "Two", "Three"
> # Длина дает векторные длины
>length(int.vec)
[1] 5
># Обратите внимание на то, что длина символического вектора является
># длиной общего количества строк, а не отдельными символами
>length(char.vec)
[1] 3

```

Отметьте длину символического вектора: в R строки рассматривают как один элемент независимо от количества символов. Существуют функции для доступа к строкам - `nchar`, чтобы получить длину, и `substr` и `strsplit`, чтобы извлекать элементы из строки, но строки отдельного символа не так непосредственно доступны, как в Python.

R обеспечивает ряд функций для векторной арифметики. Вектор может быть добавлен к или умножен на другой вектор; если они будут одинаково измерены, результат будет вычислен на поэлементной основе. Если один вектор будет меньшим, он будет повторен для создания вектора равного размера. (Вектор, длина которого не является фактором другого вектора, увеличит ошибку.) Это относится к одноэлементным векторам также: добавьте единственный элемент к более длинному вектору, и он будет добавлен к каждому элементу в векторе; умножьте – и каждый элемент будет умножен.

Векторы являются индексруемыми. К отдельным элементам можно получить доступ с помощью квадратных скобок, таким образом, `v[k]` означает *k*-й элемент *v*. Векторы также поддерживают вырезание части вектора («ranged slicing»), такое как `v[a:b]`. Отрицательный индекс устранил индексированный элемент из вектора, как в следующем блоке кода:

```

># Мы запускаем путем создания вектора из двух других
>v1 <-c(1,2,3,4,5)
>v2 <-c(6,7,8,9,10)
>v3 <-c(v1, v2)
>v3
[1] 1 2 3 4 5 6 7 8 9 10
># Обратите внимание на то, что нет никакого вложения
># Основная арифметика – умножение и сложение
>2 * v1

```

```

> # We start by creating a vector out of two others
> v1 <- c(1,2,3,4,5)
> v2 <- c(6,7,8,9,10)
> v3 <- c(v1,v2)
> v3
[1] 1 2 3 4 5 6 7 8 9 10
> # Обратите внимание, что нет вложенности
> # Основная арифметика - умножение и сложение
> 2 * v1
[1] 2 4 6 8 10
> 2 * v3
[1] 2 4 6 8 10 12 14 16 18 20
> 1 + v1
[1] 2 3 4 5 6
> v1 * v2
# Multiplication
[1] 6 14 24 36 50
# Slicing a range
> v3[1:3]
[1] 1 2 3
> # This is identical to v3[1]
> v3[1:1]
[1] 1
> v3[2:4]
[1] 2 3 4
> # Reverse the range to reverse the vector
> v3[3:1]
[1] 3 2 1
> # Use negative numbers to cut out elements
> v3[-3]
[1] 1 2 4 5 6 7 8 9 10
> v3[-1:-3]
[1] 4 5 6 7 8 9 10
> # You can use logical vectors as selectors; selection returns anything where
> # the index is true
> v3[c(T,F)]
[1] 1 3 5 7 9

```



R может построить матрицы из векторов, используя матричную функцию. Как и векторы, матрицы могут быть добавлены и умножены (между собой, с векторами и другими матрицами), выбраны и нарезаны с использованием ряда различных подходов, как показано здесь:

```

> # Матрицы # созданы с помощью матричной команды, как показано в
> # канонической форме ниже. Обратите внимание на то, что столбцы заполнены сначала.
> s<-матрица (v3,nrow=2,ncol=5)
> s
      [,1] [,2] [,3] [,4] [,5]
[1,] 1    3    5    7    9
[2,] 2    4    6    8   10
> # Добавление одиночного элемента
> s + 3
      [,1] [,2] [,3] [,4] [,5]
[1,] 1 9 25 49 81

```



```

> [2,] 4 16 36 64 100
> #, Добавляем вектор; обратите внимание, что добавление идет
> # сначала через столбцы
> s + v3

      [,1] [,2] [,3] [,4] [,5]
[1,]    2    6   10   14   18
[2,]    4    8   12   16   20
> # Добавление меньшего вектора, обратите внимание, что
> # он закидывается на матрице, столбец-первый
> s + v1

      [,1] [,2] [,3] [,4] [,5]
[1,]    2    6   10    9   13
[2,]    4    8    7   11   15
> # Разрезание; использование запятой будет казаться странным
> # большинству людей.
> # Перед запятой это строки, после запятой – столбцы.
> # Результат возвращен как вектор, вот почему "столбец" теперь
> # горизонтальный
> s [,1]
[1] 1 2 > s[1,]
[1] 1 3 5 7 9
> # Доступ к одиночному элементу
> s[1,1]
[1] 1
> # Теперь я получаю доступ к 1-м и 2-м элементам столбца из
> # первой строки; снова получаем обратно вектор
> s [1,1:2]
[1] 1 3
> # Первые и вторые элементы строки из первого столбца
> s[1:2,1]
[1] 1 2
> # Теперь я возвращаю матрицу, потому что вытягиваю два вектора
> s[1:2,1:2]
      [,1] [,2] [1,] 1 3
[2,] 2 4
> # Выбор с помощью булевых переменных, первое значение является строкой, из которой я вытягиваю
> s[c(T,F)]
[1] 1 3 5 7 9
>> s[c(F,T)]
[1] 2 4 6 8 10
> # Если я определяю другой вектор, он вытащит столбцы
> s [c(F,T),c(T,F,T,T,F)]
[1] 2 6 8

```

Список R фактически сам является вектором, состоящим из векторных элементов, каждый из которых может быть составлен из своего списка команд. Списки, как и матрицы, строятся с помощью собственной специальной команды. Списки могут быть нарезаны как вектор, хотя отдельные элементы также доступны с помощью двойных скобок. Более интересно, что списки могут быть названы; отдельным векторам может быть присвоено имя, а затем к нему обеспечен доступ с помощью оператора \$.

```

# Обзор элементов из более ранних векторов
>      v3
      [1] 1 2 3 4 5 6 7 8 9 10
>      v4
[1] "Hi"      "There"     "Kids"

```



```

# Создаем список; обратите внимание, что мы можем добавить
># произвольное число элементов.
># Каждый добавленный элемент является новым индексом.
>list.a <- список(v3,v4,c('Что'),11)
># Dump the list; отметьте индексы списка в двойных скобках.
>list.a
[[1]]
 [1] 1 2 3 4 5 6 7 8 9 10
[[2]]
 [1] "Hi" "There" "Kids"
[[3]]
 [1] "What" "The"
[[4]] [1] 11
># Списки не поддерживают векторную арифметику.
>list.a + 1
Ошибка в list.a + 1: нечисловой параметр дан бинарному оператору
# Отдельные элементы могут быть исследованы через индексацию.
# Одиночные скобки возвращают список.
list.a[1]
[[1]]
 [1] 1 2 3 4 5 6 7 8 9 10
># Двойные скобки возвращают сам элемент; обратите внимание,
# что индекс списка ([[1]]) здесь не присутствует
>list.a[[1]]
[1] 1 2 3 4 5 6 7 8 9 10
># Одиночные скобки возвращают список, а двойные скобки тогда
># возвращают первый элемент в этом одноэлементном списке.
>list.a [1] [[1]]
[1] 1 2 3 4 5 6 7 8 9 10
># Доступ с помощью двойных скобок, затем одиночная скобка в
># векторе.
>list.a[[1]] [1]
[1] 1
>list.a[[2]][2] [1] "Там"
># Мы можем изменить результаты.
>list.a[[2]][2] <- 'Ничего себе',
># Теперь мы создадим именованный список.
>list.b <- list(values=v1,rant=v2,miscellany=c(1,2,3,4,5,9,10))
># Названия параметра становятся именами элементов списка, и
># параметры являются фактическими элементами списка.
>list.b
$values
[1] 1 2 3 4 5
$rant
>6 7 8 9 10
$miscellany
[1] 1 2 3 4 5 9 10
># Доступ к названным элементам происходит с помощью знака
># доллара.
>list.b$miscellany
[1] 1 2 3 4 5 9 10
># После доступа можно использовать стандартное разрезание.
>ем
>list.b$miscellany[2]
[1] 2
># Обратите внимание на то, что индекс и имя указывают на одно

```



```
># и то же значение.
>list_b[[3]]
1 2 3 4 5 9 10
```

Понимание синтаксиса списка важно для фреймов данных, которые более подробно мы обсудим позже.

Запись функций

R-функции созданы путем привязки результатов команды `function` к символу, так:

```
>add_elements <-function(a,b) + b
>add_elements(2,3)
[1] 5
> simple_math <- function(x, y, z) {
+   t <-c (x, y)
+   z * t
+ }
```



Отметьте фигурные скобки. В R фигурные скобки используются для хранения нескольких выражений, а также они возвращают финальный статус этих выражений. Фигурные скобки могут быть использованы без функции или чего-либо еще, как показано здесь:

```
>> {8 + 7
+ 9 + 2
+ c ('hi', 'there')
+ }
[1] "hi" "there"
```

Так, в `simple_math` результаты в фигурных скобках оценены последовательно и возвращен конечный результат. Конечный результат не должен иметь никакого отношения к предыдущим утверждениям в пределах данного блока. R действительно имеет оператор возврата для управления завершением и возвратом функции, но соглашение состоит в том, чтобы не использовать его, если результаты очевидны.

Как видно из примеров, аргументы функции определены в функциональном операторе. Аргументам можно присвоить значение по умолчанию при помощи `=sign`; любой параметр, которому вы присваиваете значение по умолчанию, становится дополнительным. Назначение аргументов может быть сделано в определенном порядке или явным использованием имени аргумента, как показано здесь:

```
>Создайте функцию с дополнительным аргументом > тест <-функция (x,y=10) {x + y}
>Если параметр не будет передан, то R будет использовать значение
> по умолчанию
> тест (1)
[1] 11
> # Аргументы и значения задаются позиционно
> тест (1,5)
[1] 6
> # Значение может также быть присвоено с помощью имени параметра
> тест (1,y=9)
[1] 10
> # Для всех переменных
> тест (x=3,y=3)
[1] 6
> # Имена заменяют положение
```



```
> тест (y=8,x=4)
[1] 12
> # Значение без значения по умолчанию все еще должно быть присвоено
> тест ()
Ошибка в x + y: 'x' отсутствует
```

Функциональные возможности R позволяют вам рассматривать функции как объекты, которыми можно управлять, оценивать и применять по мере необходимости. Функции могут быть переданы другим функциям как параметры и при помощи функций `apply` и `Reduce` могут использоваться для поддержки более комплексной оценки.

```
> # Создать функцию, которую назовет другая функция
> inc.func <- function(x) {x + 1}
> dual.func <- function(y) {y(2)}
> dual.func (inc.func)
[1] 3
> # R имеет ряд различных функций применения на основе типа входа
> # (матрица, список, вектор) и выходной тип.
> test.vec <- 1:20
> test.vec
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
> # Выполнение sapply на анонимной функции; обратите внимание, что
> # функция не привязана к объекту; она существует на время выполнения.
> # Я мог также легко вызвать sapply (c,inc.func) для использования
> # функции inc.func, определенной выше.
> sapply(test.vec, function(x) x+2)
[1] 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22
> # Где sapply является классической функцией карты, Reduce – классика
> # функции свернуть/уменьшить, уменьшая вектор единственного значения.
> # В этом случае переданная функция суммирует a и b, добавляя
> # целые числа от 1 до 20, вместе выдает 210
> # Отметьте, что Reduce записано с большой буквы
> Reduce(function(a,b) a+b,1:20)
[1] 210
```

Замечание о циклах в R: циклы R (особенно для цикла) являются достаточно медленными. Многие задачи, которые реализованы одним циклом в Python или C, сделаны в R с использованием многих функциональных конструкций. Фронтэнд для этого – *sapply* и *Reduce*.

Условные выражения и итерация

Основной условный оператор в R – это `if...then...else` (если ... тогда ... еще), использует `else if` для обозначения нескольких операторов. Оператор `if` является самостоятельной функцией и возвращает значение, которое может быть оценено.

```
> # Простое если/тогда, которое распечатывает строку
> if (a==b) print ("Equivalent") else print("Not Equivalent")
[1] "Not Equivalent"
> # Мы могли бы просто непосредственно вернуть значения
> if (a==b) ("Equivalent") else ("Not Equivalent")
[1] "Not Equivalent"
> # Если/тогда это функция, таким образом, мы можем включить ее в другую
> # функцию или если/тогда
> if((if (a!=b) "Equivalent" else "Not Equivalent") == \
```

```

>"Not Equivalent") print (Really "Not Equivalent")
>a<-45
># Объединяем в цепочку вместе многократные операторы if/then,
># используя else if
>if (a==5) "Equal to five" else if (a == 20) "Equal to twenty" \
> else if (a==45)"Equal to forty five" else "Odd beastie"
>[1] "Equal to forty five"
>a<-5
>if(a==5)"Equal to five" else if (a==20) "Equal to twenty" \ else if (a==45) "Equal to
forty five" else "Odd beastie"
[1] "Equal to five"
>a<-97
>if (a==5) "Equal to five" else if (a==20) "Equal to twenty" \
> else if (a==45) "Equal to forty five" else "Odd beastie"
[1] "Odd beastie"

```

R предоставляет оператор switch (переключения) как компактную альтернативу многократным if/then пунктам. Оператор переключения использует позиционные параметры для целочисленных сравнений и дополнительные параметры для текстового сравнения.

```

># Когда switch берет число в качестве своего первого параметра,
># он возвращает параметр с индексом, который соответствует этому числу,
># таким образом, следующее возвращает второй параметр "Is"
>switch (2,"This","Is","A" "Test")
[1]
>proto<-'tcp'
># Если параметры названы, эти текстовые строки используются для
># соответствия
>switch(proto,tcp=6,udp=17,icmp=1)
[1] 6
># последний параметр является параметром по умолчанию
>proto<-'unknown'
>switch(proto, tcp=6,udp=17,icmp=1,-1)
[1]-1
># Для использования switch неоднократно свяжите его с функцией
>proto <-function(x) { switch(x, tcp=6,udp=17,icmp=1)}
>proto ('tcp')
[1] 6
>proto('udp') [1] 17
>proto('icmp')
[1] 1

```

R имеет три циклические конструкции: repeat (повторить), которая обеспечивает бесконечные циклы по умолчанию; while (в то время как), которая делает условную оценку в каждом цикле; и for (для), которая выполняет итерации по вектору. Внутренними операциями цикла управляет break (который завершает цикл) и next (который пропускает посредством итерации), как показано здесь:

```

># Цикл repeat; обратите внимание, что циклы repeat работают бесконечно,
># пока в цикле не появится оператор break. Если не определить условие,
># он будет работать вечно.
>i<-0
>repeat {
+ i <- i + 1
+ print(i)

```

```

+ if(i>4) break;
++ }
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
> # Цикл while с идентичной функциональностью; этот не требует
> # оператора завершения
> i <-1
> while( i < 6) {
+ print(i)
+ i <- i + 1
+ }
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
># Цикл for является самым компактным
> s<-1:5
> for(i in s) print(i)
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5

```

Несмотря на то что R обеспечивает эти конструкции циклического выполнения, обычно лучше избегать циклов в пользу функциональных операций, таких как `sapply`. R не является языком программирования общего назначения; он был разработан для предоставления статистическим аналитикам богатого инструментария для различных операций. R содержит огромное количество оптимизированных функций и других инструментов, доступных для управления данными. Некоторые из них мы рассмотрим далее в этой главе, но хорошая ссылка по R бесценна.

ИСПОЛЬЗОВАНИЕ РАБОЧЕЙ ОБЛАСТИ R

R предоставляет пользователям постоянное рабочее пространство, означающее, что когда пользователь выходит из сеанса R, ему предоставляется возможность для сохранения данных и переменных, которые они имеют в распоряжении для будущего использования. Это сделано в основном прозрачно в виде командной строки, как показано в примере:

```

> s <-1:15
> s
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
> t <-(s*3) - 5
> t
[1] -2 1 4 7 10 13 16 19 22 25 28 31 34 37 40
>
Сохранить образ рабочей области? [у/н/с]: у $ R - по умолчанию
> s

```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
>t
>-2 1 4 7 10 13 16 19 22 25 28 31 34 37 40
```

Каждый раз, когда вы запускаете R в конкретном каталоге, он проверяет файл рабочей области (*.RData*) и загружает его содержание, если он существует. При выходе из сеанса *.RData* будет обновлен, если требуется. Он также может быть сохранен посреди сеанса с помощью команды `save.image()`. Это может быть спасительным при опробовании новых анализов или длинных командах.

Можно получить список объектов в рабочей области с помощью функции `ls`, которая возвращает вектор имен объектов. Они могут быть удалены с помощью функции `rm`. Объекты в рабочей области могут быть сохранены и загружены с использованием функций `save` и `load`. Они берут список объектов и имя файла как параметр и автоматически загружают результаты в среду.

```
># Давайте создадим некоторые простые объекты
><-1:20
>t <- rnorm(50,10,5)
># Ls будет нам показан
>ls()
[1] "a" "t"
# Теперь мы сохраняем их
save(a,t,file='simple_data')
# мы удаляем их и смотрим
rm(a,t)
ls()
character(0)
>load('simple_data')
>ls()
[1] "a" "t"
```

Если у вас есть простой сценарий R, который вы хотите загрузить, используйте команду `source` для загрузки файла. Команда `sink` перенаправит вывод в файл.

ФРЕЙМЫ ДАННЫХ

Фреймы данных являются структурой, уникальной для R и, возможно, самой важной с точки зрения аналитика. Фрейм данных является оперативной таблицей данных: табличная структура, где каждый столбец представляет единственную переменную. В других языках фреймы данных реализованы частично при помощи массивов или хеш-таблиц, но R включает фреймы данных как базовую структуру и предоставляет средства для выбора, фильтрации и управления содержанием фрейма данных намного более сложным способом с самого начала.

Давайте начнем с создания простого фрейма данных, что можно увидеть в примере 6-1. Самый легкий способ создать фрейм данных состоит в использовании `data.frame` на множестве векторов тождественного размера.

Пример 6-1. Создание фрейма данных

```
>names<-c('Manny','Moe','Jack')
>ages<-c(25,35,90)
>states<-c('NJ','NE','NJ')
>summary.data <-data.frame(names, ages, states)
```

```
> summary.data
  names, ages, states
1  Manny  25  NJ
2   Moe   35  NE
3   Jack  90  NJ
> summary.data$names
[1] Manny Moe Jack
Levels: Jack Manny Moe
```

Здесь `data.frame` превратил каждый массив в столбец для формирования таблицы из трех столбцов и трех строк. Мы можем извлечь нужную колонку. Обратите внимание на использование термина «Levels» при обращении к вектору имен, на которые ссылается `summary.data$names`.

Факторы

В процессе составления таблицы R преобразовал строки данных в факторы, которые являются вектором категорий. Факторы могут быть созданы из строк или целых чисел, например:

```
> services<-c("http","bittorrent","smtp","http","http","bittorrent")
> service.factors <-factor(services)
> service.factors
[1] http      bittorrent smtp      http      http      bittorrent
Levels: bittorrent http smtp
> services
[1] "http" "bittorrent" "smtp" "http" "http" "bittorrent"
```

Уровни фактора описывают отдельные категории фактора.

Поведение R по умолчанию во многих функциях должно преобразовать строки в факторы. Это сделано в `read.table` и `data.frame` и управляется через параметр `stringsAsFactors`, так же как и через опцию `stringsAsFactors`.

Команда для доступа к кадрам данных – `read.table`, имеет большое разнообразие параметров для чтения различных типов данных. В примере 6-2 параметры передаются для чтения `rawcut` во входном файле `sample.txt`.

Пример 6-2. Передача опций `read.table`

```
$ cat sample.txt | cut-d '|' -f 1-4
  sIP|      dIP| sPort| dPort|
10.0.0.1| 10.0.0.2| 56968|   80|
10.0.0.1| 10.0.0.2| 56969|   80|
10.0.0.3|...
$ R -silent
> s<-read.table(file='sample.txt',header=T,sep='|',strip.white=T)
> s
```

	sIP	dIP	sPort	dPort	pro	packets	bytes	flags
1	10.0.0.1	10.0.0.2	56968	80	6	4	172	FS PA
2	10.0.0.1	10.0.0.2	56969	80	6	5	402	FS PA
3	10.0.0.3	56690	80	6	5	1247	FS FS PA	
	65.164.242.247							
4	10.0.0.4	99.248.195.24	62904	19380	6	1	407	F PA
5	10.0.0.3	216.73.87.152	56691	80	6	7	868	FS PA
6	10.0.0.3	216.73.87.152	56692	80	6	5	760	FS PA

```

7      10.0.0.5      138.87.124.42      2871      2304      6      7      603      F      PA
8      10.0.0.3      216.73.87.152      56694      80      6      5      750      FS      PA
9      10.0.0.1      72.32.153.176      56970      80      6      6      918      FS      PA
                                sTime      dur      eTime      sen      X
1      2008/03/31T      0      2008/03/31T      0      NA
    18:01:03.030      18:01:03.030

2      2008/03/31T18:01:03.040      0      2008/03/31T18:01:03.040      0      NA
3      2008/03/31T18:01:03.120      0      2008/03/31T18:01:03.120      0      NA
4      2008/03/31T18:01:03.160      0      2008/03/31T18:01:03.160      0      NA
5      2008/03/31T18:01:03.220      0      2008/03/31T18:01:03.220      0      NA
6      2008/03/31T18:01:03.220      0      2008/03/31T18:01:03.220      0      NA
7      2008/03/31T18:01:03.380      0      2008/03/31T18:01:03.380      0      NA
8      2008/03/31T18:01:03.430      0      2008/03/31T18:01:03.430      0      NA
9      2008/03/31T18:01:03.500      0      2008/03/31T18:01:03.500      0      NA

```

Обратите внимание на используемые аргументы; файл не требует пояснений. Параметр `header` (заголовок) дает R команду рассматривать первую строку файла как названия столбцов в получающемся кадре данных; `sep` определяет разделитель столбцов, в этом случае значение по умолчанию – `|`, используемый командами `SiLK`. Команда `strip.white` дает R команду отделять избыточный пробел от файла. Конечный результат состоит в том, что каждое значение считано и автоматически преобразовано в колоночный формат.

Теперь, когда у меня есть данные, я могу их отфильтровать и управлять ими, как показано в примере 6-3.

Пример 6-3. Управление данными и их фильтрация

```

> # Я могу отфильтровать записи созданием из них булевых векторов, например:
> s$dPort == 80
[1] TRUE TRUE TRUE FALSE TRUE TRUE FALSE TRUE TRUE
# Затем я могу использовать это значение для отфильтровывания строк, где
# s$dPort == 80
# Обратите внимание на запятую. Если бы я не использовал ее, то выбрал бы
# столбцы вместо строк.
> s[s$dPort==80,]

```

```

      sIP      dIP      sPort      dPort      pro      packets      bytes      flags
1      10.0.0.1      10.0.0.2      56968      80      6      4      172      FS      A
2      10.0.0.1      10.0.0.2      56969      80      6      5      402      PA      FS
3      10.0.0.3      65.164.242.247      56690      80      6      5      1247      PA      FS
5      10.0.0.3      216.73.87.152      56691      80      6      7      868      PA      FS
6      10.0.0.3      216.73.87.152      56692      80      6      5      760      PA      FS
8      10.0.0.3      216.73.87.152      56694      80      6      5      750      PA      FS
9      10.0.0.1      72.32.153.176      56970      80      6      6      918      PA      FS
                                sTime      dur      eTime      sen      X
1      2008/03/31T18:01:03.030      0      2008/03/31T18:01:03.030      0      NA
2      2008/03/31T18:01:03.040      0      2008/03/31T18:01:03.040      0      NA
3      2008/03/31T18:01:03.120      0      2008/03/31T18:01:03.120      0      NA
5      2008/03/31T18:01:03.220      0      2008/03/31T18:01:03.220      0      NA
6      2008/03/31T18:01:03.220      0      2008/03/31T18:01:03.220      0      NA
8      2008/03/31T18:01:03.430      0      2008/03/31T18:01:03.430      0      NA

```



```

9 2008/03/31T18:01:03.500    0 2008/03/31T18:01:03.500    0    NA
># Я могу также объединить правила использования | для или и/для и
>s[s$dPort==80 & s$sIP=='10.0.0.3']

      sIP      dIP    sPort dPort    pro packets    bytes    flags
3  10.0.0.3 65.164.242.247 56690    80     6         5   1247    FS    PA
5  10.0.0.3 216.73.87.152 56691    80     6         7    868    FS    PA
6  10.0.0.3 216.73.87.152 56692    80     6         5    760    FS    PA
8  10.0.0.3 216.73.87.152 56694    80     6         5    750    FS    PA

      sTime dur      eTime    sen    X
3 2008/03/31T18:01:03.120    0 2008/03/31T18:01:03.120    0    NA
5 2008/03/31T18:01:03.220    0 2008/03/31T18:01:03.220    0    NA
6 2008/03/31T18:01:03.220    0 2008/03/31T18:01:03.220    0    NA
8 2008/03/31T18:01:03.430    0 2008/03/31T18:01:03.430    0    NA

># Я могу получить доступ к столбцам с помощью их имен
>s[s$dPort==80 & s$sIP=='10.0.0.3'],[c('sIP','dIP','sTime')]
  sIP dIP sTime
3 10.0.0.3 65.164.242.247 2008/03/31T18:01:03.120
5 10.0.0.3 216.73.87.152 2008/03/31T18:01:03.220
6 10.0.0.3 216.73.87.152 2008/03/31T18:01:03.220
8 10.0.0.3 216.73.87.152 2008/03/31T18:01:03.430
># И я могу получить доступ к единственной строке
>s [1],
  sIP dIP sPort dPort    pro packets    bytes flags sTime
>1 10.0.0.1 10.0.0.2 56968    80     6       4172 FS A2008/03/31T18:01:03.030
dur      eTime    sen    X
>1 0 2008/03/31T18:01:03.030    0 NA

```

Фреймы данных R предоставляют нам то, что фактически является специальной базой данных с одной таблицей. Помимо выбора строк и столбцов, показанных в предыдущих примерах, мы можем добавить новые столбцы с помощью оператора \$.

```

># Создаем новый вектор байтов полезной нагрузки
>payload_bytes <- s$bytes - (40 * s$packets)
>s$payload_bytes <- payload_bytes
>s[0:2],[c('sIP','dIP','bytes','packets','payload_bytes')]
  sIP      dIP      bytes packets payload_bytes
10.0.0.1 10.0.0.2    172         4         12
10.0.0.1 10.0.0.2   402         5         02

```

Визуализация

R обеспечивает чрезвычайно мощные возможности визуализации в виде коробочных решений, также много стандартных визуализаций доступно как высокоуровневые команды. В следующем примере мы произведем гистограмму с помощью образца от нормального распределения и затем графически изобразим результаты на экране.

В главе 10 обсуждаются различные методы визуализации. В этом разделе мы фокусируемся на различных функциях визуализации R, включая управление изображениями, их сохранение и управление ими.

Команды визуализации

R имеет много высокоуровневых команд визуализации для графического изображения временного ряда, гистограмм и столбчатых диаграмм. Команда `plot` – это рабочая лошадка комплекта, которая может использоваться для обеспечения ряда графиков, полученных из точечных диаграмм: простой график поля точек, шаги ступеньки и ряд. Основные имена графика перечислены в табл. 6-1 и описаны командой `help`.

Таблица 6-1. Высокоуровневые команды визуализации

Команда	Описание
<code>barplot</code>	Столбчатая диаграмма
<code>boxplot</code>	Коробчатая диаграмма (или диаграмма размаха)
<code>hist</code>	Гистограмма
<code>pairs</code>	Парный график
<code>plot</code>	График поля точек и связанные графики
<code>qqnorm</code>	График QQ

Параметры визуализации

Существует два главных механизма для управления параметрами визуализации. Во-первых, почти все команды визуализации предлагают стандартный комплект опций как параметры. Главные опции показаны в табл. 6-2, а результаты их визуализации – в сопутствующем изображении, рис. 6-2.

Таблица 6-2. Общие опции визуализации

Опция	Параметр	Описание
<code>axes</code>	Булева переменная	Если правда, добавляют оси
<code>log</code>	Булева переменная	Если правда, графики в логарифмическом масштабе
<code>main</code>	Символ	Основной заголовок
<code>sub</code>	Символ	Подзаголовок для графика
<code>type</code>	Символ	Управляет типом изображенного графика
<code>xlab</code>	Символ	Метка для оси x
<code>ylab</code>	Символ	Метка для оси y

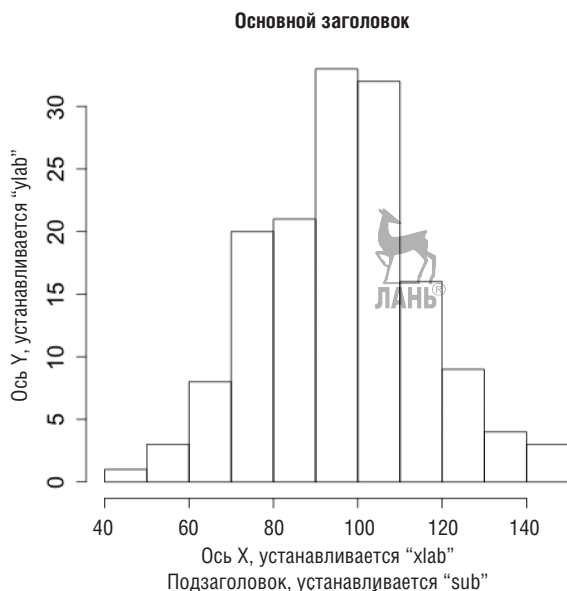


Рисунок 6-2. Визуализация опций

Опциями визуализации также управляют с помощью функции `par1` (постоянных изменений), которая обеспечивает огромное количество специальных опций для размера оси управления, типов точки, выбора шрифта и т. п.; `par` берет огромное количество опций, о которых можно прочитать через `help(par)`. В табл. 6-3 отображены некоторые наиболее важные.

```
># Мы собираемся использовать par для отображения матрицы с 3
># столбцами и 2 строками, затем заполняем 3 ячейки матрицы с различными графиками
># с помощью других par-величин
>par(mfcol=c(2,3))
># Рисуем гистограмму по умолчанию
>hist(sample_rnorm,main='Sample Histogram')
># Теперь мы перемещаемся во 2-ю строку, центральный столбец
>par(mfg=c(2,2,2,3))
># Изменяем размер осей до половины значения по умолчанию
>par(sex.axis=0.5)
># Делаем оси синими
>par(col.axis='blue')
># Делаем сам график красным
>par(col='red')
># Теперь мы графически изображаем разброс значений
>plot(sample_rnorm,main='Sample scatter')
># После того как мы графически изобразили это, оно автоматически
># переместится в 3-ю строку, 1-й столбец
># Восстановление размера оси
>par(sex.axis=1.0)
># Изменяем тип точки для scatterplot. Используем help(points) для
># получения списка чисел для PCH
```

¹ Функция `par()` используется для доступа и изменения списка графических параметров для текущего графического устройства.

```
> par(pch=24)
> plot(sample_rnorm, main='Sample Scatter with New Points')
```

Таблица 6-3. Полезные параметры паритета

Имя	Ввести	Описание
mfcol	2-integer (row, col) vector	Разбивает поле рисования на строки и столбцы, превращая в набор из ячеек
mfg	4-integer (row, col, nrows, ncols) vector	Назначает определенную ячейку в mfcol для рисования
сех ¹	Floating point	Устанавливает размер шрифта, по умолчанию – 1, таким образом определение сех=0,5 означает, что все размеры являются теперь половиной от первоначального
col	Character b ²	Цвет
lty	Number or character	Тип строки
pch	Number	Тип точки

¹ сех и col имеют много дочерних параметров: .axis, .main, .lab, и .sub, – которые влияют на соответствующий элемент; сех.main является относительным размером шрифта для заголовка, например.

² Цветные строки могут быть строкой типа red (красный) или шестнадцатеричной строкой RGB в виде #RRGGBB.

Аннотация технологии визуализации

При рисовании визуализаций я обычно предпочитаю иметь в наличии некоторую модель или аннотацию для сравнения. Например, если я сравниваю визуализацию с нормальным распределением, я хочу, чтобы соответствующее нормальное распределение на экране сравнило его с результатами гистограммы.

R обеспечивает ряд функций поддержки для рисования текста на графике. Они включают lines, points, abline, polygon и text. В отличие от высокоуровневых функций графика, их пишут непосредственно на экран, не сбрасывая изображение. В этом разделе мы покажем, как использовать lines и text для аннотирования изображения.

Мы начнем путем генерации гистограммы для общего сценария: сканирование трафика плюс трафик типичного пользователя на /22 (1024 хоста) сети. Наблюдаемый параметр является количеством хостов, и мы предполагаем, что при нормальных обстоятельствах это значение обычно является нормально распределенным средним из 280 хостов и стандартным отклонением 30. Одно из каждых 10 событий произойдет во время сканирования. Во время сканирования количество наблюдаемых хостов всегда 1024, поскольку сканер достигает всех хостов в сети.

```
> # Сначала мы моделируем типичное действие с помощью распределения
> # Гаусса через rnorm
> normal_activity <- rnorm(300, 280, 30)
> # Затем мы создаем вектор атак, где каждая атака – это 1024 хоста
> attack_activity <- rep(1024, 30)
> # Мы объединяем их вместе; поскольку мы фокусируемся на количестве
> # хостов и независимости от времени, мы не заботимся об упорядочивании
> activity_vector <- c(normal_activity, attack_activity)
> hist(activity_vector, breaks=50, xlab='Hosts observed', ylab='Probability of Occurrence',
  prob=T, main='Simulated Scan Activity')
```

Обратите внимание на параметры breaks и prob в гистограмме; breaks управляет количеством корзин в гистограмме, которая особенно важна, когда вы имеете

дело с длиннохвостым распределением, как эта модель; `rgob` строит гистограмму как плотность, а не как частоту значений.

Построим теперь соответствие заданной кривой. Для этого мы создаем вектор `x` и вектор значений `y` для функции `lines`. Значения `x` являются равномерно раз- деленными точками в диапазоне, охватываемом нашим эмпирическим распреде- лением, в то время как значения `y` получаются с помощью функции `dnorm`:

```
> xpoints<-seq(min(activity_vector),max(activity_vector),length=50)
> # Использование dnorm для вычисления соответствующих значений y,
> # учитывая канал значений x (xpoints) и модель нормального распределения
> # используя среднее и sd (ухудшение качества) от вектора действия.
> # Значение будет плохим> # fit, так как атака искажает трафик.
> ypoints<-dnorm(xpoints,mean=mean(activity_vector),sd=sd(activity_vector))
> # Графически изображаем гистограмму, которая очищает экран
> hist(activity_vector,breaks=50,xlab='Hosts observed',\
  ylab='Density',prob=T,main='Simulated Scan Activity')
> # Проводим подходящую линию с помощью lines
> lines(xpoints,ypoints,lwd=2)
> # Рисуем текст. Значение x и y получено на графике.
> text(550,0.010, "This is an example of a fit")
```

Экспорт визуализации

R-визуализации выводятся на *устройства*, которые можно вызвать при помощи ряда различных функций. Устройством по умолчанию является X11 в системах Unix, `quartz` для Mac OS X и `win.graph` для Windows. Помощь R для **устройств** (в зависимости от ситуации) предоставляет список того, что доступно на текущей платформе.

Для печати вывода R откройте файл вывода (например, `png`, `jpeg` или `pdf`) и затем запишите команды как обычно. Результаты будут записаны в файл устройства, пока вы не деактивируете его с помощью `dev.off()`. На данном этапе необходимо назвать устройство по умолчанию снова без параметров.

```
> # Вывод гистограммы к файлу 'histogram.png'
> png (file='histogram.png')
> hist(rnorm(200,50,20))
> dev.off()
> quartz()
```

АНАЛИЗ: ПРОВЕРКА СТАТИСТИЧЕСКИХ ГИПОТЕЗ

R разработан, чтобы обеспечить статистического аналитика разнообразием инстру- ментов для исследования данных. Функции программирования, обсуждаемые до сих пор в этой главе, являются средством достижения данной цели. Важными функ- циями, для которых мы хотим использовать R, являются возможности поддержки конструкции предупреждений путем идентификации статистически значимых функций (см. главу 7 для обсуждения сигналов опасности).

Определение атрибутов, полезных для сигналов тревоги, требует определения «важного» поведения, для различных степеней важности. R обеспечивает огром- ный комплект инструментов для исследования и статистического тестирования данных. Обучение использованию этих инструментов требует понимания общей тестовой статистики, которую дают инструменты R. Остаток этой главы фокуси- руется на данных задачах.

Проверка гипотез

Статистическая проверка гипотезы является процессом оценки утверждения о поведении мира, основанном на определенном наборе данных. Требование могло бы состоять в том, что данные нормально распределены или что нападения на нашу сеть происходят в течение утра. Проверка гипотез начинается с гипотезы, которую можно сравнить с моделью, а затем объявляется недействительной. Язык тестирования гипотезы часто является парадоксальным, потому что полагается на ключевое свойство наук – наука не может доказать утверждение, она может опровергнуть это утверждение или, альтернативно, не опровергнуть его. Следовательно, тесты гипотезы фокусируются на «отклонении от нулевой гипотезы».

Статистическое тестирование начинается с требования, называемого *нулевой гипотезой* (H_0). Самая основная нулевая гипотеза – это что нет никакого отношения между переменными в наборе данных. *Альтернативная гипотеза* (H_1) утверждает противоположность нулевой – что существует доказательство отношения. Нулевая гипотеза протестирована путем сравнения вероятности данных, сгенерированных моделированием процесса нулевой под предположениями, сделанными нулевой.

Например, предположим, что процесс тестирования монеты определяет, является ли она равномерно взвешенной или же взвешена пользу одной из сторон. Мы тестируем монету путем ее неоднократного подбрасывания. Нулевая гипотеза утверждает, что вероятность приземления орлом равна вероятности приземления решкой: $P = 0.5$. Альтернативная гипотеза утверждает, что взвешивание смещено к одной стороне.

Чтобы определить, взвешена ли монета, мы должны подбросить ее многократно. Вопрос в построении теста – это сколько раз мы должны подбросить монету для создания этого определения. Рисунок 6-3 показывает снижение вероятностей для комбинаций подбрасывания монеты на одно к четырем подбрасываниям¹.

1 Flip

Head 0.5	Tails 0.5
-------------	--------------

2 Flips

HH 0.25	TH 0.5	TT 0.25
------------	-----------	------------

3 Flips

HHH 0.125	HHT 0.375	TTH 0.375	TTT 0.125
--------------	--------------	--------------	--------------

4 Flips

4H 0.062	3HT 0.25	2H2T 0.375	3TH 0.25	4T 0.062
-------------	-------------	---------------	-------------	-------------

Рисунок 6-3. Модель подбрасывания для равномерно взвешенной монеты

¹ Комбинации не упорядочены, таким образом, при вычислении вероятностей получение решки, затем орла считается эквивалентным получению орла, затем решки.

Результаты следуют за биномиальным распределением, которое мы можем вычислить, используя функцию R `dbinom`.

```
> # Используем dbinom для получения вероятностей от 0 до 4 орлов,
> # учитывая, что имеется 4 подбрасывания монеты и вероятность выпадения
> # орла в отдельном подбрасывании = 0.5
> dbinom ((0:4),4,p=0.5)
[1] 0.0625 0.2500 0.3750 0.2500 0.0625
> # Результаты в таком порядке – 0 орлов, 1 орел, 2 орла, 3 орла, 4 орла
```

Чтобы определить, является ли результат значительным, мы должны определить вероятность результата, полученного случайно. В статистическом тестировании это сделано при помощи *p-value* (*p-значения*). Р-значение является вероятностью того, что *если нулевая гипотеза верна*, вы получите результат как экстремальный случай к наблюдаемым результатам. Чем меньше р-значение, тем меньше вероятность того, что наблюдаемый результат мог иметь место при нулевой гипотезе. Условно нулевая гипотеза отвергается, когда р-значение ниже 0,05.

Для понимания концепции конечности рассмотрим неуспешный биномиальный тест на одно к четырем подбрасываниям монеты. В R:

```
> binom.test(0,4,p=0.5)
Точный биномиальный тест
данные: 0 и 4
количество успехов = 0, количество испытаний = 4, р-значение = 0,125
альтернативная гипотеза: истинная вероятность успеха не равна 0,5
95%-ный доверительный интервал:
 0.0000000 0.6023646
демонстрационная оценка:
вероятность успеха
0
```

Это р-значение = 0,125 является суммой вероятностей, что подбрасывание монеты завершилось четырьмя орлами (0.0625) и четырьмя решками (также 0.0625); р-значение, в этом контексте «две решки», означает, что оба значения считаются экстремальными. Точно так же, если мы считаем для одного орла:

```
> binom.test(1,4, p=0.5)
Точный биномиальный тест
данные: 1 и 4
количество успехов = 1, количество испытаний = 4, р-значение = 0,625
альтернативной гипотезы: истинная вероятность успеха не равна 0,5
95%-ный доверительный интервал:
 0.006309463 0,805879550
демонстрационная оценка:
вероятность успеха
0.25
```



Р-значение 0.625, сумма 0,0625 + 0.25 + 0.25 + 0.0625 (все, кроме вероятности 2 орлов и 2 решек).

Тестирование данных

Один из наиболее распространенных тестов, который можно сделать с R, – это протестировать, соответствует ли конкретный набор данных распределению. Для получения информации об обнаружении опасности и аномалии, имея данные, которые следуют за распределением, нам нужно определить пороги для сигналов тревоги. Однако мы редко на самом деле встречаемся с данными, которые могут быть смоделированы с распределением, как обсуждено в главе 10. В предположении, что явление может быть удовлетворительно смоделировано распределением, можно использовать функции характеристик распределения для предсказания значений.

Классическим примером этого процесса оценки является использование среднего и стандартного отклонения для предсказания значений нормально распределенного явления. Нормальное распределение имеет функцию плотности вероятности формы:

$$\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(\chi-\mu)^2}{2\sigma^2}},$$

где μ является средним, а σ – стандартным отклонением модели.

Если трафик может быть удовлетворительно смоделирован распределением, он предоставляет нам математический инструментарий для оценки вероятности возникновения события. Шанс встретить удовлетворительную модель, как обсуждается в разделе 10, редок – как правило, только после сильной фильтрации данных и применения многократной эвристики можно извлечь что-то полезное.

Это имеет значение потому, что при использовании математики для модели, не зная, работает ли модель, вы рискуете создать неисправный сенсор. R обеспечивает огромное количество различных статистических тестов, чтобы определить, можно ли использовать модель. Ради краткости данный текст фокусируется на двух тестах, которые обеспечивают основной инструментарий. Это: *Shapiro-Wilk* (*shapiro.test*)

Тест Шапиро–Уилка – это хороший тест на соответствие нормальному распределению. Используйте его, чтобы проверить, является ли образец нормальным распределением.

Kolmogorov-Smirnov (*ks.test*)

Хороший тест соответствия для использования в отношении непрерывных распределений, таких как нормальное или равномерное.

Все эти тесты работают аналогичным образом: тестовая функция выполняется на демонстрационном наборе и другом демонстрационном наборе (представленном явно или через вызов функции). Тестовая статистика описывает качество сгенерированного соответствия и произведенное p -значение.

Тест (или критерий) Shapiro-Wilk (*shapiro.test*) является тестом нормальности и используется для проверки гипотезы H_0 : «случайная величина X распределена нормально» – и является одним из наиболее эффективных критериев проверки нормальности. Посмотрите пример 6-4 запуска теста.



Пример 6-4. Запуск теста Шапиро–Уилка (тест Шапиро)

```
> # Это хороший тест на соответствие нормальному распределению. Его применение позволяет
    проверить, нормально ли распределен набор данных.
> # функция, тест shapiro
> shapiro.test(rnorm(100,100,120))
```

Тест нормальности Шапиро–Уилка

```
данные: rnorm(100, 100, 120)
W = 0.9863, p-значение = 0.3892
> # Мы объясним эти числа через мгновение
> # Тест, чтобы увидеть, является ли функция нормально распределенной, тест shapiro
> shapiro.test(runif(100,100,120))
```

Критерий нормальности функций Шапиро–Уилка

```
данные: runif(100, 100, 120)
W = 0.9682, p-значение = 0.01605
```

Все статистические тесты дают тестовую статистику (W в тесте Шапиро–Уилка), которая сравнивается с распределением по нулевой гипотезе. Точное значение и интерпретация статистической величины специфичны для теста, и p -значение должно использоваться в качестве нормализованной интерпретации значения.

Тест Колмогорова–Смирнова (Kolmogorov-Smirnov) (*ks.test*) является простым тестом совершенства подбора распределения, который используется, чтобы определить, соответствует ли наш набор данных конкретному непрерывному распределению, такому как нормальное или равномерное распределение. Он может использоваться либо с функцией (в этом случае набор данных сравнивается с функцией), либо с двумя наборами данных (в этом случае они сравниваются друг с другом). Посмотрите тест в действии в примере 6-5.

Пример 6-5. Использование теста KS

```
> # KS тестируют в действии; давайте создадим два случайных равномерных
    > # распределения
> a.set <- runif(n=100, min=10, max=20)
> b.set <- runif(n=100, min=10, max=20)
> ks.test(a.set, b.set)
```

Тест Kolmogorov-Smirnov с двумя образцами

```
данные: a.set и b.set
D = 0.07, p-значение = 0,9671
альтернативная гипотеза: двухсторонний
> # Теперь мы сравним набор с распределением с помощью функции.
> # Обратите внимание на то, что я использую runif, чтобы получить
> # распределение и передать в тех же параметрах, как если бы я вызывал
> # punif самостоятельно
> ks.test(a.set, punif, min=10, max=20)
```

Один образец, тест Kolmogorov-Smirnov

```
данные: a.set
D = 0.0862, p-значение = 0,447
альтернативная гипотеза: двухсторонний
> # Мне нужна оценка перед использованием теста.
> # Для равномерного я могу использовать мин. и макс., как я использовал
> # бы средний и sd для нормального
> ks.test(a.set, punif, min=min(a.set), max=max(a.set))
```

Один образец, тест Kolmogorov-Smirnov



```

данные: a.set
D = 0.0829, p-значение = 0,4984
альтернативная гипотеза: двухсторонний
># Теперь тот, где я отклоняю нулевой; я буду рассматривать данные, как
># будто это нормальное распределение, и оценю снова
>ks.test(a.set,pnorm,mean=mean(a.set),sd=sd(a.set))

Один образец, тест Kolmogorov-Smirnov

данные: a.set
D = 0.0909, p-значение = 0,3806
альтернативная гипотеза: двухсторонний
># Hmm, p-значение высоко... Поскольку я не использую достаточно образцов,
># давайте сделаем это снова с 400 образцами каждый.
>a.set <-runif (400, min=10, max=20)
>b.set <-runif (400, min=10, max=20)
># Сравнение друг с другом
>ks.test (a.set, b.set)$p.value
[1] 0.6993742
># Сравнение с распределением
>ks.test(a.set,punif,min=min(a.set),max=max(a.set))$p.value
[1] 0.5499412
># Сравнение с различным распределением
>ks.test(a.set,pnorm,средний = средний(a.set),sd=sd (a.set))$p.value
[1] 0.001640407

```

Тест KS относится к слабым тестам. Сила эксперимента относится к его способности верно отклонить нулевую гипотезу. Слабые тесты требуют большего числа образцов, чем более мощные тесты. Объем выборки, особенно при работе с данными безопасности, является сложной проблемой. Большинство статистических тестов прибывает из экспериментальных лабораторий, где получение даже 60 образцов может быть чем-то вроде достижения. В то время как для анализа сетевого трафика возможно собрать огромное число образцов, тесты начнут вести себя неустойчиво с огромным объемом данных; маленькие отклонения от нормальности приведут к небольшим отклонениям от нормальности, но вы всегда можете вбросить больше данных, чтобы достигнуть нужного результата.

В моем опыте тесты распределения обычно служат для лучшей визуализации данных. В главе 10 обсудим это более глубоко.

ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ ДЛЯ ЧТЕНИЯ

1. *Patrick Burns*. The R Inferno.
2. *Richard Cotton*. Learning R: A Step-by-Step Function Guide to Data Analysis (O'Reilly, 2013).
3. *Russell Langley*. Practical Statistics Simply Explained (Dover, 2012).
4. The R Project. An Introduction to R.
5. *Larry Wasserman*. All of Statistics: A Concise Course in Statistical Inference (Springer Texts in Statistics, 2004).

Глава 7

.....

Классификация и инструменты события: IDS, AV и SEM



Эта глава посвящена разработке и использованию датчиков на основе событий, таких как вторжение системы обнаружения (IDSes). Эти системы включают пассивные датчики, такие как IDSes и большинство AVs, а также активные системы, такие как брандмауэры. Аналитически они все ведут себя так же – анализируют данные и генерируют *события* в ответ на эти данные. Создание события – это то, что отличает IDS от простого датчика создания отчетов, такого как NetFlow. Простые датчики сообщают обо всем, что они наблюдают, в то время как IDS или датчик другого класса настроен так, чтобы сообщать только об определенных пороговых событиях, поднимая тревогу всякий раз, когда достигается пороговое значение.

Множество аналитических процессов в конечном итоге приведет к некоторой форме идентификаторов. Например, вам потребуется разработать систему обнаружения abuse (нежелательной) активности на хосте. Используя некоторые разделы математики из части III, вы создадите модель так о нежелательной активности, зададите пороговые значения и сможете поднимать тревогу всякий раз, когда достигается порог. Проблема состоит в том, что эти процессы почти никогда не работают по назначению. Операционные системы IDS *очень трудно* реализовать правильно. *Не* является проблемой обнаружение; проблемой являются определение контекста и атрибуция (соотнесение), т. е. определение места, времени и прочих параметров события. Системы IDS легко сконфигурировать и использовать. Но они производят столько предупреждений, что аналитики игнорируют их, или они сконфигурированы для создания столь немногих предупреждений, что могут что-то пропустить. Разработка эффективных оповещений требует понимания того, как ids используются оперативно, почему они отказывают в качестве классификаторов и как это повлияет на аналитику.

Эта глава разделена на две части. Первая разделяет системы IDS и то, как они используются в реальности. Здесь обсуждается, как системы IDS перестают работать и как разные виды отказа влияют на анализ. Вторая часть фокусируется на конструкции лучших систем обнаружения и обсуждает повышение эффективности подписи и разные методы обнаружения аномалий.

КАК РАБОТАЕТ IDS

Все IDSes являются экспертными системами типа, называемого *бинарным классификатором*. Классификатор считывает данные и отмечает их как одну из двух

категорий. Или данные нормальны и не требуют никаких дальнейших действий, или данные говорят об атаке. Если это атака, то система реагирует определенно; датчики события генерируют событие, контроллеры блокируют трафик и т. д.

Система IDS интерпретирует данные по-другому, чем пассивные датчики, такие как NetFlow. Простой датчик оповещает относительно всего, что он контролирует, в то время как IDS сообщает только относительно событий, которые сконфигурировал на основе созданных отчетов. IDSes отличаются в зависимости от данных, которые они используют для интерпретации, и процесса, который они применяют для принятия конкретного решения.

Существует несколько проблем с классификацией, которые мы можем определить как *моральная*, *статистическая* и *поведенческая*. Моральная проблема состоит в том, что нападения могут быть неотличимы от безвредного или даже разрешенного пользовательского действия. Например, DDos-атака и флеш-толпа¹ могут выглядеть очень похожими. Статистическая проблема состоит в том, что системы IDS часто конфигурируются для создания сотен или миллионов тестов в день – при этих условиях даже низкие ложные положительные значения могут привести к намного более ложным положительным через день, чем истинные положительные значения через месяц. Поведенческая проблема заключается в том, что злоумышленники – интеллектуальная сторона, и они заинтересованы в уклонении от обнаружения и часто могут сделать это с минимальными затратами и ущербом их целям.

В этом разделе обсудим IDS и получим очень пессимистическое представление о его возможностях. Начнем с обсуждения словаря обнаружения проникновений, затем перейдем на механику бинарных классификаторов, а потом к проблеме технических систем обнаружения и влияния отказов классификаторов.

Базовый словарь

Мы можем разбить IDS вдоль двух основных осей: куда IDS помещен и как IDS принимает решения. На первой оси идентификаторы разбиваются на сетевые идентификаторы (NIDS) и хост-идентификаторы (HIDS). На второй оси идентификаторы разделяются между системами на основе сигнатур и системами, основанными на аномалиях.

NIDS эффективнее любой IDS, которая начинается с рсар данных. В домене с открытым исходным кодом это включает такие системы, как [Snort](#), Bro и Suricata. Системы NIDS работают при ограничениях, обсужденных для сетевых датчиков в главе 2, например необходимость получения трафика через зеркальное отображение портов или прямое подключение к сети в случае невозможности чтения зашифрованных данных.

HIDSes работают в домене хоста и обычно намного более вариативны, чем NIDSes. HIDS может контролировать сетевую активность, физический доступ (такой как пытается ли пользователь применять USB-устройство) и информацию от операционной системы, такую как нарушения ACL или порядок доступа к файлам.

Рисунок 7-1 показывает, как несколько общих систем IDS ложатся вдоль этих осей.

¹ «Flash Crowd» является устоявшимся выражением для очень быстрой передачи информации в социуме и возникающих вследствие этого беспорядках и панике. – Прим. ред.

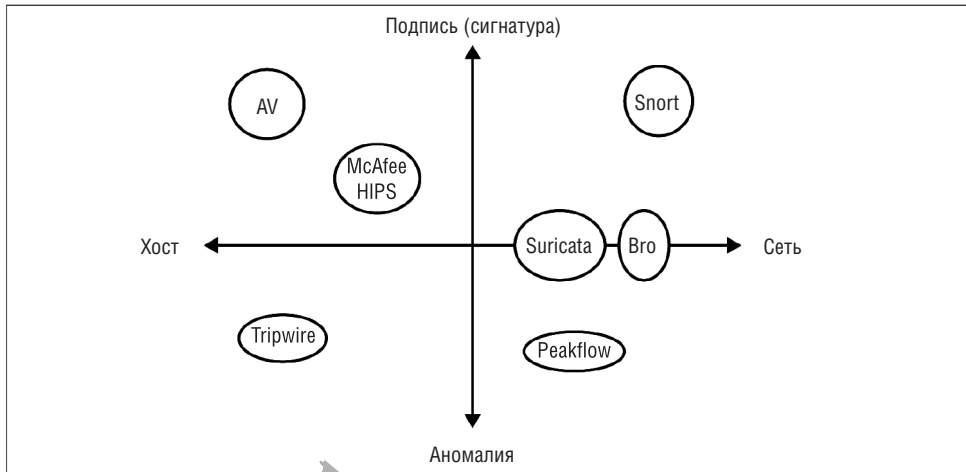


Рисунок 7-1. Отказ общего IDS

Рисунок 7-1 показывает семь примеров различного IDS. Это:

Snort

Наиболее часто используемый IDS. **Snort** является основанной на сети системой соответствия подписи, которая применяет *индивидуальные сигнатуры Snort* для идентификации вредоносного трафика. **Snort** обеспечивает обширный язык для описания сигнатур и может быть вручную сконфигурирован для добавления новых.

Bro

Сложная система анализа трафика, которая может использоваться для обнаружения проникновения с помощью сигнатур и аномалий. Bro является меньшей из IDS как язык анализа трафика. Bro была недавно перепроектирована для работы с кластерами.

Suricata

Экспериментальная IDS с открытым исходным кодом, разработанная Open Information Security Foundation с финансированием Министерства национальной безопасности (США). Suricata является самой молодой IDS из перечисленных здесь и используется для экспериментов с новыми методами обнаружения проникновений.

Peakflow

Коммерческий пакет анализа трафика, разработанный Arbor Networks. Peakflow анализирует трафик NetFlow, чтобы идентифицировать и сгладить атаки, такие как DDoS.

Tripwire

Система мониторинга целостности файлов. **Tripwire** контролирует определенные каталоги и генерирует события, когда она видит изменения содержания каталога.

AV

Антивирусные системы, такие как Symantec, ClamAV или McAfee, являются наиболее распространенными формами, основанными на подписи HIDS. Системы AV исследуют диск хоста и память на двоичные подписи вредоносного

программного обеспечения и выдают предупреждения при обнаружении подозрительных двоичных файлов.

McAfee HIPS

Предотвращение проникновения хоста McAfee (HIPS) является одним из нескольких коммерческих IPS-пакетов. HIPS-системы, такие как эта, объединяют двоичный анализ с анализом журнала для исследования нарушений ACL или подозрительных модификаций файла.

Подавляющее большинство IDSes основано на *сигнатурах (подписи)*. Основанная на сигнатурах система использует ряд правил, которые получены независимо от цели, для идентификации зловреда. Например, подпись Snort, записанная по правилам языка Snort, могла быть похожей на это:

```
alert tcp 192.4.1.0/24 any -> $HOME_NET 22 (flow:to_server,established;\ content: "root";)
```

Это предупреждение выдается, когда трафик от подозрительной сети (192.4.1.0/24) пытается проникнуть на любой хост на внутренней сети и войти в систему как root SSH. HIDS может предложить те сигнатуры, которые «выдают предупреждение, когда пользователь пытается удалить журнал безопасности». Создание и управление набором правил является важной проблемой для идентификаторов на основе сигнатур, а хорошо продуманные правила часто являются секретным соусом, который отличает различные коммерческие пакеты.

Основанный на подписи IDS только тогда выдаст предупреждения, когда он будет иметь определенные правила, чтобы сделать это. Это ограничение означает, что основанные на подписи IDSes обычно имеют высокий *ложный отрицательный* уровень, означающий, что большое количество атак идет без сообщений о них. Самая экстремальная версия этой проблемы связана с уязвимостями. AV прежде всего, но также и NIDS, и HIDS полагаются на определенные двоичные подписи для идентификации вредоносного программного обеспечения (см. раздел «Красный код и уклонение от вредоносного ПО» для более широкого обсуждения этой проблемы). Данные подписи требуют, чтобы у эксперта был доступ к эксплойту¹, в эти дни их обычно называют «нулевой день», что означает, что они освобождаются и некоторое время функционируют самостоятельно, прежде чем кто-либо получит возможность добавить сигнатуру.

Основанные на аномалии IDSes созданы обучением (или дополнительной конфигурированием) IDS на основе данных трафика для создания модели нормального действия. Как только эта модель создана, отклонения от модели становятся аномальными, подозрительными и генерируют события. Например, простой основанный на аномалии NIDS мог бы контролировать трафик к определенным хостам и генерировать событие, когда внезапно происходит скачок трафика, указывая на DDoS или другое подозрительное событие.

Основанные на аномалии IDSes используются намного меньше, чем основанный на подписи IDS, прежде всего потому, что у них есть противоположная проблема основанного на подписи IDS – высокий *ложный положительный* уровень. Основанные на аномалии IDSes известны постоянным созданием предупреждений, которые часто сокращают, чтобы произвести минимальное количество предупреждений, а не выключать его постоянно.

¹ Эксплоит (англ. exploit, эксплуатировать) – компьютерная программа, фрагмент программного кода или последовательность команд, использующие уязвимости в программном обеспечении и применяемые для проведения атаки на вычислительную систему. – *Прим. ред.*

которых было достаточно аналитикам, чтобы проверить все нотификации систем, исследовать и выработать план действий. Современные нападения в основном автоматизированы, и фактическая подрывная деятельность и перехват управлением хоста могут произойти мгновенно, если выполнены необходимые условия.

Проблема управления цифровой подписью стала значительно сложнее в прошлом десятилетии, потому что атакующим легко изменить полезную нагрузку, не изменяя функциональности вируса. При исследовании баз данных угроз, таких как Symantec (см. раздел 8), вы увидите, что существуют более сотни вариантов обыкновенных вирусов, каждый из них с различной двоичной подписью.

Что касается вирусов отказа в обслуживании, таких как Slammer¹, они в основном подавлены, поэтому я лучше всего опишу эволюционные причины. Во многом как и физический вирус, который не убивает своего хозяина, пока не начнет распространяться, пока не имел шанса на это, современные сетевые вирусы обычно ограничены в своем воспроизведении. Лучше владеть интернетом, чем уничтожить его.



Частота отказов классификатора: понимание ошибки базовой ставки

Все системы IDS являются прикладными приложениями, работающими с *классификацией*, являющейся стандартной проблемой в искусственном интеллекте и статистике. Классификатор – процесс, который берет на входе данные и распределяет их в одну, по крайней мере, из двух категорий. В случае систем IDS категории – обычно «атака» и «нормальный».

Основанные на сигнатуре и аномалиях системы IDS представляют нападения существенно различными способами, и это влияет на тип ошибок, которые они могут сделать. Основанный на сигнатуре IDS калиброван для поиска определенных странностей в поведении, таких как вредоносные сигнатуры или необычные попытки входа в систему. Основанные на аномалии IDSes обучаются на нормальном поведении и затем ищут что-либо, что не соответствует норме. Основанные на подписи IDSes имеют высокие ложноотрицательные уровни, что означает, что они пропускают много нападений. Основанные на аномалии IDSes имеют высокие ложноположительные уровни, это означает, что они считают большое число совершенно нормальных действий атакой.

IDSes являются обычно *бинарными* классификаторами, означая, что они делят данные на две категории. Бинарные классификаторы имеют два вида отказа:

Ложноположительные

Также называют *ошибкой типа I*, это происходит, когда что-то, что не имеет свойства, которое вы ищете, классифицируется как наличие свойства. Пример подобного отказа: когда электронная почта от президента вашей компании, сообщающая вам о продвижении, классифицирована как спам.

Ложноотрицательные

Также называют *ошибкой типа II*, это происходит, когда что-то, что имеет свойство, которое вы ищете, классифицировано как не имеющее этого свойства. Например, когда спам появляется в вашем ящике входящих сообщений.

Чувствительность относится к проценту положительных классификаций, которые корректны, а *специфичность* относится к проценту отрицательных классификаций, которые корректны. Идеальное обнаружение имеет наивысшую чув-



ствительность и специфичность. В худшем случае никакой уровень не выше 50 %: то же, как при подбрасывании монеты.

Большинство систем требуют определенной степени компромисса; обычно увеличение чувствительности означает также принятие более низкой специфичности. Сокращение ложных отрицательных сторон будет сопровождаться увеличением ложных положительных сторон, и наоборот.

Для описания этого компромисса мы можем использовать визуализацию, названную ROC-кривая (receiver operating characteristic, рабочая характеристика приёмника) – график, позволяющий оценить качество бинарной классификации, отображает соотношение между долей объектов от общего количества носителей признака, верно классифицированных как несущие признак. На рис. 7-2 показан пример кривой ROC.

В этом случае рабочая характеристика является количеством пакетов в сеансе и показана на горизонтальных строках в графике. На этом сайте трафик HTTP (в левом верхнем углу графика) имеет хорошее отношение истинных к ложноположительным, тогда как SMTP-трафик более сложно классифицировать правильно, а FTP – еще сложнее.

Теперь давайте зададим вопрос. У нас есть кривая ROC, и мы калибруем детектор таким образом, что 99 % имеют истинно положительный уровень и 1 % – ложноположительный уровень. Мы получаем предупреждение. Какова вероятность, что предупреждение истинно положительное? Это не 99 %; истинноположительный уровень является вероятностью того, что *если бы* нападение произошло, IDS поднял бы тревогу.

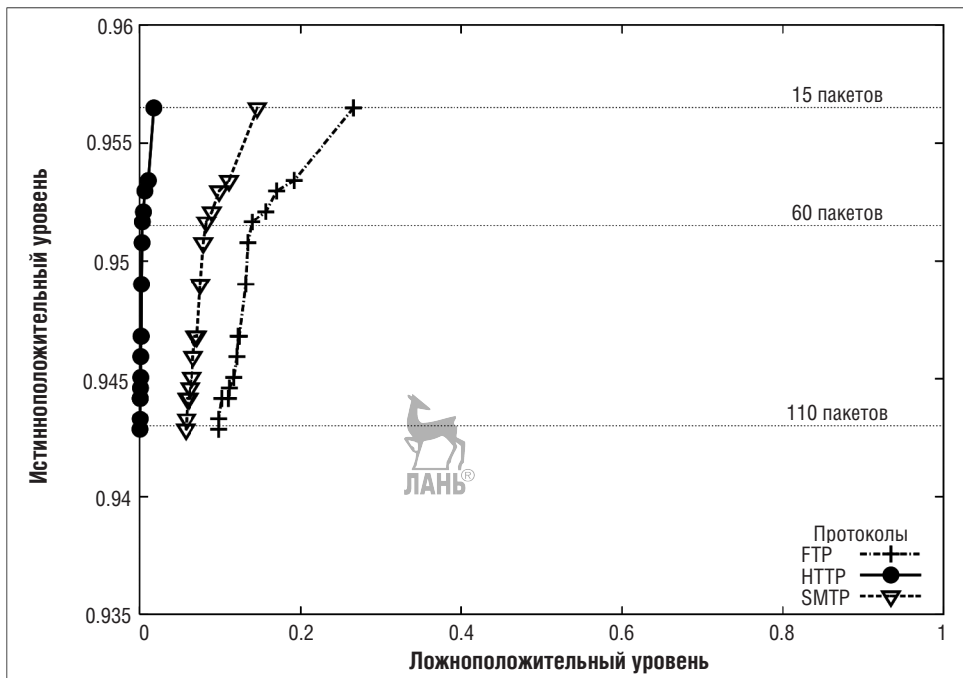


Рисунок 7-2. Кривая ROC, показывающая размер пакета сообщений, отправленных для обнаружения BitTorrent

Давайте определим тест как процесс, который IDS использует для принятия решения о данных. Например, тест может состоять из сбора 30 секунд сетевого трафика и сравнения его с прогнозируемым объемом или изучения первых двух пакетов сеанса на подозрительные строки.

Теперь предположим, что вероятность фактического нападения, происходящего во время теста, составляет 0,01 %. Это означает, что из каждых 10 000 тестов, которые проводит IDS, один будет атакой. Таким образом, из каждых 10 000 тестов мы поднимаем одну тревогу *из-за атаки* – в итоге у нас есть 99%-ный истинно-положительный уровень. Однако ложноположительный уровень составляет 1 %, который означает, что 1 % тестов поднимает тревогу даже притом, что ничего не произошло. Это означает, что для 10 000 тестов мы можем ожидать примерно 101 предупреждение: 100 ложных положительных и 1 истинное положительное, из чего следует, что вероятность тревоги *из-за нападения* составляет 1/101 или немного меньше, чем 1 %.

Эта *ошибка базовой ставки* объясняет, почему врачи не проводят каждый тест на каждом человеке. Когда вероятность реальной атаки устранена, ложные срабатывания будут легко подавлять истинные плюсы. Эта проблема усугубляется тем, что никто в здравом уме не доверяет только IDS и не делает работу в одиночку.

Применение классификации

Рассмотрим поток данных на рис. 7-3, который является простым представлением того, как IDS обычно используется для защиты.

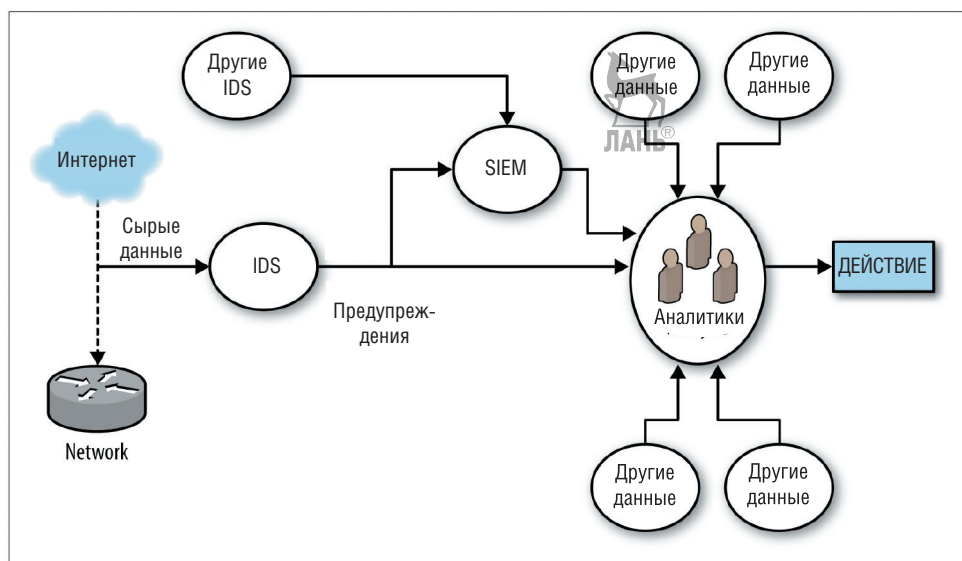


Рисунок 7-3. Простой рабочий процесс обнаружения

Рисунок 7-3 разделяет аварийную обработку на три шага: IDS получает данные, выдает предупреждение, и затем это предупреждение передает аналитикам непосредственно или через SIEM.

Как только IDS генерирует предупреждение, оно должно быть передано аналитику для дальнейших действий. Аналитики начинают исследовать предупреждение и выяснять, что оно означает. Это может быть относительно простым процессом, но часто становится расширенным и может включать много запросов. Простые запросы будут включать просмотр геолокации, владельца адреса и предыстории адреса, порождающего нападение (см. раздел 8), путем изучения полезной нагрузки события с помощью tcpdump или Wireshark. С более сложными атаками аналитики должны будут обратиться к помощи Google, новостям, блогам и доскам объявлений, чтобы идентифицировать подобные атаки или же события реального мира, ускоряющие атаку.

За исключением систем IPS, которые работают над очень сырыми и очевидными нападениями, такими как DDoS, всегда существует промежуток времени на аналитический шаг между предупреждением и действием. На данном этапе аналитики должны, получив предупреждение, определить, является ли предупреждение угрозой, если возможна угроза – то можно ли что-либо с этим сделать. Это нетривиальная проблема, рассмотрим следующие сценарии:

- IDS сообщает, что атакующий использует особую уязвимость IIS. Есть ли в сети какие-либо серверы IIS? Были ли они исправлены и, таким образом, не являются ли предметом для использования? Есть ли доказательство из других источников, что атака успешна?
- IDS сообщает, что атакующий сканирует сеть. Можем ли мы остановить сканирование? Должны ли мы обеспокоиться, учитывая, что существует другая сотня сканирований, продолжающихся прямо сейчас?
- IDS сообщает, что хост систематически выбирает через веб-сервер и копирует каждый файл. Действительно ли хост является ботом Google, и будет ли его остановка означать, что основной веб-сайт нашей компании больше не будет видим на Google?

Обратите внимание на то, что это на самом деле не сбои обнаружения. Первые два сценария представляют фактические потенциальные угрозы, но эти угрозы могут не *иметь значения*, и это решение может быть принято только через сочетание контекста и стратегических решений.

Проверка предупреждений занимает время. Аналитик способен серьезно обработать приблизительно одно предупреждение в час, а сложные события займут дни для исследования. Решите, как использовать это время, учитывая ложные положительные уровни, обсужденные ранее.

УЛУЧШЕНИЕ ПРОИЗВОДИТЕЛЬНОСТИ IDS

Существует два подхода к улучшению работы IDS. Первое должно улучшить IDS как классификатор; то есть увеличить чувствительность и специфичность. Второй путь состоит в том, чтобы уменьшить время, за которое аналитик должен обработать предупреждение путем выборки дополнительной информации, обеспечения контекста и идентификации планов действий.

Нет никаких совершенных правил для этого процесса. Например, несмотря на то что это всегда хорошая (и необходимая) цель – минимизировать ложные положительные уровни, аналитики проявят более детальный подход к этой проблеме. Например, если будет временный риск неприятного нападения, часто до-

пускают более высокий уровень ложных срабатываний, чтобы более эффективно защититься от этой атаки.

Здесь существует своего рода проблема закона Паркинсона. Все наши системы обнаружения и мониторинга предоставляют только частичную страховую защиту, потому что интернет странный, и у нас действительно нет полной ясности, что мы пропускаем. По мере того как каждый уровень улучшает процесс обнаружения, вы обнаруживаете, что есть новые, все более неприятные предупреждения для рассмотрения. Перефразируя Дональда Рамсфельда: у нас действительно есть проблема с неизвестными неизвестными.

Эта проблема неизвестных неизвестных делает ложноотрицательные стороны конкретной головной болью. По определению основанный на сигнатуре IDS не может предупредить о том, на предупреждение о чем он не был сконфигурирован. Однако большинство систем соответствия сигнатурам будут сконфигурированы для идентификации только ограниченного подмножества всех злонамеренных активностей, которые использует конкретный хост. Используя вместе IDSes, обнаруживающие на основе сигнатуры и на основе аномалии, можно, по крайней мере, начать идентифицировать мертвые зоны.

Улучшение обнаружения IDS

Улучшение IDS как классификатора включает сокращение ложноположительных и ложноотрицательных уровней. Это обычно лучше всего делается путем сокращения объема трафика, который исследует IDS. Так же как врач не запускает тест, пока нет симптома, с которым нужно бороться, мы пытаемся запустить в работу IDS, только когда у нас есть начальное подозрение, что происходит что-то странное. Много различных механизмов доступно в зависимости от того, используете ли вы IDS на основе сигнатуры или аномалии.

Противоречивое уведомление: головная боль с многократным IDSes

Специальная категория ложного отрицания включает противоречивый набор правил IDS. Предположим, вы работаете в сети с точками доступа А и В и с IDS, работающей на обеих. Если вы не сохраните правила на IDS А совместимыми с правилами на IDS В, то обнаружите, что А отправляет вам предупреждение, которое В не распознает, и наоборот.

Самый легкий способ управлять этой проблемой состоит в том, чтобы рассматривать набор правил как любой другой исходный код. Таким образом, поместите правила в систему управления версиями, удостоверьтесь, что вы фиксируете и комментируете их, и затем установите правила из своей системы управления версиями. Соблюдение правил под управлением версиями – в любом случае хорошая идея, потому что при выполнении исследования многомесячного трафика вы действительно захотите посмотреть на те старые правила, чтобы точно выяснить, что вы заблокировали в прошлом апреле.

Существует класс IDS, который, однако, делает этот тип управления особенно проблематичным. AV и некоторые другие системы обнаружения обычно являются системами черного ящика. Система черного ящика обеспечивает обновления

правил как сервис подписки, и правила обычно абсолютно недоступны администратору. Противоречивая идентификация может быть особенно проблематичной с системами черного ящика, где в лучшем случае необходимо отслеживать текущую базу правил и идентифицировать системы, которые находятся за ней¹.

Один общий механизм, характерный и для основанного на сигнатуре, и для основанного на аномалии IDSes, использует материально-технические ресурсы для создания белых списков. Чистые белые списки означают, что вы слепо доверяете всему трафику от хоста и *всегда* являются риском. Я не рекомендую только добавить хост в белый список и никогда не проверять его. Лучший подход состоит в том, чтобы использовать белый список в качестве руководства для более или менее обширного инструментария.

Например, я создаю материально-технические ресурсы всех веб-серверов в своей сети. Хост, который не является веб-сервером, *де-факто* подозрителен, если я вижу, что он обслуживает трафик HTTP. В этом случае я хочу получить существенное сокращение трафика и выяснить, *почему* это – теперь веб-сервер. В то же время для фактических веб-серверов я буду использовать свои стандартные сигнатуры.

Обычно в основанном на сигнатуре IDS может быть усовершенствована основа подписи, так чтобы правило срабатывало только для определенных протоколов или в тандеме с другими индикаторами. Например, правило для обнаружения строки полезной нагрузки «травяная добавка» на порту 25 будет отслеживать спам-сообщения с этим названием, но и внутренняя почта сообщения, такие как «мы получаем много травяной добавки», отправит в спам. Снижение ложноположительной скорости в этом случае включает в себя добавление дополнительных ограничений к совпадению, например отслеживание только почты из-за пределов сети (фильтрации по адресам). Путем совершенствования правила использовать больше выборочных выражений оператор может уменьшить ложноположительный уровень.

Как пример рассмотрим следующее (глупое) правило – определить, входит ли кто-либо в систему как root сервера SSH:

```
alert tcp any any -> any 22 (flow:to_server, established;)
```

Правило Snort состоит из двух логических разделов: заголовков и опции. Заголовок состоит из *действия* правила и адресной информации (протокол, исходный адрес, исходящий порт, адрес назначения, целевой порт). Опции состоят из ряда специфических ключевых слов, разделенных точками с запятой.

В примере выше действие – это alert, указывает на то, что Snort генерирует предупреждение и регистрирует пакет. Альтернативные действия включают log (зарегистрировать пакет без предупреждения), pass (игнорировать пакет) и drop (блокировать пакет). После действия следует строка, называющая протокол, tcp в этом случае, с uid, isrp и ip, являющимися другими опциями. Действие сопровождается источником к целевой информации, разделенной стрелкой (→) диграф. Информация об источнике может быть выражена как адрес (например, 128.1.11.3), сетевой блок (118.2.0.0/16), как выше, или any (любой) для указания на все адреса. Snort может также определить различные наборы адресов с макросами (напри-

¹ Это хороший бонус для идентификации систем, которые могут попасть под угрозу. Вредоносное программное обеспечение отключит AV самостоятельно.

мер, \$HOME_NET, чтобы указать на домашнюю сеть для IDS) для реализации сводных белых списков, как обсуждалось ранее.

Это правило выдает предупреждение, когда кто-либо успешно соединяется с ssh-сервером, который определяется слишком расплывчато. Для совершенствования правила я должен добавить дополнительные ограничения. Например, я могу ограничить его выдавать предупреждение, только если оно приходит из определенной сети и если кто-то пытается войти в систему, особенно как root.

```
alert tcp 118.2.0.0/16 any -> any 21 (flow:to_server, established; \ content:"root";
pcrc: "/user\s_root/i";)
```

После адресной информации одна или несколько *настроек*. Настройки могут использоваться для совершенствования правила, подстраивая информацию, которую правило ищет для сокращения ложноположительного уровня. Настройки могут также использоваться, чтобы добавить дополнительную информацию к предупреждению, инициировать другое правило или завершить множество иных действий.

Snort хорошо определяет более 70 опций для различных форм анализа. Краткий обзор наиболее полезных правил включает:

content (содержание)

content является повседневным¹ Snort-правилом сопоставления с образцом; оно выполняет точное сопоставление данных, переданных в настройке content, исходя из полезной нагрузки пакета. content может использовать двоичный файл и текстовые данные, включая двоичные данные в каналах. Например, content: |05 11|N|02 23| соответствует байту с содержанием 5, затем 11, потом буква N, далее байт 2, затем байт 23. Ряд других настроек непосредственно влияет на содержание, такое как depth (определяющий, где в полезной нагрузке прекратить искать) и offset (определяющий, где в полезной нагрузке начать поиск).

HTTP options (опции HTTP)

Ряд опций HTTP (http_client_body, http_cookie, http_header) извлечет релевантную информацию из пакета HTTP для анализа с помощью content.

pcrc

pcrc-опция использует регулярное выражение PCRE для сопоставления с пакетом. Регулярные выражения стоят дорого; убедитесь, что вы используете content для предварительной фильтрации трафика и пропускаете применение регулярного выражения к каждому пакету.

flags (флаги)

Проверки, чтобы видеть, присутствуют ли определенные флаги TCP.

flow (поток)

Ключевое слово flow определяет направления потока трафика, например от клиента к клиенту, от сервера или к серверу. flow также описывает определенные характеристики сессии, к примеру был ли он на самом деле установлен.

Язык Snort-правила используется несколькими другими IDS, особенно Suricata. Другие системы могут дифференцировать себя дополнительными опциями (например, Suricata имеет iprep опцию для просмотра репутации IP-адреса).

В отличие от основанных на сигнатурах систем, где вы не можете действительно пойти не так, как надо путем обсуждения правил Snort, системы обнаружения на

¹ Самым простым – bread and butter.

основе аномалии, более вероятно, будут созданы вручную (не автоматизированно). Следовательно, при обсуждении, как сделать детектор аномалии более эффективным, мы должны действовать на базовом уровне. В разделе III мы обсуждаем ряд различных числовых и поведенческих методов для реализации систем обнаружения аномалии, а также случаев для ложноположительных уровней. Однако это – подходящее место для обсуждения общих критериев создания хороших систем обнаружения аномалии.

В их самых простых формах системы обнаружения аномалии поднимают тревогу при достижении пороговых значений. Например, я мог бы попробовать обнаружить аномалию на файловом сервере путем подсчета числа байтов, загружаемых с сервера каждую минуту. Я могу сделать это, используя `gfilter` для фильтрации данных и `gscout` для подсчета трафика в единицу времени. Затем я использую R и генерирую гистограмму, показывающую вероятность, что значение выше x . Хорошая вещь в гистограммах и статистическом обнаружении аномалии состоит в том, что я управляю этим номинальным ложноположительным уровнем. Тест каждую минуту и 95%-ный порог, прежде чем поднять тревогу, означают, что я создаю три предупреждения в час; 99%-ный порог означает одно предупреждение каждые два часа.

Проблема заключается в выборе порогового значения, которое на самом деле полезно. Например, если злоумышленник знает, что я подниму тревогу, если он будет слишком занят, он может уменьшить объем действий ниже порогового значения. Этот тип уклончивости является действительно тем же, что мы видели с вирусом Code Red. Злоумышленник в этом случае мог изменить содержание буфера, не влияя на производительность вируса. Когда вы определяете явления для обнаружения аномалии, необходимо иметь в виду, как это влияет на цели атакующего; обнаружение является просто первым шагом.

У меня есть четыре эмпирических правила, которые я применяю при оценке явлений для системы обнаружения аномалии: предсказуемость, управляемые ложноположительные уровни, разрушительность и влияние на поведение зловреда.

Предсказуемость является самым основным качеством для поиска явления. Предсказуемым считается явление, значение которого эффективно сходится со временем. «Сходимость» (или конвергенция) – это что-то, о чем я должен немного беспокоиться. Можно обнаружить, что девять дней из десяти пороговым значением является x , и затем на десятый день он повышается до $10x$ по непонятной причине. Ожидайте необъяснимую странность; если можно идентифицировать и описать выбросы поведенчески, и независимо от этого, то, что остается, имеет верхний предел, который можно объяснить: у вас есть что-то предсказуемое. Ложноположительные результаты будут достигнуты во время исследования, а истинноположительные – во время обучения!

Второе правило – это управляемые ложноположительные уровни. Посмотрите на недельный трафик для любого общедоступного хоста, и вы увидите, что происходит что-то странное. Можно ли объяснить эту странность? Действительно ли это – один и тот же адрес снова и снова? Действительно ли это – обычный сервис, такой как поисковый робот, посещающий веб-сервер? Во время процесса начальной подготовки для любого детектора аномалии необходимо зарегистрироваться, запомните, сколько времени вы тратите на идентификацию и объяснение выбросов и можно ли управлять этими выбросами через белый список или другие

поведенческие фильтры. Чем меньше необходимо объяснять, тем ниже нагрузка на занятых операционных аналитиков.

Разрушительность – это то, на что злоумышленник должен влиять для достижения своих целей. Чем проще, тем лучше. Например, для загрузки трафика с веб-сервера атакующий должен связаться с веб-сервером. Он, возможно, не должен делать это с одного и того же адреса, и ему, возможно, не понадобится аутентификация, но ему нужно сбросить данные.

Наконец, существует влияние явления на поведение злоумышленника. Лучшие предупреждения – те, которые *должен* инициировать атакующий. Со временем, если детектор повлияет на него, злоумышленник будет учиться уклоняться или путать его. Мы видим, что антиспам в фильтрах и различных инструментах используется для обхода байесовских фильтров и в инсайдерских угрозах. При рассмотрении сигнала тревоги подумаем, как злоумышленник может уклониться от него, например:

Путем медленного перемещения

Может ли злоумышленник повлиять на предупреждение, если он уменьшит объем действий? Если так, каково влияние на цель атакующего? Если сканер замедляет сканирование, сколько у него времени займет отсканировать вашу сеть? Если файл leech (пиявка) копирует ваш сайт, сколько времени потребуются, чтобы скопировать весь сайт?

Путем более быстрого перемещения

Может ли атакующий обмануть систему, если он перемещается быстрее? Если он рискует обнаружением, может ли он перемещаться быстрее, чем ваша возможность блокировать его при перемещении максимально быстро?

Путем распределения нападения

Если атакующий работает с многочисленных IP-адресов, могут ли отдельные адреса проскользнуть под пороговыми значениями?

Переменное поведение

Может ли атакующий, изменяя свое поведение между подозрительным и невиновным, обмануть IDS таким образом?

Многие методы, обсужденные ранее, подразумевают различную степень неоднородности вашей системы обнаружения. Например, систему обнаружения аномалий, возможно, придется сконфигурировать индивидуально для различных хостов. Я счел полезным продвинуть эту идею о модели с подписками, где аналитики выбирают, какой хост контролировать, выбирают пороговые значения и обеспечивают средства помещения в белый и черный списки для каждого хоста, который они решают контролировать. Подписки гарантируют, что аналитик может обрабатывать каждый узел по отдельности, и в конечном итоге создать интуитивно нормальное поведение на этом хосте (например, зная, что трафик на зарплатный сервер становится аномальным каждые две недели).

Модель подписки подтверждает, что вы не можете контролировать все, и соответственно следующий вопрос – *что* точно контролировать. Разделы 13 и 15 обсуждают этот вопрос более глубоко.

Улучшение ответа IDS

IDS, особенно NIDS, был задуман как система обнаружения в реальном времени – будет действительно достаточно времени между началом нападения и заклю-

чительным использованием того, что вооруженные предупреждениями IDS защитники смогли остановить нападение до нанесения значительного ущерба. Это понятие было разработано в то время, когда атакующие могли бы использовать два компьютера, когда нападения были запущены вручную экспертами и когда вредоносное программное обеспечение было намного более примитивным. Теперь слишком часто IDS является рецептом лишь для раздражения. Это не просто случай неправильно классифицированных нападений; это – случай зловредов, атакующих хосты, которые не находятся там в надежде, что они найдут то, что можно украсть.

В какой-то момент вы сделаете IDS настолько эффективным детектором, насколько сможете, и все равно получите ложные положительные уровни, потому что существуют нормальные поведения, которые похожи на атаки, и единственный способ понять, с чем имеешь дело, – это исследовать их. Как только вы достигаете этой точки, вас оставляют с проблемой предупреждения: IDS генерируют простые предупреждения в режиме реального времени, и аналитики должны разрешить их. Сокращение рабочей нагрузки на аналитиков означает агрегирование, группировку и управление предупреждениями, так чтобы процесс проверки и ответа стал более быстрым и проводился более эффективно.

Рассматривая, как управлять предупреждением, сначала спросите, каков будет ответ на это предупреждение. Большинство CSIRTs имеют ограниченный набор мер, которые они могут принять в ответ на предупреждение, такой как переконфигурация брандмауэра или правил IPS, удаление хоста из сети для дальнейшего анализа или внесение изменений в политику безопасности. Ответы редко поступают в режиме реального времени, и определенным атакам весьма свойственно не заслужить вообще никакого ответа. Классический пример последнего случая – это сканирование: оно является вездесущим, его почти невозможно заблокировать, и существует очень мало шансов поимки преступника.

Если ответ в режиме реального времени не требуется, часто полезно свернуть оповещения, особенно с IP-адреса злоумышленника или эксплойта. Это не редкость для IDS, генерация нескольких предупреждений для одного и того же злоумышленника.

Поведение, которое не очевидно из одного предупреждения, становится более очевидным, когда оно агрегируется.

Упреждающая выборка данных

После получения предупреждения аналитики должны проверить информацию и исследовать ее. Это обычно включает такие задачи, как определение страны происхождения, целей и любого прошлого действия с этого адреса. Упреждающая выборка данной информации помогает намного уменьшить нагрузку на аналитиков.

В частности, с системами обнаружения аномалий это помогает представить варианты. Как мы уже обсудили, обнаружения аномалии являются часто пороговыми, выдавая предупреждение, после того как явление превышает порог. Вместо того чтобы просто представить отклоняющееся событие, возвратите топ-*n* список наиболее отклоняющихся (абберантных) событий с фиксированным интервалом.

Предоставление сводных данных в визуализациях, таких как графики временных рядов или графики контактов, помогает снизить когнитивную нагрузку на

аналитика. Вместо того чтобы просто производить прямой текстовый дамп информации запроса, лучше генерировать соответствующие графики. В разделе 10 это обсуждается более подробно.

Наконец, рассмотрите возможность мониторинга ресурсов, вместо того чтобы просто контролировать нападения. Большинство систем обнаружения фокусируются на поведении атакующего, таком как выдача предупреждения, когда определенная сигнатура атаки обнаружена. Вместо того чтобы фокусироваться на поведении зловреда, выдайте вашему аналитику определенные хосты в сети, чтобы наблюдать и анализировать график ресурс–аномалия. Цели более низкого приоритета должны быть защищены с помощью более строгой технологии, такой как строгие брандмауэры.

Распределение аналитиков по ресурсам, помимо реагирования на предупреждения, имеет еще одно преимущество: аналитики могут развивать знания о системах, которые они наблюдают. Ложные срабатывания – это часто выход из общих процессов, которые нелегко описать идентификаторам, таким как рост активности для файловых серверов, потому что проект пика своего роста – это обычные запросы к платежной ведомости или это сервис, популярный у определенной демографической группы. Экспертиза сокращает время аналитикам, нужно просеять данные и помочь им выбросить мелочи, чтобы сосредоточиться на наиболее значительных угрозах.

ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ ДЛЯ ЧТЕНИЯ

1. *Stefan Axelsson*. The Base-Rate Fallacy and the Difficulty of Intrusion Detection. ACM Transactions on Information and System Security, Vol. 3, Issue 3, August 2000.
2. *Brian Caswell, Jay Beale, and Andrew Baker*. Snort IDS and IPS Toolkit (Syngress, 2007).
3. *Vern Paxson*. Bro: A System for Detecting Network Intruders in Real-Time. Computer Networks: The International Journal of Computer and Telecommunications Networking, Vol. 31, Issue 23–24, December 1999.
4. *Martin Roesch*. Snort—Lightweight Intrusion Detection for Networks. Proceedings of the 1999 Large Installation Systems Administration Conference.

Глава 8

Ссылка и поиск: инструменты для выяснения, кто есть кто

Каждая строка предупреждения или файла журнала, которая сообщает о событии, предоставляет некоторую основную информацию об источнике события. Только от IP-адреса можно получить информацию о географическом положении и сделать обратный поиск DNS. В данной главе рассматриваются инструменты, которые помогают отследить идентификационные данные хоста.

Эта глава фокусируется на идее «идти» по стеку OSI, как упомянуто в разделе «Влияние уровней сети на ее оснащение» на странице 34. Мне нравится рассматривать уровень модели OSI как последовательность процессов поиска. Каждый уровень предлагает различную часть адресной информации, такой как MAC-адрес на уровне 2, IP-адрес на 3-м и порты на 4-м уровне. Эта информация перемещается между уровнями посредством различных систем ссылок: протокол определения адресов (ARP) отображает IP-адреса на MAC-адреса, DNS отображает доменные имена на IP-адреса и т. д. Снова абстракция не совершенна – перевод DNS не перемещает нас вверх или вниз по стеку OSI – но путем хождения по каждому уровню мы можем описать то, что означают адреса и когда они релевантны для данного поиска.

Остаток от этой главы структурирован следующим образом: раздел по MAC-адресам, затем IPv4 и IPv6, сопровождаемый информацией об интернет-уровнях, затем DNS, потом протоколы более высокого уровня. Наконец, наступает обсуждение других важных инструментов, которые не помещаются в модель деления на уровни, в частности репутационные базы данных, репозитории вредоносных программ.

К сожалению, некоторые наши методы поиска зависят от недостаточно хорошо поддерживаемой общедоступной базы данных, но они могут быть полезны, если вы понимаете это ограничение.

MAC и аппаратные адреса

Глава 2 обсуждает основы управления доступом к среде Media Access Controller (MAC). Адреса MAC определены в сетевом оборудовании для обеспечения локально уникального адреса для хостов на втором сетевом

уровне модели OSI сети. Большинство MAC-адресов следуют 48-разрядному стандарту расширенного уникального идентификатора (EUI – *Extended Unique Identifier*): 6 байт выражены шестнадцатеричным образом (например, 08-21-23-41-FA-BB). Аппаратные средства более современной сети могут использовать EUI-64, который добавляет дополнительные 16 бит. Когда кадр переходит от 48-разрядной системы к 64-разрядной системе, 48-разрядный адрес дополняется до 64 битов.

Рисунок 8-1 показывает, как преобразуются EUI-48 и EUI-64.

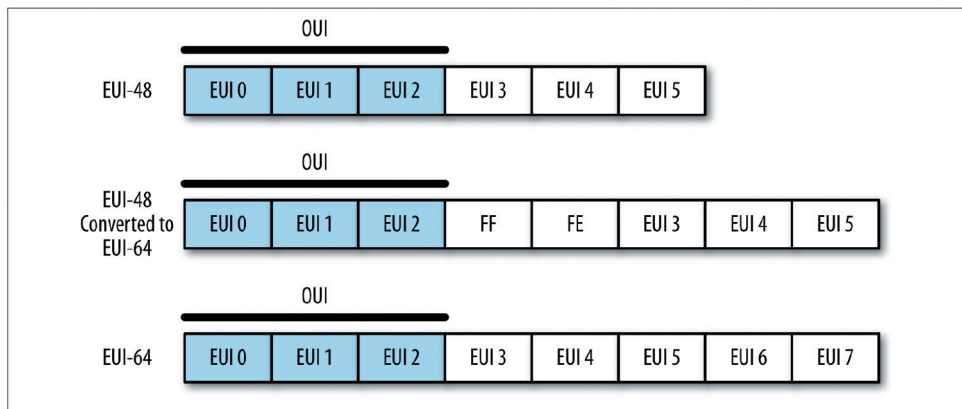


Рисунок 8-1. Стандарты EUI-48 и EUI-64

В частности, отметим две вещи. Во-первых, если EUI-48 преобразован в EUI-64, это можно определить, рассмотрев байты 3 и 4, они будут FFFE. Более важно то, что первые 3 байта являются *организационно уникальным идентификатором* (OUI – *Organizationally Unique Identifier*), который является 24-разрядным значением, присвоенным IEEE производителю оборудования. OUI's являются фиксированными порядковыми номерами, и если вы знаете OUI, можно узнать, кто произвел карту. IEEE ведет [список](#) присвоений OUI, и можно использовать поисковую систему для нахождения OUIs по компании или компании по OUI.

Например, рассмотрим следующий пакет от *pcap*:

```
$ tcpdump -c 1-e-n-r web.pcap
reading from file web.pcap, link-type EN10MB (Ethernet)
00:37:56.480768 8c:2d:aa:46:f9:71> 00:1f:90:92:70:5a, ethertype IPv4 (0x0800),
length 78: 192.168.1.12.50300> 157.166.241.11.80: flags [S],
seq 4157917085, win 65535, options [mss 1460,nop,wscale 4,nop,
nop,TS val 560054289 ecr 0, sackOK, eol], length 0
```

Сообщение идет от 8c:2d:aa:46:f9:71 до 00:1f:90:92:70:5a. Их поиск говорит нам, что 8c:2d:aa принадлежит Apple и 00-1f-90 принадлежит Actiontec Electronics, который делает маршрутизаторы Verizon FIOS.

Там меньше работы, чем вы думаете

Общий аналитический камень преткновения вырастает, когда аналитик пытается создать сложное общее решение проблемы, когда имеется только ограниченное количество опций. Чтобы использовать военный пример, вы не должны разрабатывать общее решение для идентификации авианосцев, потому что только 20 из них состоит на действительной службе. Вместо того чтобы работать над одной большой проблемой, можно решить 20 проблем, которые значительно меньше и обычно подобны.

При контакте с аппаратными системами и приложениями это часто помогает остановиться, отступить и провести некоторое исследование рынка. Проблема нередко становится меньше, когда вы узнаете, например, что, в то время как существует набор систем со встроенными веб-серверами, большинство из них использует Allegro RomPager.

MAC-адреса работают полностью в рамках локальной сети. Для передачи вне границ маршрутизатора хост должен иметь IP-адрес. Отношение между локальным MAC- и IP-адресом управляется через *протокол определения адресов (ARP)*. Отдельные хосты поддерживают *таблицы ARP*, которые содержат отображения между IP-адресами и MAC-адресами в сети. Например, на моем локальном хосте я могу запросить таблицу ARP с помощью утилиты `arp`:

```
$ arp -a
wireless_broadband_router.home (192.168.1.1) at 0:1f:90:92:70:5a on en1 ifscope
/ [ethernet]
new-host-2.home (192.168.1.3) в 0:1e:c2:a6:17:fb on naen1 ifscope[ethernet]
new-host.home (192.168.1.4) в cc:8:e0:68:b8:a4 on en1 ifscope [ethernet]
apple-tv-3.home (192.168.1.9) в 7c:d1:c3:26:35:bf on en1 ifscope [ethernet]
? (192.168.1.255) в ff:ff:ff:ff:ff:ff on en1 ifscope[ethernet]
```

Запустите `lookup`, и вы обнаружите, что мне *действительно* нравятся аппаратные средства Apple. Или я предпочитаю сохранять свои поля Windows и Linux физически соединенными проводом.

Аналитически MAC-адреса (когда вы можете получить их, и вы будете иметь их только для вашей локальной сети, как уже объяснялось) особенно полезны для идентификации и дифференциации оборудования, особенно сетевого оборудования, такого как маршрутизаторы). IP-адреса значительно более взаимозаменяемы, чем MAC-адреса, и если необходимо отследить такой мобильный актив, как ноутбук или что-либо, модулируемое через DHCP, MAC-адрес будет лучшим активом для этого.

IP-АДРЕСАЦИЯ

IP-адреса являются наиболее часто доступной частью информации о хосте и нередко единственной частью данных, которые вы будете иметь о хосте.

IP медленно мигрирует от протокола IPv4 до IPv6. IPv6 исправляет много ошибок дизайна в IPv4, самое известное – исчерпание IP-адреса. Адрес IPv4 является 32-разрядным, традиционно записанным в «точечном квадратическом» формате: четыре байта, записанных по десятичной системе, разделенных периодами (как 192.168.1.1). Во время первоначального проекта IPv4 никто не ожидал всерьез, что 4 миллиарда адресов будут исчерпаны, и многие ранние выделения адресов IPv4

были безумно щедры, как видно из основного списка /8 выделений. /8 является набором 16 миллионов + адреса (2)²⁴, все из которых имеют один и тот же первый октет, таким образом, с 9.0.0.0 до 9.255.255.255 все принадлежат IBM, например. Глядя на список, можно увидеть, что некоторые большие блоки были давным-давно присвоены компаниям, таким как Херох и Ford, которые действительно не используют полностью принадлежащее им пространство. На самом деле ситуация улучшилась за последние годы, когда несколько фармацевтических фирм, владеющих почти пустыми /8, возвратили их в IANA.

Большинство англоязычного интернета все еще работает на IPv4, в то время как в Азии и в других частях света все более и более распространен IPv6. Неравномерное выделение адресов IPv4 заставляет страны, которые присоединились к сети интернет исторически позже, быстрее создавать инфраструктуру IPv6.

Адреса IPv4, их структура и важные адреса

Адреса IPv4 могут быть выражены с помощью ряда различных нотаций. Большая часть является точечным квадратическим форматом, обсужденным ранее: четыре целочисленных значения между 0 и 255, разделенных точками. Адреса могут также быть упомянуты непосредственно как значение, обычно в шестнадцатеричном формате. Следовательно, IP-адрес 0xA1010203 – 161.1.2.3 – как точечная четверка и 2701197827 – как десятичное целое число.

Группы IP-адресов обычно описываются линейно (например, 128.2.11.3 – 128.2.3.14) или с использованием блока *Classless Internet Domain Routing (CIDR)*. Блоки CIDR, которые более подробно описаны ниже, являются механизмом для описания адресов, достижимых путем выбора конкретного маршрута. Адреса в нотации CIDR представлены *префиксом*¹, который является точечным квадратическим представлением значимых битов адреса, и затем *маской*, которая указывает, сколько битов составляет префикс¹.

Например, блок CIDR 128.2.11.0/24 состоит из всех адресов, первые 24 бита которых 128.2.11, таким образом, любой адрес от 128.2.11.0 до 128.2.11.255 находится в этом блоке.

Ряд IP-адресов или зарезервирован, или зафиксирован соглашением в конфигурации сети. Для отдельного хоста в сети самыми важными являются широковещательный адрес, шлюз и сетевая маска. Сети IP логически разделены на подсети, набор непрерывных адресов, которые могут все связаться друг с другом без потребности во внутренней маршрутизации. При конфигурировании IP-адреса этот диапазон задается с помощью *сетевой маски*, представляющей собой IP-адрес с определенным числом обнуленных младших значащих битов.

Для передачи вне его подсети хост должен будет обратиться к маршрутизатору с помощью предварительно сконфигурированного *адреса шлюза*. Адресом шлюза является просто IP-адрес интерфейса маршрутизатора к подсети. Адресам шлюза обычно присваивают самое низкое значение в подсети, но это не является обязательным требованием.

Широковещательный адрес сети установлен в маску подсети, но со всеми верхними битами хоста (например, для сети с маской подсети 192.168.1.0 широковещательный адрес 192.168.1.255). Сообщения, отправленные в широковещательный адрес, отправляются каждому целевому объекту в пределах сети. Широковещательный адрес явля-

¹ Обратите внимание на то, что префикс является эквивалентом сетевой маски подсети.

ется одним из многих адресов, который вы никогда не должны видеть вне локального сетевого трафика. Адреса, заканчивающиеся на .255, из-за отсутствия лучшего термина, довольно смешные.

Ряд адресов IPv4 зарезервирован для определенных сетевых функций. Эти адреса, в частности, предназначены для локального использования и, следовательно, не должны применяться при выходе за границы локальной сети. Наиболее важные – это:

Локальные идентификационные адреса (Local identification addresses)

Они принадлежат 0.0.0.0/8 блоку CIDR (0.0.0.0–0.255.255.255). Локальные идентификационные адреса используются для согласования во время запуска хоста, у которого еще нет IP-адреса.

Адрес обратной связи (Loopback address)

Петлевой адрес хоста 127.0.0.1. Трафик, отправленный в loopback-адрес, передают обратно в хост, минуя сеть. IANA зарезервировала весь 127.0.0.0/8 блок CIDR (127.0.0.0–127.255.255.255) для обратной связи, поэтому, как с локальной идентификацией, ничто из 127.0.0.0/8 блока CIDR не должно выйти за границы локальной сети.

RFC 1918 netblocks

Этот документ определяет ряд сетевых блоков для частного пользования. Эти адреса могут использоваться в локальных сетях с намерением никогда не взаимодействовать непосредственно с глобальным интернетом. Сетевыми блоками RFC являются 10.0.0.0/8, 192.168.0.0/16 и 172.16.0.0/12. Адреса в этих блоках часто присваиваются автоматически локальными инструментами маршрутизации или DHCP.

Групповые адреса (Multicast addresses)

Групповые адреса используются для классификации определенных групп хостов в подсети. Например, групповой адрес 224.0.0.2 является общим адресом «все маршрутизаторы», и все маршрутизаторы в подсети получают трафик, отправленный на него. Multicast-трафик в первую очередь основное внимание уделяет маршрутизации и другим протоколам управления интернетом.

Адреса IPv6, их структура и важные адреса

Одним из наиболее значимых отличий между IPv4 и IPv6 является количество адресов, которые они делают доступными. IPv6 присваивает 128 бит каждому адресу; это гарантирует большое количество адресов, но представляет некоторые проблемы в нотации (написании).

По умолчанию формат для адреса состоит из восьми 16-разрядных шестнадцатеричных значений, разделенными двоеточиями, например 2001:0010:AF3A:F B31:09A8:08A1:1098:1101. Учитывая, что это долгое и неуклюжее представление, адреса обычно представляются с помощью ряда кратких соглашений. При записи адресов IPv6 применяйте эти правила.

- Первые нули в любой группе опускаются, таким образом, 01AA:0002 может быть записан как 1AA:2.
- Последовательные группы нулей могут быть заменены парой двоеточий, таким образом, 2001:0:0:0:0:0:1 записан как 2001::1. Сокращением двойным двоеточием может использоваться только однажды, таким образом, 2001:0:0:0:11:0:0:1 будет записан как 2001::11:0:0:1.

RIRs и распределение IP-адресов

Исследование IP-адреса часто означает прослеживание цепочки владения от IANA до определенной организации. Процесс резервирования является иерархическим; на верхнем уровне распределением IP-адресов управляет Комитет по цифровым адресам в интернете (IANA). IANA является отделом интернет-корпорации по присвоению имен и адресов (ICANN) американской некоммерческой организации, отвечающей за IP-адрес управления и присвоение имени DNS.

IANA делегирует управление блоками номеров региональным интернет-реестрам (RIRs), континентальным организациям, которые справляются с выделением IP-адресов и номеров автономных систем в пределах их континента. RIRs являются посредником между IANA и различными национальными и TLD-регистраторами, которые на самом деле имеют дело с выделением адресов (см. табл. 8-1).

Таблица 8-1. RIRs

RIR	Домен	URL
ARIN	США и Канада	www.arin.net
LACNIC	Центральная и Южная Америка, Карибское море	lacnic.net
RIPE	Европа, Россия и Ближний Восток	www.ripe.net
APNIC	Азия и Океания	www.apnic.net
AfriNIC	Африка	www.afrinic.net

IANA делегирует блоки адресов к RIRs, а RIRs, в свою очередь, выделяют части этих блоков организациям в их доменах. RIRs выделяют блоки адресов своим участникам, а эти участники могут выделить подблоки или адреса, как они считают целесообразным.

Этот процесс выделения означает, что каждый IP-адрес имеет цепочку владения. Это владение начинается с IANA, выделенного одному из RIRs, и затем идет вниз через один или несколько ISPs, пока не достигает того, кто в настоящее время использует адрес. Вне заключительного ISP (обычно ниже /24 или /27) право владения адресом более однородно – редко можно связать определенный адрес с определенным человеком, если это не публичная информация через whois или ISP готов выдать эту информацию.

Как и с IPv4, множество блоков IPv6 зарезервированы для определенных функций. Наиболее важным резервированием в этом случае является 2 000::/3 (как с IPv4, нотация блока CIDR может использоваться с адресами IPv6, и маска может расширяться на 128 бит). Пространство IPv6 огромно, и чтобы помочь сохранить маршруты обоснованно близко друг к другу, *весь маршрутизируемый трафик в IPv6 должен быть в блоке 2000::/3*. Дальнейшие деления в блоке 2000::/3 обслуживаются IANA, так же как это делается с /8 реестром для IPv4. Основная ссылка доступна на странице IPv6 Global Unicast Address Assignments <http://bit.ly/ipv6-add>.

Дополнительные блоки адреса включают ::/128 и :: 1/128, которые являются неопределенным адресом и loopback-адресом (эквивалент 0.0.0.0 и 127.0.0.0 для IPv4).

Особенно интересны служебные блоки адресов 2001:758::/29 и 2001:678::/29. 2001:758::/29, в частности, присвоены *точкам обмена интернет-трафиком* (IXPs); IXP является физическим местоположением, где многочисленные ISPs соединяются друг с другом. 2001:678::/29 представляет блок независимых от провайдера адресов; для получения этих адресов пользователи могут связаться непосредственно со своим RIRs.

Для ясности сводка локальных и немаршрутизируемых адресов предоставлена в табл. 8-2.

Таблица 8-2. Известные адреса

Блок IPv4	Блок IPv6	Описание
0.0.0.0/0	::/0	Маршрут по умолчанию; адреса от этого блока не должны быть видны
0.0.0.0/32	::/128	Неопределенный адрес
127.0.0.1/8	::1/128	Адрес обратной связи
192.168.16.0/24	fc00::/7	Зарезервирован для локального трафика
10.0.0.0/8	fc00::/7	Зарезервирован для локального трафика
172.16.0.0/12	fc00::/7	Зарезервирован для локального трафика
224.0.0.0/4	ff00::/8	Групповые адреса

Проверка возможности соединения: используйте ping для соединения с адресом

Самым основным инструментом командной строки для проверки возможности соединения является `ping`. Проверка с помощью *ping-запросов* работает при помощи ICMP-сообщения (см. «Форматы пакета и фрейма» на стр. 24). `ping` отправляет эхо-запрос ICMP (тип 8, код 0) к цели. При получении сообщения эхо-запроса цель должна ответить эхо-ответом (тип 0, код 0). Пример 8-1 отображает вывод `ping` и `pcap` содержания.

Пример 8-1. Вывод `ping`

```
$ ping -c 1 nytimes.com
PING nytimes.com (170.149.168.130): 56 data bytes
64 bytes from 170.149.168.130: icmp_seq=0 ttl=252 time=29.388 мс

$ tcpdump-Xnr ping.pcap
reading from file ping.pcap, link-type EN10MB (Ethernet)
20:38:09.074960 IP 192.168.1.12 > 170.149.168.130:
    ICMP echo request, id 44854, seq 0, length 64
    0x0000: 4500 0054 0942 0000 4001 5c9b c0a8 010c E..T.B..@.\....
    0x0010: aa95 a882 0800 0fb8 af36 0000 5175 d7f1 .....6..Qu..
    0x0020: 0001 24a6 0809 0a0b 0c0d 0e0f 1011 1213 ..$.
    0x0030: 1415 1617 1819 1a1b 1c1d 1e1f 2021 2223 .....!"#
    0x0040: 2425 2627 2829 2a2b 2c2d 2e2f 3031 3233 $%&'()*+,-./0123
    0x0050: 3435 3637                                     4567
20:38:09.104250 IP 170.149.168.130 > 192.168.1.12:
    ICMP echo reply, id 44854, seq 0, length 64
    0x0000: 4500 0054 0942 0000 fc01 a09a aa95 a882 E..T.B.....
    0x0010: c0a8 010c 0000 17b8 af36 0000 5175 d7f1 .....6..Qu..
```

```

0x0020:  0001 24a6 0809 0a0b 0c0d 0e0f 1011 1213  ..$.
0x0030:  1415 1617 1819 1a1b 1c1d 1e1f 2021 2223  .....!"#
0x0040:  2425 2627 2829 2a2b 2c2d 2e2f 3031 3233  $%&'()*+,-./0123
0x0050:  3435 3637                                     4567

```

Отметьте сначала размер пакета и значения `ttl`. Эти значения обычно устанавливаются по умолчанию стеком TCP. В случае Mac OS X пакет ICMP имеет 56-байтовую полезную нагрузку, которая приводит к 84-байтовому пакету (20 байт заголовка IP, 8 байт заголовка ICMP и 56-байтовая полезная нагрузка). Тип и код в 0x0014-0x0015 (08 для запроса, 00 для ответа). После заголовка ICMP обратите внимание, что содержание пакета отражено. ICMP имеет понятие сеанса, и во многих случаях сообщения отправлены в ответ на пакеты от совершенно различных протоколов. Различные сообщения ICMP используют различную технику для указания на их место происхождения; в случае `ping` это сделано путем повторения исходного содержания пакета.

`ping` является простым приложением: отправляется эхо-запрос со встроенным идентификатором согласования. Приложение ожидает до указанного тайм-аута (обычно на порядке 4000 мс); если ответ получен за это время, ответ распечатан, и следующий пакет отправлен. `ping` – это диагностический инструмент, и любое серьезное внедрение предоставит несколько командных строк для управления составом пакетов.

Sweeping ping и ping sweeping

Это на самом деле различные условия, несмотря на то что Google запутывается при вводе их в поиск. Сканирование *ping* (или *ping sweeping*) является методом сканирования, который систематически проверяет с помощью `ping`-запросов все IP-адреса, присвоенные сети, чтобы определить, какие из них присутствуют, а какие нет. Развертка `ping` поддерживается *ntmap* и многими другими инструментами сканирования, несмотря на то что можно написать сценарий ее выполнения приблизительно за 20 секунд®.

Широкий ping (или *sweeping ping*), напротив, является последовательностью сообщений `ping`, размер которых увеличивается с каждым пакетом. *Sweeping ping* предназначены для диагностирования каналов путем идентификации управления трафиком или проблем MTU. Они включены параметром командной строки на большинстве современных реализаций *ping*.

Весьма распространены сети, блокирующие сообщения ICMP. Поэтому сканирование `ping` является посредственным инструментом для нахождения хостов в сети; прямой TCP или сканирование UDP обычно будет более эффективно.

Tracerouting

`traceroute` является инструментом и методом для идентификации маршрутизаторов, которые направляют пакеты из точки А в точку В. `traceroute` составляют последовательный список маршрутизаторов путем управления пакетом TTLs.

Поле пакета IP TTL (время жизни) является механизмом, разработанным, чтобы препятствовать исчезновению пакетов интернета навсегда. Каждый раз, когда пакет передан маршрутизатором, его значение TTL уменьшается на единицу. Когда TTL достигает нуля, адресующий маршрутизатор сбрасывает пакет и отправляет ICMP-сообщение о превышении (тип 11) времени.

```
$tracert www.nytimes.com
tracert to www.nytimes.com (170.149.168.130), 64 hops max, 52 byte packets 1
 1 wireless_broadband_router (192.168.1.1) 1,189 мс 0,544 мс 0,802 мс
 2 l100.washdc-vfttp-47.verizon-gni.net (96.255.98.1) 2,157 мс 1,401 мс 1,451 мс
 3 g0-13-2-7.washdc-lcr-22.verizon-gni.net (130.81.59.154) 3,768 мс 3,751 мс 3,985 мс
 4 ae5-0.res-bb-rtr1.verizon-gni.net (130.81.209.222) 2,029 мс 2,314 мс 2,314 мс
 5 0.xe-3-1-1.br1.iad8.alter.net (152.63.37.141) 2,731 мс 2,759 мс 2,781 мс
 6 xe-2-1-0.er2.iad10.us.above.net (64.125.13.173) 3,313 мс 3,706 мс 3,970 мс
 7 xe-4-1-0.cr2.dca2.us.above.net (64.125.29.214) 3,741 мс 3,668 мс
   xe-3-0-.cr2.dca2.us.above.net (64.125.26.241) 4,638 мс
 8 xe-1-0-0.cr1.dca2.us.above.net (64.125.28.249) 3,677 мс
   xe-7-2-0.cr1.dca2.us.above.net (64.125.26.41) 3,744 мс
   xe-1-0-0.cr1.dca2.us.above.net (64.125.28.249) 4,496 мс
 9 xe-3-2-0.cr1.lga5.us.above.net (64.125.26.102) 24,637 мс
   xe-2-2-0.cr1.lga5.us.above.net (64.125.26.98) 10,293 мс 9,679 мс
10 xe-2-2-0.mpr1.ewr1.us.above.net (64.125.27.133) 20,660 мс 10,043 мс 10,004 мс
11 xe-0-0-0.mpr1.ewr4.us.above.net (64.125.25.246) 15,881 мс 16,848 мс 16,070 мс
12 64.125.173.70.t01646-03.above.net (64.125.173.70) 30,177 мс 29,339 мс 31,793 мс
```

Как отображено в следующем блоке кода, tracert отправляет первоначальное 52-байтовое сообщение и затем продолжает получать последовательную информацию о каждом адресе, в который оно попадает по направлению к 170.149.168.130. Давайте посмотрим на полезную нагрузку более подробно.

```
$ tcpdump -nXr traceroute.pcap | more
21:06:51.202439 IP 192.168.1.12.46950> 170.149.168.130.33435: UDP, length 24
 0x0000: 4500 0034 b767 0000 0111 ed85 c0a8 010c E..4.g.....
 0x0010: aa95 a882 b766 829b 0020 b0df 0000 0000 .....f.....
 0x0020: 0000 0000 0000 0000 0000 0000 0000 0000 .....
 0x0030: 0000 0000 .....
21:06:51.203481 IP 192.168.1.1> 192.168.1.12: ICMP time exceeded in-transit,
 length 60
 0x0000: 45c0 0050 a201 0000 4001 548e c0a8 0101 E..P....@. T....
 0x0010: c0a8 010c 0b00 09fe 0000 0000 4500 0034 .....E..4
 0x0020: b767 0000 0111 ed85 c0a8 010c aa95 a882 .g.....
 0x0030: b766 829b 0020 b0df 0000 0000 0000 0000 .f.....
 0x0040: 0000 0000 0000 0000 0000 0000 0000 0000 .....
21:06:51.203691 IP 192.168.1.12.46950> 170.149.168.130.33436: UDP, length 24
 0x0000: 4500 0034 b768 0000 0111 ed84 c0a8 010c E..4.h.....
 0x0010: aa95 a882 b766 829c 0020 b0de 0000 0000 .....f.....
 0x0020: 0000 0000 0000 0000 0000 0000 0000 0000 .....
 0x0030: 0000 0000 .....
21:06:51.204191 IP 192.168.1.1> 192.168.1.12: ICMP time exceeded in-transit,
 length 60
 0x0000: 45c0 0050 a202 0000 4001 548d c0a8 0101 E..P....@. T....
 0x0010: c0a8 010c 0b00 09fe 0000 0000 4500 0034 .....E..4
 0x0020: b768 0000 0111 ed84 c0a8 010c aa95 a882 .h.....
 0x0030: b766 829c 0020 b0de 0000 0000 0000 0000 .f.....
 0x0040: 0000 0000 0000 0000 0000 0000 0000 0000 .....
LANE
```

Обратите внимание на то, что tracert отсылает сообщения UDP, запускающиеся в порте 33435 и постепенно увеличивающие номер порта на один с каждым дополнительным сообщением. Номер порта постепенно увеличивается для восстановления порядка, в котором отправлены пакеты. Обратите внимание на то, что пакет ICMP от смещения 0x001C вперед содержит исходный пакет UDP. Как отмечено выше, сообщения ICMP должны использовать много различных методов

для обеспечения контекста – сообщения об ошибках, такие как «TTL превышен», включают заголовок IP и первые 8 байт исходного пакета. Оно включает и номер исходного порта UDP. *traceroute* показывает сообщение ICMP в порядке номеров портов для определения порядка, в котором были отправлены эти сообщения.

В то время как *traceroute* использует UDP по умолчанию, тот же метод может использоваться TCP или любым другим протоколом, где существует управляемое значение (такое как количество временных портов) в первых восьми байтах полезной нагрузки IP.

ping и *traceroute* более полезны, если можно использовать их из различных местоположений. С этой целью ряд поставщиков интернет-услуг и другие организации обеспечивают *сервер-зеркала*. Зеркало является публично доступным (обычно через веб) интерфейсом к любому из многих общих интернет-приложений. Большинство зеркал управляемы NOCs или ISPs и обеспечивают доступ к многочисленным маршрутизаторам. Нет никакого стандарта для реализации, и различные зеркала предоставляют различные услуги. Исчерпывающий перечень доступен на www.traceroute.org.

Интеллект IP: геолокация и демография

Ряд баз данных и разведывательных служб предоставляют дополнительную информацию об IP-адресе. Этот тип расширенных данных включает информацию о владельце, геолокации и демографическую информацию.

Важно отличать эти расширенные данные от информации, такой как автономная система, доменное имя и whois-данные. Последний сервис необходим для функционирования сети и поддерживается интернет-организациями, связанными с ICANN. Геолокация, демографические данные и данные о владении являются интеллектуальными продуктами. Компании, которые хранят эти данные, используют множество механизмов, включая сетевое сканирование, так же как и другие старые добрые технологии для создания этих продуктов. Это приводит к нескольким важным свойствам:

- интеллект обновляется медленно, тогда как имена DNS могут измениться очень быстро. Требуются дополнительные проверки, чтобы узнать, что 128.2.11.214 больше не участвует в продаже автозапчастей, а размещает теперь вредоносное программное обеспечение;
- всегда существует определенная степень приближения. Как показывает опыт, данные разведки становятся менее точными, поскольку вы копаете вглубь для более детальной информации. Информация о стране обычно хороша, но я умеренно скептически отношусь к информации о городе за пределами США и Западной Европы, и я никогда не доверяю физическому местоположению;
- вы получаете то, за что платите. Компании, которые производят эти данные, получают клиентов, которым они нужны. Большинство компаний начали обеспечивать демографические данные для больших веб-сайтов, и все еще актуальна задача определить пределы на количество запросов, которые можно выполнить на стоимость лицензии. Вы платите за правильность, и вы платите за точность. Существуют свободные интеллектуальные базы данных, но если вы хотите получить более прекрасную деталь, чем коды страны, подготовьтесь открыть ваш кошелек.

Наиболее часто используемой ссылкой с открытым исходным кодом является **GeoIP MaxMind** <http://bit.ly/geo-ip>, который обеспечивает множество баз данных для городов, стран, областей, организаций, ISP и сетей. Они также предоставляют бесплатные услуги в форме «облегченных» баз данных для нахождения города и страны. Все их продукты являются загружаемыми базами данных и регулярно обновляются. MaxMind предоставлял эту услугу в течение многих лет наряду с рядом API в Python и других языках, которые используют для доступа к базам данных.

Расширенную информацию включают **Neustar** <http://www.neustar.biz/> и **Digital Element** <http://www.digitalenvoy.com/> агента Digital. Оба ресурса обеспечивают более точное измерение, а также дополнительные демографические данные, такие как *плотность городского населения (MSA)* (смежные области высокой плотности населения, используемые правительством для статистического анализа) и коды *Североамериканской промышленной системы классификации (NAICS)* (числовой идентификатор, сродни десятичному числу Dewey для бизнеса). Однако эти сервисы не очень дешевы.

DNS

В справедливом мире каждый IP-адрес имел бы единственное имя DNS, и процедура нахождения имени DNS, связанного с IP-адресом, была бы простым обращением к базе данных. Но этот мир не справедлив.

DNS является связующим звеном, которое делает интернет удобным для людей. Как один из наиболее старых сервисов, заставляющих интернет работать, DNS перекрывает несколько разных сервисов (особенно почту). С этой точки зрения, DNS – это распределенная база данных, которая предоставляет информацию о соответствии для многих различных отношений, в особенности соотношение между DNS и IP-адресом, имени DNS к другому имени DNS, адреса электронной почты к почтовому серверу и т. д.

Структура имени DNS

Доменное имя состоит из иерархической последовательности символов, разделенных точками, такими как *www.oreilly.com*. Доменные имена становятся более общими, если вы читаете справа налево, заканчиваясь в корневом домене (корневой домен – это почти всегда подразумевается). Доменные имена действительно имеют пределы. Общая длина имени не может превысить 253 символов, и отдельные его части ограничены 64¹.

Исторически наименования в доменах были лимитированы ограниченностью подмножества символов ASCII для имени. С 2009 г. стало возможным получить *национальные доменные имена*, которые закодированы с помощью символьных систем, таких как китайский язык, греческий язык и т. д.² Механически предел 253 символа на имя все еще поддерживается, хотя кодирование более сложное.

¹ В различных доменных зонах действуют разные ограничения на длину доменного имени. – *Прим. ред.*

² Национальные доменные имена повышают риск гомографических атак, таких как создание доменного имени, которое похоже на *oreilly.com*, но использует О на кириллице.

NICs и выделение доменного имени

Полномочия для выделения доменных имен, как и с IP-адресами, начинаются в ICANN. ICANN управляет корневой зоной и выдает *домены верхнего уровня* (TLDs), которые лежат чуть ниже (на *втором уровне*) корневого домена. Как с адресами, каждый TLD имеет руководящие полномочия *сетевого информационного центра (NIC)*. Каждый NIC устанавливает различные политики для выделения имени – например, любой может получить адрес *.com*, но только аккредитованные учебные заведения имеют право на адрес *.edu*. В зависимости от политики NIC регистрационные полномочия могут быть далее делегированы одному или нескольким *регистраторам*.

IANA определяет четыре категории TLD. Самая старая категория – *универсальный (generic) TLDs* (gTLD); это агностики – домены верхнего уровня, такие как *.com* или *.edu*. Далее идут gTLDs – *инфраструктурный TLD* с одним доменом, *.arpa*, используемый для сервиса DNS lookup. *Код страны TLD* (ccTLD) является доменом верхнего уровня с двумя буквами для стран (например, *.ie* для Ирландии). Новый набор *национальных TLD* (IDN ccTLD) позволяет использовать нелатинские символы.

Каждый TLD имеет свой собственный NIC. Таблица 8-3 ниже отображает NICs для ряда часто используемых TLDs.

Таблица 8-3. Известные NICs

TLD	NIC	URL
<i>.org</i>	Public Interest Registry	www.pir.org
<i>.biz</i>	Neustar	www.neustar.biz/enterprise/domain-name-registry
<i>.com</i>	VeriSign	www.verisigninc.com/
<i>.net</i>	VeriSign	www.verisigninc.com/
<i>.edu</i>	Educause	www.educause.ed
<i>.int</i>	IANA	www.iana.org/domains/int
<i>.fr</i>	AFNIC	www.afnic.fr/
<i>.uk</i>	Nominet	www.nominet.org.uk
<i>.ru</i>	Coordination Center for RU TLD	www.cctld.ru/en/
<i>.cn</i>	CNNIC	www1.cnnic.cn/
<i>.kr</i>	KISA	www.kisa.or.kr/

Эта иерархия серверов также служит для определения, какие серверы являются *авторитарными*. Реестры верхнего уровня наделяют субрегистраторов полномочиями путем предоставления им зон. Каждая зона имеет один главный (master) сервер, который поддерживает свои доменные имена и является авторитарным, но зоны могут быть вложены, чтобы дать возможность быть авторитарными различным серверам.

Направление DNS-запроса с использованием dig

Основной инструмент DNS-запроса – это *модуль сбора информации о доменах (dig)*, командная строка клиента DNS, которая позволяет вам запросить DNS для всех главных (major) записей. Начните путем проведения простого dig-запроса:

```
$ dig oreilly.com
dig oreilly.com

; <<>> DiG 9.8.3-P1 <<>> oreilly.com
;; global options: +cmd
;; Got answer:
;; ->> HEADER <<-opcode: QUERY, status: NOERROR, ID: 29081
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 0

; QUESTION SECTION:
;oreilly.com.          IN      A

;; ANSWER SECTION:
oreilly.com.          383     IN      A      208.201.239.101
oreilly.com.          383     IN      A      208.201.239.100

;; Query time: 10 msec
;; SERVER: 192.168.1.1#53 (192.168.1.1)
;; WHEN: Sat Jul 20 19:11:17 2013
;; MSG SIZE rcvd: 61
$ dig +short oreilly.com
208.201.239.101
208.201.239.100
```

Мы рассмотрим параметры dig и затем структуру ответа DNS. Как видно из предыдущего примера, основная команда dig предоставляет обширную информацию о запросе, начиная со списка вызванных опций, затем заголовок DNS, и потом несколько секций, соответствующих запросу. Обратите внимание на поля QUERY, ANSWER, AUTHORITY и ADDITIONAL в строке заголовка, и как они соответствуют строкам в соответствующих разделах. Поскольку этот домен не возвратил записей AUTHORITY или ADDITIONAL, ни один не показан на выходе. Запрос сопровождается блоком статистики о запросе: сервер, затраченное время и размер сообщения.

dig обеспечивает огромное количество выходных опций; предыдущий пример показал значения по умолчанию. Отдельные разделы этого ответа могут быть включены с помощью +nocomments (который уничтожает все комментарии, начинающиеся с двойной точки с запятой), +nostats (уничтожение статистики в конце) и +noquestion и +noanswer (для устранения ответов DNS). +short просто удалит весь хлам и покажет ответы.

dig – просто клиент DNS, таким образом, большая часть информации – с самого сервера DNS. dig позволяет сделать запросы к различным серверам при помощи @ в командной строке. Например:

```
$ # 8.8.8.8 is Google's public DNS server; let's query a CDN using it,
$ dig @8.8.8.8 www.foxnews.com
; <<>> DiG 9.8.3-P1 <<>> @8.8.8.8 www.foxnews.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->> HEADER <<- opcode: QUERY, status: NOERROR, id: 18702
```



```
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.foxnews.com.          IN      A

;; ANSWER SECTION:
www.foxnews.com.          282     IN      CNAME   www.foxnews.com.edgesuite.net.
www.foxnews.com.edgesuite.net. 21582 IN      CNAME   a20.g.akamai.net.
a20.g.akamai.net.         2       IN      A       204.245.190.42
a20.g.akamai.net.         2       IN      A       204.245.190.8

;; Query time: 141 msec
;; SERVER: 8.8.8.8#53 (8.8.8.8)
;; WHEN: Sat Jul 20 19:48:01 2013
;; MSG SIZE rcvd: 135
$ # Query using my default server
$ dig www.foxnews.com

; <<>> DiG, 9.8.3-P1 <<>> www.foxnews.com
;; global options: +cmd
;; Got answer:
;; ->> HEADER <<-opcode: QUERY, status: NOERROR, id: 47098
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.foxnews.com.      INA

;; ANSWER SECTION:
www.foxnews.com.          189     IN      CNAME   www.foxnews.com.edgesuite.net.
www.foxnews.com.edgesuite.net. 9699 IN      CNAME   a20.g.akamai.net.
a20.g.akamai.net.         9       IN      A       23.66.230.160
a20.g.akamai.net.         9       IN      A       23.66.230.106

;; Query time: 97 msec
;; SERVER: 192.168.1.1#53(192.168.1.1)
;; WHEN: Sat Jul 20 19:48:09 2013
;; MSG SIZE rcvd: 135
```



Как видите, запрашивая CDN-модерируемый сайт (Fox News использует Akamai), мы получим результаты в радикально различных IP-адресах для одного и того же имени. CDNs управляют DNS, чтобы гарантировать, что кеши публичных данных географически близки к действительности. Если вы не укажете сервер, использующий @, dig будет по умолчанию использовать любой сервер, на котором настроена система (например, в системах Unix это поддерживается в каталоге `/etc/resolv.conf`).

CDN – это кеширующая сеть, которая делает интернет жизнеспособным. До веба пользователь смог бы посетить четыре-пять хостов за час; после веба запрос к веб-странице может запустить сто различных запросов HTTP. Большинство этих запросов перенаправляются через DNS к кеширующимся серверам, которые расположены в географической близости.

CDNs добавляют некоторую неопределенность к веб-анализу, потому что единственный сервер CDN может содержать многочисленные веб-сайты. Как только адрес идентифицирован как CDN, выяснить точно, что это, достаточно затруднительно.

Теперь давайте посмотрим на данные DNS. DNS является федеративной системой баз данных. Таким образом, запросы идут сначала в локальный сервер DNS, который отправляет ответ, если у него есть ответ на запрос. Если сервер не имеет

информации, он использует иерархическую структуру имени для выяснения, куда отправить запрос, ожидает ответа и передает ответ обратно. DNS поддерживает много различных запросов, которые называются *ресурсными записями* (RRs), и параметры, отправленные при запросе, определяют ресурсную запись, которую требуют, а также параметры на запросы дополнительных серверов. Значения с As или CNAMEs в строках выше являются ресурсными записями.

Обратите внимание на то, что заголовок перечисляет восемь полей:

opcode (*код операции*)

Это поле было предназначено для определения многих различных действий, таких как запросы, обратные запросы и состояние сервера. На практике это всегда должно быть установлено в запросе. Существует ряд других кодов операции, но они используются для передачи информации между серверами.

status (*состояние*)

Состояние ответа. Чаще всего появляются три сообщения: NOERROR, NXDOMAIN и SERVFAIL. NOERROR указывает, что запрос был успешен, NXDOMAIN указывает, что никакой домен не был доступен, и SERVFAIL указывает, что авторитарные серверы для домена были недоступны.

id (*идентификатор*)

Идентификатор сообщения. DNS является модулируемым UDP протоколом и использует идентификаторы сообщений для отслеживания запросов и ответов.

flags (*флаги*)

Они предоставляют информацию об ответе и включают qf (устанавливается параметр high – высокий приоритет для ответа), aa (устанавливается как high, когда ответ идет с авторитарного сервера), ra (1-битовое поле, которое означает «рекурсия возможна» (recursion available). Этот бит устанавливается в 1 в отклике, если сервер поддерживает рекурсию) и rd (1-битовое поле, которое означает «требуется рекурсия» (recursion desired). Бит может быть установлен в запросе и затем возвращен в отклике. Этот флаг требует от DNS сервера обработать этот запрос самому (т. е. сервер должен сам определить требуемый IP-адрес, а не возвращать адрес другого DNS-сервера), что называется рекурсивным запросом (recursive query). Если этот бит не установлен и запрашиваемый сервер DNS не имеет авторитетного ответа, запрашиваемый сервер возвратит список других серверов DNS, к которым необходимо обратиться, чтобы получить ответ. Это называется повторяющимся запросом (iterative query)).

Оставшиеся четыре поля относятся к категориям записей, отправляемых в ответ. Вот они:

QUERY (*ЗАПРОС*)

Эта запись является просто копией исходного запроса; в этом случае вы видите, что запрос отражен в том, что **qtype** называется разделом QUESTION.

ANSWER (*ОТВЕТ*)

Содержит ответ.

AUTHORITY (*ПОЛНОМОЧИЯ*)

Зарезервированный для записей, которые идентифицируют другие серверы.

ADDITIONAL (*ДОПОЛНИТЕЛЬНЫЙ*)

Предоставляет дополнительную информацию, такую как ожидаемые ответы на будущие запросы.

Дополнительной информацией является в значительной степени функция администраторов сервера имен. Типичный пример ее использования – поиск имени для почтового сервера, идентифицируемого запросом MX:

```
$ dig +nostats +nocmd mx cmu.edu
;; Got answer:
;; -> HEADER <<- opcode: QUERY, status: NOERROR, id: 30852
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 3
;; QUESTION SECTION:
; cmu.edu.                IN      MX

;; ANSWER SECTION:
cmu.edu.                20051   IN      MX      10    CMU-MX-02.ANDREW.cmu.edu.
cmu.edu.                20051   IN      MX      10    CMU-MX-03.ANDREW.cmu.edu.
cmu.edu.                20051   IN      MX      10    CMU-MX-04.ANDREW.cmu.edu.
cmu.edu.                20051   IN      MX      10    CMU-MX-01.ANDREW.cmu.edu.

;; ADDITIONAL SECTION:
CMU-MX-03.ANDREW.cmu.edu. 20412   IN      A       128.2.155.68
CMU-MX-01.ANDREW.cmu.edu. 20232   IN      A       128.2.11.59
CMU-MX-02.ANDREW.cmu.edu. 20051   IN      A       128.2.11.60
```

Теперь давайте обсудим, что на самом деле означают эти ресурсные записи. DNS имеет вверх 20 ресурсных записей для различных функций. Главные:

A

Содержит в себе IP-адрес сервера. При отсутствии A-записей на домене доступ по протоколу HTTP к нему невозможен.

AAAA

Аналогична A, однако содержит в себе IP-адрес для домена, но для IPv6.

CNAME

Связывает два имени, каноническое имя и псевдоним (одноуровневая переадресация).

MX

Возвращает mailserver для домена.

PTR

Соответствие адреса имени – обратное соответствие для A и AAAA.

TXT

Содержит произвольные текстовые данные.

NS

Описывает сервер имен для адреса.

SOA

Предоставляет информацию об авторитативном сервере имен для адреса.

dig запускает все ресурсные записи с теми же четырьмя значениями: имя, время жизни (TTL), класс и идентификатор для RR (например: cmu.edu, 20051, IN, MX). Имя передано с запросом. TTL указывает, как долго (в секундах) значению имени можно доверять; DNS полагается в большой степени на кеширование, и TTL предоставляет инструкции относительно того, когда обновить кеш. Класс

почти неизменно будет IN (интернет); другие имена классов возможны, но вне области исследования этой книги.

A и AAAA обеспечивают основную функциональность DNS: они связывают запрошенное имя с IP-адресом. Записи A обеспечивают адреса IPv4, а записи AAAA – адреса IPv6. По умолчанию dig запрашивает A записи, в то время как другие типы записи определяются путем добавления их к командной строке, как показано здесь:

```
$ dig +nocomment +noquestion +nostats +nocmd www.google.com
www.google.com. 55 IN A 74.125.228.81
www.google.com. 55 IN A 74.125.228.83
www.google.com. 55 IN A 74.125.228.84
www.google.com. 55 IN A 74.125.228.80
www.google.com. 55 IN A 74.125.228.82
$ dig +nocomment +noquestion +nostats +nocmd aaaa www.google.com
www.google.com. 18 IN AAAA 2607:f8b0:4004:802::1014
```

Обратите внимание на то, что *round robin DNS allocation* (от англ. round-robin – циклический) – алгоритм распределения нагрузки распределённой вычислительной системы методом перебора и упорядочения её элементов по круговому циклу. В *round robin DNS* то же доменное имя присвоено нескольким IP-адресам. Следовательно, когда запрос выбирает IP-адрес для контакта с именем, он эффективно выбирает имя случайным образом из множества мишеней. Алгоритм *round robin DNS* является одной из возможных точек взломов DNS, он делает обратный поиск (IP-адресов по именам) невероятно раздражающим.

Отметьте также короткие значения TTL. Если конкретный сервер Google падает (недоступен), TTL гарантирует, что через 55 секунд у пользователя есть хорошие шансы контакта с другим сервером.

Каноническое имя (CNAME) записей используется для соединения псевдонима и канонического имени. Например, рассмотрим поиски для *www.oreilly.com*:

```
dig +nocomment +noquestion +nostats +nocmd www.oreilly.com
www.oreilly.com. 3563 IN CNAME oreilly.com.
oreilly.com. 506 IN A 208.201.239.101
oreilly.com. 506 IN A 208.201.239.100
```

Как видно, имя *www.oreilly.com* на самом деле указывает на *oreilly.com*. *www.oreilly.com* не имеет IP-адреса; он указывает на *oreilly.com*, и это имя имеет IP-адрес. Канонические имена используются для ярлыков (как в предыдущем примере), а также для управления распространением контента. Пример с использованием Fox News показал, как Akamai сначала изменяет все сайты Fox News на его собственные сетевые имена с помощью CNAME.

DNS обеспечивает функции поиска для электронной почты при посредстве записи почтовой маршрутизации (MX). Записи MX фиксируют адреса почтовых серверов для конкретного домена. Например, если я решаю отправить по почте *jbro@andrew.cmu.edu*, я могу найти почтовый сервер, для того чтобы сделать это путем поиска записей MX для *cmu.edu*:

```
$dig +noquestion +nostats +nocmd mx cmu.edu
;; Got answer:
;; ->> HEADER <<- opcode: QUERY, status: NOERROR, id: 49880
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 2
;; ANSWER SECTION:
```

```

cmu.edu. 21560 INMX10CMU-MX-03.ANDREW.cmu.edu.
cmu.edu. 21560 INMX10CMU-MX-04.ANDREW.cmu.edu.
cmu.edu. 21560 INMX10CMU-MX-01.ANDREW.cmu.edu.
cmu.edu. 21560 INMX10CMU-MX-02.ANDREW.cmu.edu.

```

```

;; ADDITIONAL SECTION:
CMU-MX-01.ANDREW.cmu.edu. 21519 IN A 128.2.11.59
CMU-MX-02.ANDREW.cmu.edu. 21159 IN A 128.2.11.60

```

Записи MX включают имя сервера (такое как CMU-MX-03.ANDREW.cmu.edu), а также приоритетное значение для почтового сервера. Взвешенное значение используется для выбора почтового сервера: почтовые клиенты должны выбирать почтовые серверы в порядке возрастающего приоритета (т. е. 1 должен быть выбран прежде 10).

Примечательны в этом примере А записи, помещенные в дополнительный раздел. Эти записи разрешают адреса CMU-MX-02 и CMU-MX-01. Это – сознательное решение администраторов DNS с сервера CMU включить данную информацию и уменьшить количество сделанных поисков.

Записи NS используются для нахождения авторитативного сервера имен для зоны. Например, для O'Reilly медиа:

```

$ dig +nostat ns oreilly.com

; <<> DiG, 9.8.3-P1 <<> +nostat ns oreilly.com
;; global options: +cmd
;; Got answer:
;; -> HEADER <<- opcode: QUERY, status: NOERROR, id: 32310
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;oreilly.com.                IN      NS

;; ANSWER SECTION:
oreilly.com.                 3600    IN      NS      nsautha.oreilly.com.
oreilly.com.                 3600    IN      NS      nsauthb.oreilly.com.

```

Теперь снова посмотрите на запись NS для сайта, управляемого CDN, такого как Fox News:

```

$ dig +nostat ns foxnews.com

; <<> DiG, 9.8.3-P1 <<> +nostat ns foxnews.com
;; global options: +cmd
;; Got answer:
;; -> HEADER <<- opcode: QUERY, status: NOERROR, ID: 38538
;; flags: qr rd ra; QUERY: 1, ANSWER: 8, AUTHORITY: 0, ADDITIONAL: 5

;; QUESTION SECTION:
;foxnews.com.                IN      NS

;; ANSWER SECTION:
foxnews.com.                 300     IN      NS      usc2.akam.net.
foxnews.com.                 300     IN      NS      ns1.chi.foxnews.com.
foxnews.com.                 300     IN      NS      ns1-253.akam.net.
foxnews.com.                 300     IN      NS      dns.tpa.foxnews.com.
foxnews.com.                 300     IN      NS      usw1.akam.net.
foxnews.com.                 300     IN      NS      usw3.akam.net.
foxnews.com.                 300     IN      NS      asia3.akam.net.
foxnews.com.                 300     IN      NS      usc4.akam.net.

;; ADDITIONAL SECTION:

```

usw1.akam.net.	28264	IN	A	96.17.144.195
usw3.akam.net.	50954	IN	A	69.31.59.199
asia3.akam.net.	28264	IN	A	222.122.64.134
usc4.akam.net.	28264	IN	A	96.6.112.196
usc2.akam.net.	88188	IN	A	69.31.59.199

Обратите внимание на то, что в этом случае авторитативные серверы имен в основном принадлежат *akam.net* (Akamai). Fox News размещен на CDN Akamai, и Akamai изменяет названия хостов по мере необходимости для повышения производительности.

Записи SOA содержат итоговую информацию об авторитативном сервере для домена. Эти записи обычно встречаются во время неудачных поисков. Когда адрес не найден, вместо него возвращается информация SOA для сервера этой зоны.

```
dig @8.8.4.4 +multiline +nostat zlkoriomgk.com
; <<> DiG, 9.8.3-P1 <<> @8.8.4.4 +multiline +nostat zlkoriomgk.com
;(1 server found)
;; global options: +cmd
;; Got answer:
;; -> HEADER <<- opcode: QUERY, status: NXDOMAIN,id: 11857
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 0
;; QUESTION SECTION:
; zlkoriomgk.com.          IN      A
;; AUTHORITY SECTION:
com.                      899 IN SOA a.gtld-servers.net. nstld.verisign-grs.com. (
                           1374373035; serial
                           1800 ; refresh (30 minuiies)
                           900  ; retry (15 minuiies)
                           604800 ; expire (1 week)
                           86400 ; minimum (1 день)
                           )
```

Поле SOA начинается с исходного хоста и сопровождается контактным электронным адресом. После этого адреса прибывает порядковый номер, который указывает, сколько раз исходный файл был изменен, и затем статистика тайм-аута. Обратите внимание на `+multiline option for dig`; она обеспечит многокомпонентную строку, более человекочитаемый вывод для записи SOA.

Поле TXT является подстановочным полем, используемым для любого текстового вывода, который администратор сервера испытывает желание передавать. Например, Google передает строки для управления Google Apps:

```
$ dig +short txt google.com
"v=spf1 include:_spf.google.com ip4:216.73.93.70/31 ip4:216.73.93.72/31 ~all"
```

Обратный поиск DNS

Обратный поиск является процессом восстановления имени DNS по IP-адресу. Например, если я хочу узнать, кто владеет 208.201.139.101, я делаю это, используя `dig -x`:

```
$ dig +nostat -x 208.201.139.101
; <<> DiG, 9.8.3-P1 <<> +nostat -x 208.201.139.101
;; global options: +cmd
;; Got answer:
;; -> HEADER <<- opcode: QUERY, status: NOERROR, id: 7519
```

```
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;; QUESTION SECTION:
;101.139.201.208.in-addr.arpa. IN PTR
;; ANSWER SECTION:
101.139.201.208.in-addr.arpa. 21600 IN PTR host-d101.studley.com.
```



Обратные поиски являются запросами для получения имен DNS из IP-адресов. Обратите внимание на то, что раздел вопроса не запрашивает IP-адрес, 208.201.139.101, а запрашивает 101.139.201.208.in-addr.arpa, который перечисляет поля IP-адреса в обратном порядке. Когда DNS делает обратный поиск, он создает специальное доменное имя для запросов в in - addr . arpa TLD¹. Строкой цифр и периодов, используемых для обратного поиска, является исходный инвертированный IP-адрес. Потому что имена DNS и IP-адреса определены противоречащим способом. Имя DNS становится более точно определенным (от TLD до домена к отдельному хосту) путем чтения справа налево, в то время как IP-адреса более точно определены, читая слева направо.

Обратные поиски являются клуджем (нелепое, неуклюжее, но удивительно эффективное решение проблемы). Обратите внимание на то, что записью, возвращенной в ответе, является указатель запись записи (PTR). Записи PTR автоматически не создаются из канонической A записи, но вместо этого регистрируются отдельно через NIC. Что более важно, нет никакого требования, чтобы запись PTR быть зарегистрированной, и отношение между именами и IP-адресами в лучшем случае слабое.

Например, рассмотрим CDN. Если я ищу один из IP-адресов Fox News, такой как 23.66.230.66, я получаю это:

```
dig +nostat +nocmd-x 23.66.230.66
;; Got answer:
;; -> HEADER <<- opcode: QUERY, status: NOERROR, id: 56379
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;; QUESTION SECTION:
;66.230.66.23.in-addr.arpa. IN PTR
;; ANSWER SECTION:
66.230.66.23.in-addr.arpa. 290 IN
PTR a23-66-230-66.deploy.static.akamaitechnologies.com.
```



CDN становится информационным тупиком; ответ от обратного поиска не имеет никакого значимого отношения к именам в исходном запросе.

Как правило, сведения DNS лучше всего собирать во время исходного запроса. Причиной этого является неопределенность обратных поисков. Однако, даже если обратные поисковые запросы работали отлично, злоумышленники часто используют очень недолговечные имена. Где это возможно, запишите используемые доменные имена, (например, URL-адрес в журналах HTTP), чтобы не пытаться реконструировать их постфактум.

¹ *arpa* официально обозначает область Address и Routing Parameter. Это имя является бэкронимом, потому что сокращение первоначально означало *управление проектами перспективных исследований*, агентство DoD, которое первоначально финансировало разработку интернета. *arpa* официально обозначает область Address и Routing Parameter.

Использование whois для нахождения владельца

В то время как DNS может предоставить информацию об имени домена, суть информации о владельцах обеспечивается whois. Это – федеративный протокол (RFC 3921), который перечисляет предполагаемых владельцев имен DNS. Стандарт запроса whois на домен возвратит информацию о владельце и контактную информацию для домена, как показано в примере 8-2.

Пример 8-2. Запрос whois для oreilly.com

```
$whois oreilly.com
```

```
<шаблон>
```

```
Доменное имя: OREILLY.COM
Регистратор: GODADDY.COM, LLC
Сервер Whois: whois.godaddy.com
Направление URL: http://registrar.godaddy.com
Сервер имен: NSAUTHA.OREILLY.COM
Сервер имен: NSAUTHB.OREILLY.COM
Состояние: clientDeleteProhibited
Состояние: clientRenewProhibited
Состояние: clientTransferProhibited
Состояние: clientUpdateProhibited
Дата обновления: 26 мая 2012
Дата создания: 27 мая 1997
Дата истечения срока: 26 мая 2013
```

```
<больше шаблона>
```

```
Зарегистрированный через: GoDaddy.com, LLC (http://www.godaddy.com)
```

```
Доменное имя: OREILLY.COM
```

```
Созданный на: 26 мая 97
```

```
Истекает на: 25 мая 13
```

```
Последнее обновление на: 26 мая 12
```

```
Владелец домена:
```

```
O'Reilly Media, Inc.
```

```
1005 Gravenstein Highway North
```

```
Sebastopol, California 95472
```

```
United States
```

```
Административный контакт:
```

```
nic-ac@oreilly.com
```

```
O'Reilly Media, Inc.
```

```
1005 Gravenstein Highway North
```

```
Sebastopol, California 95472
```

```
United States
```

```
+1.7078277000 Факс - +1.7078290104
```

```
Технический контакт:
```

```
nic-tc@oreilly.com
```

```
O'Reilly Media, Inc.
```

```
1005 Gravenstein Highway North
```

```
Sebastopol, California 95472
```

```
United States
```

```
+1.7078277000 Факс - +1.7078290104
```

```
Доменные серверы в перечисленном порядке:
```

```
NSAUTHA.OREILLY.COM
```

```
NSAUTHB.OREILLY.COM
```



Можно заметить, что запись whois для домена возвращает огромное количество шаблонов информации. Вы также увидите, что возвращенная информация не имеет никакого конкретного фиксированного формата – whois-информацией является электронный эквивалент учетной карточки 3×5. В зависимости от того, кто владеет карточкой и как он решает администрировать ее, можно получить номера телефона и биографии или совсем ничего.

Хороший способ почувствовать разницу в регистрации – взглянуть на регистрационные файлы для разных стран. Нет никакой центральной whois-базы данных – вместо этого в зависимости от домена высшего уровня whois-информация может сохраняться любым из ряда whois-серверов. Например, российские whois-данные (.ru-домен) поддерживаются *whois.ripn.net*, французские – *lvs-vip.nic.fr*, и бразильский – *registro.br*. К счастью, хорошие люди на *whois-servers.net* обеспечивают псевдонимы для каждой страны и TLD, и в зависимости от вашей whois-реализации информация уже может сформироваться в пригодный для вас файл.

Как минимум, любая whois-реализация обеспечит способность определить сервер поиска с помощью параметра *-h*. Таким образом, *whois-h ru.tld-servers.net* идентично *whois-h whois.ripn.net*. Несколько whois-реализаций предлагает специфичную для страны *-c* опцию, делая *whois-c RU* идентичной обеим из предыдущих примеров.

В дополнение к предоставлению информации о доменных именах whois также полезен для обеспечения информацией о распределении адресов и владельцев. Если whois вызван IP-адресом, а не именем, как в примере 8-3, то будет предоставлена информация об организации, которая владеет этим адресом, часто в форме netblock. Например, если я ищу whois-информацию для Voila, французская поисковая система, то получу различную информацию в зависимости от того, смотрю ли я на RIPE (европейский реестр верхнего уровня) или французский NIC. RIPE, который более информативен; французский NIC дает значительно меньше информации.

Пример 8-3. Использование whois с IP-адресом

```
$dig +short voila.fr
193.252.148.80
% $ whois-h whois.ripe.net 193.252.148.80%
% Это – сервис запросов к базе данных RIPE.
% Объекты хранятся в формате RPSL.
%
% База данных RIPE подчиняется следующим правилам и условиям.
% См. http://www.ripe.net/db/support/db-terms-conditions.pdf
%
% Примечание: этот вывод был отфильтрован.
% чтобы получить вывод для обновления базы данных, используйте флаг "-B".
%
% Информация связана с '193.252.148.0 - 193.252.148.255'
%
% abuse для '193.252.148.0 - 193.252.148.255' является 'gestionip.ft@orange.com'
inetnum:      193.252.148.0 - 193.252.148.255
netname:      ORANGE-PORTAILS
descr:        France Telecom
descr:        интернет-порталы для разных сервисов
страна:       FR
```



```

админ-с:      WPTR1-RIPE
тех-с:        WPTR1-RIPE
состояние:    ПРИСВОЕННЫЙ РА
комментарии:  по взлому, спаму или проблемам безопасности
комментарии:  отправлять почту в abuse@orange.fr
mnt-by:       FT-BRX
источник:     RIPE # фильтрованный
роль:         Wanadoo Portails техническая роль
адрес:        France Telecom - OPF/Portail/DOP/Hebex
адрес:        48, rue Camille Desmoulins
адрес:        92791 Issy Les Moulineaux Cedex 9
адрес:        FR
телефон:      +33 1 58886500
номер факса:  +33 1 58886680
админ-с:      WPTR1- RIPE
тех-с:        WPTR1- RIPE
nic-hdl:      WPTR1- RIPE
mnt-by:       FT-BRX
источник:     RIPE # фильтрованный

```

% Этот запрос обслужен версией 1.60.2(WHOIS4) службы запроса базы данных RIPE

```
$ whois-h fr.whois-servers.net 195.152.120.129
```

%%

%% Это - AFNIC Whois сервер.

%%

%% полный формат даты : DD/MM/YYYY

%%

%% короткий формат даты : DD/MM

%% версия : FRNIC-2.5

%%

%% Права ограничены авторским правом.

%% См. http://www.afnic.fr/afnic/web/mentions-legales-whois_en

%%

%% Используйте '-h'-опцию, чтобы получить больше информации об этом сервисе.

%%

```
%% [96.255.98.126 REQUEST]>> 195.152.120.129
```

%%

```
%% RL, сетевой [#####] - RL IP [#####.]
```

Вы обнаружите, что ситуация инвертирована с азиатской информацией. ARNIC whois часто довольно редок, но whois-записи на уровне страны обычно информативны.

Информация о Whois особенно полезна, когда вы не можете вытащить много полезных данных из реверсного поиска DNS. Если вы не можете найти определенное доменное имя, можно использовать whois, чтобы, по крайней мере, найти блок адресов, на которых размещен домен.

ДОПОЛНИТЕЛЬНЫЕ ССЫЛОЧНЫЕ ИНСТРУМЕНТЫ

В дополнение к информации о сети и маршрутизации существует ряд общедоступных сайтов, содержащих информацию об использовании, атаках и репутации важных IP-адресов. Эти сайты являются обычно небольшими, работают на добровольных началах и имеют достаточное количество обращений к ним.

DNSBLs

Черный список DNS (DNSBL) является основанной на DNS базой данных IP-адресов, используемой прежде всего в качестве метода борьбы против спама. Первые DNSBLs были на самом деле реализованы с помощью BGP и были предназначены для активного обхода маршрутов, связанных с IP-адресами спаммера. DNSBLs вместо этого модерируются DNS, они служат репутационными базами данных для почтового программного обеспечения. Например, агент передачи почты может консультироваться с DNSBL, чтобы определить, является ли передающий IP спаммером, и реагирует соответственно.

DNSBLs работают путем обеспечения функциональности стиля обратного поиска на их серверах DNS. Например, я могу искать эхо-адрес на DNSBL, используя dig:

```
$ dig 2.0.0.127.sbl.spamhaus.org
; <<> DiG, 9.8.3-P1 <<> 2.0.0.127.sbl.spamhaus.org
;; global options: +cmd
;; Got answer:
;; -> HEADER <<- opcode: QUERY, status: NOERROR, id: 45434
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
;; QUESTION SECTION:
;2.0.0.127.sbl.spamhaus.org. IN
;; ANSWER SECTION:
2.0.0.127.sbl.spamhaus.org. 300 IN A 127.0.0.2
;; Query time: 39 мс
;; SERVER: 192.168.1.1#53 (192.168.1.1)
;; WHEN: Sun Jul 28 15:10:23 2013
;; MSG SIZE rcvd: 60
```

Адрес, который я намеревался запросить, был 127.0.0.2. Обратите внимание на то, что, как с обратным поиском, я инвертирую IP-адрес. После инвертирования адреса я присоединяю его к названию списка и запроса. Этот процесс фактически является обратным поиском и не полагается на жестко закодированный .arpa TLD. Вместо этого ответ обеспечивается записью A, предоставленной SBL-сервером Spamhaus.

DNSBLs отличаются в зависимости от списка и провайдера. Провайдеры могут обеспечить несколько различных форм списков для различных категорий трафика. Различные провайдеры будут также обеспечивать различные политики для добавления или удаления адресов в DNSBL. То, как различные организации управляют *исключением из списка* (удалением адреса), радикально влияет на характер списка. Одни автоматически пропускают адрес постоянное число дней после последнего злоупотребления; другие требуют ручного вмешательства.

Некоторые известные DNSBLs включают:

Spam and Open Relay Blocking System (SORBS)

Обеспечивает более 15 различных DNSBLs, которые классифицируют хосты в зависимости от типов их поведения. SORBS особенно полезен для категоризации динамических адресов, таких как коммутируемый доступ и адреса DSL через специализированный список, динамический список пользователя и хоста (DUHL).

Spamhaus

Некоммерческая частная компания, которая производит много отличных черных и белых списков. Обычно используемые списки Spamhaus являются PBL (адреса конечного пользователя), SBL (адреса спама), и XBL (украденные IP-адреса и роботы). Эти списки доступны как единственный объединенный сервис, ZEN.

SpamCop

В настоящее время принадлежащий Cisco Systems, SpamCop начался как частное предприятие и в конечном счете стал частью почтовой системы репутации IronPort. Сейчас SpamCop обеспечивает один общедоступный список, черный список SpamCop (SCBL).

DNSBLs полезны как категоризированный источник враждебной активности. Используя DNSBL, аналитик может определить, делал ли конкретный адрес что-то враждебное в другом месте в интернете и, возможно, каким действием это было. Они добавляют более обоснованную информацию о поиске, обсужденную ранее, путем анализа активности сайта в прошлом.

DNSBLs разработаны, чтобы быть инструментами в реальном времени, они работают прежде всего с почтовыми агентами, а не для поддержки судебного анализа. Записи изменятся быстро и непредсказуемо, таким образом, адрес сможет быть распознан DNSBL как враждебный во время события, но вычеркнут из списка, если аналитик исследует его позже. Большинство черных списков продает некоторые данные или дампы данных, которые могут быть использованы в судебных целях.



Глава 9

.....

Больше инструментов

Как было разобрано в начале книги, существует много инструментов, которые вы закончите использовать, применив в одной или двух определенных целях. В этом разделе я обсуждаю другие инструменты, которые считаю удобными для анализа, и включаю краткое объяснение того, как их использовать.

Многие из этих инструментов довольно мощны – намного больше, чем можно описать в сводке на три страницы. Я затрону каждый из этих инструментов очень кратко и попытаюсь предоставить пример каждому. Однако будьте готовы искать дополнительный материал и дополнительную документацию.

Визуализация

В то время как R является моим основным инструментом для визуализации графика, существует несколько дополнительных инструментов, которые удобны при определенных обстоятельствах. Graphviz является инструментарием для визуализации графиков. Gnuplot является служебным ножом инструментов графического изображения: мощный, пригодный для написания сценариев и глубоко недружественный.

Graphviz

Graphviz – пакет расположения и визуализации графика. Первоначально разработанный AT&T Labs, пакет теперь выпущен в соответствии с лицензией Eclipse и активно поддерживается.

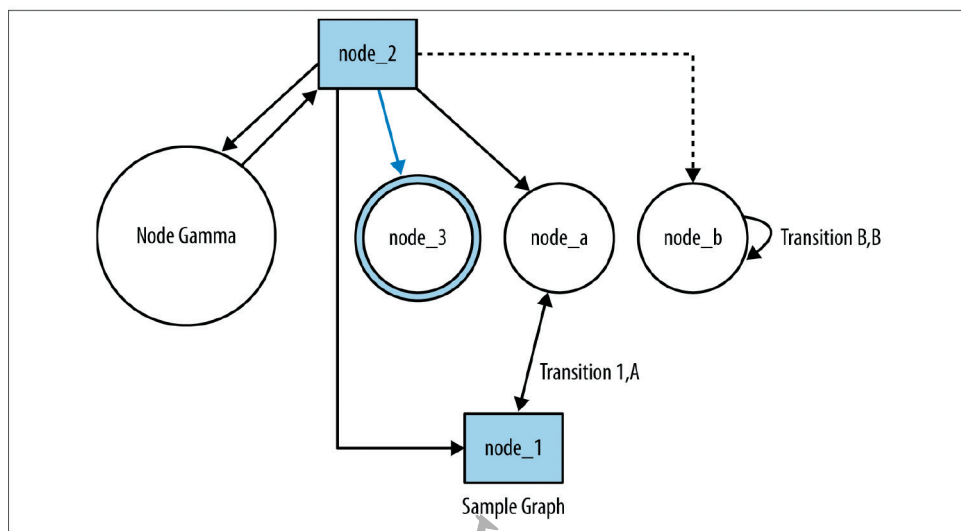
Graphviz является на самом деле комплектом инструментов, каждый из которых обеспечивает различный механизм, для того чтобы автоматически разметить графики. С каждым инструментом вы обеспечиваете спецификацию графика, и инструмент автоматически размечает график на основе спецификации. Графики определены через язык, названный *точкой*, которая определяет узлы различных атрибутов, затем соединяемых линком. Команда точки в качестве примера и вывод показаны в примере 9-1 с результатами, проиллюстрированными на рис. 9-1.

Пример 9-1. Демонстрационный график в точке

```

# Это простой dotfile, показывает основные характеристики графика digraph sample_graph {
# Узлы определены командой узла, если примечание маркировало отдельно
# Их метки являются их именами
node [shape=circle] node_a, node_b;
# Форма автоматически будет кругом
node [label = "Node Gamma"] node_c;
# Атрибуты узла переданы по линии, поэтому если я хочу
# избежать того, чтобы что-то было названо 'гаммой узла', я
# должен сбросить метку к имени узла
node [shape=square, label="\N"] node_1, node_2;
node [shape=doublecircle] node_3;
# Граничные атрибуты помещены в квадратные скобки;
# метка является текстовой меткой для графика
node_1-> node_a [label="Transition 1,A"];
node_a-> node_1;
node_b-> node_b [label= " Transition B,B"];
node_c-> node_2;
node_2-> node_1;
# Цвет управляется атрибутом цвета
node_2-> node_3 [color= "blue"];
node_2-> node_a;
# Стиль позволяет определить отмеченный точкой, полужирный и т. д.
node_2-> node_b [style = "dotted"];
node_2-> node_c;
label="Sample Graph";
fontsize=14;
}

```

**Рисунок 9-1.** Полученный макет в точке

Очень легко преобразовать записи журнала из их собственных форматов в формат dot, и получающиеся графики могут часто использоваться для визуального

выражения функций, таких как центральные узлы. Пример 9-2 показывает код, который преобразует страницы HTTP и сайты реферера в ссылки, а затем отображает прогресс серфинга с использованием dot. График в качестве примера показан на рис. 9-2.

Пример 9-2. Преобразование записи блога в Dot-графики

```
#!/usr/bin/env python
#
> log2dot.py
>
> Ввод:
> Файлы журнала от stdin. Мы предполагаем, что эти файлы были обработаны,
> чтобы предоставить URL и Referer URL
#
> Вывод
> Stdout производит Dot-файл, который может быть запущен graphviz, import sys re
host_id = re.compile('^http://([^/]+)/.*$')
pairs = {} nodes = {}
def graph_output(nodes, pairs): graph_
    header = """
    digraph graph_output { rotate
        = 90; size="7.5,10";
        """
    print graph_header
    a = nodes.keys()
    a.sort()
    for i in a:
        print "node [shape = circle] i;" a =
pairs.keys()
    a.sort(), for
I in a:
        for j in pairs[i].keys():
            # Печатает каждую ссылку, затем помечает ее числом совпадений print'%s
            ->%s [label="%d"];'% (i,j,pairs[i][j])
        print "}"
if __ name __ == '__ main __':
    for i in sys.stdin.readlines(): values =
        i[:-1].split()
        host = values[-2][:-1]
        referrer = values[-1]
        if host_id.match (referrer):
            refname = host_id.match (referrer).groups()[0]
    else:
        refname = referrer a =
        host.split('.'), if a[0] ==
        'www':
            host = '.'.join(a[1:])
            a = refname.split ('.'), if
            a[0] == 'www':
                refname = '.'.join(a[1:])
                host = host.replace ('-', '_') host =
                host.replace ('.', '_')
                refname = refname.replace ('-', '_') refname
                = refname.replace ('.', '_') nodes[host] = 1
                nodes [refname] = 1
```

```

if pairs.has_key(refname):
    if pairs[refname].has_key(host):
        pairs[refname][host] += 1
    else:
        pairs[refname][host] = 1
else:
    pairs[refname] = {host:1}
graph
output (nodes, pairs)

```

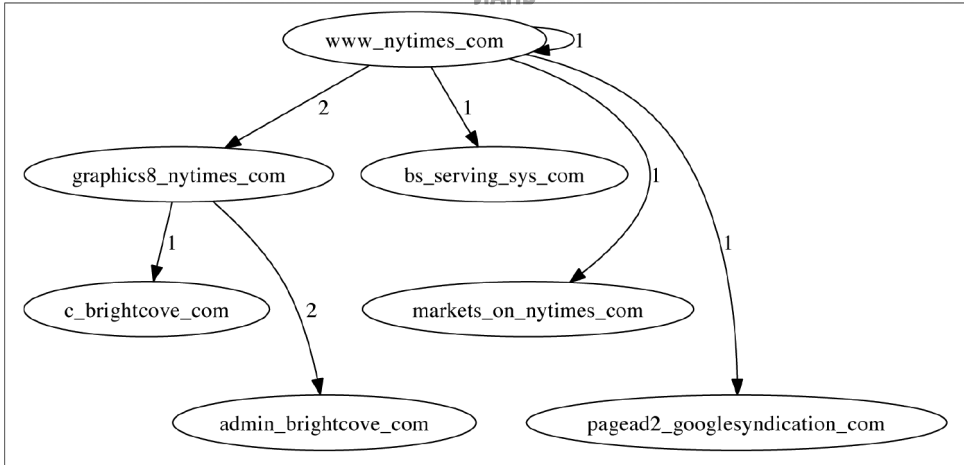


Рисунок 9-2. Пример вывода log2dot-сценария

Связь и зондирование

Сетевой мониторинг, как я уже обсуждал в этой книге, в основном пассивен. Есть ряд ситуаций, когда требуется более активный мониторинг и тестирование. Инструменты в этом разделе используются для активной работы в сети.

В контексте этой книги я сосредоточился на инструментах, которые используются для активного дополнения мониторинга инфраструктуры. Они используются для предоставления примеров сессий (*netcat*), дополнения пассивного мониторинга с активным зондированием (*nmap*) или для предоставления специально созданных сеансов, тестирующих конкретные конфигурации мониторинга (*Scapy*).

netcat

netcat является инструментом командной строки Unix, который перенаправляет вывод в сокеты UDP и TCP. Netcat позволяет быстро внедрять клиенты, серверы, прокси и сканеры портов.

Простейшим вызовом *netcat* является хост-порт *netcat*, который создает TCP-сокеты для указанного хоста и номера порта. Входные данные могут быть переданы в *netcat*, а выходные данные – с использованием стандартного перенаправления Unix:



```
$echo "GET /" | netcat www.oreilly.com 80
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html> <head>
...
```

В данном примере мы используем netcat для извлечения индексной страницы веб-сайта. GET / является стандартным синтаксисом HTTP¹. Если вы знаете, как создать сеанс с помощью конкретного протокола, такого как HTTP, SMTP и т. п., можно отправить его через netcat для создания клиента.

На том же принципе можно использовать netcat для захвата баннера (см. главу 15). Например, я могу захватить ssh баннер путем отправки фиктивного сеанса через netcat к серверу SSH:

```
$ echo "WAFFLES" | nc fakesite.com 22
SSH-2.0-OpenSSH_6.2
Protocol mismatch.
```

Обратите внимание на использование nc в примере; на большинстве netcat-пакетов эти два приложения будут псевдонимами друг для друга. По умолчанию netcat открывает TCP-соединение, это можно изменить с помощью опции -u.

netcat предоставляет ряд параметров командной строки для лучшего управления инструментом. Например, для улучшения захвата баннера мы можем использовать диапазон портов:

```
echo "WAFFLES" | nc-w1-v fakesite.com 20-30
fakesite.com [127.0.0.1] 21 (ftp) open
220 fakesite.com NcFTPD Server(licensed copy) ready.
500 Syntax error, command unrecognized.
fakesite.com [127.0.0.1] 22 (ssh) open
SSH-2.0-OpenSSH_6.2
```



-v-опция указывает на многословность, добавляя строки, показывающие открываемые порты. Команда -w1 определяет 1-секундное ожидание, а 20-30 определяют для проверки 20–30 портов.

Простое сканирование портов может быть сделано с помощью опции -z, которое просто показывает, открыто ли соединение. Например:

```
$ nc-n-w1-z-vv 192.168.1.9 3689-3691
192.168.1.9 3689 (daap) открыто
192.168.1.9 3690 (svn): нет соединения
192.168.1.9 3691 (magaya-сеть): нет соединения
Общих полученных байтов: 0
Общие отправленные байты: 0
```

В данном случае сканирует Apple TV.

netcat является очень удобным инструментом для захвата баннера и внутренней аналитики, он позволяет вам довольно быстро создать специального клиента для любого приложения. Когда новые внутренние сайты идентифицированы, netcat может использоваться, чтобы отсканировать и зондировать их для получения дополнительной информации, если лучший инструмент не доступен.

¹ HTTP является чрезвычайно надежным протоколом и допускает любую комбинацию попыток поднять сессию, так что это своего рода идеальный плацдарм для разных примеров.

nmap

Пассивный анализ безопасности работает только тогда, когда каждая программа внутренней безопасности имеет хотя бы один доступный инструмент сканирования. Сетевой картопостроитель (nmap) является лучшим доступным инструментом сканирования открытого исходного кода.

Причина использовать nmap или любой другой инструмент сканирования состоит в том, что эти инструменты содержат огромный объем информации об уязвимостях в операционных системах. Цель любого усилия по сканированию состоит в том, чтобы получить аналитику о целевом хосте или сети. Простое полуоткрытое сканирование может быть легко реализовано с помощью практически любой линейной команды, профессиональные инструменты сканирования извлекают пользу из экспертных систем, которые могут совместить захват баннеров, анализ пакетов и другие методы идентификации информации о хосте. Например, разберем простое сканирование nmap на Apple TV, использованное в предыдущем примере (адрес 192.168.1.9):

```
$ nmap -A 192.168.1.9
Starting Nmap 6.25 (http://nmap.org) at 2013-07-28 19:44 EDT
Nmap scan Report for Apple-TV-3.home (192.168.1.9)
Host is up (0,0058s latency).
Not shown: 995 closed ports
PORT      STATE SERVICE      VERSION
3689/tcp   open  opendaap     Apple iTunes DAAP 11.0.1d1
5000/tcp   open  openrtsp     Apple AirTunes rtspd 160.10 (Apple TV)
| rtsp-methods:
|__ ANNOUNCE,SETUP, RECORD,PAUSE, FLUSH, TEARDOWN, OPTIONS, \GET_PARAMETER, SET_PARAMETER,
POST, GET
7000/tcp   open  http         Apple AirPlay httpd
| http-methods: Potentially risky methods: PUT
| _See http://nmap.org/nsedoc/scripts/http-methods.html
| _http-title: Site doesn't have a title.
7100/tcp   openhttp     Apple AirPlay httpd
|_http-methods: No Allow or Public header in OPTIONS response(status code 400)
|_http-title: Site doesn't have a title
62078/tcp  open  tcpwrapped
Service Info: OSs: OS X, Mac OS X; Device: media device;
CPE: cpe:/o:apple:mac_os_x
Обнаружение служб выполнено. Сообщите о любых неправильных результатах по http://nmap.org/submit/.
Nmap done: 1 IP-address (1 host up) scanned in 69,63 seconds
```

Сканирование nmap содержит информацию об открытых портах, версии программного обеспечения сервера на этих портах, потенциальных рисках и дополнительных данных, таких как строка CPE¹.

Аналитически инструменты сканирования используются сразу после того, как в сети обнаружен новый хост для выяснения, что же это за зверь. В частности, это сделано при помощи следующего процесса:

¹ CPE является проектом NIST, обеспечивающим общую основу для описания платформ.

- 1) проверьте данные потока, чтобы увидеть, появляются ли на экране новые комбинации хоста и порта или сети;
- 2) если новые хосты найдены, запустите `npmp` на хостах, чтобы определить, что они работают;
- 3) если `npmp` не может идентифицировать службу на порту, запустите `nc`, чтобы захватить основные баннеры; тогда сможете узнать, что это за новый порт.

Scapy

Scapy – это библиотека анализа пакетов и управления ими для Python. Используя Scapy, можно разложить пакеты на удобные для Python структуры, визуализировать их содержимое и создать новые корректные IP-пакеты, которые можно внедрять в коллекции пакетов. Scapy служит мне незаменимым инструментом преобразования и управления `tcpdump`-записями.

Scapy обеспечивает удобное для Python представление `tcpdump` данных. Загрузив данные в виде элементов, их можно просмотреть с помощью ряда функций дисплея или исследовать различные уровни каждого пакета, представленные в словаре. В примере 9-3 мы считываем и исследуем содержимое некоторого пакета с помощью библиотеки Scapy, которая выводит текстовые метки и сопровождающее их изображение. Результат показан на рис. 9-3.

Пример 9-3. Чтение и исследование содержимого пакета

```
>>> # загружаем файл дампа с помощью rdpcap
>>> s=rdpcap('web.pcap')
>>> # ищем первый пакет с полезной нагрузкой TCP
>>> for i in range(0,100):
...     if len(s[i][TCP].payload) > 0:
...         print i
...         break
...
63
>>> # выводим его содержимое с помощью show()
>>> s[63].show()
### [Ethernet] ###
  dst= 00:1f:90:92:70:5a
  src= 8c:2d:aa:46:f9:71
  type= 0x800
### [IP] ###
  version= 4L
  ihl= 5L
  tos= 0x0
  len= 1110
  id= 10233
  flags= DF
  frag= 0L
  ttl= 64
  proto= tcp
  checksum= 0xbe42
  src= 192.168.1.12
  dst= 157.166.241.11
  \options\
```



```

#### [ TCP ]###
sport= 50300
dport= http
seq= 4157917086
ack= 3403794807
dataofs= 8L
reserved= 0L
flags= PA
window= 8235
chksum= 0x5dd5
urgptr= 0
options= [('NOP', None), ('NOP', None), ('Timestamp', (560054364, 662137900))]
#### [Raw] ###
load= 'GET / HTTP/1.1\r\nHost: www.cnn.com \r\nConnection:...'
>>> # вывести содержимое с помощью PDFdump
>>> s[63].pdfdump('http.pdf')

```

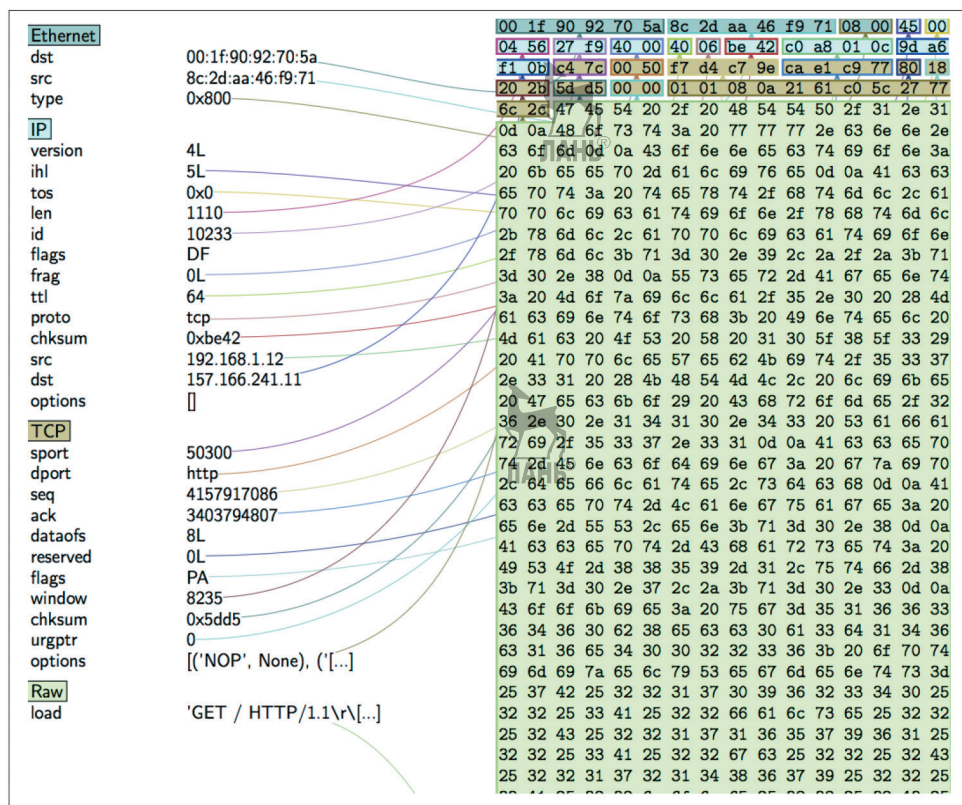


Рисунок 9-3. После полной установки Scapy может произвести графическое дизассемблирование пакета

Я использую Scapy в основном для преобразования и переформатирования записи `tcpdump`. Следующий пример демонстрирует применение библиотеки. Сценарий в примере 9-4 осуществляет колоночный вывод содержимого `pcap`-файлов подобно тому, как это делает `gswcut`.

Пример 9-4. Сценарий tcrcut.py


```
#!/usr/bin/env python
#
# tcrcut.py
#
# Этот сценарий принимает tcrcdump-файл, как введено, и выводит его содержимое на экран
# в формате, подобном gwcut.
# Поддерживает только девять полей и не выводит приглашения
# к вводу ради простоты примера.
#
# Ввод:
# tcrcut.py data_file
#
# Вывод:
#
# Поколоночный вывод в stdout

from scapy.all import *
import sys, time
header = '%15s|%15s|%5s|%5s|%5s|%15s|' % ('sip', 'dip', 'sport', 'dport',
    'proto', 'bytes')
tfn = sys.argv[1]
pcap_data = rdpcap(tfn)
for i in pcap_data:
    sip = i[IP].src
    dip = i[IP].dst
    if i[IP].proto == 6:
        sport = i[TCP].sport
        dport = i[TCP].dport
    elif i[IP].proto == 17:
        sport = i[UDP].sport
        dport = i[UDP].dport
    else:
        sport = 0
        dport = 0
    bytes = i[IP].len
    print "%15s|%15s|%5d|%5d|%5d|%15d" % (sip, dip, sport, dport, i[IP].proto, bytes)
```



Также я использую Scapy, чтобы сгенерировать данные для тестирования сеанса. Например, внедряя новую систему журналирования, я генерирую сеанс с помощью rscap и запускаю его с этой системой журналирования, затем настраиваю сеанс с помощью Scapy и наблюдаю, как мои изменения влияют на записи в журнале.

ИССЛЕДОВАНИЕ ПАКЕТОВ И ПОЛУЧЕНИЕ СПРАВОЧНОЙ ИНФОРМАЦИИ

Все инструменты, обсуждаемые в этом разделе, предназначены для проведения расширенного анализа пакетов. Wireshark – это, пожалуй, самый полезный инструмент исследования пакетов, а *geoip* – удобный справочный инструмент, помогающий выяснить источник трафика.

Wireshark

Я не собираюсь занимать много места для описания Wireshark, потому что, как Snort и nmap, это один из наиболее известных и хорошо документированных инструментов анализа трафика. Wireshark – это анализатор протокола с графическим интерфейсом, предлагающий возможности для исследования пакетов и сбора статистики по ним, а также ряд инструментов для исследования данных.

Истинная мощь Wireshark заключена в его обширной библиотеке *диссекторов* для анализа содержимого пакетов. Диссектор – это набор правил и процедур для разложения содержимого пакетов в структуру и восстановления сеанса внизу. На рис. 9-4 показано, как Wireshark может извлечь и вывести на экран содержимое сеанса HTTP.

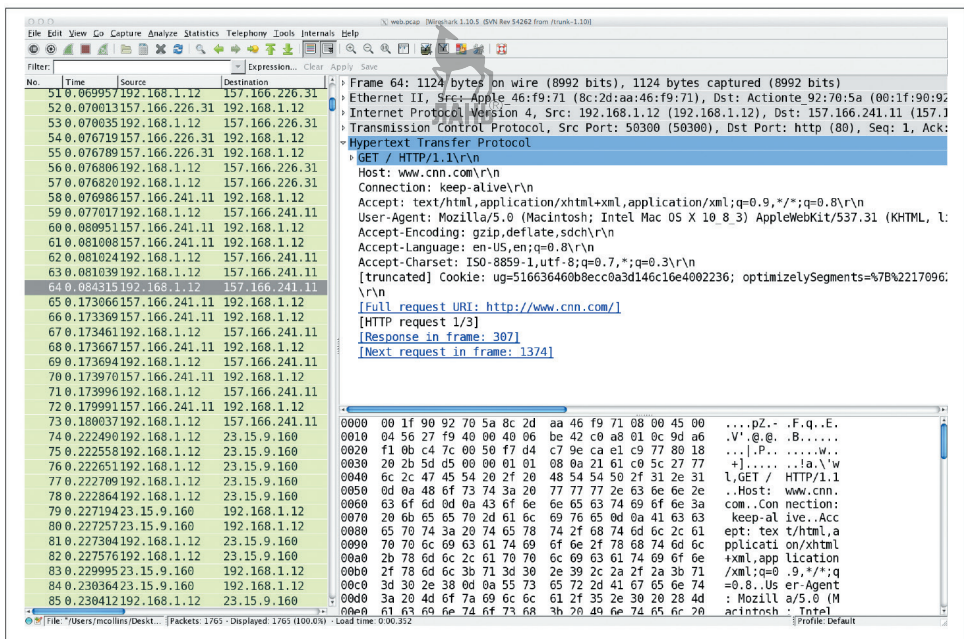


Рисунок 9-4. Пример реконструкции сеанса с помощью Wireshark

GeoIP

Службы геолокации принимают IP-адреса и возвращают координаты физического местоположения систем с этими адресами. Геолокация – сложный процесс: исследователи начинают с определения номера сетевой карты NIC и затем используют разнообразные подходы, от определения задержек на передачу до звонков в компании, чтобы выяснить их почтовые адреса.

MaxMind GeoIP (<http://www.maxmind.com/>) – это бесплатная по умолчанию база данных геолокации. Бесплатная версия (GeoLite) позволит узнать город, страну и информацию об ASN.

Applied Security (<http://www.appliedsec.com/>) предлагает превосходную GeoIP-библиотеку для Python (pygeoip (<http://bit.ly/pygeoip>), также доступную

в *pip*). *pygeoip* работает и с коммерческими, и с бесплатными экземплярами базы данных. Вот пример сценария *pygeoip_lookup.py*, демонстрирующий API библиотеки:

```
#!/usr/bin/env python
#
# pygeoip_lookup.py
#
# Принимает произвольные IP-адреса, передает
# их в базу данных maxminds geoip и возвращает
# код страны.
include sys,string,pygeoip
gi_handle = None
try:
    geoip_dbfn = sys.argv[1]
    gi_handle = pygeoip.GeoIP(geoip_dbfn, pygeoip.MEMORY_CACHE)
except:
    sys.stderr.write("Specify a database\n")
    sys.exit(-1)
for i in sys.stdin.readlines():
    ip = [:-1]
    cc = gi_handle.country_code_by_addr(IP)
    print "%s%s" % (ip, cc)
```



Геолокация – большой бизнес, и в мире существует несколько коммерческих баз данных геолокации. MaxMind предлагает свою собственную, а среди других можно назвать IP Intelligence Neustar, Akamai и Digital Envoy.

NVD, вредоносные сайты и C*E

Национальная база данных уязвимостей (*National Vulnerability Database, NVD*) – это государственная служба, поддерживаемая Национальным институтом по стандартизации и технологиям (*National Institute of Standards and Technology, NIST*), созданная для учета и классификации уязвимостей программных и аппаратных систем. Проект NVD не раз переименовывался в течение многих лет, и под его эгидой было создано несколько различных компонентов в базе данных. Наиболее важные из них с разными именами, начинающимися с С и заканчивающимися на Е, были созданы в MITRE:

CVE

База данных *Common Vulnerabilities and Exposures* (перечень типовых уязвимостей и рисков) перечисляет уязвимости программного обеспечения и эксплоиты.

CPE

База данных *Common Platform Enumeration* (перечень типовых платформ) предлагает механизм описания программных платформ с использованием иерархических строк. Записи в CVE используют CPE для ссылки на конкретные версии программного обеспечения, подверженные уязвимостям, описанным в CVE.

CCE

Common Configuration Enumeration (перечень типовых конфигураций) перечисляет и описывает конфигурации программного обеспечения, такого как Apache. CCE все еще находится в разработке.

NVD управляет всеми этими перечнями по протоколу автоматизации управления данными безопасности (Security Content Automation Protocol, SCAP), обеспечивающему автоматическую настройку безопасности. Для анализа база данных CVE является самой важной частью всего этого сумасшествия. Для единственной уязвимости могут иметься десятки и сотни разных эксплоитов, но все они будут связаны номером уязвимости в CVE.

В дополнение к разработкам, которые финансируются правительством, существует много других перечней эксплоитов. В их числе:

BugTraq IDs

BugTraq – это список рассылки уязвимостей, в котором публикуются сообщения от большого числа независимых исследователей о новых эксплоитах и уязвимостях. BugTraq использует простой числовой ID и поддерживает список (<http://bit.ly/vuln-list>) всех вновь выявленных уязвимостей. Отчеты об уязвимостях в BugTraq часто дублируются в NVD.

OSVDB

База данных уязвимостей, которая поддерживается фондом *Open Security Foundation* (OSF), некоммерческой организацией, созданной для управления сведениями об уязвимостях.

Symantec Security Response (<http://bit.ly/sec-resp>)

Этот сайт содержит базу данных и сводную информацию о каждой вредоносной программе, произведенную программным обеспечением AV Symantec.

Центр угроз McAfee (<http://bit.ly/mcafee-threat>)

Центр угроз компании McAfee служит той же цели, что и сайт Symantec, – это интерфейс к базе данных с перечнем известных на настоящий момент угроз и вредоносного программного обеспечения, которые может идентифицировать программное обеспечение AV McAfee.

Перечень Касперского с описанием угроз безопасности (<http://bit.ly/securelist>)

Список сигнатур уязвимостей, известных лаборатории Касперского.

Эти базы данных более полезны для исследователей вредоносного программного обеспечения, усилия которых сосредоточены на исследовании эксплоитов и методов взлома. С точки зрения анализа сетевой безопасности эти сайты прежде всего полезны тем, что помогают определить векторы распространения вредоносного программного обеспечения и, следовательно, позволяют получить хорошее первое представление о том, как будет выглядеть его трафик. Например, узнав, что компоненты вредоносного программного обеспечения распространяются по HTTP и NetBIOS, вы сможете определить круг сетевых служб и номера портов, которым следует уделить особое внимание.

Поисковые системы, списки рассылки и люди

Вот чем отличается хороший аналитик от посредственного: посредственный аналитик получит данные от *rsar* или из блогов и сделает вывод на основании полученных данных. Хороший аналитик будет искать другую информацию в блогах, в списках рассылки или общаясь с аналитиками на форумах.

Компьютерная безопасность – это постоянно меняющееся поле, а нападения – постоянно движущаяся цель. Аналитик может быстро самоуспокоиться, потому

что существует так много простых нападений, которые легко выявляются и контролируются. Но злоумышленники продолжают развиваться и все время используют новые инструменты и подходы. Интернет-трафик изменяется по многим причинам, большинство из которых нетехнические, и объяснение некоторым скачкам я нашел в списках рассылки, таких как NANOG, а также на первой странице «Нью-Йорк таймс».



ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ ДЛЯ ЧТЕНИЯ

1. *Лаура Чаннелл (Laura Chappell) и Гералд Комбс (Gerald Combs)*. Wireshark 101: Essential Skills for Network Analysis.
2. Graphviz (<http://www.graphviz.org/>).
3. *Гордон «Федор» Лайон (Gordon «Fyodor» Lyon)*. Nmap Network Scanning: проект Nmap Project, 2009.
4. Проект Nmap Project (<http://www.insecure.org/>).
5. Scapy (<http://bit.ly/scapy>).
6. Wireshark (<http://www.wireshark.org/>).



ЧАСТЬ III

АНАЛИТИКА



В предыдущих двух частях книги мы обсудили типы данных, которые можно собрать, и инструменты для управления этими данными. В этой части мы сфокусируемся на получении этих данных и проведении их исследований.

Каждая глава части III посвящена различным видам математических и аналитических методов, которые могут использоваться с данными. Основное внимание сосредоточено на предоставлении информации, которая наиболее важна для безопасности или является основополагающей. Прежде всего остановимся на процессе *исследовательского анализа данных (EDA)*, описанном в главе 10. В главах 11, 12, 13 и 14 рассмотрены примеры поведений, связанные с нападениями, и обсуждаются способы их использования для создания предупреждений или использования в судебной экспертизе и расследовании. В главе 15 рассмотрена проблема отображения сети с применением методов, описанных в предыдущих главах, для обеспечения ситуационной осведомленности.





Глава 10

Разведочный анализ данных и визуализация



Разведочный анализ данных (Exploratory Data Analysis, EDA) – это процесс исследования набора данных без выдвижения предварительных гипотез и предположений о данных и их характере. Данные, получаемые на практике, часто искажены, имеют сложную структуру и потому требуют обстоятельной фильтрации и классификации для идентификации явлений, которые можно использовать для определения аномалий и расследования. Атакующие и сам интернет являются движущейся целью, и аналитики сталкиваются с постоянным притоком странностей. Поэтому разведочный анализ (EDA) является непрерывным процессом.

Смысл разведочного анализа заключается в том, чтобы составить как можно более полное мнение о данных, перед тем как задействовать математические методы. Чтобы было понятнее, зачем это нужно, решим простое статистическое упражнение. В табл. 10.1 представлены четыре набора данных, все они включают два вектора, X и Y . Для каждого набора данных вычислите следующие значения:

- среднее для X и Y ;
- дисперсию для X и Y ;
- корреляцию между X и Y .



Таблица 10-1. Четыре набора данных

I		II		III		IV	
X	Y	X	Y	X	Y	X	Y
10.0	8.04	10.0	9.14	10.0	7.46	8.0	6.58
8.0	6.95	8.0	8.14	8.0	6.77	8.0	5.76
13.0	7.58	13.0	8.74	13.0	12.74	8.0	7.71
9.0	8.81	9.0	8.77	9.0	7.11	8.0	8.84
11.0	8.33	11.0	9.26	11.0	7.81	8.0	8.47
14.0	9.96	14.0	8.10	14.0	8.84	8.0	7.04
6.0	7.24	6.0	6.13	6.0	6.08	8.0	5.25
4.0	4.26	4.0	3.10	4.0	5.39	19.0	12.50
12.0	10.84	12.0	9.13	12.0	8.15	8.0	5.56

I		II		III		IV	
X	Y	X	Y	X	Y	X	Y
7.0	4.82	7.0	7.26	7.0	6.42	8.0	7.91
5.0	5.68	5.0	4.74	5.0	5.73	8.0	6.89

Вы обнаружите, что среднее, дисперсия и корреляция идентичны во всех четырех наборах данных, но даже при простом просмотре чисел возникают смутные сомнения. А если их визуализировать, вы легко заметите, насколько они непохожи. Графики на рис. 10-1 показывают, как различаются этих наборы данных своей структурой. *Квартет Энскомба (Anscombe Quartet)* был специально разработан для иллюстрации важности применения графиков в анализе и влияния выбросов (как в наборе IV) на свойства всего набора данных.

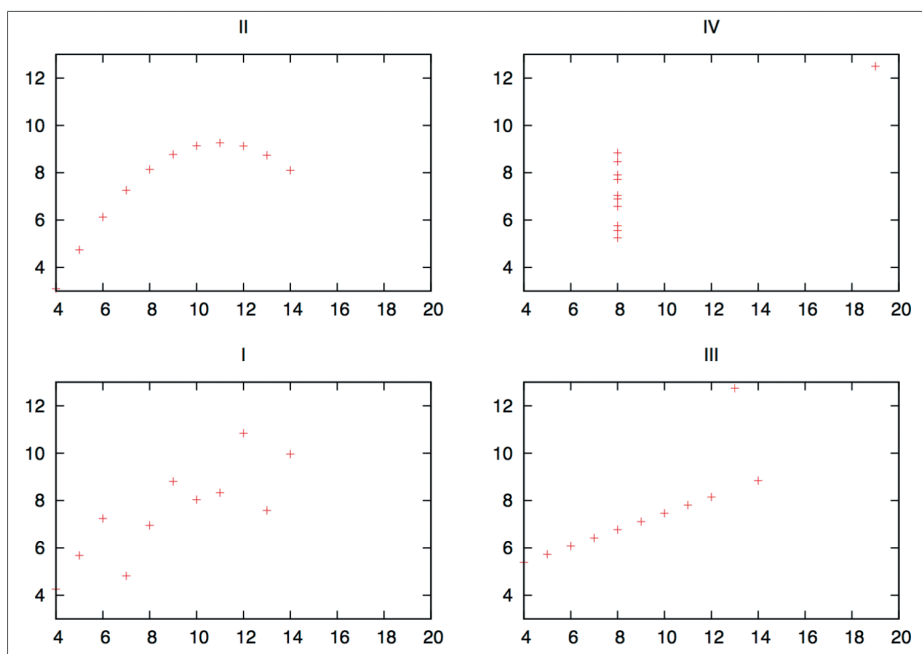


Рисунок 10-1. Результат визуализации квартета Энскомба (Anscombe Quartet)

Как показывает этот пример, простая визуализация позволяет выявить важные характеристики наборов данных, которые не выявляются путем вычисления простых статистик. Классическая ошибка при проведении статистического анализа заключается в том, что математический аппарат включается в работу до знакомства с данными. Например, аналитики часто вычисляют среднее и стандартное отклонение, чтобы определить пороговое значение (обычно отстоящее примерно на 3,5 стандартного отклонения от среднего). При этом они обычно исходят из предположения нормального распределения данных в наборе; если это не так (что, впрочем, бывает редко), то более простые вычисления могут дать более эффективные результаты.

ЦЕЛЬ РАЗВЕДОЧНОГО АНАЛИЗА: ПРОВЕДЕНИЕ АНАЛИЗА

Смысл любого разведочного анализа – приблизиться к модели; эта модель может быть формальным представлением данных или чем-то простым, как: «поднимаем тревогу, когда наблюдается слишком много чего-то» (где «слишком много» и «чего-то», конечно, выражены количественно). Далее мы обсудим четыре основные цели анализа данных применительно к информационной безопасности: формирование тревог, расследование, конструирование системы защиты и ситуативная осведомленность.

При использовании для формирования тревог аналитический процесс включает генерирование некоторого числа, сравнение его с типичным показателем работы модели и определение *необходимости привлечения внимания аналитика*. Аномалия необязательно является нападением, и нападение необязательно заслуживает ответа. Хорошая тревога основана на явлениях, предсказуемых при нормальных обстоятельствах, когда защищающийся может что-то предпринять, а атакующий должен преодолеть сопротивление для достижения своих целей.

Задача в оперативной информационной безопасности – *не создать тревоги*, а сделать их управляемыми. Первое, что должен сделать аналитик, получив тревогу, – обеспечить контекст: убедиться в реальности угрозы, гарантировать, что она актуальна, определить степень ущерба и рекомендовать действия, которые следует предпринять. Важной проблемой являются ложные тревоги, которые обычно появляются из-за отсутствия полной информации по всем видам отказов для формирования тревог. Хороший анализ может увеличить эффективность тревог. Более подробно об этом рассказывается в главе 7.

В большинстве случаев проводимый анализ относится к категории расследований, которые проводятся после появления события. Расследование может быть начато в ответ на любую информацию: тревоги, сигналы системы определения вторжений (Intrusion Detection System, IDS), сообщения пользователя или газетные статьи¹.

Расследование начинается со сбора некоторой первичной информации, например с определения IP-адреса зараженного компьютера или враждебного веб-сайта. Опираясь на нее, расследующий должен узнать как можно больше о нападении – степень нанесенных повреждений, какие действия выполнялись атакующим, хронологию основных событий, имеющих отношение к нападению. Расследование часто является самой информационно емкой работой из всего, что делает аналитик, поскольку включает сопоставление данных из многочисленных источников, от журналов трафика до результатов опроса сотрудников и просмотра архивов данных, сохраненных несколько лет назад.

Тревоги и расследования являются реактивными мерами, но аналитик может также использовать данные проактивно и создавать защитные механизмы. Как аналитики мы обладаем рядом инструментов, таких как рекомендованные политики, правила брандмауэра и аутентификация, которые можно использовать для реализации защиты, но есть одна сложность – эти меры существенно ограничены. С точки зрения пользователя безопасность определяется рядом правил, которые ограничивают их действия для предотвращения некоторых абстрактных неприятностей.

¹ Нет ничего лучше, чем начать расследование, прочитав статью о злоумышленниках в «Нью-Йорк таймс».

В информационной безопасности люди всегда являются последней линией обороны. Если меры безопасности внедряются спустя рукава или бессистемно, это поощряет конфликтные отношения между системными администраторами и пользователями, и очень быстро все начинает улетать в порт 80. С помощью анализа можно определить разумные ограничения, которые будут препятствовать злоумышленникам и не лягут тяжким бременем на пользователей.

Тревоги, расследования и модернизация составляют единый цикл: обнаружение нападений, анализ нападений и восстановление нормальной работы после нападений. Однако весь этот цикл зависит от использования знаний. Использование знаний в форме учетных и справочных данных, прошлой истории и даже телефонных книг помогает справиться с накатывающимися бедствиями.

Знания влияют на все. Например, почти все системы определения вторжений (особенно системы управления электронными ключами) фокусируются на содержимом пакета, не зная, например, что эксплойт для IIS, который они благополучно идентифицировали, был нацелен на Amiga 3000, где действует Apache¹. В системах IDS ложная тревога обычно является признаком первых попыток нападения. Ведение учетной и справочной информации является необходимым первым шагом к разработке эффективных тревог; многие нападения порождают отказы, которые можно идентифицировать по контексту и ликвидировать до того, как дело дойдет до привлечения аналитиков.

Хорошие учетные и справочные данные и прошлая история могут также использоваться для ускорения расследования. Многие расследования ссылаются на различные источники данных для контекста, и эта информация предсказуема. Например, если у меня есть внутренний IP-адрес, я должен знать, кому принадлежит компьютер с этим адресом и какое программное обеспечение на нем установлено.

Для использования знаний требуется возможность получения данных из разных источников и размещения их в одном месте. Такая информация, как перечень сервисных сетей доступа (Access Service Network, ASN), данные службы whois и даже простые номера телефонов, часто хранится в десятках, если не в сотнях баз данных, которые могут изменяться с течением времени и накладывать свои внутренние ограничения. Состояние внутренней сети нередко столь же хаотично, если не больше, потому что почти всегда люди начинают запускать сетевые сервисы, о которых никто не знает. Бывает, сам процесс идентификации ресурсов на оперативном уровне помогает управлять сетью и решать проблемы ИТ в целом.

Рассматривая данные, не забывайте о цели анализа. В конечном итоге вы должны выяснить причины появления тревоги, восстановить хронологию событий или понять, можно ли добавить новое правило брандмауэра и не вызвать дополнительных проблем.

Порядок выполнения разведочного анализа

На рис. 10-2 показан обобщенный порядок выполнения разведочного анализа в сфере информационной безопасности. Как видите, в основе разведочного анализа лежит цикл, включающий извлечение явления и его исследование.

¹ Да, такая комбинация действительно существует.

Разведочный анализ начинается с простого вопроса, например: «На что похоже типичное действие?» Вопрос управляет выбором данных. Например, чтобы ответить на вопрос «можно ли идентифицировать трафик BitTorrent по размеру пакета?», может понадобиться получить образцы трафика с известными трекерами BitTorrent или протекающего через порты 6881–6889 (зарезервированные за протоколом BitTorrent).

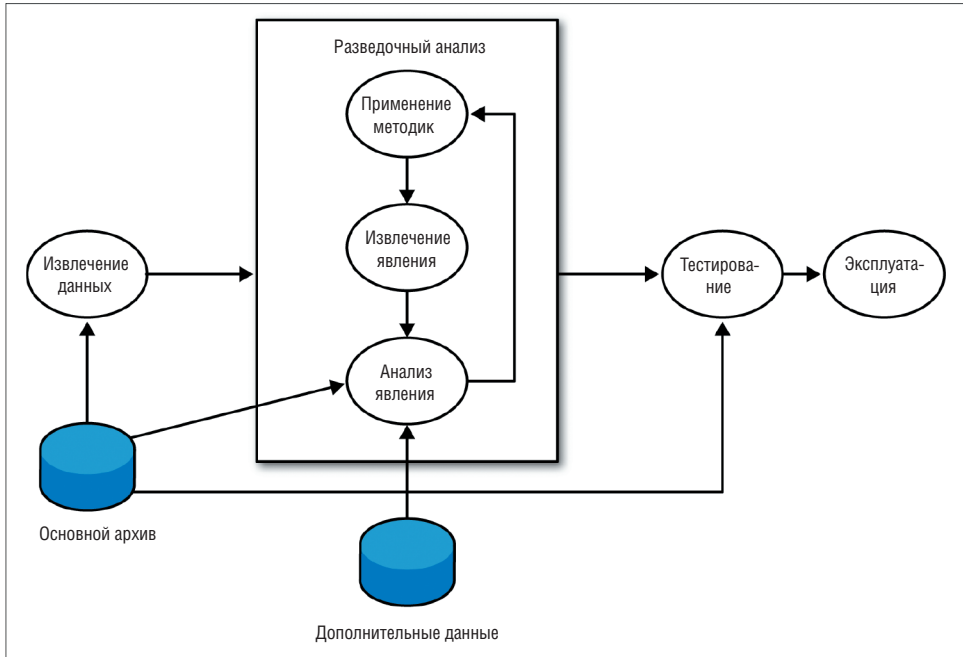


Рисунок 10-2. Порядок выполнения разведочного анализа

В цикле разведочного анализа аналитик повторяет три шага: обобщение и исследование данных с помощью выбранной методики, идентификация явления в данных и последующее его исследование. *Методика разведочного анализа* – это процесс получения набора данных и его обобщения некоторым способом, чтобы человек смог идентифицировать явления для дальнейшего исследования. Многие методики разведочного анализа основаны на визуализации, и большая часть этой главы посвящена инструментам визуализации. Также к методикам разведочного анализа относятся такие виды анализа данных, как кластеризация и классические статистические методы, например регрессионный анализ.

Методики разведочного анализа позволяют выявить характерные признаки, которые потом можно использовать для извлечения конкретных явлений из исходных данных, чтобы исследовать их более подробно. Например, исследуя трафик, протекающий через порты 6881–6889, аналитик обнаружил, что многие пакеты имеют от 50 до 200 байт полезной нагрузки. Он возвращается к исходным данным и, руководствуясь этой информацией и при помощи Wireshark, узнает, что эти пакеты являются управляющими пакетами протокола BitTorrent.

Этот цикл, применение методик / извлечение / анализ, может повторяться бесконечно; поиск явлений и умение вовремя остановиться являются искусством, которым овладевают с опытом. Анализу подвергается огромное количество ложных явлений, потому что даже самые эффективные начальные критерии часто слишком широки и подвержены ложным выводам. В процессе разведочного анализа часто приходится обращаться к многочисленным источникам данных. Например, аналитику, исследующему трафик BitTorrent, может потребоваться изучить описание протокола или самому запустить клиента BitTorrent, чтобы определить наличие выявленных свойств в наблюдаемых данных.

В какой-то момент цикл разведочного анализа должен остановиться. В этот момент, как правило, на руках у аналитика будет все необходимое, чтобы ответить на поставленный вопрос. Например, при поиске периодических явлений, таких как подключение к центральному серверу ботнета, можно использовать автокорреляцию, анализ Фурье или просто посчитать интервалы времени. Когда у аналитика появляется несколько вариантов ответа, встает проблема выбора, разрешение которой определяется тестированием и эксплуатационными требованиями.

В процессе тестирования должны использоваться методики, разработанные во время разведочного анализа и наиболее точно соответствующие эксплуатационным требованиям. Эта фаза процесса включает создание тревог и отчетов. За дополнительной информацией о критериях хороших тревог обращайтесь к главе 7, где описываются приемы обнаружения аномалий.

ПЕРЕМЕННЫЕ И ВИЗУАЛИЗАЦИЯ

Наиболее доступной и широко используемой методикой разведочного анализа является визуализация. Инструменты визуализации выбираются, исходя из характера исследуемых данных и целей анализа. Обычно к каждой конкретной задаче можно применить несколько способов визуализации. Чтобы понять характер данных, начинать следует с изучения *переменных*.

Переменная – это характеристика объекта, которую можно измерить или посчитать, например вес или температура. Переменные могут отличаться для разных сущностей или изменяться со временем; рост человека изменяется с возрастом, и разные люди имеют разный рост.

Есть четыре категории переменных, знакомые читателям, изучавшим курс элементарной статистики. Я кратко рассмотрю их здесь в порядке убывания строгости.

Интервал

Интервал используется, когда имеет смысл говорить о разности между двумя значениями, но бессмысленно говорить об их отношении. Наиболее распространенной формой интервала является время начала события в данных сетевого трафика. Например, одно событие может быть зарегистрировано через 100 секунд после полуночи, а другое – через 200 секунд после полуночи. Второе событие произошло после первого, но в отсутствие реального понятия «точки отсчета» бессмысленно говорить, что оно произошло «вдвое позже».

Отношение

Отношение подобно интервалу, но дополнительно имеет значимое понятие «точки отсчета», которое позволяет рассуждать об отношении с точки зрения

умножения и деления. Примером отношения может служить число байтов в пакете. Например, один пакет может содержать 200 байт, а другой 400 байт. По аналогии с интервалами, об этих пакетах можно сказать, что один больше другого, а также что второй пакет «вдвое больше» первого.

Порядок

Данные поддерживают понятие «порядка», но интервалы между соседними порядковыми значениями не фиксированы. К этой категории относятся оценки потребителей. Оценка 5 выше, чем 4, а оценка 4 выше, чем 3, из этого следует, что 5 выше, чем 3. Но мы не можем утверждать, что степень удовлетворенности потребителя увеличивается в равной степени, когда оценка повышается с 3 до 4 и с 4 до 5. (Многие часто допускают ошибку, рассматривая такие оценки как переменные отношения или интервала и основывая свои вычисления на этом предположении.)

Номинал

Термин «номинал» – это всего лишь название, применяемое к нечисловым данным. К номиналу неприменимо понятие «порядок». К данным этого типа можно отнести имена хостов и сервисов (веб, электронная почта и т. д.).

Данные, определяемые числами, необязательно являются порядковыми. Например, порты являются номинальными данными. Порт 80 не «выше», чем порт 25; в таких ситуациях о числах лучше думать как об альтернативных названиях порта HTTP, порта SMTP и т. д.

Переменные, описывающие интервал, отношение и порядок, также называют *количественными*, а номинальные переменные – *качественными*. Интервалы и отношения можно дополнительно классифицировать на дискретные и непрерывные. Дискретные переменные имеют конечное число распознаваемых значений, тогда как непрерывные переменные – бесконечное. В данных сетевого трафика почти все величины дискретны. Например, пакет может содержать 9 или 10 байт полезной нагрузки, но не дробное число байтов. Даже такие значения, как время начала, дискретны, даже при наличии возможности определять время с очень высокой точностью. Непрерывные переменные являются главным образом производными, как, например, среднее число байтов в пакете.

Визуализация одномерных данных:

ГИСТОГРАММЫ, ГРАФИКИ КВАНТИЛЕЙ И КОРОБЧАТЫЕ ДИАГРАММЫ

Опираясь на тип переменной, мы можем выбрать разные методы визуализации. Проще всего визуализируются *одномерные* данные, состоящие из одной наблюдаемой переменной. Примерами одномерных измерений могут служить: число байтов в пакете или количество соединений в течение определенного интервала.

Гистограммы

Гистограмма – фундаментальный способ графического представления отношений и интервалов; она описывает, как часто переменная принимает каждое возможное значение. Гистограмма изображается в виде ряда *столбиков*, представляющих дискретные диапазоны значений и *частоты*. То есть если система получает пакеты

с разной скоростью от 0 до 10 000 пакетов в секунду, можно нарисовать гистограмму с 10 столбиками для диапазонов от 0 до 999, от 1000 до 1999 и т. д. Частота – это число, описывающее, сколько раз наблюдаемая величина попала в диапазон столбика.

Создание гистограммы

Основой для создания гистограммы является множество количественных наблюдений. Вот как можно быстро создать гистограмму в оболочке интерпретатора R:

```
> sample <- rnorm(10,25,5)
> sample
[1] 30.79303 25.52480 22.29529 29.20203 21.88355 19.73429 24.99312
[2] 20.79997 22.24344 24.29335
> hist(sample)
```

Функция `rnorm` в языке R принимает размер выборки, среднее значение и стандартное отклонение и генерирует случайную последовательность. Функция `hist`, как это принято в R, избавляет вас от рутины и, например, автоматически определяет количество столбиков.

Дополнительно функция `hist` может принимать следующие параметры:

`prob` (булево значение)

Если имеет значение `True`, гистограмма будет построена так, чтобы общая площадь была равна 1. Если имеет значение `False`, гистограмма будет отражать частоты.

`breaks` (может принимать разные типы значений)

Параметр `breaks` управляет распределением данных по диапазонам. Если передать в этом параметре числовое значение, оно будет интерпретироваться как количество столбиков. Если передать вектор, его элементы будут использоваться как границы диапазонов. Также в этом параметре можно передать строку с названием предопределенного алгоритма или указатель на функцию.

Гистограмма – ценный инструмент для анализа данных, потому что помогает понять структуру распределения переменной и заложить основу для дальнейших исследований. Например, гистограмма помогает определить *модальное значение*, тот есть значение, появляющееся наиболее часто. Модальные значения в гистограммах наблюдаются как пики. Обычно анализ гистограммы проводят, чтобы получить ответы на два вопроса:

1. Является ли распределение нормальным или каким-то другим известным распределением?
2. Каковы модальные значения?

В качестве примера такого подхода к анализу рассмотрим гистограмму на рис. 10-3. На этой гистограмме распределения размеров сообщений в потоке данных BitTorrent можно видеть заметный пик для диапазона от 78 до 82 байт. Этот пик – характерная особенность протокола BitTorrent и является результатом обращения одного узла BitTorrent к другому за информацией о наличии у того конкретной части файла и возврата отрицательного ответа.

Знание модальных значений позволяет формулировать новые вопросы. Определив модальные значения в распределении, можно вернуться к исходным данным и изучить записи, содержащие эти модальные значения. В примере на рис. 10-3 можно было бы рассмотреть записи со вторым модальным значением (пик 250–255) и посмотреть, имеет ли соответствующий им трафик какие-либо

отличительные характеристики – длительность взаимодействий, связь с пустыми адресами и т. д. Модальные значения управляют рассуждениями.

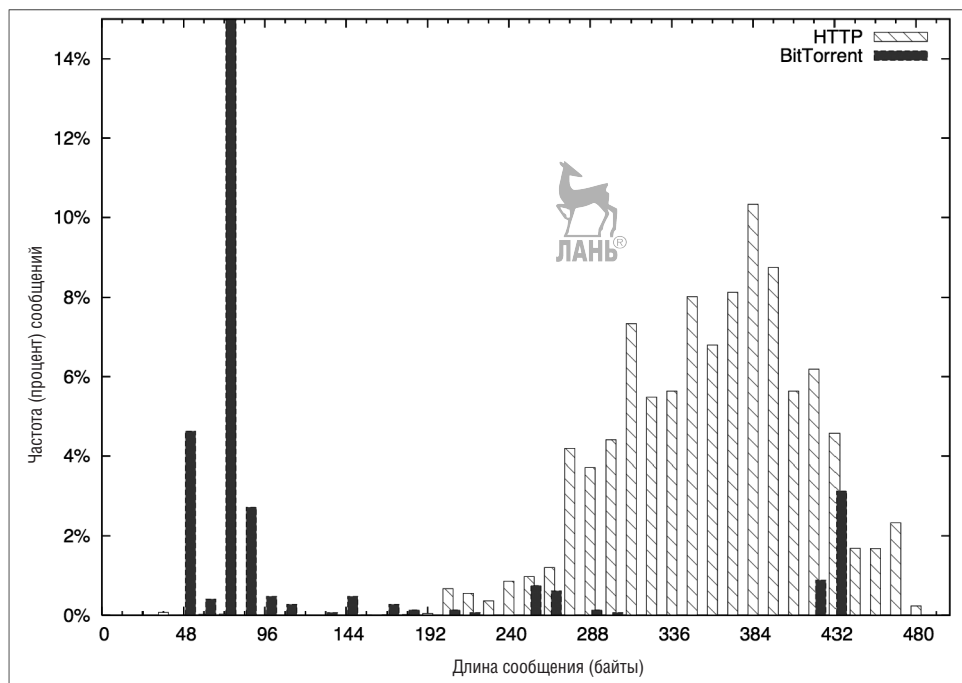


Рис. 10-3. Распределение размеров сообщений BitTorrent

Этот процесс, состоящий из визуализации с последующим возвратом к исходным данным за более подробной информацией, может служить хорошим примером итеративного подхода к анализу, изображенного на рис. 10-2. Разведочный анализ – это циклический процесс, следуя которому, аналитики неоднократно возвращаются к источнику (или нескольким источникам), чтобы понять отличительные особенности.

Столбиковые диаграммы

Столбиковая диаграмма как аналог гистограммы используется для анализа одномерных качественных данных. Так же как гистограмма, она показывает, как часто встречаются наблюдаемые значения с помощью столбиков. На рис. 10-4 показан пример такой диаграммы, отражающей, как часто наблюдаются обращения к разным службам в данных сетевого трафика.

Разница между столбиковыми диаграммами и гистограммами заключается в определении столбиков. Гистограммы используются для представления количественных данных, которые можно группировать в диапазоны и изображать эти диапазоны в виде столбиков. Выбор границ диапазонов является произвольным, и их можно менять, чтобы получить более наглядное представление. Столбиковые гистограммы используются для представления качественных значений, которые являются дискретными, перечисляемыми и часто не поддерживают понятия упорядочения. Отсутствие упорядочения является особой проблемой при

работе с несколькими столбиковыми диаграммами – если это ваш случай, следите за тем, чтобы на всех диаграммах качественные признаки отображались в одном и том же порядке и включались нулевые значения.

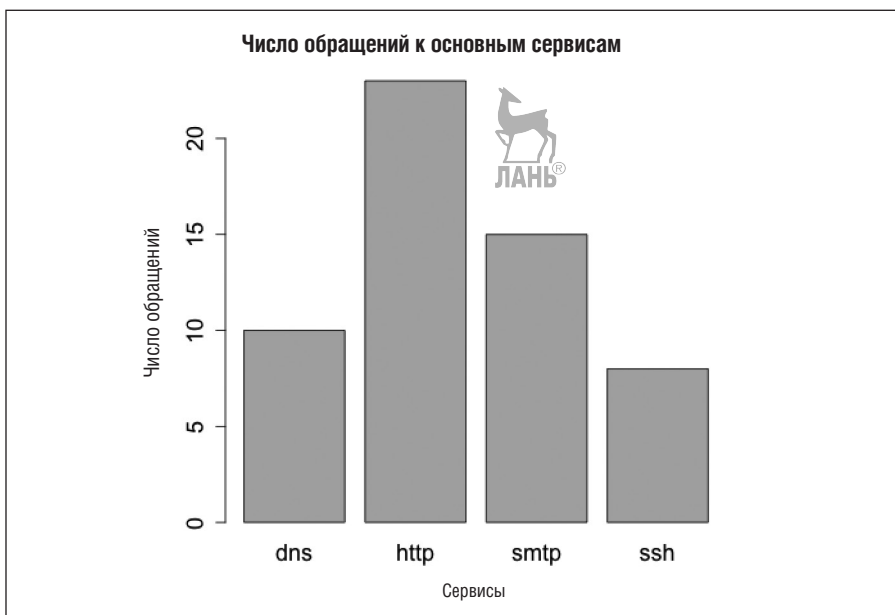


Рис. 10-4. Столбиковая диаграмма, показывающая распределение числа обращений к основным сервисам

В аналитических исследованиях предпочтительнее использовать не круговые, а столбиковые диаграммы. Связано это с тем, что порой очень непросто заметить тонкие различия в размерах секторов, которые в столбиковых диаграммах намного более очевидны.

Графики квантилей

График квантилей (Quantile-Quantile, QQ) позволяет сравнить распределение двух переменных. Это двумерный график, вдоль оси x которого откладываются значения одного из распределений, нормализованные как квантили, а вдоль оси y – значения второго распределения, тоже нормализованного как квантили. Например, если я разобью каждое распределение на 100 процентилей, тогда первой точкой на графике будет первый перцентиль из каждого распределения, 50-й точкой – 50-й перцентиль, и т. д.

На рис. 10-5 и 10-6 показаны два графика квантилей, а за ними – код для их создания. Эти графики, сгенерированные с помощью функции `qqplot`, изображают результат сравнения двух фактических распределений с теоретическим нормальным распределением. Первый график показывает ожидаемый результат для двух похожих распределений – значения образуют диагональ. Здесь можно видеть некоторые отклонения, но они незначительны. Сравните эти результаты с результатами на втором рисунке, где сопоставляются нормальное и равномерное распределения; здесь на концах графика наблюдаются значительные отклонения.

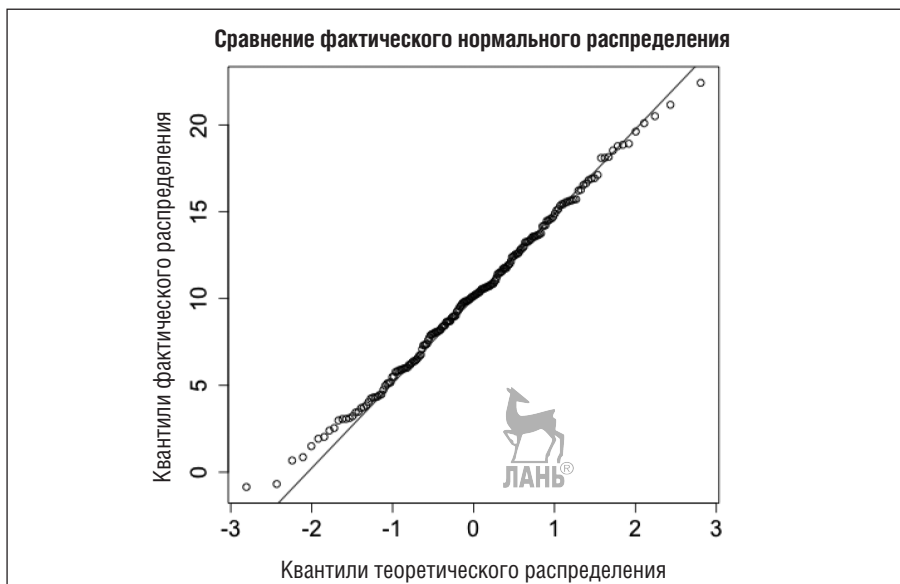


Рис. 10-5. Пример сопоставления теоретического нормального распределения с близким к нему фактическим распределением

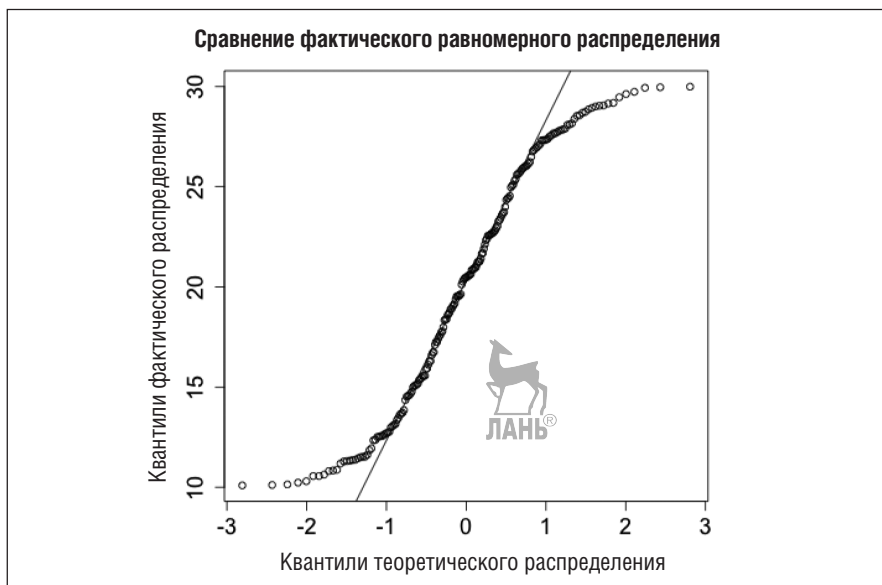


Рис. 10-6. Пример сопоставления теоретического нормального распределения с фактическим равномерным распределением

```
> # Сгенерировать равномерное и нормальное распределения
> set.normal <- rnorm(n = 200, mean=10, sd = 5)
> set.unif <- runif (n = 200, min = 10, max = 30)
> # Построить график сравнения нормального распределения с нормальным
> qqnorm(set.normal, main='QQ Plot Against a Normal Dist')
```

```
> qqline(set.normal)
> # То же самое для равномерного распределения
> qqnorm(set.unif,main='QQ Plot Against a Uniform Dist')
> qqline (set.unif)
```

В языке R есть несколько функций для построения графиков квантилей. Наиболее важными являются: `qqnorm`, которая строит график сравнения набора данных с нормальным распределением; `qqplot`, которая строит график сравнения любых двух наборов данных; и `qqline`, которая рисует базовую линию.

Действительно ли это распределение нормальное?

В главе 6 и в этой главе мы обсудили несколько методов, помогающих определить, является ли распределение данных в наборе нормальным, или, точнее, можно ли использовать нормальное распределение для аппроксимации фактических данных. Аппроксимация распределения, если она возможна, позволяет нам использовать множество дополнительных инструментов. Проблема в том, что в исходных сетевых данных конкретное распределение встречается реже, чем йети. В число упомянутых методов входят:

- критерий Шапиро–Уилка (пример 6-4), проверяющий статистическую нормальность;
- критерий Колмогорова–Смирнова (пример 6-5), критерий общего согласия;
- гистограммы (раздел «Гистограммы» выше), метод визуализации распределения;
- графики квантилей (раздел «Столбиковые диаграммы» выше), метод сравнения распределения данных с нормальным.

Из всех доступных инструментов наиболее предпочтительными я считаю методы визуализации (гистограммы и графики квантилей). Мой интерес к определению распределения – чисто прагматический. Я ищу разумные пороговые значения и что-то, что хорошо укладывается в математические законы, чтобы можно было использовать другие инструменты, так как мы не имеем возможности проводить измерения с более высокой точностью. Обычно идентифицировать злоумышленника довольно легко, если выбрать правильную метрику. Классическая ошибка, которую допускают, используя средние значения и стандартные распределения без первичного исследования данных, состоит в том, что большинство наборов сетевых данных имеют много выбросов. Эти выбросы приводят к появлению абсурдно больших стандартных отклонений, и получающийся в результате порог инициирует тревоги только в случае вопиющих событий.

Пятичисловая сводка и коробчатая диаграмма

Пятичисловая сводка (five-number summary) – это стандартный набор статистических значений, описывающих выборку данных. Он включает следующие пять значений:

- минимальное значение в выборке;
- первый квартиль выборки;
- второй квартиль, или медиана, выборки;
- третий квартиль выборки;
- максимальное значение в выборке.

Квартили – это точки, делящие выборку на четверти, то есть эти пять чисел определяют минимальное значение, 25%-ный порог, медиану, 75%-ный порог и максимальное значение. Пятичисловая сводка позволяет быстро получить общее представление о выборке.

Графически отобразить пятичисловую сводку можно с помощью коробчатой диаграммы (рис. 10-7), которую также называют *ящичной диаграммой*. Коробчатая диаграмма состоит из пяти вертикальных линий, по одной для каждого значения в пятичисловой сводке. Три линии в центре соединены горизонтальными линиями и образуют подобие *коробки*, а внешние линии соединены с коробкой перпендикулярами – *усиками*.

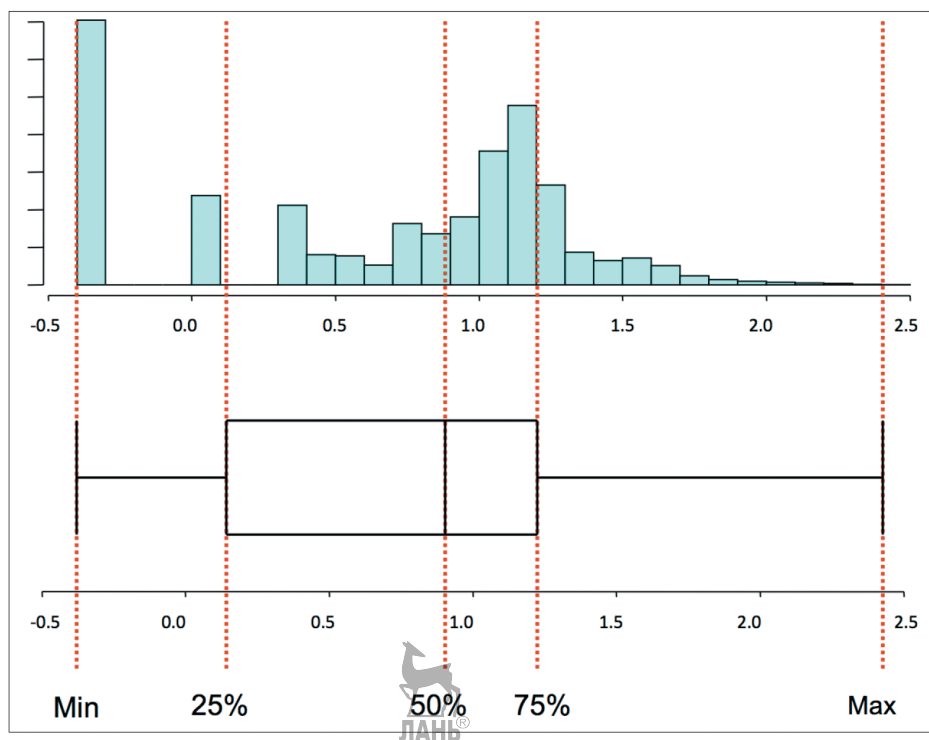


Рис. 10-7. Коробчатая диаграмма и соответствующая ей гистограмма

Создание коробчатой диаграммы

В языке R пятичисловую сводку можно получить с помощью команды `fivenum`, как показано в следующем примере.

```
> s<-rnorm(100,mean=25,sd=5)
> fivenum(s)
[1] 14.61463 22.26498 24.50200 27.43826 37.99568
```

Сама коробчатая диаграмма создается командой `boxplot` (результат которой показан на рис. 10-8):

```
> boxplot(s)
```

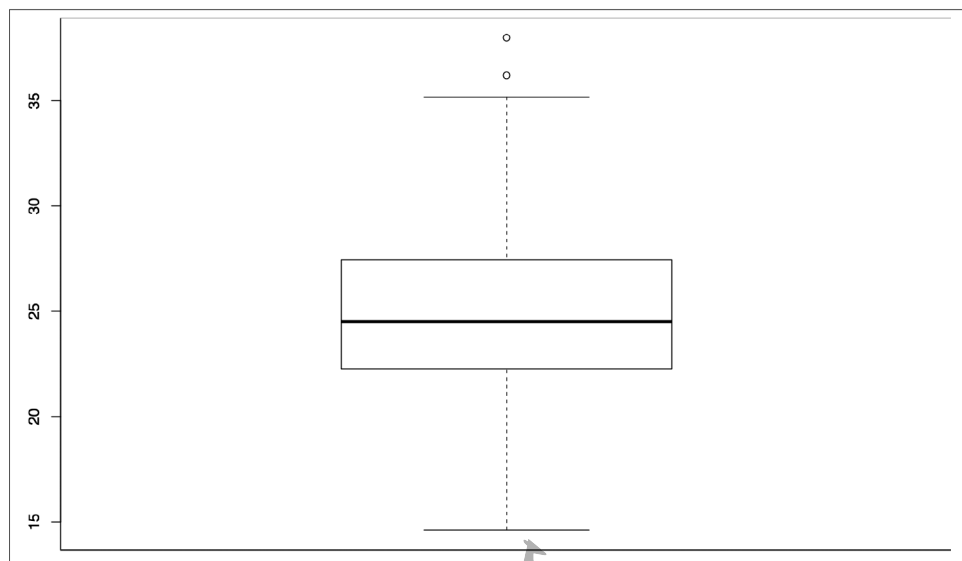


Рис. 10-8. Пример коробчатой диаграммы

Обратите внимание, что на этом графике присутствует несколько точек за пределами усов. Это *выбросы*. Они находятся далеко за пределами первого и третьего квартилей. По умолчанию низкое значение считается выбросом, если расстояние от него до первого квартиля более чем в 1,5 раза превышает межквартильный размах (разность между первым и третьим квартилями). Точно так же высокое значение считают выбросом частью, если расстояние от него до третьего квартиля более чем в 1,5 раза превышает межквартильный размах.

Функция `boxplot` имеет дополнительные параметры:

`notch` (булево значение)

Если имеет значение `True`, в диаграмму будет добавлена отметка, соответствующая медиане. Если на двух графиках метки не накладываются – это явный признак, что их медианы отличаются.

`range` (число)

Описывает, как далеко должны простираться усики. По умолчанию этот параметр принимает значение 1.5, как было описано выше. Если передать в этом параметре значение 0, усики будут иметь длину, достаточную, чтобы никакие значения не считались выбросами.

При работе с пятичисловыми сводками нередко добавляют среднее значение (как на рис. 10-9). Поэтому вам часто будут попадаться коробчатые диаграммы, включающие метку в виде дополнительного символа, обычно `x`, обозначающую среднее значение. В языке R имеется возможность выводить несколько графиков на одном холсте, например:

```
>boxplot(s)
>points(mean(s), pch='x')
```

Здесь параметр `pch` определяет вид метки, параметр `mean(s)` – в данном случае символ `x`.

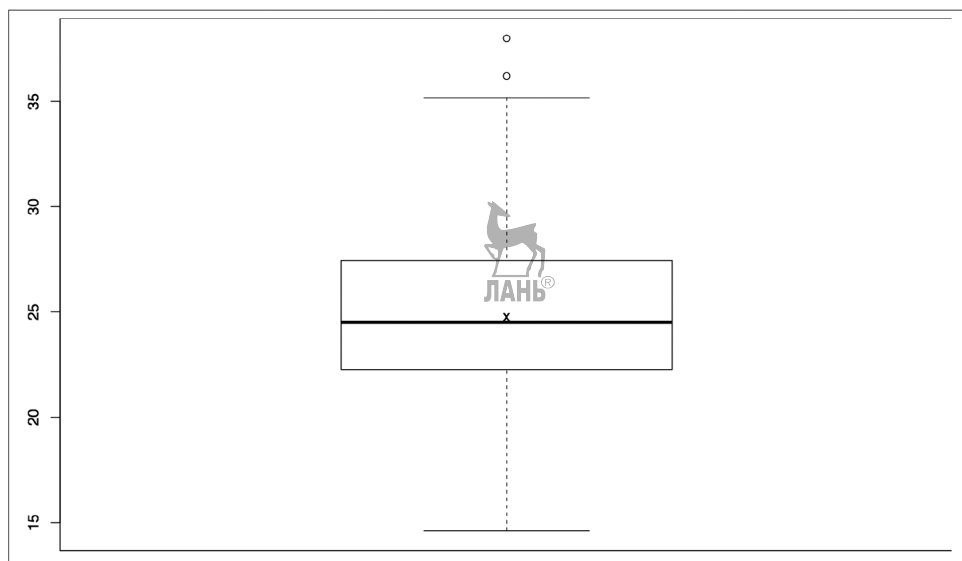


Рис. 10-9. Коробчатая диаграмма со средним значением

Функция `boxplot` может принимать несколько векторов, что делает ее удобным инструментом для быстрого сравнения нескольких дискретных наборов данных. Например, чтобы идентифицировать несколько разных явлений в наборе данных, можно выделить для каждого отдельный столбец. Следующий пример демонстрирует эту возможность с использованием искусственно подготовленных данных и данных сканирования, отображая две коробчатые диаграммы бок о бок (рис. 10-10).

```
> nonscan<-rnorm (100,mean=150,sd=30)
> scan<-runif(50,min=254,max=255)
> boxplot(nonscan,scan,names=c('nonscan', 'scan'))
```

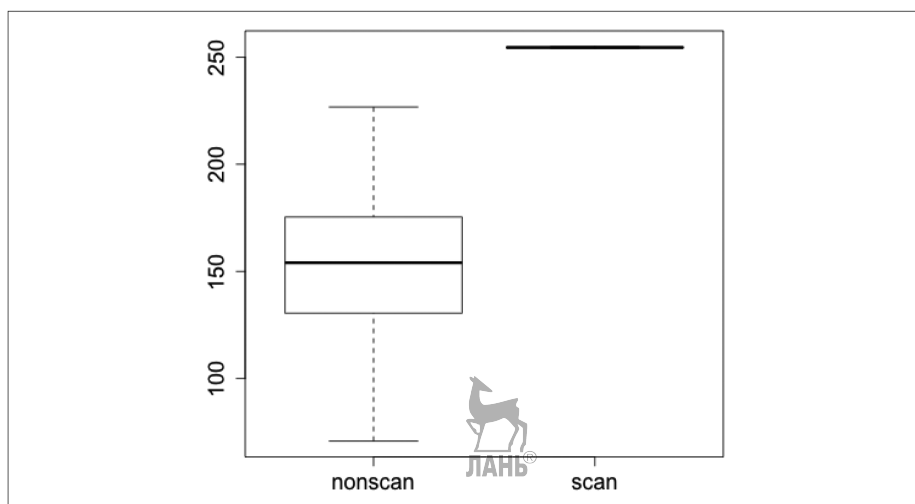


Рис. 10-10. Две коробчатые диаграммы бок о бок

Коробчатые диаграммы редко бывают полезными сами по себе. При анализе единственной выборки гистограммы обычно дают больше информации. Коробчатые диаграммы приобретают ценность, когда требуется сравнить разные выборки, потому что для сравнения гистограмм в этом случае потребуется больше усилий.

Визуализация двумерных данных

Двумерными называют данные, когда каждая точка в пространстве измерений характеризуется двумя переменными. Примерами двумерных данных могут служить: число байтов и пакетов, полученных или переданных в разные интервалы времени (две количественные переменные), или количество пакетов на протокол (количественная и качественная переменные). Для визуализации двумерных данных обычно используются диаграммы рассеяния (когда имеются две количественные переменные), многокомпонентные коробчатые диаграммы (в случае когда одна переменная количественная, а другая качественная) и таблицы сопряженности (в случае двух качественных переменных).

Диаграммы рассеяния

Для визуализации количественных двумерных данных обычно используются диаграммы рассеяния, иллюстрирующие взаимосвязь между двумя порядковыми, интервальными или относительными переменными. Основная задача анализа диаграмм рассеяния состоит в выявлении структуры среди шума. Типичными признаками структур на диаграммах рассеяния являются кластеры, пустые области, линейные взаимосвязи и выбросы.

Давайте для начала рассмотрим диаграмму рассеяния для абсолютно несвязанных данных. На рис. 10-11 показано, как выглядит диаграмма рассеяния шума, в данном случае полученного путем генерирования двух случайных выборок с нормальным распределением. На этом графике нет ничего интересного.

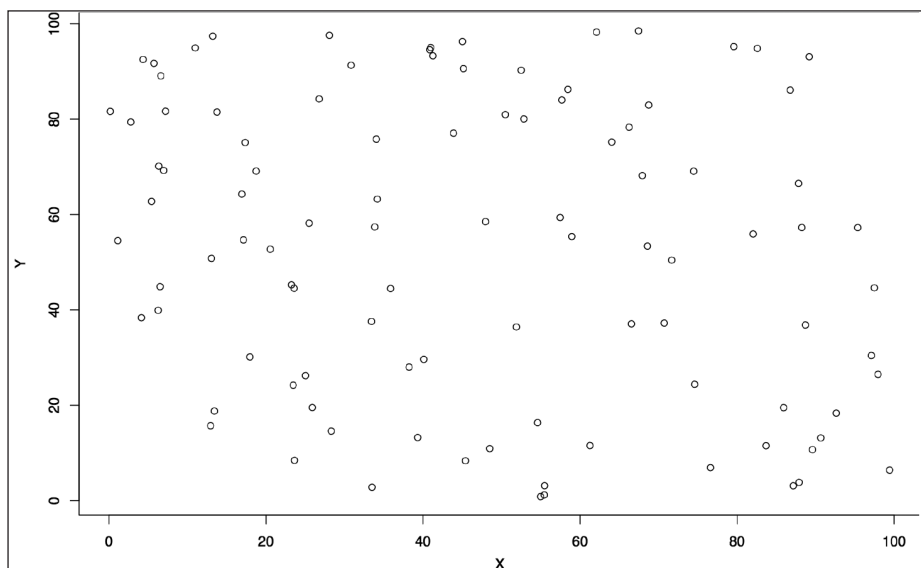


Рис. 10-11. Диаграмма рассеяния шума

Кластеры и пустые пространства наблюдаются на диаграмме рассеяния как области с разной плотностью точек. Диаграмма на рис. 10-11 имеет равномерную плотность разброса точек, координаты которых случайны и никак не связаны друг с другом. Наличие связи между переменными отражается на плотности распределения точек на графике. На рис. 10-12 показано, как выглядят кластеры и пустые области. В этом примере наблюдается заметное скопление точек в нижнем левом квадранте и почти полное их отсутствие в верхнем правом квадранте.



Рис. 10-12. Кластеры и пустые области

Линейные зависимости, как подсказывает название, на диаграмме рассеяния выглядят как линии. Силу связи между переменными можно оценить по плотности точек вокруг линии. На рис. 10-13 показан пример трех простых линейных зависимостей в форме $y = kx$. Сила связи между переменными уменьшается сверху вниз.

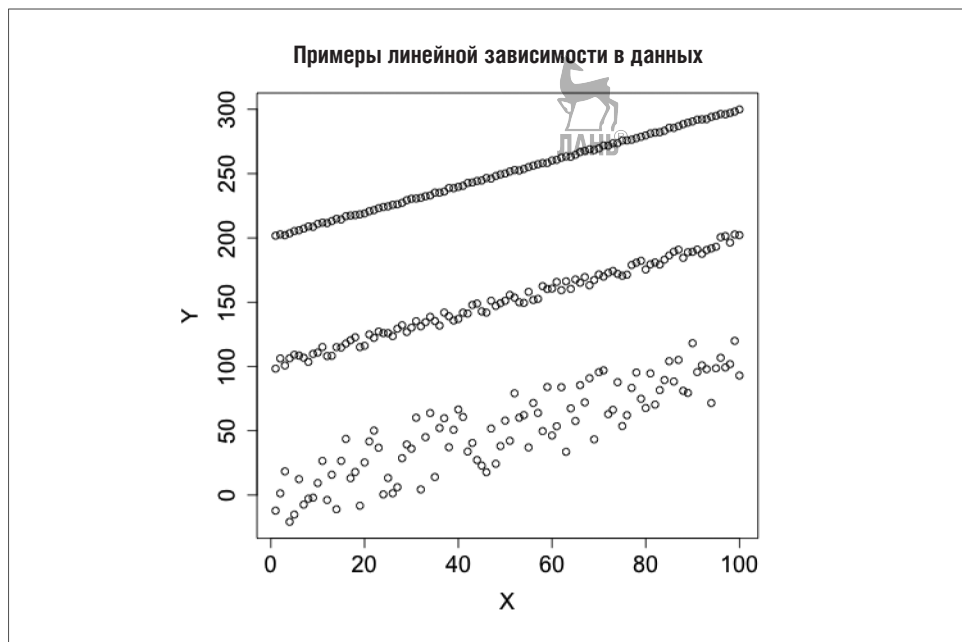


Рис. 10-13. Линейные зависимости между данными

Таблицы сопряженности

Таблицы сопряженности используются для визуализации измерений, состоящих из качественных данных. Таблица сопряженности – это обычная матрица, строки которой соответствуют значениям одной переменной, столбцы – значениям другой переменной, а в ячейках указывается количество наблюдений, когда оба значения наблюдались вместе. В зависимости от реализации таблицы сопряженности могут включать итоговые строку и столбец, содержащие суммы всех значений в столбце и строке соответственно.

В языке R таблицы сопряженности создаются командой `table`. Она возвращает таблицу, для которой затем можно получить итоговые строку и столбец:

```
# Пример создания таблицы на языке R из двух векторов
# с именами хостов и сервисов
> hosts[0:3]
[1] "A" "B" "A"
> services [0:3]
[1] "http" "dns" "smtp" "ssh"
> # Создание таблицы, векторы с именами хостов и сервисов
> # не обязательно должны иметь одинаковую длину
> info.table<-table(hosts,services)
> info.table
      services
hosts dns http smtp ssh
A      2   15   10   0
B      6    5    3    4
C      3    3    1    2
```

```
> # Получить итоговые данные можно вызовом margin.table
> margin.table(info.table)
[1] 54
> margin.table(info.table, 1)
hosts
  A B C
27 18 9
> margin.table(info.table, 2)
services
dns http smtp ssh
 11  23  14  6
```



Визуализация многомерных данных

Многомерными называют наборы данных, содержащие, по крайней мере, три переменные в каждой точке наблюдения. Визуализация многомерных данных – это больше методика, чем определенный набор графиков. В большинстве случаев визуализация многомерных данных заключается в создании двумерного графика, в который затем, тем или иным способом, добавляется дополнительная информация. Обычно для этого используются разные цвета, значки, изображения и анимация.

Хорошая визуализация многомерных данных отображает информацию из всех наборов данных и не перегружает исследователя деталями. Мы легко можем изобразить дюжину различных наборов данных на одной диаграмме, но результаты часто оказываются обескураживающими.

Самый простой подход к визуализации многомерных данных заключается в наложении друг на друга наборов данных на одной диаграмме с использованием различных меток или цветов, указывающих на набор данных. Как показывает опыт, на диаграмме можно изобразить до четырех серий данных, не вызывая перегрузки читающего. Выбирая цвета и метки, руководствуйтесь следующими правилами:

- не используйте желтый цвет; он очень похож на белый и часто неотличим от него на распечатках и мониторах;
- выбирайте символы, непохожие друг на друга; лично я часто использую пустые кружки, заштрихованные кружки, треугольники и крестики;
- выбирайте цвета, далеко отстоящие друг от друга на цветовом диске: я предпочитаю красный, зеленый, синий и черный;
- избегайте сложных символов; многие пакеты для создания графических диаграмм предлагают множество похожих друг на друга звездочек;
- будьте последовательны в выборе цветов и символов. Проще говоря, если вы решите изобразить HTTP красным, а FTP – треугольниками, это плохой выбор.

За дополнительной информацией о средствах визуализации многомерных данных в языке R обращайтесь к разделу «Аннотация технологии визуализации» в главе 6.

Альтернативой отображению нескольких наборов на одной диаграмме является создание нескольких небольших диаграмм рядом друг с другом. Такие графики обычно называют матрицами, или решетками, диаграмм. Хорошим примером может служить решетка, сгенерированная командой `pairs`. Получив массив данных, `pairs` генерирует матрицу, как показано на рис. 10–14, – для каждой пары перемен-

ных создается своя диаграмма рассеяния. Каждая диаграмма отражает связь пары переменных, и, как демонстрирует этот пример, в такой матрице легко заметна зависимость между статьями и их объемом, тогда как все остальные переменные выглядят независимыми друг от друга.

Поддержка матриц попарных диаграмм в R – очень мощное средство исследования данных и хороший пример выразительной силы визуализации многомерных данных. Отображая несколько простых диаграмм в виде матрицы с четко определенной и ясной структурой, можно быстро обработать огромный объем данных. Но, создавая такие матрицы диаграмм, не забывайте об их простоте – учитывайте, как они используют полезную площадь.

Я считаю, что в большинстве случаев матрицы диаграмм являются наилучшим вариантом графического представления многомерных данных, потому что они обеспечивают ясный и простой механизм отображения отношений между переменными. Компактное размещение графиков на рис. 10–14 – важная конструктивная особенность, достойная, чтобы обратить на нее внимание. Матрицы диаграмм обычно включают избыточный объем метаданных (например, оси, штрихи и метки). Чтобы избавиться от этой проблемы, используйте как можно меньше элементов представления данных в графиках: убирайте избыточные оси и внутренние метки и штрихи.

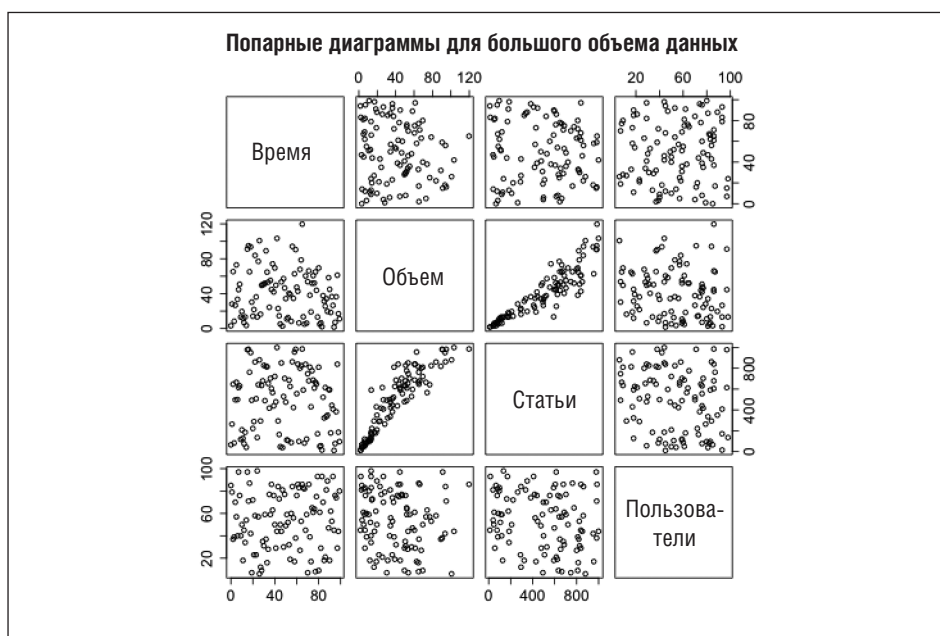


Рис. 10-14. Матрица диаграмм

Под анимацией я подразумеваю именно то, что следует из названия: сначала создается множество изображений, а затем они отображаются, сменяя друг друга. По своему опыту могу сказать, что анимация – не самый лучший выбор. Она уменьшает объем информации, непосредственно наблюдаемой аналитиком, из-за чего тот постоянно должен помнить, что он видел на предыдущих шагах.

Оперативная визуализация

Разведочный анализ и визуализация являются составными частями исследовательского процесса и, как таковые, иногда имеют недостаточную точность. В процессе разведочного анализа исследователь сталкивается с большим количеством тупиковых ситуаций и ложных выводов. Поэтому перед внедрением аналитического процесса в эксплуатацию необходимо модифицировать процедуры визуализации, чтобы добавить в них действия и ответы. Дополнительные обработка и модификация для совершенствования визуализации необходимы для плодотворной работы. Далее описываются правила и примеры хорошей и плохой визуализации, а также способы решения проблем, возникающих при визуализации данных в сфере информационной безопасности.

Правило первое: изображайте аномалии на графиках отдельно от обычных данных

Конструируя оперативные диаграммы для службы информационной безопасности, необходимо предусматривать возможность появления аномалий и их наглядное отображение – в конце концов, смысл поиска опасных событий как раз и заключается в определении таких явлений. Функции отображения, такие как автомасштабирование, могут работать против вас, скрывая данные о странностях. Например, пример аномальных событий на рис. 10-15. Этот график имеет две аномалии, но одна из них остается незаметной из-за необходимости отобразить вторую.



Рис. 10-15. Влияние автомасштабирования на визуализацию аномалий

Есть две стратегии отображения таких пиков. Первая – использовать *логарифмическое масштабирование* вдоль зависимой оси (y). При логарифмическом масштабировании линейная шкала заменяется логарифмической. Например, риски

на оси y , соответствующие величинам 10, 20, 30, 40, заменяются рисками с отметками 10, 100, 1000, 10 000. На рис. 10-16 показан график того же явления с логарифмической осью y . Использование логарифмического масштаба уменьшит различия между главной аномалией и остальными данными.

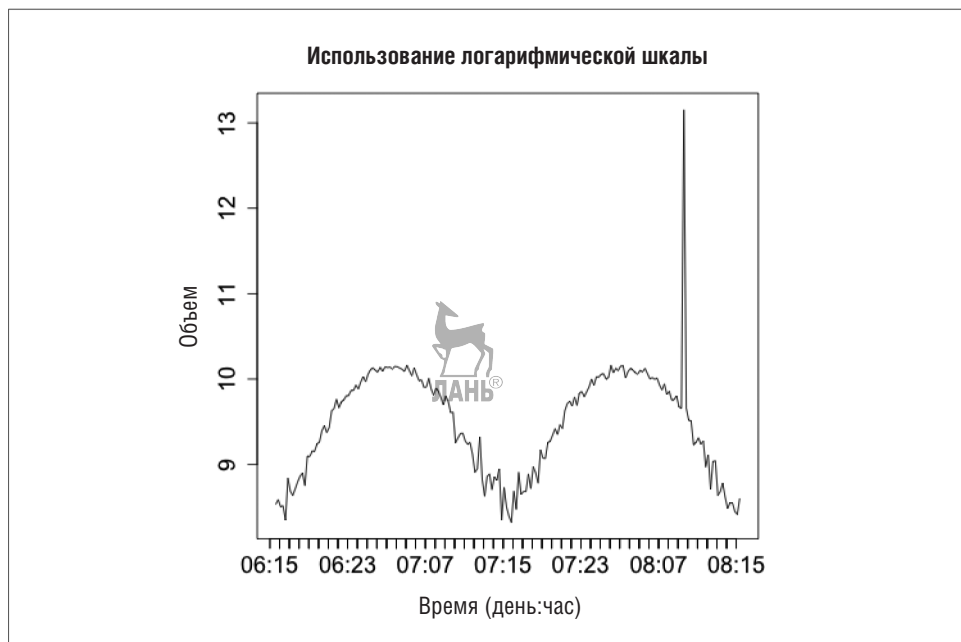


Рис. 10-16. Использование логарифмической шкалы для ограничения влияния больших выбросов

Логарифмическую шкалу удобно использовать для разведочного анализа, и многие инструменты предлагают такую возможность. В языке R, например, достаточно передать команде создания диаграммы параметр `log` с именем оси, для которой должен быть установлен логарифмический масштаб (например, `log = "y"`).

Однако сам я не люблю использовать логарифмический масштаб в процедурах оперативной визуализации, которые разрабатываю для службы эксплуатации. При использовании логарифмического масштаба возможна потеря информации о типичных явлениях, например кривая, соответствующая типичному трафику, на рис. 10-16 оказалась деформированной логарифмическим масштабом. Кроме того, иногда бывает трудно объяснить, что такое логарифмический масштаб, и я не люблю объяснять это снова и снова. Когда кто-то постоянно рассматривает одни и те же данные, я предпочел бы отображать их в линейном масштабе.

По этим причинам я стараюсь последовательно использовать линейный масштаб на графиках, выявляя и удаляя аномальные выбросы. Мы видели пример такого подхода на рис. 10-8, где R автоматически выделил выбросы в изображении

коробчатой диаграммы. Разрабатывая диаграммы для оперативной визуализации, я оцениваю диапазон значений на графике и обычно устанавливаю верхнюю границу, равной 98-му процентилю наблюдаемых данных. Когда в наблюдениях появляется аномалия, я изображаю их отдельно и по-другому, не так, как типичные данные, чтобы подчеркнуть, что это – аномалия, как показано на рис. 10-17.

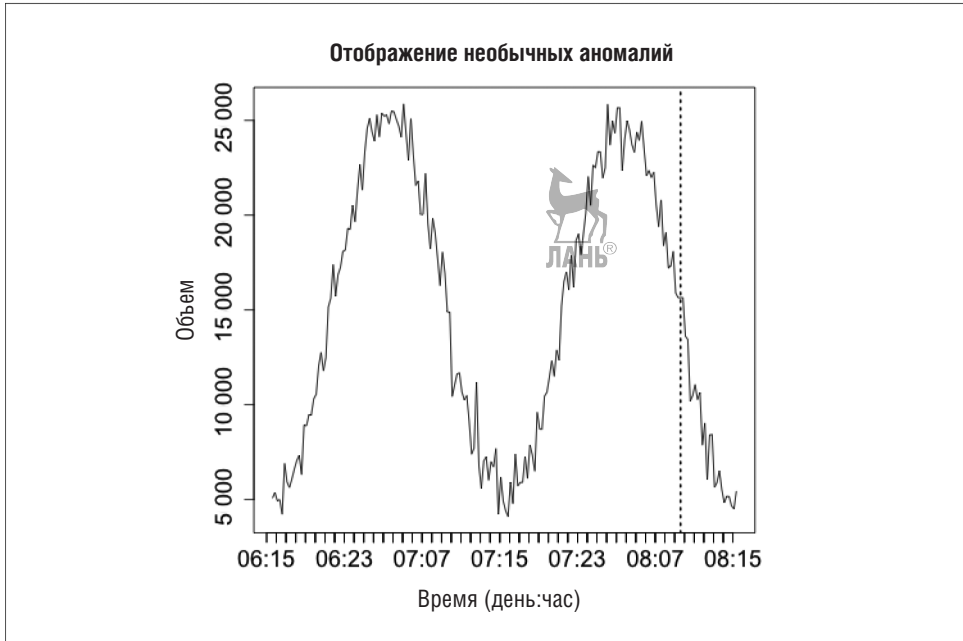


Рис. 10-17. Отделение аномалий от нормальных данных

Аномалия на рис. 10-17 изображена одной пунктирной линией, подсказывающей, что этот пик не уместается в координатную сетку. Вторая аномалия (наблюдается в 7:11) не обнаруживается аналитическим процессом, но визуализация делает ее заметной. Однако пунктирная линия, соответствующая аномалии на графике, выглядит абсолютно бессмысленной из-за отсутствия пояснительной информации, и это ведет нас к правилу два.

Правило два: подписывайте аномалии

Приняв на вооружение первое правило, вы, безусловно, выработаете некоторые критерии, помогающие отличать аномалии от обычного трафика. Целью процедур оперативной визуализации является помощь в обнаружении аномалий, поэтому к ним применимы те же правила, что и при построении систем обнаружения вторжений (см. главу 7), – упреждающая выборка данных для уменьшения времени реакции оператора. Например, на рис. 10-18 аномалия снабжена описанием причин, вызвавших ее, и некоторой дополнительной информацией.

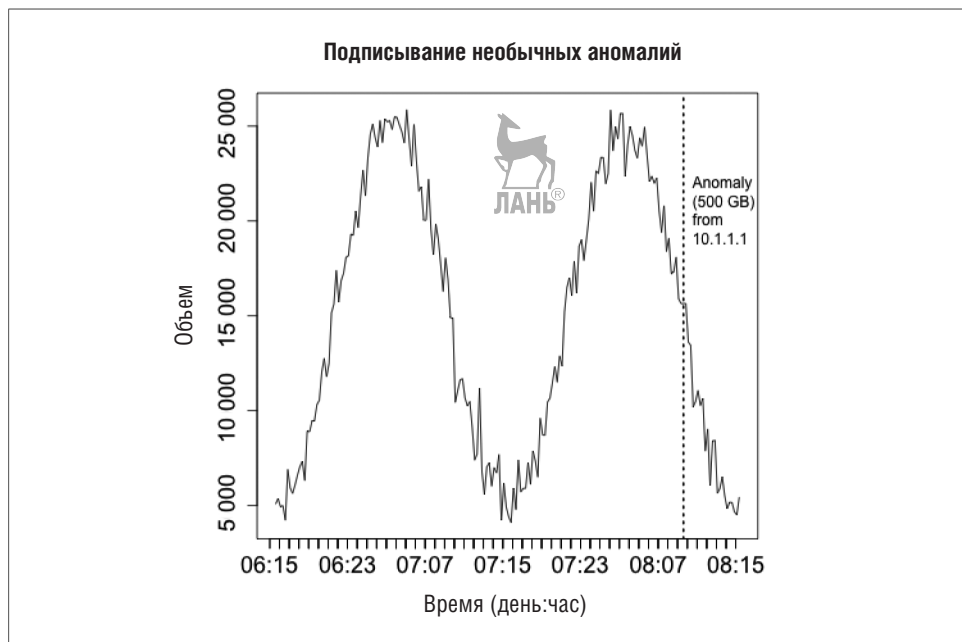


Рис. 10-18. Описание аномалий в помощь расследованию

Описание аномалий на графике может пригодиться для быстрой оценки ситуации, но когда аномалий слишком много (а внедрив первое правило, следует ожидать, что аномалий может быть очень много), такое решение может оказаться неприемлемым. Это можно наблюдать на рис. 10-18, где подпись, хотя и информативна, занимает примерно одну пятую доступного горизонтального пространства. Лучший способ – добавить описание аномалий в отдельной таблице, поместив ее рядом с графиком. Это позволит вам вывести столько данных, сколько понадобится.

Правило три: используйте линии тренда, отделяйте артефакты от наблюдений

Оперативная визуализация должна использовать методы суммирования и сглаживания, помогающие аналитику обрабатывать данные и не утонуть в деталях, и в то же время представлять фактические события, имевшие место, и не додумывать за него. Поэтому, разрабатывая оперативную визуализацию, я предпочитаю добавлять в графики исходные данные, а также дополнительные сглаживающие линии. На рис. 10-19 показан простой пример такого подхода к визуализации. Здесь для сглаживания аномалий я использовал скользящее среднее.



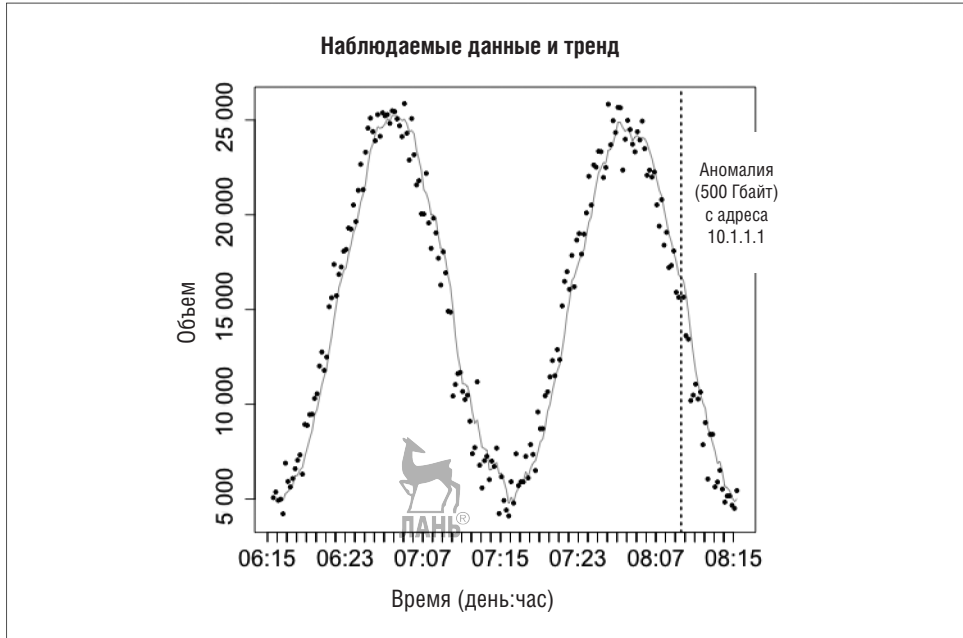


Рис. 10-19. Сглаживание непосредственных наблюдений скользящим средним

При создании такой диаграммы важно гарантировать, что аналитик сможет четко различать исходные данные и артефакты, добавленные вами в помощь анализу. Так же, как того требует первое правило, необходимо учитывать влияние аномалий – они не должны влиять на сглаженную кривую.

Правило четыре: будьте последовательны

Методы визуализации основаны на нашем врожденном умении подмечать шаблоны. Однако это может сыграть злую шутку с аналитиком. Например, если вы решили изобразить трафик HTTP красной линией, а на другом графике в том же пакете визуализации выбрали красный цвет для представления входящего трафика, кто-то обязательно решит, что это тот же трафик HTTP.

Правило пять: добавляйте контекстную информацию

Кроме подписей для аномалий, желательно включать в графики дополнительную ненавязчивую контекстную информацию, которая может помочь упростить анализ. Например, на рис. 10-20 в график добавлены некоторые серые области, выделяющие рабочие часы.



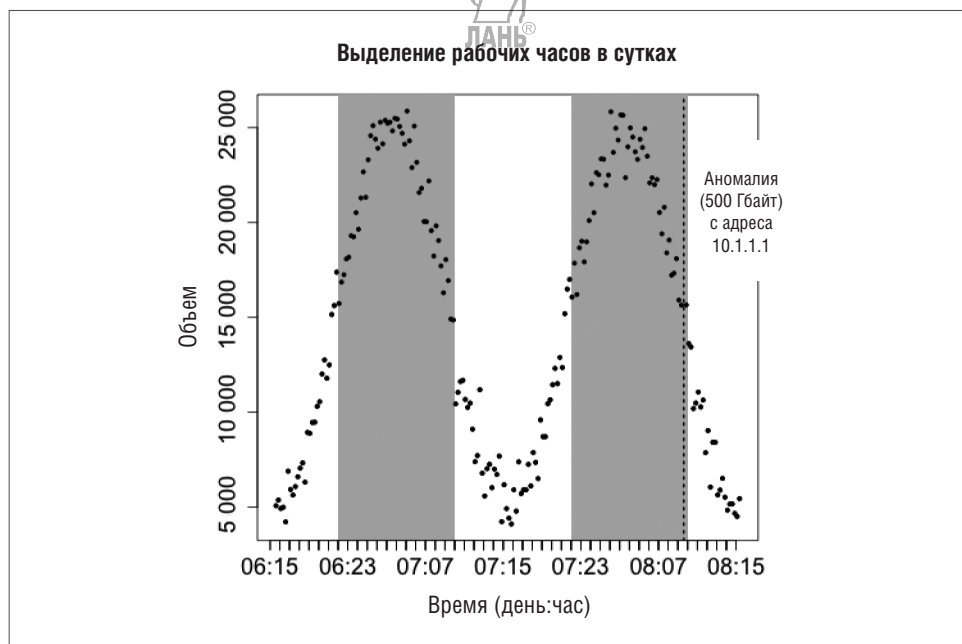


Рис. 10-20. Выделение цветом рабочих часов в сутках

Правило шесть: избегайте кричащих эффектов в пользу выразительности

Наконец, важно отметить, что оперативная визуализация должна обрабатывать данные быстро и многократно. Это не витрина для демонстрации инновационных графических возможностей. Цель оперативной визуализации – представить информацию быстро и ясно. Чрезмерное увлечение графическими эффектами, такими как анимация, выбор необычных цветов и т. п., увеличивает время, необходимое для обработки изображения, не добавляя новой информации.

Будьте особенно осторожны, используя в визуализации метафоры из киберпространства или реального мира. Прихотливые изыски *быстро* надоедают, и в данном случае мы не имеем дела с материальным миром. Метафоры, такие как «открытие стола» или «отпирание всех дверей в здании» (я видел такие визуализации в своей практике и предпочел бы говорить о них поменьше), часто выглядят неплохо на первый взгляд, но нередко они требуют применения сложных анимационных эффектов (которые занимают время) и теряют в информативности. Отдавайте предпочтение простым, выразительным и серьезным изображениям.

Правило семь: при выполнении продолжительных операций выводите подсказку о ходе процесса

Когда я выполняю запросы SiLK, я обычно добавляю ключ `--print-file`, не потому, что хочу сообщить, к каким файлам происходит доступ, а чтобы добавить индикатор, иллюстрирующий ход выполнения операции, и не возникало ощущения, что система зависла. Конструируя визуализацию, важно знать, сколько времени она будет выполняться, и сообщить пользователю, что процесс подготовки данных для визуализации продолжается.

ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ ДЛЯ ЧТЕНИЯ

1. *Грэг Конти (Greg Conti)*. Security Data Visualization: Graphical Techniques for Network Analysis (No Starch Press, 2001).
2. Руководство NIST по разведочному анализу данных (<http://1.usa.gov/ex-data-an>).
3. *Кэти О'Нил (Cathy O'Neil)* и *Рэйчел Шатт (Rachel Schutt)*. Doing Data Science (O'Reilly, 2013, http://oreil.ly/doing_data_science)¹.
4. *Эдвард Тафт (Edward Tufte)*. The Visual Display of Quantitative Information (Graphics Press, 2001).
5. *Джон Тукей (John Tukey)*. Exploratory Data Analysis (Pearson, 1997).



¹ *О'Нил Кэти, Шатт Рэйчел*. «Data Science. Инсайдерская информация для новичков, 2019, Питер, ISBN: 978-5-4461-0622-6. – Прим. перев.

Глава 11



О «прощупывании»

К настоящему моменту мы обсудили ряд методов сбора и анализа данных. Теперь познакомимся с поведением атакующего.

Вспомните наше обсуждение различий между аномалиями и сигнатурами. Основное внимание в этой книге уделяется поиску надежных механизмов обнаружения аномалий, а чтобы найти эти механизмы, мы должны уметь определять типичные признаки действий атакующего. *Признаки первых попыток проникновения, или **прощупывание***, о которых рассказывается в этой главе, как раз и являются такими признаками.

Под первыми попытками проникновения, или **прощупыванием**, подразумевается последовательность систематических неудач при соединении с целевой системой с использованием ссылок. Такими ссылками могут быть IP-адреса, URL или адреса электронной почты. *Подозрительными* такие попытки делает то, что законному пользователю не нужно действовать методом проб и ошибок – работодатель должен сообщить ему все необходимые ссылки. Когда вы поступаете на работу в новую компанию, вам *сообщают* имя почтового сервера – вы не должны угадывать его.

У атакующих нет доступа к этой информации. Они вынуждены строить предположения, попытаться украсть или как-то иначе выяснить эти данные о системе, и на этом этапе они совершают ошибки. Часто эти ошибки повторяются систематически. Выявление таких ошибок и отделение их от невинных промахов – важный первый шаг в процессе анализа.

В этой главе мы рассмотрим шаблоны поведения обычных пользователей, которые нарушаются атакующими. Эта глава объединяет все, о чем рассказывалось в предыдущих главах, включая сведения об электронной почте, сетевом трафике и анализе социальных сетей.



Модели нападения

Прежде чем начинать разговор о шаблонах поведения атакующего, нужно договориться о терминологии. Существует множество статей и исследований *моделей нападения*, в которых предпринимается попытка разбить процесс взлома на ряд дискретных этапов. Эти модели имеют самую разную сложность, от относительно простых и линейных действий до чрезвычайно сложных *древовидных атак*, которые пытаются выявить все уязвимости и использовать соответствующие эксплоиты. Начну с изложения простой, но гибкой модели, которая содержит шаги, характерные для большинства нападений.

Разведка

Атакующий разведывает цель. В зависимости от типа нападения разведка может включать поиск информации в поисковых системах, использование приемов социальной инженерии (публикация сообщений на форумах с целью найти и подружиться с пользователями в сети) или активное сканирование с помощью nmap и других подобных инструментов.

Диверсия, или вторжение

Атакующий запускает эксплоит против целевой системы и захватывает управление. Это может быть сделано с помощью удаленного эксплоита, путем отправки троянского файла или даже взломом пароля.

Конфигурация

Атакующий перенастраивает целевую систему под свои нужды. Он может отключать антивирусные пакеты, устанавливать дополнительное вредоносное программное обеспечение, использовать ресурсы системы и ее возможности и/или устанавливать дополнительные средства защиты, чтобы не позволить другим злоумышленникам захватить систему.

Эксплуатация

После всего вышеперечисленного атакующий приступает к использованию хоста для своих целей. Характер эксплуатации зависит от первоначальных намерений атакующего (подробнее об этом чуть ниже).

Распространение

Если это возможно, атакующий будет использовать хост для нападения на другие хосты. Хост может использоваться в роли прокси-сервера для нападения на соседей (например, на другие хосты за брандмауэром, находящиеся в сети 192.168.0.0/16).

Эта модель не идеальна, но достаточно хорошо описывает поведение атакующего без лишних технических деталей. Всегда имеют место некоторые общие тонкости, например:

- при распространении червей и проведении фишинговых атак злоумышленники полагаются на пассивные эксплоиты и простые приемы социальной инженерии. Эти нападения основываются на желании жертвы щелкнуть на ссылке или получить файл, для чего необходимо, чтобы приманка (имя файла или история, окружающая его) была достаточно привлекательна для щелчка. К слову, когда писались эти строки, я наблюдал серию фишинговых атак с использованием кредитных рейтингов в качестве приманки – сначала мне сообщили, что мой кредитный рейтинг вырос, а потом я получил злое предупреждение о последствиях недавно упавшего кредитного рейтинга. В одноранговых сетях атакующие часто распространяют троянцев с названиями популярных игр или альбомов для привлечения жертв. Даже в этом случае все еще могут проводиться «разведочные» мероприятия. Фишинговые атаки, проводимые в рамках многих АРТ-атак (Advanced Persistent Threat – целевые кибератаки), часто зависят от исследования привычек целевой аудитории и выявления жертв, которые с наибольшей вероятностью откликнутся на специально созданное письмо;

- черви часто объединяют разведку и вторжение в один этап. Некоторые примеры такого подхода будут показаны ниже в этой главе (в частности, в примере 11-1), где атакующий просто запускает эксплойты против известных PHP URL без предварительной проверки их существования.

Атакующий не тот, кем вы его считаете: заинтересованные и незаинтересованные атакующие

Размышляя об атакующих, мы склонны считать их технически грамотными людьми, выясняющими слабые места в сети с целью кражи файлов или другой информации. Это – классический пример *заинтересованного атакующего*, который хочет вторгнуться и захватить управление важной сетью с целью кражи денег или данных, достижения уважения в определенных кругах и бог знает чего еще.

В настоящее время подавляющее большинство нападений проводится *незаинтересованными* атакующими, которые хотят вторгнуться в как можно большее число хостов и не заботятся об особенностях каждого. Незаинтересованные нападения обычно автоматизированы и не обращают внимания на большое количество отказов. Из-за этого этапы разведки и вторжения часто объединяются вместе. Червь, действующий автоматически, может просто предпринять попытку атаки на каждый хост, встречающийся на его пути, независимо от того, уязвим он или нет.

Незаинтересованные атакующие полностью полагаются на свои инструменты и ожидают, что рано или поздно им встретится уязвимый хост. В большинстве случаев им даже неинтересно знать о существовании конкретного хоста, пока не получат управление над ним. На первых порах незаинтересованные атакующие собирали роботов для проведения DDoS-атак на сети. Создатели таких роботов захватывали управление над примерно десятком машин, устанавливали на них программное обеспечение для DDoS-атак и затем запускали лавинные рассылки пакетов SYN против целевых хостов. По мере расширения возможностей подключения расширялись также возможности и гибкость ботнетов – атакующие начали устанавливать программное обеспечение, играющее роль прокси, получали изображения с подключенных веб-камер и продавали их порносайтам, устанавливали спам-роботов и осуществляли массу других вредоносных действий.

Проще говоря, незаинтересованные атакующие действуют подобно комбайнам, обрабатывающим большие площади. Незаинтересованный атакующий запускает сценарий, затем фильтрует его результаты и смотрит, что он получил. Хост имеет веб-камеру и расположен в общежитии колледжа? Источник порно. Хост имеет много дискового пространства и широкий канал? Файловый сервер. Хост является домашней машиной? Кейлоггер.

Такой подход, напоминающий сбор урожая, обычно свидетельствует о том, что атакующему безразлично, на какой хост он посягает. В первые годы использования SCADA-систем было очевидно, что атакующие понятия не имели, что за хост перед ними. Они видели лишь хост с Windows, набором непонятных приложений и дополнительными каталогами. Даже сейчас не редкость, когда атакующий захватывает управление медицинской аппаратурой и использует ее в качестве ботнета.

В последние годы в «конфигурации» хостов стали включать их роли: кто им владеет, для чего используется и чем можно похвастаться, захватив управление им. Например, если две страны, враждебно настроенные друг к другу, имеют общую границу, резидентные хакерские группы будут портить сайты в стране-противнике. Министерства обороны многих стран поддерживают буквально тысячи сайтов, от серверов разведывательных служб до компьютеров в начальных школах. В их числе нетрудно найти уязвимый сайт и затем объявить всему миру о «взломе». Это то, что нужно иметь в виду.



ПРОЩУПЫВАНИЕ: НЕВЕРНАЯ КОНФИГУРАЦИЯ, АВТОМАТИЗАЦИЯ И СКАНИРОВАНИЕ

Под *прощупыванием* подразумевается любая неудачная попытка хоста обратиться к ресурсу. В TCPощупывание означает, что хост не смог обратиться к конкретной комбинации адрес/порт хоста, тогда как в HTTPощупывание означает неспособность получить доступ к URL. Отдельные попыткиощупывания – вполне ожидаемое явление. Но если наблюдаются многократно повторяющиеся попыткиощупывания, это должно вызывать беспокойство. Прощупывание может быть обусловлено несколькими причинами: ошибки в процессе поиска или в настройках, работа автоматизированного программного обеспечения и сканирование.

Ошибки в процессе поиска

Признакиощупывания обычно появляются, когда искомый ресурс просто отсутствует. Это может быть временное явление из-за ошибки в адресе или из-за того, что искомый ресурс никогда не существовал.

Имейте в виду, что люди *редко* вводят адреса вручную. Большинство пользователей никогда не вводят IP-адрес непосредственно, обычно они полагаются в этом на сервер имен DNS. Также пользователи редко вручную вводят URL-адреса (кроме доменов верхнего уровня), предпочитая щелкать на ссылках или копировать их из других приложений. Когда кто-то вводит неправильный адрес или URL, это обычно вызывает ошибку поиска в дальнейшей цепочке протоколов.

Когда цель перемещается, *ошибки в адресе* становятся обычным явлением. В этом случае цель *действительно* существует, просто источник ошибки имеет неверную информацию об адресе. Например, атакующий может ввести устаревшее имя или IP-адрес после перемещения хоста.

Каждая сеть имеет неиспользуемые IP-адреса и номера портов. Например, адресное пространство /24 (класс C) позволяет выделить хостам 254 адреса (еще два зарезервированы для особых целей), но обычно в сети используется только часть из них. Неиспользуемые адреса и/или номера портов называют *темным пространством*. Законные пользователи редко пытаются получить доступ к темному пространству, а атакующие почти всегда делают это. Однако «стук в дверь» неиспользуемого IP-адреса или порта не является опасным сам по себе, это настолько распространенное явление, что отслеживать его не стоит.

Ошибки в адресе – это довольно типичное явление, не ограничивающееся одним или двумя пользователями. Классический пример ошибки в адресе – кто-то

пытается послать сообщение в список и допускает опечатку в URL. Когда такое случается, вы не видите одной или двух ошибок и не видите отдельных ошибок. Вы видите, что та же самая бессмысленная строка возникает много раз, поступая от десятков и даже сотен сайтов. Если вы видите большое количество идентичных попытокощупывания, предпринятых разными сайтами и содержащих явную орфографическую ошибку, то велика вероятность, что ошибка обусловлена банальной причиной, такой как неправильно настроенный сервер DNS, ошибка в настройках перенаправления на веб-сервере или неправильно указанный адрес в электронном письме.

Автоматизация

Люди нетерпеливы. Часто, не сумев добраться до сайта с первой попытки, они могут попробовать еще раз, а потом уходят и находят лучшее применение своему времени. Автоматизированные системы, напротив, повторяют попытки для надежности и часто возвращаются через сравнительно короткие промежутки времени, чтобы посмотреть, заработал ли целевой хост.

С точки зрения характеристик сетевого трафика это означает, что протокол, управляемый человеком (SSH, HTTP, Telnet), почти всегда будет иметь более низкую частоту отказов, чем автоматизированные протоколы (SMTP, BitTorrent).

Сканирование

Сканирование – это наиболее распространенная форма атакующего трафика, наблюдаемого в сети. Если у вас есть нетривиальный сегмент IP-пространства (скажем, /24 или больше), вы каждый день будете обнаруживать тысячи попыток сканирования.

Сканирование является одним из основных источников фиктивных показателей безопасности. Если классифицировать сканирование как атаку, то можно утверждать, что вы отражаете несколько тысяч атак в день. Вы ничего не будете предпринимать для предотвращения этих атак, но цифра в несколько тысяч выглядит внушительно. Сканирование – это легкое, веселое и глупое развлечение для дилетантов.

Представьте, что ваша сеть – это двумерная решетка, где ось x соответствует IP-адресам, а ось y – портам. Решетка будет иметь $65\,536\,k$ ячеек, где k – общее количество IP-адресов. Теперь, каждый раз, когда сканер попадает в цель (комбинация IP-адрес/порта), отметьте ячейку. Если вам интересно узнать обо всех возможностях одного хоста, можете попытаться подключиться к каждому порту, в результате получится одна вертикальная линия на решетке – *вертикальное сканирование*. Кроме вертикального сканирования производится *горизонтальное сканирование*, когда атакующий пытается подключиться к каждому хосту в сети, но только к определенному порту.

Обычно те, кто занимаются защитой, выполняют вертикальное сканирование, а атакующие – горизонтальное. Разница обусловлена ситуативными причинами – атакующий сканирует сеть горизонтально, потому что его не интересуют цели, не имеющие уязвимостей, которыми он мог бы воспользоваться. Если атакующего интересует конкретная цель, он может выполнить вертикальное ее сканирование. Защищающиеся сканируют по вертикали, потому что не могут предугадать, какие порты попыбует использовать атакующий.

Если атакующему известна структура сети, он может использовать *список первоочередных целей*, список IP-адресов систем, которые, как он знает или подозревает, могут быть уязвимы. Пример типичной атаки по списку первоочередных целей описан Алатом (Alata) и Дасье (Dacier): атакующий начинает со слепого сканирования сети, чтобы выявить хосты с открытыми портами SSH, а затем использует список таких хостов для атаки подбора пароля¹.

ОПРЕДЕЛЕНИЕ ПОПЫТОК ПРОЩУПЫВАНИЯ

Определение попытки прощупывания выполняется в два этапа. На первом определяется, что с точки зрения протокола означает ошибка доступа к ресурсу. То есть как «выглядит» ошибка доступа. На втором этапе определяется, является ли ошибка постоянной или временной, глобальной или локальной.

Прощупывание TCP: диаграмма состояний

Чтобы уметь определять ошибки соединения по протоколу TCP, необходимо иметь некоторое представление о возможных его состояниях и как он работает. Как уже говорилось выше, TCP создает иллюзию потокового протокола, действующего поверх пакетного протокола IP. Моделирование потока производится с использованием диаграммы состояний TCP, показанной на рис. 11-1.

В нормальных условиях сеанс TCP состоит из последовательности пакетов квитирования, устанавливающих начальное состояние.

- Клиент выполняет переход из состояния SYN_SENT (отправляет начальный пакет SYN) в состояние ESTABLISHED (получает пакет SYN|ACK от сервера, отправляет ACK в ответ), а затем приступает к обычному обмену данными в рамках сеанса.
- Сервер выполняет переход из состояния LISTEN в состояние SYN_RCVD (получает пакет SYN и отправляет SYN|ACK), а затем в состояние ESTABLISHED (получает ACK).
- Закрытие соединения с обеих сторон влечет отправку как минимум двух пакетов (переход из состояния CLOSE_WAIT в состояние LAST_ACK или из состояния FIN_WAIT_1 в состояние CLOSING/FIN_WAIT_2 и затем TIME_WAIT).



¹ Алат И. и др. Lessons learned from the deployment of a high-interaction honeypot. EDCC, 2006.

реализации стека протоколов; например, в системах Linux число повторных попыток обычно равно 3 и определяется переменной `tcp_retries1`. В системах Windows это значение хранится в значении `TcpMaxConnectRetransmissions`, в ветке `HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters` реестра.

Аналитик может распознать попытку прощупывания, анализируя разные показатели, в зависимости от данных, которыми располагает оператор, и необходимой степени точности. При этом используются такие методики, как использование карты сети, анализ *двунаправленного трафика* и исследование *однонаправленного потока* на активность. Каждая методика имеет свои сильные и слабые стороны, о которых я расскажу далее.

Карты сети

Лучшим инструментом для выявления попыток прощупывания является актуальная и точная карта сети. Имея карту сети перед глазами, прощупывание можно идентифицировать по единственному пакету, тогда как при анализе трафика TCP требуется отыскать запросы и повторные попытки.

Следует, однако, отметить, что карта сети не является точным отражением сетевой информации – это лишь модель сети, построенная за некоторое время до события. В самом худшем случае карта сети с DHCP имеет ограниченный срок службы, но даже в сети со статической адресацией регулярно будут появляться новые службы и хосты. При использовании карты сети регулярно проверяйте ее актуальность с помощью одного из методов, перечисленных в этом разделе.

Фильтрация однонаправленного потока

Если у вас есть доступ к обеим сторонам сеанса (то есть к клиенту и серверу), определение завершенных сеансов – это всего лишь вопрос объединения двух сторон. Однако и в отсутствие этой информации все еще возможно понять, являются ли пакеты частью целого сеанса.

По своему опыту могу сказать, что потоки позволяют определять попытки прощупывания эффективнее, чем отдельные пакеты. Атакующий не взаимодействует с самим сервисом, и его поток не несет полезной нагрузки. В то же время идентификация прощупывания включает поиск нескольких пакетов с идентичными адресами, которые появляются примерно в одно и то же время, что является классическим определением потока.

В зависимости от объема необходимой информации и требуемой точности для идентификации ошибок в потоках TCP можно использовать ряд некоторых эвристик. К ним можно отнести просмотр флагов, подсчет пакетов или величины полезной нагрузки.

Флаги – хороший индикатор прощупывания, но их использование осложняется массой нюансов, благополучно используемых сканерами для выявления конкретной реализации стека IP. Как было показано на рис. 11-1, клиент отправляет флаг ACK только после получения начального SYN + ACK от сервера. Если ответа нет, клиент не должен отправлять флаг ACK; как следствие потоки SYN без флага ACK являются хорошим индикатором прощупывания. Конечно, есть *вероятность*, что ответ выпал за пределы тайм-аута анализатора потока, но на практике это случается редко.

Атакующие часто специально подготавливают пакеты с неправильными комбинациями флагов, чтобы определить конфигурацию стека и брандмауэра. Самой

известной из таких комбинаций является пакет «Рождественская елка» (такое название он получил, потому что в нем включены все флаги, как огоньки на рождественской елке) с установленными флагами SYN ACK FIN PUSH URG RST. Также часто используются другие комбинации флагов, в которых установлены оба флага, SYN и FIN. В работе долгоживущих протоколов (таких как SSH) нередко встречаются пакеты, включающие только флаг ACK. Эти пакеты служат для поддержки TCP-соединения в открытом состоянии и не являются признакамиощупывания.

Еще одно необычное поведение, не являющеесяощупыванием, – *обратная рассылка*. Обратная рассылка случается, когда хост открывает соединение с существующим сервером, используя поддельный адрес, и сервер отправляет соответствующий ответ на этот поддельный адрес. Одиночные пакеты SYN, ACK и RST, которые не достигли цели, скорее всего, являются признаками обратной рассылки.

Простым, хотя и грубым признаком, что поток определяет полноценный сеанс, является количество пакетов. Легитимный сеанс TCP требует наличия не менее трех пакетов накладных расходов, прежде чем в его рамках будет осуществлена передача фактических данных. Кроме того, большинство реализаций стеков устанавливают число повторов от трех до пяти пакетов. Эти правила позволяют определить простой фильтр: потоки TCP, включающие пять или менее пакетов, почти наверняка являются попыткамиощупывания.

Анализ размера потока можно дополнить анализом отношения размера пакета к количеству пакетов. Пакеты TCP SYN содержат блок параметров TCP (<http://bit.ly/tcp-para>) переменной длины. В случае неудачной попытки соединения хост будет повторно отправлять пакеты SYN с одними и теми же параметрами. Следовательно, если поток с N пакетами SYN является попыткойощупывания, то можно ожидать, что общее количество отправленных байтов равно $n \times (40 + k)$, где k – общий размер блока параметров.

Сообщения ICMP иощупывание

Фактическая цель протокола ICMP – информировать пользователя об ошибке, возникшей при попытке установить соединение. Хосту должны отправляться три ICMP-сообщения (destination unreachable – адресат недостижим), чтобы сообщить, что целевая сеть (код 0), хост (код 1) или порт (код 3) недостижимы для клиентского пакета. Протокол ICMP также предусматривает сообщения, указывающие, что маршрут неизвестен (код 7) или административно запрещен (код 13).

За исключением ping-запросов, сообщения ICMP посылаются *в ответ* на ошибки, возникшие в других протоколах. Некоторые сообщения, такие как «хост недостижим» или «сеть недостижима», отправляются из другой точки, отличной от адреса назначения, – обычно ближайшим маршрутизатором. Сообщения ICMP тоже могут фильтроваться в зависимости от политик сети и, следовательно, не улавливаться вашими сенсорами.

Эта асимметрия означает, что при выявлении попытокощупывания по трафику ICMP продуктивнее искать *ответ*. Если вы видите внезапный всплеск ICMP-сообщений, исходящих из маршрутизатора, это верный признак, что адресат, которому отправляются сообщения, пытался исследовать сеть этого маршрутизатора. После этого вы можете проанализировать трафик хоста и определить, с какими хостами он взаимодействовал и нет ли чего подозрительного.

Вас просканировали, держите «медаль»

В настоящее время сканирование стало настолько обыденным, хотя и неприятным, явлением, что его стали считать не атакой, а своеобразной формой интернет-погоды. Готов биться об заклад, что вы в основном сканируете TCP-порты 80, 443, 22, 25 и 135.

То есть сам факт сканирования не представляет интереса, но в его определении все же есть одна *ценность*. В первую очередь это проблема оптимизации. Как обсуждалось в главе 4, данные сканирования можно отбросить на заключительных этапах обработки, чтобы уменьшить количество записей, которые аналитик видит в основном потоке данных. Для тех, кто управляет крупными сетями, проблема присутствия данных сканирования встает особенно остро – простое линейное сканирование сети /16 сгенерирует 65 535 потоков для каждого порта, который решено будет проверить. Сможете ли вы заметить в этом шуме восемь потоков, соответствующих долгоживущим сеансам SSH?

Фильтрацию данных сканирования лучше всего делать на уровне IP, потому что если хост сканирует сеть, он, вероятно, делает это незаконно. Определите адреса, откуда осуществляется сканирование, и удалите *весь* трафик, исходящий с этих адресов. Затем этот массив трафика можно проанализировать и выявить, какие порты подвергались сканированию, какие эксплойты использовались (если идентифицируются), и сравнить виды сканирования, выполнявшиеся в разные моменты времени. Списки «топ-*n*» обычно малополезны для анализа сканирования, потому что первые пять позиций остаются довольно статичными в течение последних пяти лет.

В рабочем окружении я обычно не слишком забочусь о точном определении потока трафика и предпочитаю фильтровать мелкие детали, деля трафик TCP на *короткие* и *длинные* файлы, а затем использовать длинные файлы в качестве исходных данных для анализа. В тех случаях, когда мне действительно нужно проанализировать короткие файлы, я просто извлеку их из хранилища, хотя вероятность, что анализ коротких взаимодействий действительно имеет смысл и *весь трафик с этого хоста находится в коротком файле*, практически равна нулю.

С аналитической точки зрения данные сканирования более полезны для определения того, кто откликнулся на сканирование, а не кто его произвел. Обычно атакующие будут сканировать вашу сеть гораздо активнее и гораздо чаще, чем ваш собственный персонал, занимающийся поддержкой сети, а это означает, что, выявляя хосты, откликнувшиеся на сканирование, вы, скорее всего, обнаружите новые системы и службы задолго до следующей ревизии.

Можно предположить, что в анализе трафика сканирования есть некоторая ценность. Институт SANS и множество других организаций хранят текущую статистику сканирования в центре Internet storm center (<https://isc.sans.edu/>). Однако, чтобы представлять истинную ценность, анализ не должен ограничиваться фактами сканирования наиболее популярных: 22, 25, 80, 443 и 139.

Определение прощупывания в UDP

Выявить неудачные попытки соединения в трафике UDP часто бывает невозможно. Протокол TCP гарантирует доставку данных или возврат сообщения об ошибке, тогда как протокол UDP не дает вообще никаких гарантий. Если *сервис*

обеспечивает некоторую форму гарантий при работе с протоколом UDP, то это характерная особенность самого сервиса. Лучшими способами идентификации ошибок UDP являются карты сетей и анализ трафика ICMP.

ПРОЩУПЫВАНИЕ НА УРОВНЕ СЕРВИСОВ

Прощупывание на уровне сервисов обычно имеет место при сканировании, использовании автоматизированных эксплойтов и разнообразных инструментов поиска. В отличие отощупывания на уровне сети,ощупывание на уровне сервисов обычно идентифицируется вполне однозначно, поскольку в большинстве основных сервисов есть коды ошибок, которые регистрируются и могут использоваться для определения незаконных соединений.

Прощупывание HTTP

Как вы помните, каждая транзакция HTTP возвращает трехзначный код состояния. Коды из семейства 4xx зарезервированы для описания ошибок клиентов. Двумя наиболее важными и распространенными ошибками в семействе 4xx являются коды 404 (страница не найдена) и 401 (несанкционированный доступ).

Код 404 указывает, что ресурс по указанному URL-адресу был недоступен, и является самой распространенной ошибкой в HTTP. Пользователи часто вызывают ошибку 404 вручную, например когда ошибочно вводят сложный URL. Также нередко проблемы возникают из-за неправильной конфигурации, например когда кто-то публикует несуществующий URL-адрес.

Ошибки, вызванные публикацией неправильного адреса URL или опечатками, идентифицируются довольно просто. Во-первых, опечатки довольно редки. URL-адреса с опечатками редко повторяются – если пользователь допускает опечатку, то каждый раз опечатки будут немного отличаться. Кроме того, поскольку опечатки являются *индивидуальными* ошибками, одна и та же опечатка едва ли появится из нескольких источников. Если вы видите одну и ту же ошибку, исходящую из разных местоположений, это, скорее всего, является результатом публикации ошибочного URL. Публикацию ошибочного URL можно идентифицировать изучением HTTP-заголовка *Referer*. Если этот заголовок ссылается на подконтрольный вам сайт, найдите и исправьте ошибку на этом сайте.

Третьим распространенным источником ошибок 404 являются боты, сканирующие HTTP-сайты на наличие известных уязвимостей. Поскольку большинство современных HTTP-сайтов построены на комплексе приложений, они часто наследуют уязвимости от одного или нескольких своих приложений. Эти уязвимости хорошо известны и, следовательно, являются предметом интереса для ботов всех мастей. Все URL-адреса, перечисленные в примере 11-1, связаны с phpMyAdmin – известным инструментом управления базами данных MySQL.

Пример 11-1. Попытки ботнетовощупать известные адреса URL

```
223.85.245.54 - - [16/Feb/2013:20:10:12 -0500]
"GET /pma/scripts/setup.php HTTP/1.1" 404 390 "-" "ZmEu"
223.85.245.54 - - [16/Feb/2013:20:10:15 -0500]
"GET /MyAdmin/scripts/setup.php HTTP/1.1" 404 394 "-" "ZmEu"
188.230.44.113 - - [17/Feb/2013:16:54:05 -0500]
"GET http://www.scanproxy.net:80/p-80.html HTTP/1.0" 404 378 "-"
194.44.28.21 - - [18/Feb/2013:06:20:07 -0500]
"GET /w00tw00t.at.blackhats.romanian.anti-sec:) HTTP/1.1" 404 410 "-" "ZmEu"
```



```

194.44.28.21 - - [18/Feb/2013:06:20:07 -0500]
"GET /phpMyAdmin/scripts/setup.php HTTP/1.1" 404 397 "-" "ZmEu"
194.44.28.21 - - [18/Feb/2013:06:20:08 -0500]
"GET /phpmyadmin/scripts/setup.php HTTP/1.1" 404 397 "-" "ZmEu"
194.44.28.21 - - [18/Feb/2013:06:20:08 -0500]
"GET /pma/scripts/setup.php HTTP/1.1" 404 390 "-" "ZmEu"
194.44.28.21 - - [18/Feb/2013:06:20:09 -0500]
"GET /myadmin/scripts/setup.php HTTP/1.1" 404 394 "-"
    
```

В отличие от ошибок с кодом 404, описанных выше, ошибка 404, обусловленная сканированием, обычно идентифицируется по полному отсутствию связи со структурой сети. Атакующие догадываются, что в сети что-то есть, и следуют документации и обычной практике, пытаясь добраться до уязвимости.

Ошибки 401 – это ошибки аутентификации. Основным их источником является механизм аутентификации HTTP, который вы никогда не должны использовать. Механизм аутентификации 401 был включен в стандарт HTTP очень давно¹ и передает пароли в кодировке base64 в незашифрованном виде для аутентификации доступа пользователя к защищенным каталогам.

Использование этого механизма для проверки подлинности доступа является катастрофой, и он не должен применяться современными веб-серверами. Если вы увидели ошибку 401 в своих системных журналах, определите и устраните их источник на своем сервере. К сожалению, обычная аутентификация HTTP все еще используется в некоторых встроенных системах как единственная доступная форма аутентификации².

Веб-сканеры и robots.txt

Поисковые системы используют автоматизированные инструменты, которые называют *веб-сканерами*, *пауками* или *роботами*, для исследования веб-сайтов и идентификации контента для поиска. Эти сканеры могут действовать очень агрессивно, копируя содержимое сайта; владельцы веб-сайтов могут подсказывать сканерам, какие разделы сайта те могут исследовать, используя *стандарт исключений для роботов*, или *robots.txt*. Стандарт определяет файл (с именем *robots.txt*) для сканеров, содержащий инструкции о том, к каким файлам они могут или не могут обращаться.

Хост, который не обращается к файлу *robots.txt*, а сразу начинает шарить по сайту, заслуживает пристального внимания. Кроме того, инструкции в *robots.txt* носят рекомендательный характер; ничто не мешает сканеру проигнорировать его, и нередко неэтичные или новые сканеры игнорируют инструкции.

Также не редкость, когда прощупывание сайта маскируется под действия веб-робота. Обычным веб-роботам свойственны две черты: они используют строку User-Agent, уникальную для робота, и рассылают запросы из фиксированного диапазона IP-адресов². Большинство поисковых систем публикуют свои диапазоны адресов, чтобы не позволить злоумышленникам маскироваться под их роботов; эти диапазоны адресов могут изменяться, поэтому регулярно проверяйте сайты, такие как Robots Database (<http://bit.ly/web-robots>) и List of User-Agents (<http://www.user-agents.org/>).

¹ См. RFC 1945 (<http://bit.ly/rfc-1945>) и RFC 2617 (<http://bit.ly/rfc-2617>).

² Заметным исключением из этого правила является Googlebot. Инструкции по идентификации Googlebot можно найти по адресу: <http://bit.ly/verify-google>.

Прощупывание SMTP

SMTP-ошибка, которой мы должны уделить внимание, возникает, когда хост отправляет почту на несуществующий адрес. В зависимости от конфигурации SMTP-сервера это влечет одно из трех последствий: отклонение, возврат или (в случае настройки всеобщей ловушки) перенаправление на учетную запись ловушки. Все эти события должны регистрироваться сервером SMTP, который принимает окончательное решение о маршрутизации.

При анализе попытокощупывания SMTP возникает та же проблема, что и при анализе всего SMTP-трафика: спам. В сообщениях SMTP присутствует масса недействительных адресов получателей, потому что спамеры отправляют почту на все мыслимые адреса¹. Как следствие могут существовать относительно безобидные ошибки (опечатки в адресах), но они тонут в общей массе спама. Кроме того, искать признаки злонамеренногоощупывания в трафике электронной почты фактически бессмысленно, потому что спамеры не проверяют существование адресов; они запоминают их.

Однако есть одна веская причина для анализа ошибочных SMTP-адресов: выявление попыток обмана. В нескольких фишинговых сообщениях я видел, как атакующие заполняют поле To: несколькими реалистичными, но недействительными адресами. Я предполагаю, что адреса либо устарели, либо были добавлены намеренно, чтобы придать почте видимость законности.

Анализ попытокощупывания

Пока какой-нибудь гениальный исследователь не придумает лучший способ, выявление попыток сканирования будет сводиться к проверке наличия X событий во временном окне с размером Y.

— Стивен Норткут (Stephen Northcutt)

Для уведомления о попытках сканирования, спаме и других явлениях, когда атакующий почти ничего не знает о целевой сети, можно использовать предупреждения оощупывании.

Создание предупреждений оощупывании

Цель определения попытокощупывания состоит в том, чтобы поднять тревогу, когда есть подозрение, чтоощупывание выполняется не случайно. Для этого нужно сначала собрать события, идентифицируемые какощупывание, используя правила, описанные выше в этой главе, и проанализировать их, в том числе:

- 1) создать или просмотреть карту целей, чтобы определить, сумел ли атакующий достичь реальной цели;
- 2) исследовать трафик на наличие отказов в подключении. Вот несколько примеров таких отказов;
- 3) асимметричные сеансы TCP или сеансы без флага ACK;
- 4) HTTP-ошибки 404;
- 5) записи о возврате в журнале сервера электронной почты.

¹ Однажды я открыл свой почтовый ящик, которым никогда не пользовался, и обнаружил в нем 3000 спам-сообщений.

Безобидные факты, идентифицируемые как прощупывание, обычно являются результатом ошибок в конфигурации или в выборе цели. Например: если в DNS для имени *destination.com* IP-адрес сменился с А на В, то пока изменение не распространится по всей системе DNS, пользователи будут пытаться подключаться к адресу А вместо В. Ошибки этого типа непротиворечивы и имеют разные исходящие адреса. Вернемся к имени *destination.com*: если адрес А больше не используется и в сети имеется «темный» адрес С (то есть адрес, не имеющий доменного имени); пользователи могут случайно пытаться подключаться к адресу А в течение какого-то времени, но они не будут подключаться к С. Под подозрение должны попадать пользователи, которые обращаются к разным несуществующим адресам; хост может попытаться подключиться к А из-за ошибки в конфигурации, он может случайно попытаться подключиться к С, но если он пытается подключиться и к А, и к С, то, скорее всего, он нащупывает цель.

Поэтому, как говорит Норткат, умение отличать злонамеренное прощупывание от безобидных ошибок заключается в умении определить количество допустимых событий, при превышении которого следует поднять тревогу. Чтобы определить такой, можно:

- 1) вычислить ожидаемое количество ошибок, исходя из числа узлов в сети, с которыми пользователь должен связаться в течение фиксированного периода времени;
- 2) или использовать последовательную проверку гипотез – статистический метод вычисления вероятности, что явление пройдет или провалит определенную проверку несколько раз. Этот подход впервые предложил Джейеун Юнг (Jaeyeon Jung) в своей статье «Fast Portscan Detection Using Sequential Hypothesis Testing», опубликованной в 2004 году¹;
- 3) поднимать тревогу при каждой попытке пользователя подключиться к «темному» адресу.

Дело в том, что атакующие, как правило, не имеют особых причин для избирательности. Если кто-то сканирует сеть, он постарается быстро опробовать все возможные адреса. Статистические методы более полезны для *быстрого* обнаружения попыток атаки и, следовательно, больше подходят в системах активной защиты, а не в создании тревог.

Расследование попыток прощупывания

Сканирование как *таковое* не представляет большого интереса. На планете много бездельников, которые сканируют интернет, и многие из них делают это по нескольку раз в день. Некоторые черви осуществляли сканирование (например, Code Red и SQLSlammer, появившиеся еще в Юрском периоде) на протяжении многих лет без какого-либо заметного эффекта. Сканирование – это как дождь: оно обязательно случится, и главная наша задача – не предотвратить его, а определить ущерб, который оно наносит.

Получив сигнал о попытке сканирования, необходимо найти ответы на несколько основных вопросов:

¹ Юнг, Джейеун (Jung, Jaeyeon) и др. Fast Portscan Detection Using Sequential Hypothesis Testing. Статья была представлена на симпозиуме IEEE по безопасности и конфиденциальности. Окленд, Калифорния, май 2004.

1. Какие хосты ответили сканеру? Я понимаю, что сканеры могут исследовать мое «темное пространство», как им заблагорассудится. Но меня по-настоящему беспокоит наличие хостов в моей сети, *вступивших в диалог* со сканером, и что они сделали после этого. Более конкретно:
 - а) состоялся ли какой-то более или менее продолжительный диалог между сканером и каким-нибудь хостом? Обычно атакующее программное обеспечение выполняет сканирование и внедрение эксплойта в ходе двух-этапного процесса. Поэтому, увидев попытку сканирования, я сразу задаю себе вопрос: закончилось ли оно до внедрения эксплойта?
 - б) состоялся ли после этого какой-нибудь подозрительный диалог у хостов, ответивших сканеру? Под подозрение попадают попытки связаться с внешними хостами (особенно, если это внутренний сервер), получение файлов и взаимодействие через необычные порты.
 2. Обнаружил ли что-то сканер в моей сети, о чем я не знал? Учетная информация имеет свойство постоянно устаревать, а нападения происходят *постоянно*. Учитывая это, имеет смысл воспользоваться результатами тяжелой работы сканера для собственной выгоды. Более конкретно:
 - а) нашел ли сканер ранее неизвестные хосты? Это – пример неизвестной информации;
 - б) нашел ли сканер ранее неизвестные сервисы?
 3. Что еще сделал сканер? Обычно боты делают несколько дел сразу, поэтому желательно проверить, сканировались ли другие порты, производились ли другие виды зондирования и были ли попытки проведения атак иных типов.
- Вот еще несколько хороших вопросов о *прощупывании* в целом:
- 1) что еще было сделано в ходе *прощупывания*? Если с одного и того же адреса была отправлена почта нескольким адресатам, это, скорее всего, спамер, который, подобно сканеру, использует бота в качестве инструмента.
 - 2) было ли *прощупывание* направлено на какие-то конкретные цели? Обычно такая конкретизация наблюдается в отношении адресов электронной почты, потому что IP-адреса извлекаются из гораздо меньшего пула. Существуют ли в вашей сети какие-то типовые адреса? Такие адреса обычно считаются хорошими кандидатами для дальнейшего развития атаки.

Проектирование сети для извлечения пользы от *прощупывания*

Прощупывание обычно производится с целью определить общую конфигурацию сети. Часто атакующие сканируют хорошо известные порты, такие как 22, предполагая найти соответствующие сервисы. Эти попытки подтвердить предположения можно использовать для размещения в сети более чувствительных инструментов, например осуществляющих полный перехват пакетов.

Поскольку злонамеренное сканирование предполагает типовые настройки, вы можете немного усложнить жизнь злоумышленникам, определив необычные настройки для своей сети:

Измените схему адресации

В большинстве случаев сканирование производится линейно: атакующий *прощупывает* адрес X, затем X + 1 и т. д. Большинство администраторов и реализаций DHCP тоже назначают адреса линейно. Часто в сетях /24 или /27 верхняя половина остается полностью темной. Переупорядочение адре-

сов так, чтобы равномерно распределить их по сети, или оставление больших пустых промежутков в сети – это простой метод создания темного пространства.

Сместите цели

Выбирая номера портов для назначения сервисам, обычно руководствуются общепринятыми соглашениями, и большинство современных приложений имеют возможность работать с сервисами, расположенными на необычных портах. Переназначение портов, особенно для внутренних сервисов, которые не должны быть доступны внешнему миру, является простым способом сбить с толку простые сканеры.

ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ ДЛЯ ЧТЕНИЯ

1. Джейеун Юнг (*Jaeyeon Jung*), Верн Паксон (*Vern Paxson*), Артур У Бергер (*Arthur W. Berger*) и Хари Балакришнан (*Hari Balakrishnan*). Fast Portscan Detection Using Sequential Hypothesis Testing. Материалы симпозиума IEEE по безопасности и конфиденциальности, 2004.



Глава 12



Анализ объема и времени

В этой главе мы рассмотрим явления, которые можно идентифицировать, анализируя изменение объема трафика с течением времени. Под «объемом» можно понимать простое количество байтов или пакетов, или это может быть количество IP-адресов, пересылающих файлы. Анализируя наблюдаемый трафик, можно получить множество разной и полезной информации о трафике, в частности:

Тревожные сигналы

Когда кто-то регулярно пытается подключиться к вашему хосту, это можно расценивать как тревожный сигнал.

Извлечение файлов

Массовые загрузки файлов могут говорить о том, что кто-то крадет ваши внутренние данные.

Отказ в обслуживании (Denial of Service, DoS)

Препятствование вашим серверам оказывать услуги пользователям.

Данные об объеме трафика часто бывают искажены. Большинство наблюдений, которые можно подсчитать непосредственно, например количество байтов за разные периоды времени, сильно различаются, и часто между количеством событий и их значимостью нет прямой связи. Другими словами, между количеством байтов и важностью событий редко существует значимая связь. Эта глава покажет вам, как идентифицировать необычные явления с помощью сценариев и методов визуализации, но, чтобы уметь отличать опасные явления от неопасных, требуется определенный опыт.



ВЛИЯНИЕ РАБОЧИХ ЧАСОВ НА ОБЪЕМ ТРАФИКА

Основная часть трафика в корпоративной сети обусловлена производственной деятельностью сотрудников, поэтому объем трафика будет изменяться в примерном соответствии с началом и концом рабочего дня. Трафик будет уменьшаться вечером, увеличиваться после 08:00, достигать пика около 13:00 и опять уменьшаться около 18:00.

Чтобы увидеть, насколько существенное влияние на объем трафика оказывают рабочие часы, взгляните на график на рис. 12-1. Этот график демонстрирует активность распространения почтового червя SoBIG.F в сети SWITCH (<http://www.>

switch.ch/) в 2003 году. SWITCH – это образовательная сеть Швейцарии и Лихтенштейна, которая формирует значительную часть национального трафика в Швейцарии. График на рис. 12-1 показывает общий объем SMTP-трафика за две недели. Распространение SoBIG началось ближе к концу этого периода. Но в первую очередь я хотел бы отметить характер нормальной активности в начале графика. Обратите внимание, что в каждый будний день наблюдается пик с выемкой, попадающей на обеденное время. Отметьте также, что в выходные дни объем трафика значительно ниже.

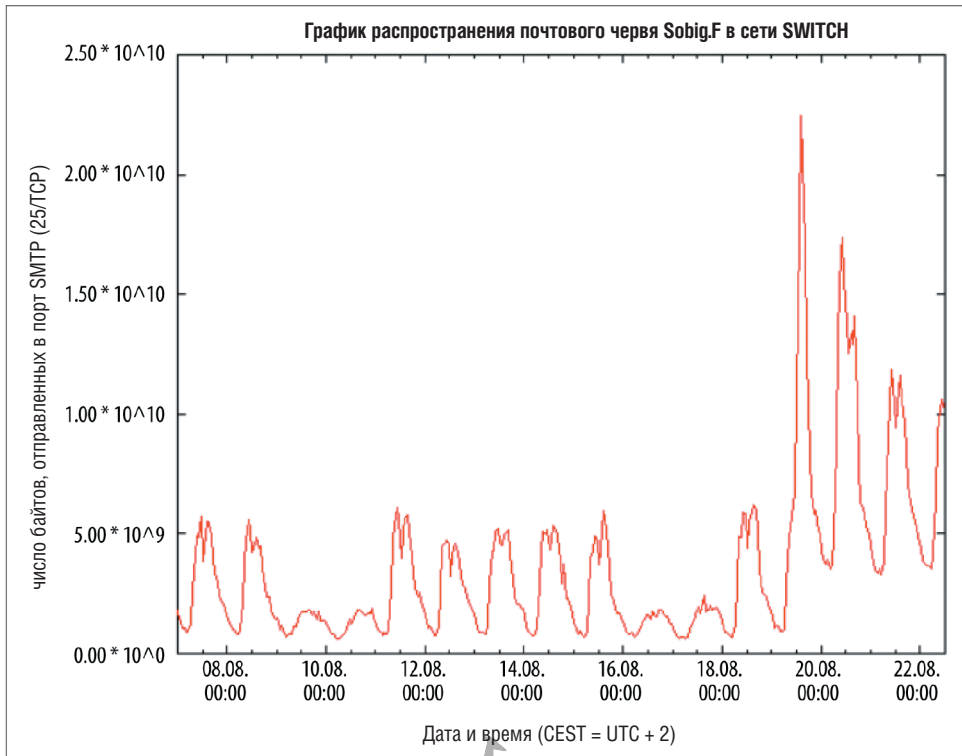


Рис. 12-1. Трафик электронной почты и признаки распространения червя в сети SWITCH (иллюстрацию любезно предоставил доктор Арно Вагнер (Arno Wagner))

Это явление имеет социальные корни; зная примерно, где находится хост, за которым вы наблюдаете (дом, работа, школа), а также местный часовой пояс, можно довольно точно прогнозировать события и объем трафика. Например, вечером, когда люди отдыхают и смотрят телевизор, значительную часть трафика генерируют компании, занимающиеся распространением потокового видео.

Есть целый ряд полезных эмпирических правил, помогающих учитывать расписание рабочих дней для выявления аномалий. К ним относятся: учет периодов активности, учет внутреннего расписания организации и учет часового пояса. В этом разделе мы рассмотрим лишь самые простые методы, основанные на эмпирическом подходе к анализу временных рядов; описание более продвинутых методов вы найдете в книгах, которые я буду упоминать.

Анализируя информацию о трафике, я предпочитаю разделить ее на рабочие и нерабочие периоды. Гистограмма на рис. 12-2 показывает, как это может повлиять на распределение объема трафика – в данном случае наблюдаются два пика, соответствующих рабочим и нерабочим периодам. Моделирование этих двух периодов по отдельности позволит более точно оценить объем без сложных математических вычислений, используемых при анализе временных рядов.

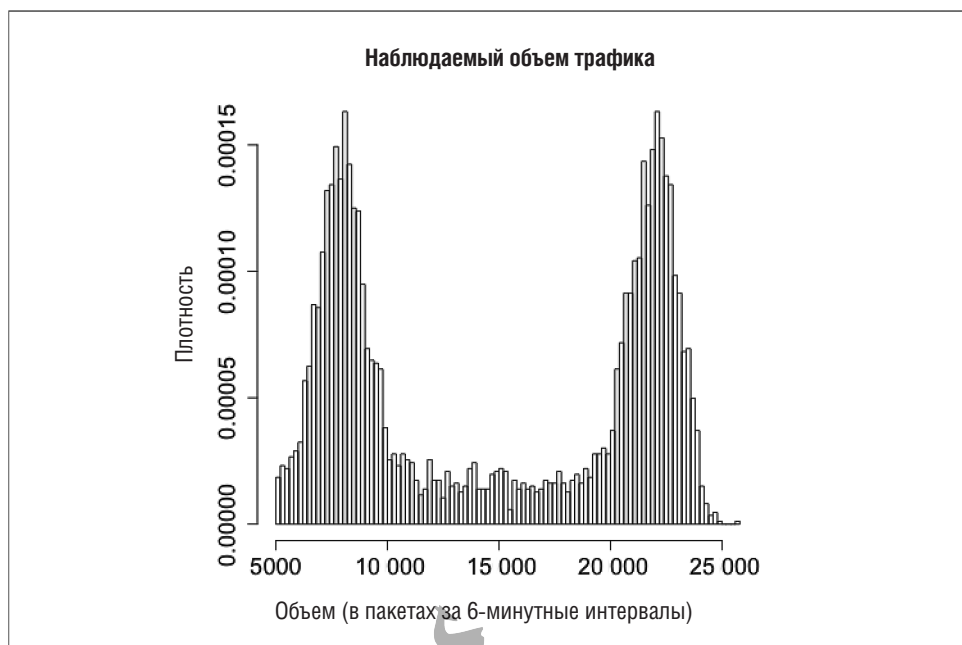


Рис. 12-2. Распределение трафика в обычной сети; пик справа – рабочий день, пик слева – вечер

Для определения рабочих и нерабочих периодов используйте график работы самой организации. Если в вашей компании есть особые праздники, например выходной в день рождения основателя, учитывайте их как нерабочие периоды. Кроме того, некоторые подразделения организации могут работать круглосуточно, а другие – только с 8 до 17. Если есть подразделения, работающие круглосуточно, посмотрите, как меняется трафик в течение смен – вы почти наверняка заметите повторяющуюся картину, когда в начале смены сотрудники входят в системы, проверяют электронную почту, проводят совещания и затем начинают работать.

Объем трафика в нерабочие дни

Знание нерабочих периодов очень ценно. Например, чтобы идентифицировать соединения с внешними системами, утечку файлов и другие подозрительные действия, я выделяю из общего потока информации именно нерабочие часы. В эти периоды меньше трафика, меньше людей, и если кто-то не знает о внутреннем распорядке компании, его будет намного легче поймать в эти периоды, чем в бурные периоды рабочего времени.

Именно поэтому я предпочитаю учитывать особые нерабочие часы в расписании компании. Злоумышленник легко сможет спрятать свой трафик, действуя в периоды с 8:00 до 17:00, с понедельника по пятницу, но если злоумышленник не знает, что компания не работает в День святого Свитина, то он с большей вероятностью попадетсЯ.

Я видел, как это отражается на трафике, генерируемом злоумышленниками внутри компании, которые выводят секретные сведения. Они часто стараются действовать по вечерам и выходным, чтобы их коллеги ничего не заподозрили и не стали задавать неудобных вопросов, и эти их действия довольно четко отражаются в журналах трафика.

Бизнес-процессы являются распространенным источником ошибочных выводов при анализе трафика. Например, я видел корпоративную сеть, где раз в две недели происходил внезапный всплеск трафика с определенным сервером. Как оказалось, на сервере размещалась информация о зарплате, и каждый сотрудник компании заходил на него каждую вторую пятницу. В остальные дни этот сервер был никому не интересен. Явления, происходящие еженедельно, раз в две недели или раз в месяц, почти наверняка связаны с внутренними процессами предприятия и должны идентифицироваться как таковые для учета в будущем.

ТРЕВОЖНЫЕ СИГНАЛЫ

Тревожные сигналы – это факты систематического и регулярного контакта с хостом. Например, ботнеты периодически опрашивают свои серверы для получения новых инструкций. Это особенно верно для современных ботнетов, которые используют протокол HTTP. Такое поведение можно выявлять по наличию информационных потоков, возникающих через регулярные промежутки времени, между зараженными системами в вашей сети и неизвестным адресом за ее пределами.

Однако существует множество законных видов деятельности, которые тоже генерируют систематический и регулярный трафик. Например:

Трафик поддержки соединений

В течение продолжительных сеансов, например интерактивных сеансов SSH, системы регулярно посылают пустые пакеты, помогающие поддерживать открытым соединение с целью.

Обновление программного обеспечения

Большинство современных приложений поддерживают возможность автоматической проверки обновлений. Например, антивирус регулярно загружает обновления сигнатур вредоносных программ.

Новости и погода

Многие новостные, метеорологические и другие интерактивные сайты регулярно обновляют страницу, пока она открыта у клиента.

Выявление тревожных сигналов обычно выполняется в два этапа. Первый этап включает идентификацию последовательных событий. Примером может служить сценарий *find_beacons.py*, представленный в примере 12-1. Сценарий *find_beacons.py* принимает поток записей с информацией о пакетах и раскладывает пакеты по

«корзинам» одинакового размера. Каждый вход состоит из двух полей: IP-адрес, определяющий источник события, и время начала потока, возвращаемое `gwsut`. Для упорядочивания трафика по IP-адресу источника и времени используется `gwsort`.

Сценарий вычисляет медиану в массиве «корзин» и оценивает каждый IP-адрес по доле «корзин», находящихся в пределах некоторого допуща от этой медианы. Если выяснится, что основная масса «корзин» сосредоточена вблизи медианы, значит, вы столкнулись с регулярно повторяющимся событием.

Пример 12-1. Простой тревожных сигналов

```
#!/usr/bin/env python
#
#
# find_beacons.py
#
# входные данные:
#   gwsort --field=1,9 | gwsut --no-title --epoch --field=1,9 | <stdin>
# порядок использования:
# find_beacons.py precision tolerance [epoch]
#
# precision: целое число, определяющее размер "корзины" (в секундах)
# tolerance: вещественное число, величина допуща, которая
# выражается как доля от медианы, например 0.05 означает все, что попадает
# в диапазон (median - 0.05*median, median + 0.05*median)
# epoch: время начала формирования "корзин"; если не задано,
# по умолчанию выбирается время, соответствующее полуночи.

# Это очень простой детектор тревожных сигналов, который разбивает трафик на
# "корзины" с емкостью [precision]. По расстоянию между крайними корзинами
# определяется медиана, которая затем используется для оценки. Если все расстояния
# оказываются внутри диапазона отклонений tolerance от медианы, трафик считается
# тревожным.

import sys

if len(sys.argv) >= 3:
    precision = int(sys.argv[1])
    tolerance = float(sys.argv[2])
else:
    sys.stderr.write("Specify the precision and tolerance\n")

starting_epoch = -1
if len(sys.argv) >= 4:
    starting_epoch = int(sys.argv[3])

current_ip = ''

def process_epoch_info(bins):
    a = bins.keys()
    a.sort()
    distances = []
    # Создать таблицу расстояний между "корзинами"
    for i in range(0, len(a) - 1):
        distances.append(a[i + 1] - a[i])
```

```

distances.sort()
median = distances(len(distances)/2)
tolerance_range = (median - tolerance * median, median + tolerance * median)
# Теперь проверим "корзины"
count = 0
for i in distances:
    if (i >= tolerance_range[0]) and (i <= tolerance_range[1]):
        count+=1
return count, len(distances)

bins = {} # Контрольный список "попаданий" в "корзины".
          # В действительности это множество,
          # что упрощает его использование.
results = {} # Ассоциативный массив с результатами анализа,
             # заполняется при построении окончательного отчета

# Сначала сценарий читает входные данные; для каждой строки создает таблицу
# тревожных событий. Тревожные события – это просто признаки, что
# трафик 'имел место' во время X. Продолжительность трафика, как часто он
# наблюдался и сколько пакетов включал – ничего из этого не учитывается.
# Значение имеет только сам факт наличия трафика.
for i in sys.stdin.readlines():
    ip, time = i.split('|')[0:2]
    if ip != current_ip:
        results[ip] = process_epoch_info(bins)
        bins = {}

    if starting_epoch == -1:
        starting_epoch = time - (time % 86400) # Sets it to midnight of that day
    bin = (time - starting_epoch) / precision
    bins[bin] = 1

a = bins.sort()
for i in a:
    print "%15s|%5d|%5d|%8.4f" % (ip, bins[a][0], bins[a][1],
                                100.0 * (float(bins[a][0])/float(bins[a][1])))

```

Второй этап обнаружения тревожных сигналов (как обычно) – оценка выявленных событий. Как было показано выше, периодический трафик генерирует огромное количество вполне законных приложений: NTP и протоколы маршрутизации, а также антивирусные инструменты, которые регулярно «звонят домой» для обновления информации. SSH тоже может порождать периодический трафик, потому что этот протокол нередко используется администраторами для выполнения периодических действий и операций.

РЕЙДЕРСТВО – НЕСАНКЦИОНИРОВАННОЕ КОПИРОВАНИЕ ФАЙЛОВ

Хищение данных по-прежнему является основной формой атаки на базы данных или веб-сайты, *особенно* если веб-сайт является внутренним или защищенным ресурсом. Из-за отсутствия лучшего термина я буду использовать термин *рейдерство* для обозначения несанкционированного копирования содержимого веб-сайта или базы данных с целью последующего распространения или продажи информации. Разница между рейдерством и законным доступом зависит от

конкретных обстоятельств, поскольку целью любого сервера является обработка данных.

Очевидно, что попытки несанкционированного копирования данных (рейдерства) должны приводить к изменению объема трафика. Обычно рейдерство проводится быстро (нередко в моменты, когда заканчивается рабочий день и работники собираются домой) и с использованием автоматизированных инструментов, таких как *wget*. Рейдерские атаки могут проводиться тоньше, но для этого требуется, чтобы у атакующего было достаточно времени и терпения для медленного извлечения данных.

Анализ объема трафика – один из самых простых способов идентификации рейдерской атаки. Для этого первым делом нужно построить модель типичного трафика, порождаемого хостом. Сценарий *calibrate_raid.py* в примере 12-2 определяет пороговые значения объема и создает таблицу результатов для построения графика.

Пример 12-2. Сценарий обнаружения рейдерской атаки

```
#!/usr/bin/env python
#
# calibrate_raid.py
#
# входные данные:
#   ничего
# выходные данные:
#   выводит в stdout отчет с временными рядами и оценками объема трафика
# порядок использования:
# calibrate_raid.py start_date end_date ip_address server_port period_size
#
# start_date: начальная дата для анализа
# end_date: конечная дата для анализа
# ip_address: адрес исследуемого сервера
# server_port: порт исследуемого сервера
# period_size: размер периодов для моделирования
#
# Этот сценарий генерирует для заданного IP-адреса временные ряды
# (с помощью gwscount) и определяет ожидаемые пороговые значения для
# диапазона 90–100 % трафика. Результаты вычислений можно использовать для
# построения графиков, чтобы упростить выявление отклонений и аномалий.
#
import sys,os,tempfile

start_date = sys.argv[1]
end_date = sys.argv[2]
ip_address = sys.argv[3]
server_port = int(sys.argv[4])
period_size = int(sys.argv[5])

if __name__ == '__main__':
    fh, temp_countfn = tempfile.mkstemp()
    os.close(fh)
    # Обратите внимание, что IP-адрес и порт сервера передаются в фильтр
    # как исходящие адрес и порт. Сценарий выявляет потоки, ИСХОДЯЩИЕ ИЗ
    # сервера, то есть факты отправки данных. Если передать IP-адрес и порт
    # сервера в параметрах daddress/dport, сценарий зафиксирует входящий
    # трафик, направленный от клиентов к серверу (который обычно намного
```

```

# меньше по объему).
#
os.system(('rfilter --saddress=%s --sport=%d --start-date=%s ',
          '--end-date=%s --pass=stdout | rwcourt --epoch-slots',
          ' --bin-size=%d --no-title > %s') % (
          ip_address, server_port, start_date, end_date, period_size,
          temp_countfn))

# Небольшое примечание к выполняемой здесь фильтрации. Можно было бы
# ограничить rfilter и заставить его учитывать только сеансы,
# в ходе которых передается 4 пакета или больше, чтобы исключить
# трафик сканирования. Однако этот трафик очень невелик, и поэтому
# я решил не вводить такого ограничения.

# Загрузить файл с результатами фильтрации в память и пранализировать его
#

a = open(temp_countfn, 'r')

# Предполагается, что результаты будут использоваться для построения
# гистограммы, поэтому необходимо определить минимальное и максимальное
# значения
min = 99999999999L
max = -1
data = {}
for i in a.readlines():
    time, records, bytes, packets = map(lambda x:float(x),
                                         i[:-1].split('|')[0:4])

    if bytes < min:
        min = bytes
    if bytes > max:
        max = bytes
    data[time] = (records, bytes, packets)

a.close()
os.unlink(temp_countfn)

# Подготовить данные для построения гистограммы с hist_size слотами
histogram = []
hist_size = 100
for i in range(0, hist_size):
    histogram.append(0)
bin_size = (max - min) / hist_size
total_entries = len(data.values())
for records, bytes, packets in data.values():
    bin_index = (bytes - min)/bin_size
    histogram[bin_index] += 1

# Вычислить пороги в диапазоне от 90 до 100 %
thresholds = []
for i in range(90, 100):
    thresholds.append(0.01 * i * total_entries)
total = 0
last_match = 0 # индекс в массиве thresholds, где мы остановились

# Шаг 1, вывести пороги

```

```

for i in range(0, hist_size):
    total += histogram[i]
    if total >= thresholds[last_match]:
        while thresholds[last_match] < total:
            print "%3d%% | %d" % (90 + last_match, (i * bin_size) + min)
a = data.keys()
a.sort()
for i in a:
    print "%15d|%10d|%10d|%10d" % (i, data[i][0], data[i][1], data[i][2])

```

Визуализация играет важную роль при калибровке пороговых значений для обнаружения рейдерских атак и других аномалий. В главе 10 мы обсуждали проблему со стандартными отклонениями, и гистограмма – это самый простой способ определить, является ли распределение хотя бы отдаленно напоминающим гауссово. По своему опыту могу сказать, что рейдерству регулярно подвергается огромное число сервисов. Наиболее яркими примерами могут служить веб-пауки и интернет-архив. Если сайт является исключительно внутренним, по ошибке за рейдерство можно принять резервное копирование и внутреннее зеркалирование.

Визуализация позволит идентифицировать эти отклонения. Пример на рис. 12-3 показывает, что в подавляющем большинстве сеансов объем трафика ниже примерно 1000 Мбайт за 10 мин, но те немногие выбросы, превышающие 2000 Мбайт за 10 мин, будут определены как проблемные сценарием *calibrate_raids.ru* и большинством других алгоритмов анализа. После идентификации законных выбросов их можно занести в белый список и прекратить учитывать их, добавив исключение в команду фильтрации с помощью параметра `--not-dipset`. После этого можно использовать `gswcount` для настройки простого механизма оповещения.

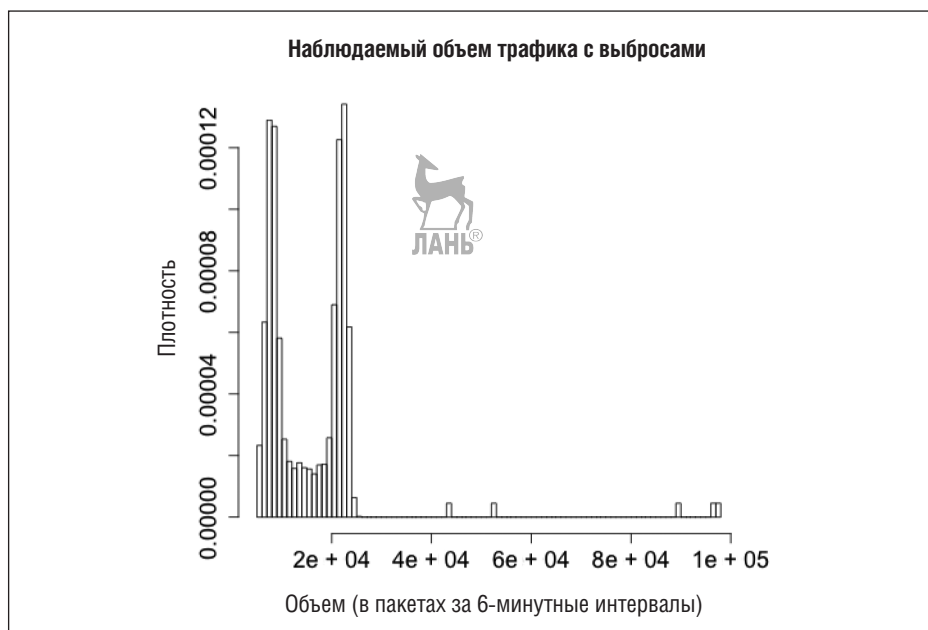


Рис. 12-3. Трафик с выбросами; определение причины выбросов может уменьшить число предупреждений

Локальность

Локальность – это тенденция ссылок (ячеек памяти, URL-адресов, IP-адресов) объединяться в кластеры. Например, если проанализировать список веб-страниц, посещавшихся пользователем в течение некоторого времени, можно обнаружить, что большинство страниц находится на узком круге сайтов (пространственная локальность) и что пользователи, как правило, посещают одни и те же сайты снова и снова (временная локальность). Локальность – это хорошо известное понятие в информатике, служащее основой для механизмов кеширования, организации сетей доставки контента (Content Delivery Network, CDN) и настройки обратных прокси-серверов.

Анализ локальности может с успехом дополнять анализ объема трафика, учитывая предсказуемость пользователей. Пользователи посещают узкий круг сайтов и общаются с узким кругом людей, и хотя изменения иногда случаются, мы все равно можем смоделировать это поведение, используя *рабочий набор*.

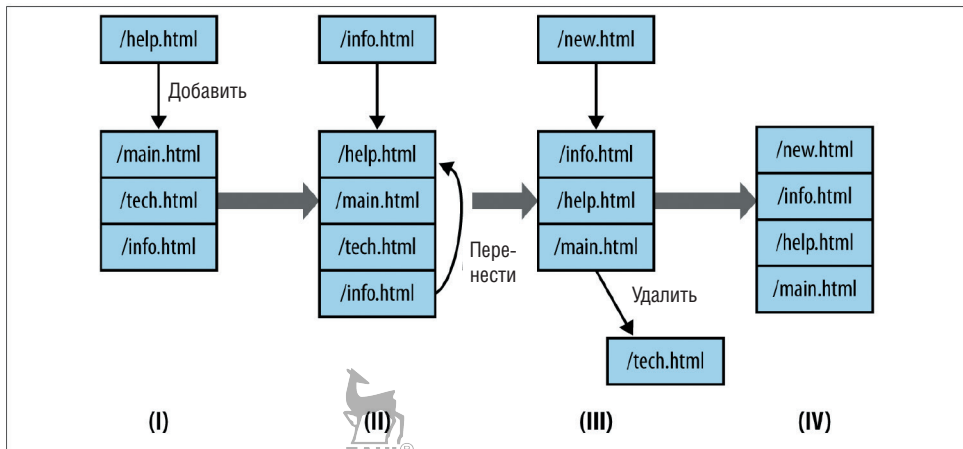


Рис. 12-4. Использование рабочего набора

На рис. 12-4 представлен пример использования рабочего набора. В этом примере рабочий набор реализован как очередь LRU (Least Recently Used – остается недавно использовавшийся) фиксированного размера (в данном случае очередь хранит четыре ссылки). Этот рабочий набор отражает характер веб-серфинга. Для этого из файла журнала HTTP-сервера извлекаются URL-адреса и добавляются в стек. Рабочие наборы хранят только одну копию каждой встречавшейся ссылки, поэтому данный набор на рис. 12-4, включающий четыре ссылки, будет показывать только четыре ссылки. Получив очередную ссылку, рабочий набор выполняет одно из трех действий:

- 1) если в наборе есть незаполненные ячейки, новая ссылка добавляется в конец очереди (переход из состояния I в состояние II);
- 2) если очередь заполнена и такая ссылка уже имеется, эта ссылка перемещается в конец очереди;
- 3) если очередь заполнена и такая ссылка отсутствует, тогда ссылка добавляется в конец очереди, а ссылка в голове очереди удаляется.

Код в примере 12-3 иллюстрирует реализацию LRU-модели рабочего набора на Python.

Пример 12-3. Вычисление рабочего набора

```
#!/usr/bin/env python
#
#
# Определение локальности хоста методом анализа рабочего набора.
# входные данные:
#     stdin – последовательность тегов
#
# аргументы командной строки:
#     первый: размер очереди рабочего набора
import sys

try:
    working_set_depth = int(sys.argv[1])
except:
    sys.stderr.write("Specify a working_set depth at the command line\n")
    sys.exit(-1)

working_set = []
i = sys.stdin.readline()
total_processed = 0
total_popped = 0
unique_symbols = {}
while i != '':
    value = i[:-1] # Удалить перевод строки \n
    unique_symbols[value] = 1 # Добавить символ
    total_processed += 1
    try:
        vind = working_set.index(value)
    except:
        vind = -1

    if (vind == -1):
        # Значение отсутствует в кеше LRU; удалить значение,
        # которое дольше всех не использовалось, и добавить
        # текущее в конец
        working_set.append(value)
        if len(working_set) > working_set_depth:
            del working_set[0]
        working_set.append(value)
        total_popped += 1
    else:
        # Самое недавно использовавшееся значение; перенести в конец очереди
        del working_set[vind]

# Вычислить вероятность замены
p_replace = 100.0 * (float(total_popped)/float(total_processed))

print "%10d %10d %10d %8.4f" % (total_processed, unique_symbols,
                                working_set_depth, p_replace)
```

На рис. 12-5 показан пример, как выглядят рабочие наборы. Этот график отражает вероятность замены значения в рабочем наборе в зависимости от его раз-

мера. Здесь для сравнения показаны два разных набора: полностью случайный, в котором ссылки выбираются из 10 млн символов, и набор, моделирующий активность реального пользователя с применением распределения Парето. Модель Парето достаточно точно описывает типичную активность пользователей, хотя и немного *менее* стабильна, чем пользователи в обычных условиях.

Обратите внимание, что в модели Парето присутствует «колено», тогда как график случайной модели остается линейным на уровне 100 %. Рабочие наборы обычно имеют некоторый идеальный размер, по достижении которого дальнейшее увеличение размера очереди не дает никакой выгоды. Это колено отражает данное явление – как видите, вероятность замены плавно уменьшается слева от колена, но остается постоянной правее его.

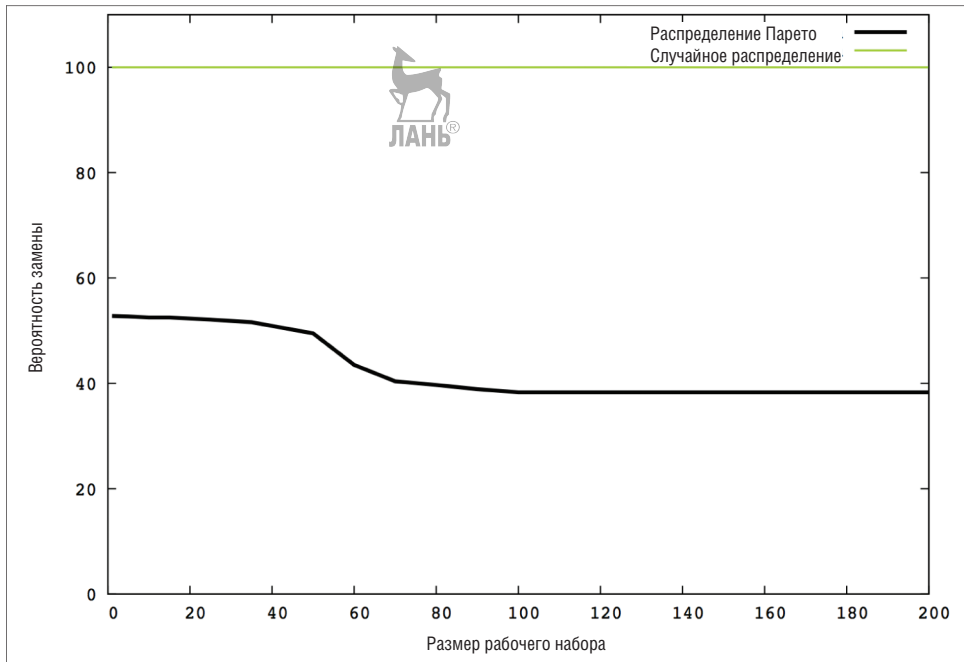


Рис. 12-5. Анализ рабочего набора

Ценность рабочих наборов в том, что после калибровки они позволяют выразить привычки пользователя двумя параметрами: размером очереди и вероятностью появления ссылки, которая приведет к замене элемента в очереди.

Отказ в обслуживании, флешмобы и исчерпание ресурсов

Отказ в обслуживании (Denial of Service, DoS) – это цель, а не стратегия. Атаки типа «отказ в обслуживании» приводят к тому, что атакованный хост оказывается недостижимым для удаленных пользователей. Большинство DoS-атак реализуются как распределенные атаки (Distributed Denial of Service, DDoS), когда атакующий использует сеть захваченных им хостов для организации атаки. Есть множество способов реализовать DoS-атаку. Вот некоторые из них:

Исчерпание уровня обслуживания

Целевой хост предоставляет некоторый общедоступный сервис. Используя ботнет, атакующий запускает множество клиентов, каждый из которых обращается к сервису и выполняет некоторую тривиальную операцию (например, получает домашнюю страницу веб-сайта).

SYN flood

SYN flood – классическая DDoS-атака. Суть ее заключается в следующем: атакующий создает большое число клиентов, каждый из которых просто открывает соединение с TCP-портом жертвы, посылая пакеты SYN, и оставляет соединение открытым.

Исчерпание полосы пропускания

Проводя атаку этим способом, атакующий отправляет массивный поток мусорного трафика в целевой хост, намереваясь забить канал между маршрутизатором и целевым хостом.

Не следует также игнорировать возможность инсайдерской атаки, когда атакующий может просто подойти к физическому серверу и выключить его.

Все эти способы приводят к одному и тому же результату, но каждый будет выглядеть по-разному в сетевом трафике и требует разных способов противостояния. Точный объем ресурсов, необходимых атакующему, зависит от способа реализации DDoS-атаки. Как правило, чем выше уровень атаки в модели OSI, тем большую нагрузку она оказывает на цель и тем меньше ботов требуется атакующему. Например, атака, направленная на исчерпание пропускной способности, оказывает влияние и на маршрутизатор и фактически исчерпывает пропускную способность маршрутизатора. Атака SYN flood, классическая DDoS-атака, направлена на исчерпание стека TCP жертвы. На более высоких уровнях такие инструменты, как Slowloris (<http://ha.ckers.org/slowloris>), создают частичные HTTP-соединения, истощая ресурсы веб-сервера.

С точки зрения атакующего, чем выше уровень, тем лучше – тем меньше ресурсов требуется для атаки, а значит, требуется меньше ботов, и высокоуровневый сеанс с большей вероятностью будет пропущен брандмауэром, который может блокировать пакеты, созданные для атаки на уровень IP или TCP.

DDoS и инфраструктура маршрутизации

DDoS-атаки, направленные конкретно на инфраструктуру маршрутизации, влекут сопутствующий ущерб. Рассмотрим простую сеть, подобную той, что изображена на рис. 12-6; жирная линия показывает направление атаки на подсеть С. Атакующий исчерпывает не только пропускную способность в точке С, но и пропускную способность маршрутизатора. Следовательно, хосты в сетях А и В не смогут выйти в интернет и увидят, что их входящий интернет-трафик упал до нуля.

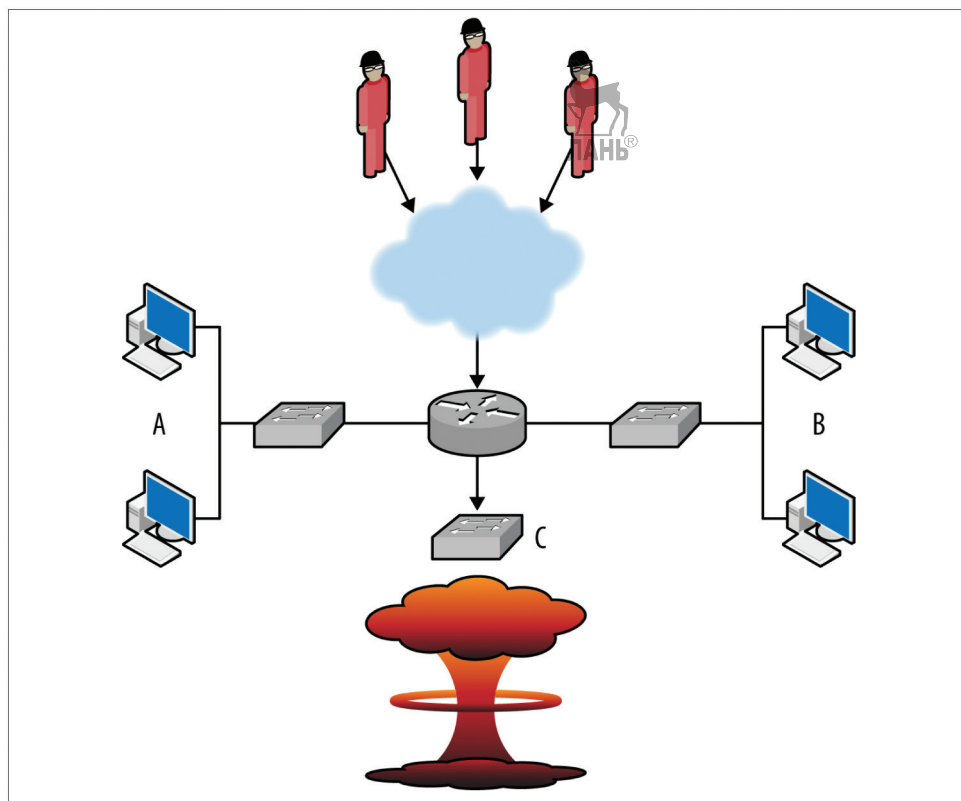


Рис. 12-6. Сопутствующий ущерб от DDoS-атаки

Проблемы этого типа не являются чем-то необычным для сервисов в одной сети и лишний раз подчеркивают, что защита от DDoS-атак является задачей сетевой инфраструктуры. С этой точки зрения интересно посмотреть, как совмещаются DDoS-атаки и облачные вычисления. Облачные вычисления позволяют запускать распределенные службы в инфраструктуре интернета, что позволяет атакующему приобрести достаточный объем ресурсов для вывода из строя своей жертвы.

Нередко за DoS-атаки ошибочно принимают внезапные *флешмобы* и обрывы кабелей. Флешмоб – это внезапный приток легального трафика в ответ на какое-то объявление или уведомление. Иногда флешмобы называют *слеш-дот-эффектом* (SlashDot effect), *фаркингом* (farking) или *Reddit-эффектом*, что достаточно хорошо объясняет происходящее.

Эти разные явления обычно легко различить на графике входящего трафика. На рис. 12-7 показаны некоторые идеализированные ситуации, объясняющие суть явлений.



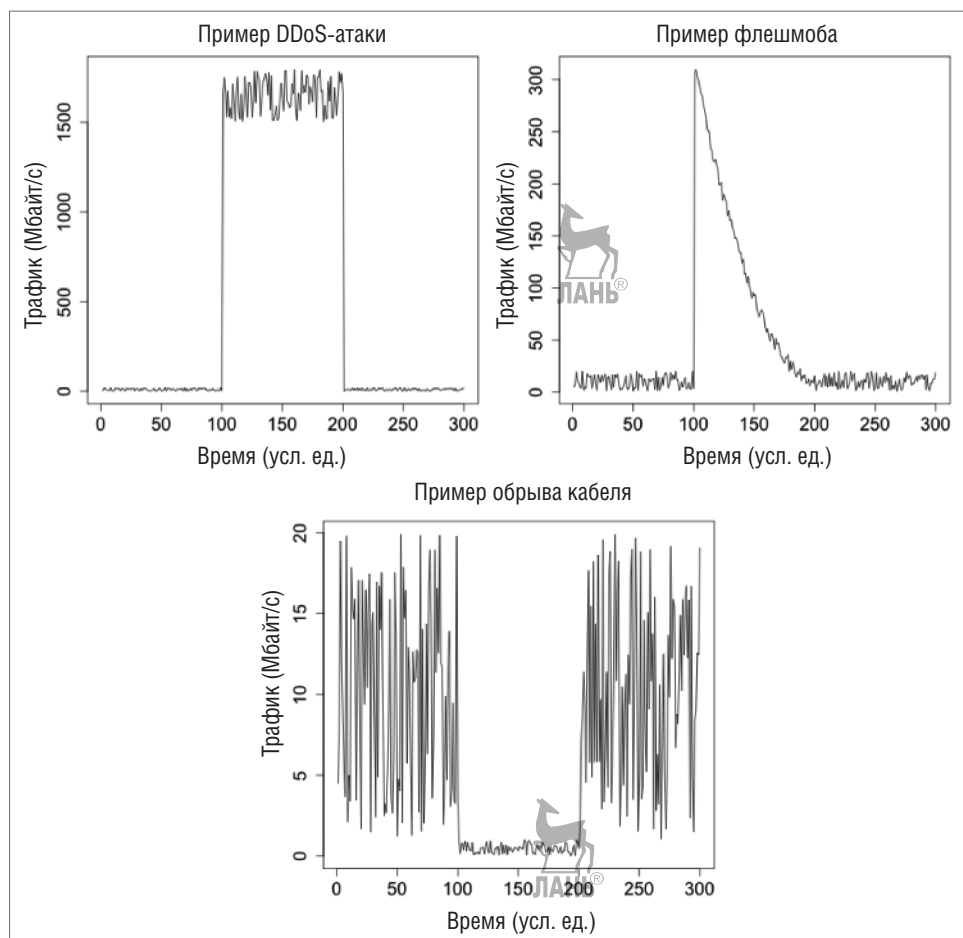


Рис. 12-7. Разные ситуации, приводящие к исчерпанию полосы пропускания

Графики на рис. 12-7 описывают три случая исчерпания полосы пропускания: DDoS-атака, наплыв посетителей (флешмоб) и обрыв кабеля или другой сбой инфраструктуры. Они показывают, как меняется входящий трафик в каждом случае, и отражают явления, вызвавшие проблемы.

DDoS-атаки носят механический характер. Обычно атакующий трафик резко возрастает, а потом так же резко уменьшается, в моменты, когда атакующий посылает команды в сеть ботов. Когда DDoS-атака запускается, она почти мгновенно потребляет всю доступную пропускную способность. На многих графиках DDoS-атаки верхний предел определяется возможностями сетевой инфраструктуры: если ваш канал имеет ширину 10 Гбит, максимум на графике составит 10 Гбит. DDoS-атаки также отличаются *равномерностью*. После начала они, как правило, не меняют уровень воздействия. В большинстве случаев атакующие вовлекают в атаку избыточное число ботов. Затем они постепенно исключают лишние боты, но оставляют достаточное их количество, чтобы использовать всю доступную пропускную способность.

Противостояние DDoS-атакам – это соревнование на выносливость. Лучшая защита – выделить достаточную пропускную способность до начала атаки. Когда атака действительно случится, лучшее, что можно сделать, – попытаться определить закономерности в трафике и заблокировать те виды трафика, которые наносят наибольший урон, например определить основную аудиторию целевого сервера, и ограничить трафик к основной аудитории. Аудиторию можно идентифицировать, к примеру по IP-адресам, сетевым блокам, коду страны или языку. Важно, чтобы основная аудитория имела как можно меньше совпадений с набором атакующего. Сценарий в примере 12-4 реализует механизм упорядочения /24, различая два вида пользователей: исторических, которым вы доверяете, и новых, которых вы подозреваете в участии в DDoS-атаке.

Фальсифицированные атаки *иногда* можно идентифицировать по некоторому недостатку в их реализации. Например, генератор случайных чисел для подделки может выбрать для всех адресов значение x.x.x.1.

Пример 12-4. Пример сценария для упорядочения блоков

```
#!/usr/bin/env python
#
# ddos_intersection.py
#
# входные данные:
#   нет
# выходные данные:
#   Отчет, сравнивающий количество адресов в двух наборах, упорядоченный
#   по наибольшему числу хостов в наборе А, отсутствующих в наборе В.
#
# порядок использования:
# ddos_intersection.py historical_set ddos_set
#
# historical_set: набор исторических данных с внешними адресами,
# которые исторически взаимодействовали с конкретным хостом или сетью
# ddos_set: набор данных о DDoS-атаке на хост
# Для простоты сценарий предполагает, что анализируются сети /24.
#
import sys,os,tempfile

historical_setfn = sys.argv[1]
ddos_setfn = sys.argv[2]
blocksize = int(sys.argv[3])

mask_fh, mask_fn = tempfile.mkstemp()
os.close(mask_fh)
os.unlink(mask_fn)

os.system(('rwssettool --mask=24 --output-path=stdout %s | ' +
          ' rwssetcat | sed 's/$/\24/' | rwssetbuild stdin %s') %
          (historical_setfn, mask_fn))

bins = {}
# Прочитать и сохранить все сети /24 в исторических данных
a = os.popen(('rwssettool --difference %s %s --output-path=stdout | ',
              'rwssetcat --network-structure=C') % (mask_fn, historical_setfn), 'r')
```

```

# Первая колонка - исторические данные, вторая - DDoS
for i in a.readlines():
    address, count = i[:-1].split('|')[0:2]
    bins[address] = [int(count), 0]

a.close()
# Повторить то же для данных из набора DDoS
a = os.popen(('rwssetool --difference %s %s --output-path=stdout | ',
            'rwssetcat --network-structure=C') % (mask_fn, ddos_setfn), 'r')
for i in a.readlines():
    address, count = i[:-1].split('|')[0:2]
    # Я снова нахожу пересечение с файлом маски, сгенерированным из
    # первоначального файла, поэтому любой адрес, присутствующий в файле,
    # уже будет присутствовать в ассоциативном массиве bins
    bins[address][1] = int(count)
#
# Теперь упорядочим содержимое bins. Этот сценарий реализует подход на основе
# белого списка - адреса в историческом наборе являются кандидатами на
# добавление в белый список, а все остальные адреса будут блокироваться.
# Мы упорядочиваем блоки-кандидаты по количеству исторических адресов.
#
address_list = bins.items()
address_list.sort(lambda x,y:(y[1][0]-x[1][0])-(y[1][1]-x[1][1]))
print "%20s|10s|10s" % ("Block", "Not-DDoS", "DDoS")
for address, result in address_list:
    print "%20s|10d|10d" % (address, bins[address][0], bins[address][1])

```

Этот тип фильтрации эффективен, если атака направлена на определенный сервис, например веб-сервер. Если цель атакующего – переполнить пропускную способность интерфейса маршрутизатора, тогда защитные мероприятия должны проводиться перед ним.

Как отмечалось в главе 11, люди нетерпеливы, в отличие от машин, и эта их черта позволяет легко отличить флешмоб от DDoS-атаки. Как показано на графике флешмоба на рис. 12-7, когда происходит какое-то событие, наблюдается начальный всплеск объема трафика, за которым тут же следует быстрое падение. Падение происходит потому, что люди, столкнувшись с невозможностью попасть на целевой сайт, перешли куда-то еще, чтобы повторить попытку позже.

Флешмоб – это общественное явление, когда по какой-то причине *кто-то* обозначил цель. В результате часто удается выяснить причину флешмоба. Например, по HTTP-заголовку Referer в журнале можно узнать ссылку на сайт. Поиск в интернете по ссылке часто помогает найти источник. Также неожиданный наплыв посетителей может быть обусловлен публикацией в прессе или в новостях информации о вашем сайте.

Обрыв кабеля и другие физические неисправности оборудования вызывают фактическое падение трафика. Это видно на рис. 12-7, где трафик внезапно падает до нуля. Когда это происходит, первым делом следует попробовать сгенерировать некоторый трафик к целевому хосту и убедиться, что проблема на самом деле заключается в оборудовании, а не обусловлена неисправностью в детекторе. После этого нужно подключить к сети резервную систему и исследовать причину сбоя.

DDoS и усиливающие факторы

Функционально DDoS-атаки – это войны на истощение: какой объем трафика атакующий сможет послать в цель и как цель сможет компенсировать его, чтобы сохранить пропускную способность? Атакующие могут повысить эффективность своей атаки, применяя различные стратегии: использовать больше ресурсов, атаковать на разных уровнях сетевого стека и задействовать инфраструктуру интернета для нанесения дополнительного ущерба. Каждая из этих стратегий эффективно использует усиливающие факторы, увеличивающие хаос при том же количестве ботов, находящихся под контролем атакующего.

Процесс получения ресурсов зависит от самого атакующего. В современном интернете имеется развитый рынок, где можно арендовать и использовать бот-сети. Альтернативный подход, применяемый, в частности, некоторыми членами группы «Anonymous», заключается в привлечении добровольцев. В «Anonymous» было разработано семейство инструментов для DDoS-атак на JavaScript и C# под названием «LOIC» (Low Orbit Ion Cannon – низкоорбитальное ионное орудие). Инструменты в семействе LOIC реализованы довольно примитивно. Возможно, они не предназначены для чего-то большего, учитывая их хактивистскую¹ аудиторию.

Эти методы основаны на асимметрии затрат на отправляющей и принимающей стороне: атакующий манипулирует операциями так, что затраты на обработку каждого соединения на сервере получаются выше, чем затраты на клиенте. Из этого следует, что архитектурные решения напрямую влияют на уязвимость системы для высокоуровневых DDoS-атак².

Также для проведения атак злоумышленники могут использовать инфраструктуру интернета. Обычно для этого используется отвечающий сервис, который можно использовать для послышки ответа по поддельному целевому адресу. Классический пример такого нападения – смурф-атака, когда хост А, желая вызвать отказ в обслуживании сайта В, отправляет поддельный ping-пакет на широковещательный адрес. Каждый хост, получивший пакет (то есть все хосты, потому что пакет послан на широковещательный адрес), возвращает ответ целевому хосту, заваливая его ответными пакетами. Наиболее распространенная современная форма этой атаки использует *отражение DNS*: злоумышленник отправляет поддельный запрос в DNS-резолвер, который затем отправляет в ответ чрезмерно информативный и большой пакет.

ПРИМЕНЕНИЕ АНАЛИЗА ОБЪЕМА И ЛОКАЛЬНОСТИ

Для обнаружения явлений, обсуждаемых в этой главе, используется несколько различных подходов. Вообще говоря, проблема заключается не столько в их обнаружении, сколько в разделении вредоносных и законных действий, которые часто

¹ <https://ru.wikipedia.org/wiki/Хактивизм>. – Прим. перев.

² Истории известны интересные примеры, подтверждающие это. Факсы подвергались атакам «черный факс», когда злоумышленник отправлял полностью черную страницу, чтобы опустошить запас тонера.

имеют схожие черты. В этом разделе мы обсудим несколько способов создания детекторов и уменьшения ложных срабатываний.

Сбор данных

Информация о трафике часто сильно искажена, и между общим объемом трафика и объемом злонамеренного трафика нередко нет никакой корреляции. Атакующий может управлять сетью с помощью *ssh* и генерировать намного меньше трафика, чем законный пользователь, отправляющий электронное письмо с объемным вложением. Основные искажения в данные вносят мусорный трафик, например созданный попытками сканирования, и другое фоновое излучение (подробнее об этом рассказывается в главе 11).

Наиболее очевидные характеристики, которые используются при анализе объема, – это количество байтов и пакетов за некоторый период. Они тоже часто настолько искажены, что лучше всего использовать их только для выявления DDoS- и рейдерских атак.

Поскольку базовые характеристики сильно искажены и легко могут быть интерпретированы неверно, я предпочитаю работать с искусственными характеристиками, такими как поток. С помощью NetFlow я группирую трафик в сеансы, а затем фильтрую потоки по различным признакам, таким как:

- оставляю трафик только с законными узлами (не с темным пространством); для этого необходимо иметь актуальную карту сети, как обсуждается в главе 15;
- выделяю короткие сеансы TCP (четыре пакета или меньше) или ищу другие признаки законности сеанса, например наличие флага PSH; более подробно этот вопрос обсуждается в главе 11;
- выделяю в трафике команды, признаки прощупывания и передачу файлов. Этот подход, о котором подробнее рассказывается в главе 14, помогает разбить трафик на разные классы, часть из которых встречается довольно редко;
- использую простую классификацию на основе пороговых значений объемов. Например, вместо количества байтов записываю объем потоков как 100, 1000, 10 000 и 100 000+. Это помогает уменьшить шум, с которым приходится иметь дело.

Всякий раз, выполняя такую фильтрацию, важно не просто отбрасывать данные, а разбивать их на классы. Например, анализируя пороговые объемы, классифицируйте трафик по границам 1–100, 100+, 1000+, 10 000+ и 100 000+ в виде отдельных временных рядов. Причиной такого подхода к разделению данных является обычная паранойя. Каждый раз, вводя жесткое правило фильтрации для данных, которые будут игнорироваться, вы открываете злоумышленнику возможность замаскировать свои действия под игнорируемые данные.

Вместо объемов можно подсчитывать такие значения, как количество IP-адресов, обращающихся к хостам в сети, или количество извлеченных уникальных URL-адресов. В вычислительном отношении эти значения являются более дорогостоящими, поскольку требуют различать отдельные значения; это можно сделать с помощью такого инструмента, как *fwset* из комплекта *SiLK*, или с помощью ассоциативного массива. Подсчет адресов, как правило, дает более стабильные результаты, чем подсчет объемов, и даже простое разделение хостов, подвергшихся сканированию (опять же), здорово помогает уменьшить шум.

Пример 12-5 демонстрирует, как применить фильтрацию и разделение к потоку данных, чтобы получить временные последовательности.

Пример 12-5. Применение вывода простого временного ряда

```
#
# gen_timeseries.py
#
# Генерирует временные последовательности, читая информацию о потоках,
# и разбивает данные, в данном случае, на короткие (<= 4 пакетам) и длинные
# (> 4 пакетов) потоки TCP.
#
# выводит
# Время <байты> <пакеты> <адреса> <длинные байты> <длинные пакеты> <длинные адреса>
#
# На входе принимает
# gwcut --fields=sip,dip,bytes,packets,stime --epoch-time --no-title
#
# Предполагается, что записи следуют в хронологическом порядке, то есть
# ни одна следующая запись не имеет значение stime меньше,
# чем предыдущая запись.

import sys
current_time = sys.maxint
start_time = sys.maxint
bin_size = 300 # Для удобства используются пятиминутные интервалы
ip_set_long = set()
ip_set_short = set()
byte_count_long = 0
byte_count_short = 0
packet_count_long = 0
packet_count_short = 0
for i in sys.stdin.readlines():
    sip, dip, bytes, packets, stime = i[:-1].split('|')[0:5]
    # Преобразовать нецелочисленные значения
    bytes, packets, stime = map(lambda x: int(float(x)), (bytes, packets, stime))
    # Проверить время текущего интервала; если необходимо, создать новый интервал
    # и очистить его содержимое
    if (stime < current_time) or (stime > current_time + bin_size):
        ip_set_long = set()
        ip_set_short = set()
        byte_count_long = byte_count_short = 0
        packet_count_long = packet_count_short = 0
        if (current_time == sys.maxint):
            # Установить время по границе 5 минут от текущего значения.
            # Это делается для того, чтобы значения времени всегда были
            # кратными пяти минутам и не было таких значений, как
            # t + 307, t + 619 и т. д.
            current_time = stime - (stime % bin_size)
    else:
        # Вывести результаты
        print "%10d %10d %10d %10d %10d %10d %10d" % (
            current_time, len(ip_set_short), byte_count_short,
            packet_count_short, len(ip_set_long), byte_count_long,
            packet_count_long)
        current_time = stime - (stime % bin_size)
```

```

else:
    # Вместо вывода просто добавим данные.
    # Сначала определим вид потока: длинный или короткий
    if (packets <= 4):
        # короткий поток
        byte_count_short += bytes
        packet_count_short += packets
        ip_set_short.update([sip,dip])
    else:
        byte_count_long += bytes
        packet_count_long += packets
        ip_set_long.update([sip,dip])

if byte_count_long + byte_count_short != 0:
    # Окончательный вывод результатов
    print "%10d %10d %10d %10d %10d %10d %10d" % (
        current_time, len(ip_set_short), byte_count_short,
        packet_count_short, len(ip_set_long), byte_count_long,
        packet_count_long)

```

Следите за тем, как вы разделяете и анализируете трафик. Например, если вы решили рассчитать пороговые значения для сигнала тревоги только для сеансов из Болгарии, с объемом трафика не менее 100 байт, обязательно используйте этот же подход для вычисления пороговых значений в будущем, задокументируйте это обстоятельство и опишите причину своего выбора.

Создание тревог на основе объема

Самый простой способ реализовать сигнал тревоги на основе объема – это рассчитать гистограмму и выбрать пороги на основе вероятности их превышения. Сценарий *calibrate_raid.py*, представленный в примере 12-2, является отличным образцом такого расчета порогов. При генерировании тревог необходимо учитывать время суток, как обсуждалось в разделе «Влияние рабочих часов на объем трафика» в начале главы, а также использовать несколько моделей; применение единственной модели обычно не позволяет достичь необходимой точности. Кроме того, выбирая пороги, поразмышляйте о влиянии необычно *низких* значений и необходимости их исследования.

Учитывая большой уровень шума в данных об объеме трафика, можно ожидать значительного количества ложных тревог. Большинство ложных тревог, генерируемых на основе анализа объема, обусловлены законной деятельностью хостов, связанной с копированием или архивированием информации с целевого сайта. Здесь вполне могут пригодиться некоторые из методов снижения ложных срабатываний в системе определения вторжений (Intrusion Detection System, IDS), обсуждавшиеся в разделе «Улучшение ответа IDS», в главе 7; в частности, добавление аномалий в белый список после того, как выяснится, что их источники безобидны.

Создание тревог из тревожных сигналов

Тревожные сигналы используются для обнаружения хостов, которые постоянно взаимодействуют с другими хостами. Тревожные сигналы часто используются для выявления злонамеренных действий и главным образом – для идентификации

взаимодействий с управляющим сервером ботнета. Для этого обычно определяются хосты, которые осуществляют *систематические* взаимодействия в пределах временного окна, как показано в примере сценария *find_beacons.py*.

Огромное число тревожных сигналов оказываются ложными, потому что множество вполне законных действий, таких как обновление программного обеспечения, обновление баз данных антивирусов и даже задания *cron*, использующие соединения через SSH, выполняются систематически и регулярно. Поэтому для надежной фильтрации тревожных сигналов необходимо владеть актуальной информацией о подобных законных действиях. Получив уведомление о тревожном сигнале, нужно оценить законность действий хоста, вызвавших появление сигнала, что нетрудно сделать, если тревожный сигнал вызван действиями по известному протоколу, обменивается данными с законным хостом или имеются какие-то другие доказательства, что трафик *не* вызван взаимодействиями с центральным сервером ботнета. Убедившись в законности, параметры соответствующего тревожного сигнала (адрес и, возможно, порт) следует зафиксировать, чтобы предотвратить дальнейшие ложные срабатывания.

Также важно проверить хосты, которые *должны* генерировать тревожные сигналы, но этого не происходит. Это особенно важно при работе с антивирусами, потому что атакующие часто отключают антивирусы при захвате нового хоста. Проверка, что все хосты, которые должны загружать обновления, делают это, является полезной альтернативой тревогам.

Создание тревог по признакам локальности

Локальность является мерой привычек пользователя. Модель рабочего набора позволяет обозначить круг этих привычек и выявить любые нарушения данного круга. Большую часть времени люди предсказуемы, но иногда они отправляют по почте новые контакты или посещают новые веб-сайты. Поэтому тревоги, основанные на оценке локальности, весьма полезны для выявления изменений в привычках и различения обычных пользователей веб-сайта от совершающего нападение на него, или для определения, как меняется аудитория сайта во время DDoS-атаки.

Локальность является полезным дополнением к выявлению рейдерских атак на основе объема трафика. Хост, совершающий рейдерскую атаку на сайт или иным образом сканирующий его, будет демонстрировать минимальную локальность, поскольку будет стараться посетить все страницы сайта как можно быстрее. Для идентификации рейдерской атаки посмотрите, к каким страницам или сервисам обращается хост и как быстро он это делает.

Наиболее распространенными источниками ложных срабатываний в этом случае являются поисковые системы и интернет-роботы, такие как Googlebot. Хорошего робота можно идентифицировать по заголовку User-Agent, и если хост *не* идентифицируется как робот по этому заголовку, его можно считать потенциально опасным.

Модель рабочего набора можно применить не только к обычным пользователям, но также к серверам. Такой рабочий набор будет значительно больше, чем профиль пользователя, но его можно использовать для определения основной аудитории веб-сайта или сервера SSH.

Инженерные решения

Обнаружение рейдерских атак – хороший пример ситуации, когда можно применить анализ и, вероятно, когда лучше *не* заниматься конструированием детектора. Гистограммы, сгенерированные с помощью *calibrate RAID.py* или путем вычисления ожидаемого объема, который извлекается пользователем за день, в конечном счете определяют, сколько данных пользователь реально получит с сервера.

Эту информацию можно использовать также для установления ограничений скорости на серверах. Вместо того чтобы генерировать тревогу, когда пользователь превышает этот порог, достаточно просто задействовать модуль ограничения скорости (например, Apache Quota), чтобы ограничить аппетиты пользователя. Если вас волнует проблема недовольства пользователей, установите пороговое значение равным 200 % от максимального наблюдаемого значения и определите тех, кому требуются специальные разрешения, чтобы превысить даже этот высокий порог.

Данный подход наиболее эффективен для защиты сервера, который хранит объем данных, радикально превышающий среднюю потребность любого отдельного пользователя. Если люди используют менее мегабайта трафика в день, тогда как на сервере хранятся гигабайты данных, вы получите легко защищаемую цель.

Дополнительные материалы для чтения

1. *Аврил Коглан (Avril Coghlan)*. A Little Book of R for Time Series.
2. *Джон МакХью (John McHugh) и Кэрри Геймс (Carrie Gates)*. Locality: A New Paradigm in Anomaly Detection: материалы семинара по новым парадигмам безопасности 2003 года.



Глава 13

.....

Анализ графа

Граф – это математическая конструкция, состоящая из одного или нескольких узлов (*вершин*), соединенных друг с другом *связями* (*ребрами*). Графы позволяют эффективно описать взаимодействия без риска заблудиться в мелких деталях. Их можно использовать для моделирования соединений и их всестороннего представления, абстрагируясь от таких деталей, как размеры пакетов и длительность сеанса. Кроме того, такие атрибуты графа, как центральность, можно использовать для идентификации критических узлов в сети. Наконец, многие важные протоколы (в частности, SMTP и протоколы маршрутизации) полагаются на алгоритмы, моделирующие конкретную сеть в виде графа.

Эта глава целиком посвящена аналитическим свойствам графов. Сначала мы узнаем, что такое граф, а затем исследуем основные атрибуты: кратчайший путь, центральность, кластеры и коэффициенты кластеризации.

АТРИБУТЫ ГРАФА: ЧТО ТАКОЕ ГРАФ?

Граф – это математическое представление совокупности объектов и их взаимосвязей. Первоначально графы были придуманы Леонардом Эйлером (Leonhard Euler) в 1736 году для решения задачи пересечения мостов Кенигсберга. С тех пор графы начали использоваться для моделирования всего и вся – от выявления основных членов заговоров до частоты звуков, произносимых человеком. Графы – чрезвычайно мощный и гибкий описательный инструмент, и эта мощь и гибкость обусловлены их огромной функциональностью. Исследователи в области математики, инженерии и социологии выявили обширный набор атрибутов графов, которые можно использовать для моделирования различных форм поведения. Приступая к использованию графов, первым делом нужно решить, какие атрибуты понадобятся и как их получить. Далее будет представлено подмножество атрибутов, демонстрирующих, что можно получить с помощью графов. Это подмножество было выбрано с учетом их прямого отношения к моделированию трафика, о котором мы поговорим позже. Любая хорошая книга по теории графов описывает намного больше атрибутов, поэтому рано или поздно вам будет куда обратиться за дополнительной информацией.

Граф, как минимум, состоит из узлов и связей (*вершин* и *ребер*), где *связи* – это соединения между двумя узлами. Ребро может быть *направленным* или *ненаправленным*; если ребро направленное, то у него имеется *начальная* и *конечная* вершины.

Традиционно граф состоит только из направленных или только ненаправленных ребер. Такие графы называют *ориентированными* и *неориентированными* соответственно. Если граф является неориентированным, тогда каждая вершина характеризуется *степенью* – количеством ребер, связанных с этой вершиной. Вершины в ориентированном графе имеют *степень входа* – количество ребер, для которых данная вершина является конечной, и *степень выхода* – количество ребер, для которых данная вершина является начальной¹.

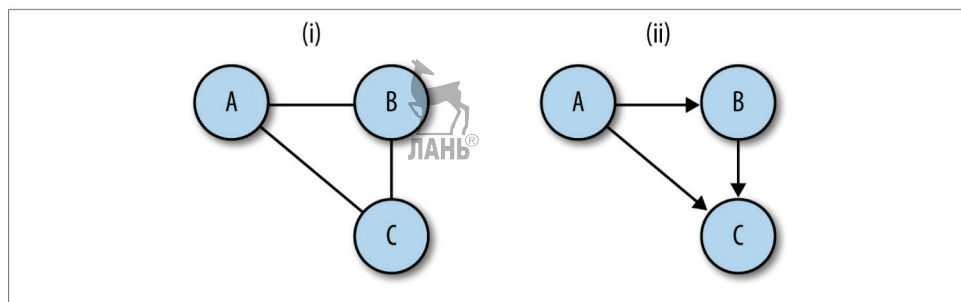


Рисунок 13-1. Неориентированный и ориентированный графы: слева (i) изображен неориентированный граф, в котором каждая вершина имеет степень 2; справа (ii) изображен ориентированный граф: вершина A имеет степень выхода 2 и степень входа 0; вершина B имеет степень выхода 1 и степень входа 1; вершина C имеет степень выхода 0 и степень входа 2

В журналах сетевого трафика можно найти множество кандидатов для преобразования в графы. Например, IP-адреса можно использовать как вершины, а наличие трафика между ними – как ребра. В журналах HTTP-серверов вершинами могут служить отдельные страницы, связанные заголовками Referer. В журналах почтового сервера вершинами могут быть адреса электронной почты, а ребрами – электронные письма. Все, что можно представить как передачу сообщения из точки A в точку B, является хорошим кандидатом.

Примечание к программному коду, представленному в этом разделе книги: он предназначен в основном для образовательных целей, поэтому, чтобы более ясно и четко показать, как работают различные алгоритмы или числа, я не использовал оптимизации и ловушки исключений, которые обязательно предусмотрел бы в промышленном коде. Это особенно важно для анализа графов, потому что алгоритмы обработки графов часто очень дорогостоящие в вычислительном отношении. Для анализа графом имеется несколько хороших библиотек, способных обрабатывать сложные графы гораздо эффективнее, чем тот код, который я здесь представлю.

Сценарий в примере 13-1 может создавать ориентированные или неориентированные графы из списков пар (например, из вывода команды `gwcut --field = 1,2 --no-title --delim = ' '`). Есть несколько методов реализации графов; в данном случае я использую *списки смежности*, которые, на мой взгляд, наиболее очевидны. В списках смежности каждая вершина хранит таблицу всех своих ребер.

¹ Иногда степени входа и выхода называют полустепенями захода и исхода соответственно. – Прим. перев.

Пример 13-1. Простые графы

```
#!/usr/bin/env python
#
# basic_graph.py
#
# Библиотека
# Предлагает:
#   Реализации объектов графов с конструкторами, принимающими файл потока
#
import os, sys

class UndirGraph:
    """ Класс неориентированного, невзвешенного графа
        также служит базовым классом для всех других
        классов графов в этой главе """

    def add_node(self, node_id):
        self.nodes.add(node_id)

    def add_link(self, node_source, node_dest):
        self.add_node(node_source)
        self.add_node(node_dest)
        if not self.links.has_key(node_source):
            self.links[node_source] = {}
        self.links[node_source][node_dest] = 1
        if not self.links.has_key(node_dest):
            self.links[node_dest] = {}
        self.links[node_dest][node_source] = 1
        return

    def count_links(self):
        total = 0
        for i in self.links.keys():
            total += len(self.links[i].keys())
        return total/2 # Компенсация удвоения ребер в неориентированном графе

    def neighbors(self, address):
        # Возвращает список всех вершин, связанных ребрами с данной вершиной.
        # Если таких вершин нет, возвращает пустой список (технически такое
        # невозможно из-за правил конструирования, но чем черт не шутит).
        if self.nodes.has_key(address):
            return self.links[address].keys()
        else:
            return None

    def __str__(self):
        return 'Undirected graph with %d nodes and %d links' %
            (len(self.nodes), self.count_links())

    def adjacent(self, sip, dip):
        # Обратите внимание, что это класс неориентированного графа,
        # поэтому достаточно проверить ребра начальной вершины.
        if self.links.has_key(sip):
            if self.links[sip].has_key(dip):
                return True
```



```

def __init__(self):
    #
    # Этот граф реализован с помощью списков смежности; каждая вершина
    # имеет ключ в хеш-таблице links, значением которого является другая
    # хеш-таблица.
    #
    # Таблица nodes избыточна для неориентированных графов, потому что
    # связь между X и Y предполагает связь между Y и X, но в реализации
    # ориентированных графов эта избыточность позволит увеличить скорость
    # поиска конкретного узла.
    self.links = {}
    self.nodes = set()

class DirGraph(UndirGraph):

    def add_link(self, node_source, node_dest):
        # Обратите внимание, что, в отличие от неориентированного графа,
        # здесь добавляются ребра только для одного направления
        self.add_node(node_source)
        self.add_node(node_dest)
        if not self.links.has_key(node_source):
            self.links[node_source] = {}
        self.links[node_source][node_dest] = 1
        return

    def count_links(self):
        # В ориентированном графе метод count_links должен подсчитывать
        # ht,hf иначе, чем в родительском классе неориентированного графа.
        total = 0
        for i in self.links.keys():
            total += len(self.links[i].keys())
        return total

if __name__ == '__main__':
    #
    # Это "заглушка", которая будет выполнена, если сценарий использовать не
    # как библиотеку, а как самостоятельную программу. Она создаст и выведет
    # неориентированный граф, получив множество пар (source, dest),
    # разделенных пробелами
    #
    a = sys.stdin.readlines()
    tgt_graph = DirGraph()
    for i in a:
        source, dest = i.split()[0:2]
        tgt_graph.add_link(source, dest)
    print tgt_graph
    print "Links:"
    for i in tgt_graph.links.keys():
        dest_links = ' '.join(tgt_graph.links[i].keys())
        print '%s: %s' % (i, dest_links)

```


Конструирование графа и его атрибуты

Приступая к работе с графами, у многих возникает соблазн сразу начать строить сложные отношения между сетевыми атрибутами, например определять направления от клиента к серверу или взвешивать ребра, соответствующие трафику между узлами.

Я считаю, что усилия, расходуемые на конструирование сложных отношений, обходятся слишком дорого и не стоят того. Лучше начать с простого графа и изучить его атрибуты, а не пытаться создать сложное графовое представление. Вот два простых правила для преобразования исходных данных в графы:

Определите взаимодействия

Ребро должно представлять взаимодействие между двумя узлами; например, ребро можно создать, только если поток между хостами насчитывает 10 или более пакетов и присутствует флаг АСК. Это позволит исключить попытки сканирования и неудачные попытки входа в систему.

Определите, как идентифицировать вершины

Вершины могут быть представлены IP-адресами хостов или комбинациями IP-адреса и номера порта. Мне показалось удобным разбить порты на сервисы (порты с номерами до 1024 представляют конкретные сервисы; все остальные номера соответствуют клиентам) и использовать комбинации из IP-адресов и названий сервисов.

МЕТКИ, ВЕС И ПУТИ

Путь в графе – это набор ребер, соединяющих два узла. В ориентированном графе пути следуют за направлениями ребер, тогда как в неориентированном графе разрешается двигаться в любом направлении. Особое значение при анализе графа имеют *кратчайшие пути*, которые, как следует из названия, представляют минимальный набор ребер, необходимых для перехода из точки А в точку В (см. пример 13-2).

Пример 13-2. Поиск кратчайшего пути в графе

```
#!/usr/bin/env python
#
# arsp.py – реализация взвешенных путей и алгоритма Дейкстры

import sys,os,basic_graph

class WeightedGraph(basic_graph.UndirGraph):

    def add_link(self, node_source, node_dest, weight):
        # Взвешенные двунаправленные ребра, обратите внимание,
        # что теперь значение ребра выбирается равным его весу
        # вместо 1, как делалось в невзвешенном графе, где
        # все ребра имеют одинаковый вес.
        self.add_node(node_source)
        self.add_node(node_dest)
        if not self.links.has_key(node_source):
            self.links[node_source] = {}
```

```

if not self.links[node_source].has_key(node_dest):
    self.links[node_source][node_dest] = 0
self.links[node_source][node_dest] += weight
if not self.links.has_key(node_dest):
    self.links[node_dest] = {}
if not self.links[node_dest].has_key(node_source):
    self.links[node_dest][node_source] = 0
self.links[node_dest][node_source] += weight

def dijkstra(self, node_source):
    # Для вершины node_source создает карту путей во все остальные вершины
    D = {} # Временная таблица расстояний
    P = {} # таблица предшествующих путей

    # Таблица предшественников основана на уникальном свойстве
    # кратчайших путей: все вложенные пути в кратчайшем пути сами
    # являются кратчайшими путями, то есть если путь (B, C, D)
    # кратчайший из точки A в точку E, тогда (B, C)
    # является кратчайшим путем из A в D. Остается только сохранить
    # предшествующий путь и вернуться назад.

    infy = 999999999999 # Представление бесконечности
    for i in self.nodes:
        D[i] = infy
        P[i] = None

    D[node_source] = 0
    node_list = list(self.nodes)
    while node_list != []:
        current_distance = infy
        current_node = None
        # Шаг 1, найти ближайшую вершину, в первом вызове это будет
        # сама вершина node_source как единственная, расстояние до
        # которой D = 0
        for i in node_list:
            if D[i] < current_distance:
                current_distance = D[i]
                current_node = i
        node_index = node_list.index(i)
        del node_list[node_index] # Удалить ее из списка
        if current_distance == infy:
            break # Были исследованы все пути из вершины,
            # все остальные пути принадлежат другому компоненту
        for i in self.neighbors(current_node):
            new_distance = D[current_node] + self.links[current_node][i]
            if new_distance < D[i]:
                D[i] = new_distance
                P[i] = current_node
            node_list.insert(0, i)
    for i in D.keys():
        if D[i] == infy:
            del D[i]
    for i in P.keys():
        if P[i] is None:
            del P[i]
    return D,P

def apsp(self):

```

```
# Вызвать метод dijkstra для каждой пары вершин,
# чтобы создать таблицу всех кратчайших путей
apsp_table = {}
for i in self.nodes:
    amsp_table[i] = self.dijkstra(i)
return amsp_table
```



В альтернативном алгоритме поиска кратчайших путей используется *взвешивание*. Во взвешенном графе ребрам присваивается числовой вес. В этом случае кратчайший путь – не просто наименьшее количество ребер на пути из точки А в точку В, а набор ребер с наименьшим общим весом, как показано на рис. 13-2.

Кратчайшие пути являются фундаментальными строительными блоками в анализе графов. В большинстве сервисов маршрутизации, таких как Open Shortest Path First (OSPF), целью является поиск кратчайших путей. В результате в большинстве случаев анализ графа начинается с построения таблицы всех кратчайших путей между всеми узлами с использованием алгоритма All Pairs Shortest Paths (APSP) в графе. Код в примере 13-2 иллюстрирует использование алгоритма Дейкстры для вычисления кратчайших путей во взвешенном неориентированном графе.

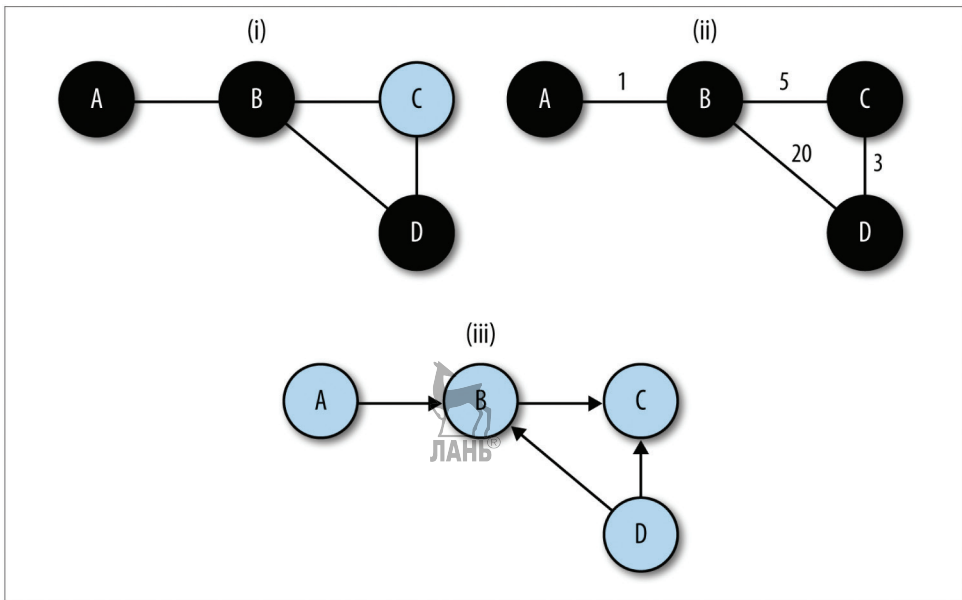


Рис. 13-2. Веса и пути, кратчайший путь из А в D: (i) в неориентированном невзвешенном графе кратчайший путь определяется наименьшим количеством промежуточных вершин, (ii) во взвешенном графе самый короткий путь обычно имеет наименьший общий вес, (iii) в ориентированном графе кратчайший путь может отсутствовать

Алгоритм Дейкстры поиска кратчайшего пути может работать с любыми графами, ребра в которых имеют положительный вес. Поиск кратчайшего пути играет важную роль в ряде областей, поэтому было создано огромное количество разнообразных алгоритмов, учитывающих особенности графов с разной структурой

и обладающих вершинами, хранящими разную информацию о графе.

Кратчайшие пути эффективно определяют расстояния между вершинами в графе и служат строительными блоками для ряда других атрибутов. Одним из примеров таких атрибутов является *центральность* (см. пример 13-3). Понятие центральности возникло в сфере анализа социальных сетей; анализ социальных сетей моделирует отношения между сущностями с использованием графов и конструирует атрибуты графа, показывающие взаимосвязи между этими сущностями в массе. Центральность, которая может оцениваться по-разному, служит показателем важности вершины для структуры графа.

Пример 13-3. Вычисление центральности

```
#!/usr/bin/env python
#
#
# centrality.py
#
# вычисляет оценки центральности для набора данных
#
# входные данные:
# Список пар (source, destination), разделенных пробелами.
# Веса ребер не задаются явно, а определяются по числу появления пары
#
# порядок использования:
# calc_centrality.py n
# n: целое число элементов в возвращаемом отчете
#
# выходные данные:
# Отчет с 7 колонками: ранг | вершина, центральная по посредничеству |
# количество кратчайших путей | вершина, центральная по степени | величина степени |
# вершина, центральная по близости | оценка близости

import sys,string
import apsp

n = int(sys.argv[1])

closeness_results = []
degree_results = []
betweenness_results = []

target_graph = apsp.WeightedGraph()

# Загрузка графа
for i in sys.stdin.readlines():
    source, dest = i[:-1].split()
    target_graph.add_link(source, dest, 1)

# Вычисление центральности по степени; самый простой случай,
# потому что просто требуется найти вершину с наибольшей степенью
for i in target_graph.nodes:
    degree_results.append((i, len(target_graph.neighbors(i))))

apsp_results = target_graph.apsp()
# Вычисление центральности по близости
```



```

for i in target_graph.nodes:
    dt = apsp_results[i][0] # Таблица расстояний
    total_distance = reduce(lambda a,b:a+b, dt.values())
    closeness_results.append((i, total_distance))

# Вычисление центральности по посредничеству

bt_table = {}
for i in target_graph.nodes:
    bt_table[i] = 0

for current_node in target_graph.nodes:
    # Реконструировать кратчайшие пути из таблицы предшественников;
    # для каждой записи в таблице выполнить обход в обратном направлении
    # от данной вершины до соответствующей исходной вершины, чтобы получить
    # кратчайший путь, затем подсчитать число вершин в этом пути в
    # основной таблице bt
    pred_table = apsp_results[i][1] # Таблица предшественников уже есть
    sp_list = apsp_results[i][0]
    if current_node in sp_list.keys():
        path = []
        for working_node in sp_list.keys():
            if working_node != current_node:
                # Если достигнут текущий узел, увеличить
                # оценку центральности по посредничеству
                for i in path:
                    bt_table[i] += 1
            else:
                path.append(working_node)
                working_node = pred_table[working_node]

for i in bt_table.keys():
    betweenness_results.append((i, bt_table[i]))

# Упорядочить таблицы. Не забывайте, что лучшей центральностью по
# посредничеству и степени считается большая оценка, а центральностью
# по близости – меньшая оценка
degree_results.sort(lambda a,b:b[1]-a[1])
betweenness_results.sort(lambda a,b:b[1]-a[1])
closeness_results.sort(lambda a,b:a[1]-b[1])

print "%5s|%15s|%10s|%15s|%10s|%15s|%10s" %
    ("Rank", "Between", "Score", "Degree", "Score", "Close", "Score")
for i in range(0, n):
    print "%5d|%15s|%10d|%15s|%10d|%15s|%10d" %
        ( i + 1, str(betweenness_results[i][0]),
          betweenness_results[i][1], str(degree_results[i][0]),
          degree_results[i][1], str(closeness_results[i][0]),
          closeness_results[i][1])

```

В этой книге мы рассмотрим три показателя центральности: *по степени*, *по близости* и *по посредничеству*. Степень – самая простая мера центральности; в неориентированном графе центральность узла по степени определяется его степенью.

Центральности по близости и по посредничеству связаны с кратчайшими путя-

ми. Центральность по близости представляет легкость передачи информации от определенной вершины к любой другой вершине в графе. Центральность вершины по близости определяется как суммарное общее расстояние между этой и всеми другими вершинами в графе. Вершина с *наименьшим* суммарным расстоянием получает *наибольшую* оценку центральности по близости.

Центральность по посредничеству, как и центральность по близости, является функцией кратчайших путей. Центральность по посредничеству отражает вероятность, что вершина войдет в состав кратчайшего пути между любыми двумя конкретными вершинами. Центральность по посредничеству вычисляется путем создания таблицы всех кратчайших путей и последующего подсчета количества путей, в которых присутствует эта вершина.

Центральности являются относительными мерами. С практической точки зрения их лучше использовать в качестве оценок в алгоритмах ранжирования. Например, если некоторая веб-страница получила высокую оценку центральности по посредничеству, это означает, что большинство пользователей при серфинге посещают ее, возможно, потому, что она играет роль шлюза или является важным индексом. Если при исследовании шаблонов серфинга выяснится, что некоторый узел имеет высокую центральность по близости, это может служить признаком, что узел является источником важных новостей или информации.

Компоненты и возможность соединения

Если в неориентированном графе имеется путь между двумя вершинами, они считаются *связанными*. Множество всех узлов, имеющих пути друг к другу, образуют *связанный компонент*. В ориентированных графах понятию связности соответствуют термины *слабо связанные* (если путь можно проложить, только игнорируя направления ребер) и *сильно связанные* (если пути существуют с учетом направлений ребер).

Граф можно разбить на компоненты с помощью поиска в ширину. *Поиск в ширину* (Breadth-First Search, BFS) выполняется выбором узла, изучением всех его соседей, а затем изучением всех соседей этих соседей. В отличие от него, *поиск в глубину* (Depth-First Search, DFS) проверяет одного соседа, затем соседа этого соседа и т. д. Код в примере 13-4 демонстрирует применение поиска в ширину для разбиения графа на компоненты.

Пример 13-4. Разбиение графа на компоненты

```
#!/usr/bin/env python
#
#

import os,sys, basic_graph

def calculate_components(g):
    # Создает таблицу компонентов, выполняя поиск в ширину.
    component_table = {}
    unfinished_nodes = {}
    for i in g.nodes.keys():
        unfinished_nodes[i] = 1
    node_list = [g.nodes.keys()[0]]
```



```

component_index = 1
while node_list != []:
    current_node = node_list[0]
    del node_list[0]
    del unfinished_nodes[current_node]
    for i in g.neighbors(current_node):
        component_table[i] = component_index
        node_list.insert(0, i)
    if node_list == [] and len(unfinished_nodes) > 0:
        node_list = [unfinished_nodes.keys()[0]]
return component_table

```

Коэффициент кластеризации

Другой способ оценить отношения между узлами в графе – вычислить *коэффициент кластеризации*. Коэффициент кластеризации – это вероятность, что любые два соседа конкретной вершины в графе являются соседями по отношению друг к другу. В примере 13-5 показан фрагмент кода, вычисляющий коэффициент кластеризации.

Пример 13-5. Вычисление коэффициента кластеризации

```

def calculate_clustering_coefficients(g):
    # Коэффициент кластеризации вершины – это доля его соседей,
    # которые являются соседями друг для друга
    node_ccs = {}
    for i in g.nodes.keys():
        mutual_neighbor_count = 0
        neighbor_list = g.neighbors(i)
        neighbor_set = {}
        for j in neighbor_list:
            neighbor_set[j] = 1
        for j in neighbor_list:
            # Получить их соседей и определить, сколько из них
            # присутствует во множестве
            new_neighbor_list = g.neighbors[j]
            for k in new_neighbor_list:
                if k != i and neighbor_list.has_key(k):
                    mutual_neighbor_count += 1

        # Найти коэффициент, разделив на d*(d-1)
        cc = float(mutual_neighbor_count)/((float(len(neighbor_list)) *
                                           (len(neighbor_list) - 1 )))
        node_ccs[i] = cc

    total_cc = reduce(lambda a,b:node_ccs[a] + node_ccs[b], node_ccs.keys())
    total_cc = total_cc/len(g.nodes.keys())
    return total_cc

```

Коэффициент кластеризации является полезной мерой «равноправности» («peerishness»). Граф чистой клиент-серверной сети будет иметь коэффициент кластеризации, равный нулю, – клиенты взаимодействуют только с серверами, а серверы – только с клиентами. Нам удалось добиться определенного успеха при использовании коэффициента кластеризации в качестве меры влияния спама

на большие сети. В качестве примера на рис. 13-3 показано влияние закрытия McColo, провайдера пуленепробиваемого хостинга, на структуру трафика SMTP в большой сети. После закрытия McColo коэффициент кластеризации для SMTP вырос примерно на 50 %.

На первый взгляд связь между равноправностью и спамом неочевидна; электронная почта (SMTP), так же как DNS и другие ранние интернет-сервисы, сильно ориентирована на совместное использование ресурсов. Хосты в сети могут выступать и в роли клиентов SMTP, и в роли серверов и способны взаимодействовать друг с другом. Спамеры, однако, действуют как этикие *суперклиенты* – они взаимодействуют с серверами, но никогда не выступают в роли сервера для кого-то. Такое поведение проявляется в виде низкого коэффициента кластеризации. Уберите спамеров, и сеть SMTP начинает больше походить на одноранговую сеть, а коэффициент кластеризации возрастает.

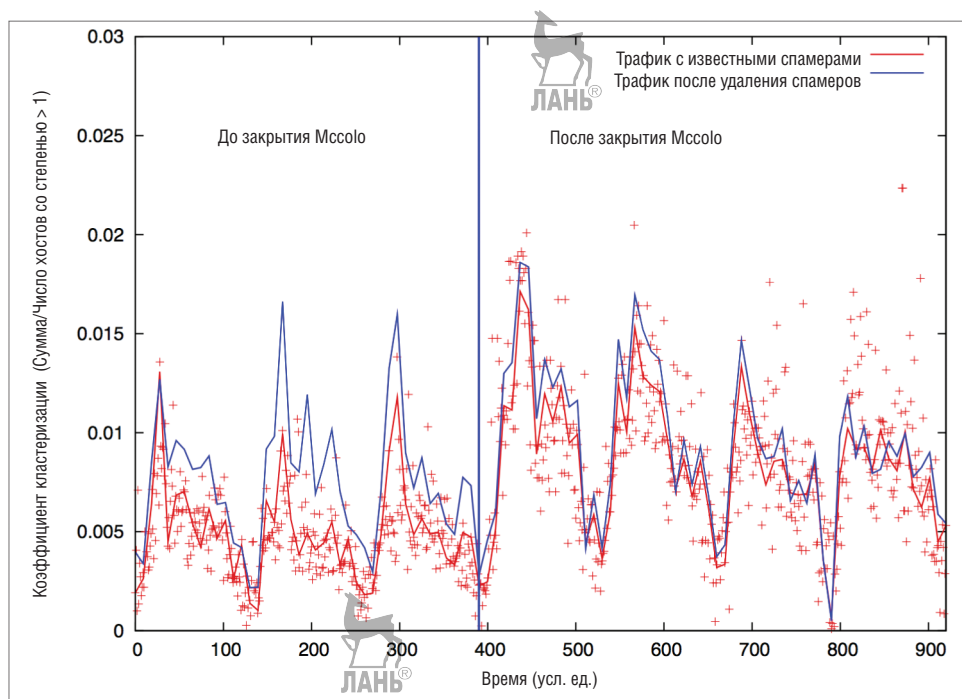


Рис. 13-3. Коэффициент кластеризации и большие почтовые сети

АНАЛИЗ ГРАФОВ

Анализ графа можно использовать для разных целей. Метрики центральности могут пригодиться и при проектировании, и при расследовании, а компоненты и атрибуты графа можно использовать для генерирования тревог.

Создание тревог с использованием анализа компонентов

В главе 11 мы обсудили механизмы, основанные на незнании атакующими структуры сети, как, например, обнаружение фактов слепого сканирования и т. д.

Связанные компоненты позволяют смоделировать невежество атакующих другого типа. Атакующий может знать, где находятся различные серверы и системы в сети, но не знает, как они связаны друг с другом. Организационную структуру можно выявить путем определения связанных компонентов, и некоторые атаки могут использовать знание целей, чтобы определить, как они связаны друг с другом, путем изучения этих компонентов.

Чтобы было понятнее, как это явление можно использовать в качестве сигнала тревоги, рассмотрим пример сети на рис. 13-4. Сеть в этом примере состоит из двух отдельных компонентов (пусть это будут производственный отдел и отдел маркетинга), которые почти не взаимодействуют между собой.

Когда появляется атакующий и пытается установить связь с хостами в сети, он объединяет эти два компонента, создавая один огромный компонент, который отсутствует в обычных условиях.

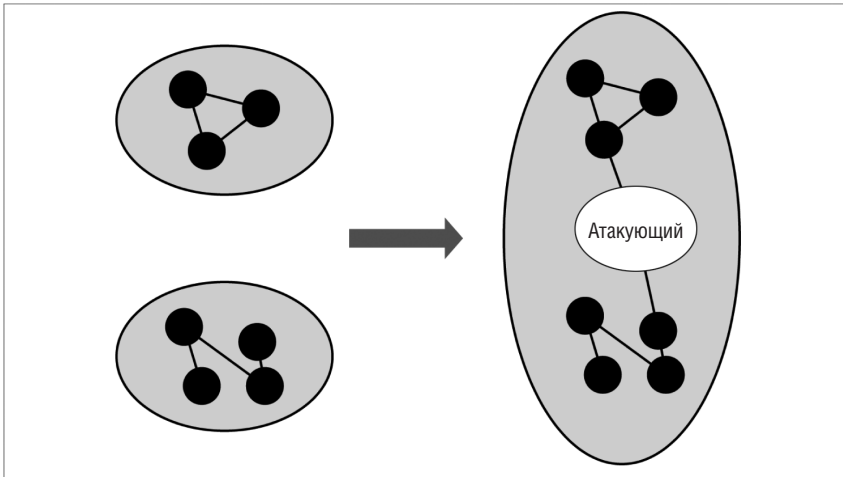


Рис. 13-4. Атакующий искусственно связывает два компонента

Для реализации тревоги этого типа нужно сначала определить сервис, по которому сеть можно разделить на несколько компонентов. Хорошими кандидатами являются такие сервисы, как SSH, требующие от пользователей пройти некоторую процедуру входа; когда в сети имеется разграничение права доступа, некоторые пользователи не будут обращаться к некоторым ее сегментам, что приводит к разбиению сети на отдельные компоненты. SMTP и HTTP обычно не подходят на эту роль, хотя иногда можно использовать и их, если рассматривать лишь серверы, требующие аутентификации пользователя, и ограничить анализ *только* этими серверами (например, с помощью IPSet).

После определения набора серверов нужно определить компоненты для мониторинга. И после этого вычислить их размеры – количество узлов в компоненте как функцию от времени, необходимого для его получения (например, 60 секунд работы *netflow*). Распределение, скорее всего, будет зависеть не только от времени, выделенного для сбора трафика, но и от времени суток. Учет времени суток (как описано в главе 12) может существенно помочь в этом.

Есть два способа идентификации компонентов: по размеру или по списку узлов внутри компонентов. В случае выбора способа идентификации по размеру можно просто определять размер самого большого компонента, второго по величине, и т. д. Этот подход прост и надежен, но малочувствителен к тонким атакам. Обычно самый большой компонент составляет более одной трети от общего числа вершин в графе, поэтому атака должна быть довольно агрессивной, чтобы нарушить устоявшийся размер компонента. Альтернативный подход включает идентификацию узлов в компонентах (например, компонент А – это компонент с узлом 127.0.1.2).

Использование оценок центральности при расследовании

Центральность можно использовать для идентификации наиболее важных узлов в сети и узлов, которые обмениваются данными в гораздо меньших объемах, чем способен определить анализ трафика.

Рассмотрим атаку, когда злоумышленник заражает один или несколько хостов в сети вредоносным программным обеспечением. После этого зараженные узлы начинают взаимодействовать с управляющим сервером, который ранее отсутствовал в трафике. Более подробно этот сценарий изображен на рис. 13-5; до заражения хостов А, В и С каждый из них имел определенную меру центральности по степени. После заражения появился новый узел (Mal), получивший самую высокую оценку центральности.

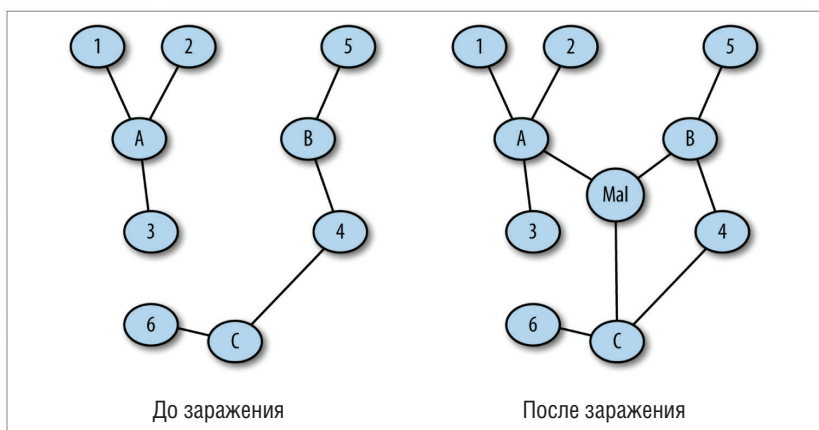


Рис. 13-5. Анализ центральности в расследовании

Этот вид анализа можно выполнить путем разделения данных трафика на два набора: *до события* и *после события*. Например, после обнаружения в сети вредоносного программного обеспечения можно взять трафик, предшествующий моменту заражения, и создать набор *до события*, а из трафика, следующего за моментом заражения, создать набор *после события*. Вычислив центральность узлов, можно выявить управляющий сервер.

Использование поиска в ширину при расследовании

После того как вы определите, что в вашей сети действует злонамеренный хост, следующий шаг – выяснить, с какими хостами он взаимодействует. После этого

можно повторить процесс, чтобы узнать, с кем взаимодействуют эти хосты, чтобы выявить другие потенциальные жертвы.

Этот итеративный процесс реализуется как поиск в ширину. Вы начинаете с одного узла, оцениваете подозрительность поведения всех его соседей, а затем повторяете процесс для их соседей (см. пример 13-6). Этот тип расследования на основе графов может помочь выявить другие зараженные узлы, потенциальные жертвы и другие системы в сети, требующие расследования или анализа.

Пример 13-6. Исследование соседей в сети

```
#!/usr/bin/env python
#
# Это несколько упрощенный пример использования поиска в ширину
# для выявления других хостов, использующих протокол BitTorrent.
# Используются следующие критерии обхода:
#   А взаимодействует с В по портам 6881-6889
#   А и В пересылают друг другу большие файлы (> 1 Мбайт)
#
# Цель примера - показать возможность использования любых критериев
# для конструирования графа.
#
## Порядок использования:
#
# crawler.py seed_ip datafile
#
# seed_ip - ip-адрес пользователя, про которого известно, что
# он использует протокол bittorrent для пересылки файлов

import os, sys, basic_graph

def extract_neighbors(ip_address, datafile):
    # По заданному ip-адресу ip_address определяет смежные узлы,
    # выявляя потоки, в которых присутствует этот адрес.
    # Другой адрес в паре считается соседом.
    a = os.popen("""rwfilter --any-address=%s --sport=1024-65535 --dport=1024-65535 \
        --bytes=1000000- --pass=stdout %s | rwfilter --input=stdin --aport=6881-6889 \
        --pass=stdout | rwuniq --fields=1,2 --no-title""") % (ip_address, datafile), 'r')

    # Обратите внимание на довольно строгие определения портов - все
    # порты в верхней части диапазона. Причина в том, что в зависимости от
    # реализации сетевого стека порты 6881-6889 (применяемые протоколом BitTorrent)
    # могут использоваться как временные. Разбивая клиентские порты в первом
    # вызове фильтра, я гарантирую, что по ошибке не начну учитывать,
    # например, веб-сеансы, использующие порт 6881.
    # Ограничение размера в 1 Мбайт тоже помогает ограничить анализ
    # фактами передачи больших файлов.
    neighbor_set = set()
    for i in a.readlines():
        sip, dip = i.split('|')[0:2].strip()
        # Проверить, является указанный IP-адрес источником или приемником потока,
        # в любом случае я добавляю взаимодействующий с ним адрес
        # во множество соседей (то есть если ip_address == sip, я добавляю dip)
        if sip == ip_address:
            neighbor_set.add(dip)
        else:
            neighbor_set.add(sip)
```

```

a.close()
return neighbor_set

if __name__ == '__main__':
    starting_ip = sys.argv[1]
    datafile = sys.argv[2]
    candidate_set = set([starting_ip])
    while len(candidate_set) > 0:
        target_ip = candidate_set.pop()
        target_set.add(target_ip)
        neighbor_set = extract_neighbors(target_ip, datafile)
        for i in neighbor_set:
            if not i in target_set:
                candidate_set.add(i)
    for i in target_set:
        print i

```

Использование анализа центральности для проектирования

Учитывая ограниченность ресурсов мониторинга и внимания аналитиков, для эффективного мониторинга сети необходимо определить критически важные хосты и выделить ресурсы для их защиты и наблюдения. В любой сети существует огромная разница между хостами, которые нужны людям, по их словам, и хостами, которые они фактически используют. Использование анализа трафика для определения критически важных хостов помогает понять, что важно на бумаге и что важно фактически.

Центральность является одним из нескольких показателей, которые можно использовать для оценки важности. К их числу можно отнести подсчет количества хостов, посетивших сайт (что фактически является централизацией по степени), и анализ объема трафика. Анализ центральности является хорошим дополнением к анализу объема.

Дополнительные материалы для чтения

1. Майкл Коллинз (*Michael Collins*) и Майкл Рейтер (*Michael Reiter*). Hit-list Worm Detection and Bot Identification in Large Networks Using Protocol Graphs: материалы симпозиума по последним достижениям в обнаружении вторжений 2007 года.
2. Томас Кормен (*Thomas Cormen*), Чарльз Лизерсон (*Charles Leiserson*), Рональд Ривест (*Ronald Rivest*) и Клиффорд Штейн (*Clifford Stein*). Introduction to Algorithms, Third Edition. MIT Press, 2009¹.
3. igraph (<http://bit.ly/igraph-pack>, библиотека поддержки графов для R).
4. Лун Ли (*Lun Li*), Дэвид Олдерсон (*David Alderson*), Рейко Танака (*Reiko Tanaka*), Джон С. Дойл (*John C. Doyle*) и Уолтер Виллингер (*Walter Willinger*). Towards a Theory of Scale-Free Graphs: Definition, Properties, and Implications (Extended Version).
5. Neo4j (<http://www.neo4j.org/>).
6. Networkx (<http://networkx.github.io/>, библиотека поддержки графов для Python).

¹ Кормен Томас Х. Алгоритмы. Вводный курс. М.: Вильямс, 2016. ISBN 978-5-8459-1868-0, 978-5-8459-2073-7. – Прим. перев.

Глава 14

.....

Идентификация приложения

Раньше не составляло труда идентифицировать приложения в сетевом трафике; достаточно было узнать номер порта или посмотреть пару заголовков пакетов, чтобы понять, что это такое. Но в последние годы идентификацию стало проводить все сложнее и сложнее, отчасти потому, что пользователи стали стремиться скрыть определенные классы трафика (например, BitTorrent), а также из-за того, что сторонники конфиденциальности настаивают на расширении применения шифрования.

Однако есть еще методы, позволяющие идентифицировать трафик, которые не зависят от полезной нагрузки. Большинство протоколов имеют четко определенную последовательность и определенное предсказуемое поведение, благодаря чему отпадает необходимость исследовать полезную нагрузку. Только на основании информации о хостах, участвующих во взаимодействиях, и размеров пакетов можно получить удивительное количество информации.

Эта глава разбита на два основных раздела. Первый посвящен методам идентификации протоколов, от наиболее очевидных до более сложных, таких как поведенческий анализ. Во втором разделе обсуждается содержание баннеров приложений и некоторые методы поиска поведенческой информации и информации, полезной для анализа.

МЕХАНИЗМЫ ИДЕНТИФИКАЦИИ ПРИЛОЖЕНИЙ

В совершенно безопасной и защищенной вычислительной среде можно просто заглянуть в конфигурационный файл на каждом сервере, и он сообщит вам весь трафик, который генерирует сервер. К сожалению, есть много скрытых способов сгенерировать трафик, которые подрывают эту простую стратегию. В сети могут присутствовать хосты, о которых вы не знаете, запущенные пользователями с невинными или не очень невинными целями. Администраторы или обычные пользователи могут запускать сервисы без добавления в конфигурацию запуска. А законные серверы могут быть захвачены злоумышленниками и использованы для целей, которые вы никогда не подразумевали. Многие методы, описанные в этом разделе, обычно используются злоумышленниками, у которых нет доступа к конфигурационным файлам серверов, но это не значит, что вы не должны использовать их, чтобы узнать, что происходит на самом деле.

Номер порта

Номера портов – это первый способ проверить использованный сервис, и, несмотря на отсутствие технических требований, чтобы конкретный сервис использовал конкретный порт, есть общепринятые соглашения, которые определяют такие требования. Организация IANA ведет общедоступный реестр (<http://bit.ly/port-list>) номеров портов и соответствующих им сервисов. Хотя назначение номеров портов фактически является произвольным, и пользователи активно интересуются возможностью уклонения от обнаружения за счет использования ранее не применявшихся номеров портов (или даже за счет использования номеров портов других сервисов), через хорошо известные порты по-прежнему протекает достаточно официального и невинного трафика, чтобы использовать их номера как механизм первичной идентификации протоколов. Методы, которые мы обсудим далее в этом разделе, часто используют номера портов для подтверждения на стороне пользователя. Например, пользователь, подключающийся к порту 80, фактически утверждает, что он взаимодействует с веб-сервером.

Номера портов назначаются довольно хаотично, потому что достаточно просто выбрать номер и надеяться, что никто другой его не использует. Официальный реестр, поддерживаемый организацией IANA (<http://bit.ly/iana-port>), регламентирует распределение портов между протоколами, разрабатывавшимися как часть процесса RFC. В числе других реестров и списков можно назвать страницу в Википедии¹ (<http://bit.ly/tcp-udp-ports>), справочник SpeedGuide.net (<http://bit.ly/sg-ports>) и сайт SANS Internet Storm Center (<http://bit.ly/sans-isc>), где можно найти таблицы с номерами и краткими описаниями портов.

Таким образом, большое число портов зарезервировано за определенными приложениями и еще большее их число традиционно используется для других приложений, но лишь небольшое число приложений действительно имеет значение. В табл. 14-1 перечислены порты, которым я уделяю наибольшее внимание, с кратким описанием, объясняющим причину.

Таблица 14-1. Порты, заслуживающие особого внимания

Порт	Сервис	Описание
<i>Святая троица</i>		
80/tcp	HTTP	В настоящее время HTTP – это не только базовый протокол почти для всего в интернете, но также часто имитируемый протокол. Пользователи пускают трафик на порт 80, чтобы обойти ограничения брандмауэра.
25/tcp	SMTP	Электронная почта – один из важнейших сервисов после HTTP и также один из самых часто подвергающихся нападению.
53/udp	DNS	Еще один важный и основополагающий протокол; нападения на DNS серьезно вредят сети.
<i>Инфраструктура и управление</i>		
179/tcp	BGP	Основной протокол межсетевой маршрутизации
161-162/udp	SNMP	Простой протокол управления сетью; используется для управления маршрутизаторами и другими сетевыми устройствами

¹ Страница в русскоязычной Википедии: https://ru.wikipedia.org/wiki/Список_портов_TCP_и_UDP. – Прим. перев.



Порт	Сервис	Описание
22/tcp	SSH	«Рабочая лошадка» администрирования
23/tcp	Telnet	Если я вижу трафик Telnet, я разрываю соединение. Это устаревший протокол, и вместо него следует использовать другие протоколы, например SSH
123/udp	NTP	Протокол сетевого времени; используется для координирования сетевых часов
389/tcp	LDAP	Облегченный протокол доступа к каталогам; управляет каталогом сервисов
<i>Передача файлов</i>		
20/tcp	FTP-data	Используется протоколом FTP в паре с портом 21
21/tcp	FTP	Порт управления FTP. Это еще один сервис, который я останавливаю, если вижу его. Вместо него лучше использовать SFTP
69/tcp	TFTP	Протокол тривиальной передачи файлов; в основном используется системными администраторами; желательно, чтобы трафик TFTP никогда не выходил за пограничный маршрутизатор.
137-139/tcp и udp	NETBIOS	NetBios – это инфраструктура для обмена служебными сообщениями, в частности обеспечивает возможность совместного доступа к файлам для Windows и Unix (через Samba). В прошлом подвергался массовым нападениям.
<i>Электронная почта</i>		
143/tcp	IMAP	Протокол доступа к электронной почте интернета; один из двух стандартных протоколов электронной почты
110/tcp	POP3	Протокол почтового отделения; еще один стандартный протокол электронной почты
<i>Базы данных</i>		
1521/tcp	Oracle	Основной порт сервера Oracle
1433/tcp и udp	SQL Server	Порт Microsoft SQL Server
3306/tcp	MySQL Server	Порт по умолчанию для MySQL
5432/tcp	Postgresql Server	Порт по умолчанию для Postgresql
<i>Совместный доступ к файлам</i>		
6881-6889/tcp	BitTorrent	Порты BitTorrent по умолчанию для подключения клиентов
6346-6348/tcp и udp	Gnutella	Порты по умолчанию для Bearshare и Limewire в файлообменной сети Gnutella
4662/tcp и udp	eDonkey	Порт по умолчанию для клиентов файлообменной сети eDonkey

В системах Unix и Windows порты назначаются в файле `/etc/services` (в Unix) и в `\WINDOWS\SYSTEM32\DRIVERS\ETC\SERVICES` (в Windows). Как можно видеть в примере 14-1, файл содержит простую базу данных, в которой указываются названия сервисов и соответствующие им номера портов.

Пример 14-1. Содержимое файла `/etc/services`

```
$ # Вывод содержимого /etc/services без информации в заголовке
$ cat /etc/services | egrep -v '^#' | head -10
rtmp      1/udp    # Протокол поддержки таблиц маршрутизации
tcpmux    1/udp    # Протокол мультимплексирования TCP-порта между сервисами
```

tcprmx	1/tcp	# Протокол мультимплексирования TCP-порта между сервисами
nbp	2/ddp	# Протокол привязки имени
compressnet	2/udp	# Утилита управления
compressnet	2/tcp	# Утилита управления
compressnet	3/udp	# Процесс сжатия
compressnet	3/tcp	# Процесс сжатия
echo	4/ddp	# Эхо-протокол AppleTalk
rje	5/udp	# Дистанционный ввод заданий

Имена из файла *services* используются в *getportbyname* и других функциях поиска портов для идентификации протоколов. Это, конечно, не означает, что пользователи действительно вызывают эти сервисы, просто *services* указывает, какие порты должны использоваться сервисами. Например, чтобы получить список всех сервисов, выполняющихся на хосте, я выполняю команду *netstat -a*, как показано в примере 14-2.

Пример 14-2. Утилита *netstat* и файл */etc/services/*

На данном хосте действует веб-сервер *django*, использующий порт 8000

\$ *netstat -a | grep LISTEN*

```
tcp4      0      0 localhost.irdmi    *.*      LISTEN
tcp46     0      0 *.8508             *.*      LISTEN
tcp46     0      0 *.8507             *.*      LISTEN
```

\$ *cat /etc/services | grep irdmi*

```
irdmi2    7999/udp    # iRDMI2
irdmi2    7999/tcp    # iRDMI2
irdmi     8000/udp    # iRDMI
irdmi     8000/tcp    # iRDMI
```

Утилита *netstat* обращается к файлу */etc/services*, чтобы определить имя сервиса по номеру порта. Однако нет никакой гарантии, что сервис, о котором сообщает *netstat*, действительно является сервисом с этим именем – в моем примере я использовал веб-сервер *Django*.

В подобных ситуациях вполне уместно удариться в параноидальный бред, свойственный аналитикам сетевого трафика. Утилита *netstat* – отличный инструмент для выявления открытых портов на хосте, но если вы хотите больше уверенности, просканируйте машину по вертикали и сравните результаты.



Назначение портов

В любых двусторонних взаимодействиях по протоколам TCP и UDP используется два порта: *порт сервера* используется клиентом для отправки трафика на сервер и *порт клиента* используется сервером для ответа. Клиентские порты недолговечны и выбираются из пула *временных портов*; размер и распределение пула зависят от реализации стека TCP и пользовательской конфигурации.

Существует несколько соглашений о назначении портов. Наиболее важным является разделение номеров портов на два диапазона: ниже 1024 и выше. Почти во всех операционных системах, чтобы открыть порт с номером ниже 1024, требуются привилегии суперпользователя root или администратора. Это означает, что только администратор сможет запустить сервис, такой как веб-сервер или сервер электронной почты. Но это свойство делает сервисы на этих портах особенно привлекательными для злоумышленников, потому что взлом этих процессов дает привилегии root.

Обычно порты с номерами ниже 1024 используются только для создания серверных сокетов. Это не означает, что их нельзя использовать для клиентов, но такое решение будет противоречить стандартной практике и выглядеть слегка сумасшедшим, потому что нет никакого смысла использовать клиентский порт с привилегиями root. Временный порт может иметь любой номер выше 1024, но и в их назначении действует ряд соглашений.

Организация IANA выделила для временных портов (<http://bit.ly/iana-port>) стандартный диапазон (от 49152 до 65535). Однако этот диапазон все еще находится в процессе согласования, и разные операционные системы могут использовать разные диапазоны. В табл. 14-2 перечислены общие соглашения о назначении портов.

Таблица 14-2. Правила назначения портов в разных операционных системах

Операционная система	Диапазон по умолчанию	Управляемый
Windows, вплоть до версии XP	1025–5000	Частично, вплоть до MaxUserPort в <i>Tcpip\Parameters</i>
Windows, от версии Vista и выше	49 152–65 535	Да, через netsh
Mac OS X	49 152–65 535®	Да, через группу параметров <i>net.inet.ip.portrange</i> в <i>sysctl</i>
Linux	32 768–65 535	Да, через параметры <i>/proc/sys/net/ipv4/ip_local_port_range</i>
FreeBSD	49 152–65 535	Да, через группу параметров <i>net.inet.ip.portrange</i> в <i>sysctl</i>

Идентификация приложений по баннерам

Извлечение идентификационных маркеров (banner grabbing) – баннеров – и сопутствующий прием получения цифрового отпечатка операционной системы относятся к методам сканирования, которые применяются для получения информации о сервере и операционной системе. Они основаны на общепринятом соглашении, что при первом обращении к приложению оно идентифицирует себя. Большинство серверных приложений в ответ на попытку открыть соединение посылают

свой протокол, текущую версию и другую информацию о конфигурации. Если они не посылают эту информацию автоматически, ее часто можно запросить явно.

Извлечение баннеров можно выполнить вручную с помощью любого инструмента, связывающего «клавиатуру с сокетом», например *netcat* (дополнительную информацию можно найти в главе 9). В примере 14-3 показан активный способ извлечения баннеров с использованием *netcat*. Обратите внимание, что я могу получать информацию о серверах, фактически не используя их рабочий протокол.

Пример 14-3. Примеры активного извлечения баннеров с помощью *netcat*

```
# Открыть соединение с сервером SSH
# Обратите внимание, что я получаю информацию, фактически
# не взаимодействуя с сервером.
$ netcat 192.168.2.1 22
SSH-2.0-OpenSSH_6.1
^C

# Открыть соединение с сервером IMAP.
# И снова я не взаимодействую с почтовым сервером.
$ netcat 192.168.2.1 143
* OK [CAPABILITY IMAP4rev1 LITERAL+ SASL-IR LOGIN-REFERRALS
  ID ENABLE STARTTLS AUTH=PLAIN AUTH=LOGIN] Dovecot ready.
```

Альтернативой активному извлечению баннеров является пассивное извлечение, которое можно организовать с помощью *tcpdump*. Поскольку баннеры на самом деле являются простым текстом, появляющимся в начале сеанса, перехват содержимого первых пяти или шести пакетов позволит получить ту же информацию.

Сценарий *bannergrab.py* (в примере 14-4) извлекает баннеры из файлов сеансов, полученных с помощью Scapy (см. главу 9). Он никак не анализирует содержимое баннеров, а только извлекает и выводит их для просмотра. Как можно судить по результатам, представленным в примере 14-5, эта информация может оказаться очень полезной.

Пример 14-4. Сценарий извлечения баннеров с помощью *scapy*

```
#!/usr/bin/env python
#
#
# bannergrab.py
# Это приложение использует пакет Scapy, чтобы загрузить и вывести
# баннеры сервера и клиента. Для этого оно читает файлы с
# содержимым сеансов, извлекает текст ASCII и выводит его на экран.
#

from scapy.all import *
import sys
sessions = {}

packet_data = rdpcap(sys.argv[1])
for i in packet_data:
    if not sessions.has_key(i[IP].src):
        sessions[i[IP].src] = ''
    try:
        sessions[i[IP].src] += i[TCP].payload.load
```

```

except:
    pass

for j in sessions.keys():
    print j, sessions[j][0:200]

$ bannergrab.py ssh.dmp
WARNING: No route found for IPv6 destination :: (no default route?)
192.168.1.12
216.92.179.155 SSH-2.0-OpenSSH_6.1

```



Пример 14-5 иллюстрирует информацию, собранную для сайта www.cnn.com:

Пример 14-5. Получение по запросу от cnn.com

```

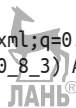
57.166.224.246 HTTP/1.1 200 OK
Server: nginx
Date: Sun, 14 Apr 2013 04:34:36 GMT
Content-Type: application/javascript
Transfer-Encoding: chunked
Connection: keep-alive
Vary: Accept-Encoding
Last-Modified: Sun
157.166.255.216
157.166.241.11 HTTP/1.1 200 OK
Server: nginx
Date: Sun, 14 Apr 2013 04:34:27 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: keep-alive
Set-Cookie: CG=US:DC:Washington; path=/
Last-Modified

66.235.155.19 HTTP/1.1 302 Found
Date: Sun, 14 Apr 2013 04:34:35 GMT
Server: Omniture DC/2.0.0
Access-Control-Allow-Origin: *
Set-Cookie: s_vi=[CS]v1|28B31B23851D063C-60000139000324E4[CE];
Expires=Tue, 14 Apr 2
23.6.20.211 HTTP/1.1 200 OK
x-amz-id-2: 287K0oW3vWNpotJGpn0RaXExCzKkFJQ/hkpAXjWUQTb6hSBzDQioFUoWYZMRCq7V
x-amz-request-id: 8B6B2E3CDBC2E300
Content-Encoding: gzip
ETag: "e5f0fa3fbe0175c47fea0164922230d4"
Acc

192.168.1.12 GET / HTTP/1.1
Host: www.cnn.com
Connection: keep-alive
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_8_3) AppleWebKit
23.15.9.160 HTTP/1.1 200 OK
Server: Apache
Last-Modified: Wed, 10 Apr 2013 13:44:28 GMT
ETag: "233bf1-3e803-4da01de67a700"
Accept-Ranges: bytes
Content-Type: text/css
Vary: Accept-Encoding
Content-Encoding

63.85.36.42 HTTP/1.1 200 OK

```



```

Content-Length: 43
Content-Type: image/gif
Date: Sun, 14 Apr 2013 04:34:36 GMT
Connection: keep-alive
Pragma: no-cache
Mechanisms for Application Identification | 285
Expires: Mon, 01 Jan 1990 00:00:00 GMT
Cache-Control: priv

138.108.6.20 HTTP/1.1 200 OK
Server: nginx
Date: Sun, 14 Apr 2013 04:34:35 GMT
Content-Type: image/gif
Transfer-Encoding: chunked
Connection: keep-alive
Keep-Alive: timeout=20

```



В предыдущем примере клиент находится в середине дампа (адрес 192.168.1.12). Обратите внимание на большое количество веб-серверов; это типичное явление для современных веб-сайтов, когда в конструировании одной страницы могут участвовать десятки серверов. Также посмотрите, какая информация предоставляется: сервер посылает характеристики содержимого, имя сервера и некоторые конфигурационные данные. Клиент посылает информацию о допустимых форматах данных, а также заголовок User-Agent, к обсуждению которого мы еще вернемся.

Извлечение баннеров реализуется очень просто. Сложность заключается лишь в *интерпретации содержимого баннеров*. Разные приложения могут иметь в корне отличные баннеры с совершенно разной структурой.

Идентификация приложений по поведению

В отсутствие полезной информации в пакетах часто трудно идентифицировать *тип* приложения, однако даже в таких случаях доступно огромное количество информации о *поведении* приложения. Целью поведенческого анализа является поиск подсказок о характере приложения путем изучения таких свойств, как размеры пакетов и сбои соединения.

Размеры пакетов в любом протоколе IP ограничены *максимальным размером блока передачи* (Maximum Transmission Unit, MTU) и максимальным размером кадра, определяемым протоколом уровня 2. Когда IP пытается отправить пакет с размером больше MTU, тот делится на несколько пакетов с размером, равным MTU. То есть в данных *tcpdump* и NetFlow вы увидите максимальный размер пакета, равный минимальному значению MTU на маршруте, пройденном этим пакетом. Поскольку в интернете преобладает Ethernet, размеры пакетов ограничиваются величиной в 1500 байт.

Это ограничение можно использовать, чтобы разделить сетевой трафик на четыре основные категории:

Прощупывание

Как рассказывалось в главе 11, этот трафик состоит из неудачных попыток установить соединение с целью.

Управляющий трафик

Небольшие пакеты фиксированного размера, отправляемые клиентами и серверами в начале сеанса.



Обмен сообщениями

Пакеты с различными размерами меньше MTU, передаваемые туда/обратно между клиентами и серверами. Обмен сообщениями характерен для протоколов обмена мгновенными сообщениями, таких как ICQ и AIM, а также для протоколов, обменивающихся командами, таких как SMTP и BitTorrent.

Передача файла

Асимметричный трафик, когда одна сторона отправляет пакеты с размерами, почти совпадающими с размером MTU, а другая сторона посылает в ответ подтверждения ACK. Характерен для SMTP, HTTP и FTP.

Пакеты управляющего трафика представляют, пожалуй, наибольший интерес с точки зрения идентификации сервиса, потому что часто имеют размеры, характерные для сервиса. Управляющие сообщения нередко реализуют шаблон некоторой формы с конкретными областями, заполняемыми пробелами. Поэтому даже при попытке замаскировать трафик под другой сервис его принадлежность к конкретному типу сервиса иногда можно установить по размеру.

Гистограммы, о которых рассказывалось в разделе «Гистограммы» главы 10, можно использовать для сравнения протоколов по размерам их управляющих сообщений. Для примера рассмотрим рис. 14-1. Это гистограммы распределения коротких потоков (менее 1000 байт) от клиентов к BitTorrent и веб-серверам.

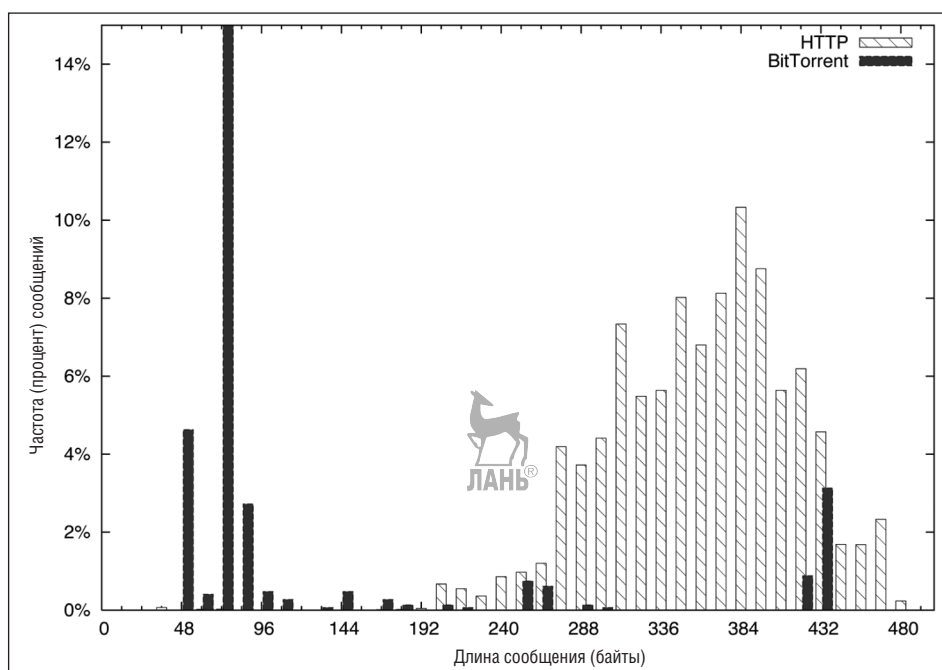


Рис. 14-1. Гистограммы для сравнения коротких потоков BitTorrent и HTTP

Задача веб-клиента главным образом состоит в том, чтобы выполнить запрос HTTP GET и затем получить файл. Как видно на рис. 14-1, размеры пакетов с запросами GET имеют почти нормальное распределение в диапазоне между 200 и 400 байт. А размеры пакетов протокола BitTorrent, напротив, имеют огромный

пик между 48 и 96 байт, что обусловлено большим количеством 68-байтных сообщений подтверждения.

Гистограммы можно проверить визуально, как на рис. 14-1, или численно, рассчитав расстояние L1 (манхэттенское расстояние, <http://bit.ly/l1-norm>). Для гистограммы расстояние L1 вычисляется как сумма разностей между каждым столбиком. После нормализации получается значение между 0 и 2, где 0 указывает, что две гистограммы идентичны, а 2 – что две гистограммы являются полными противоположностями. В примере 14-6 показано, как вычислить расстояние L1 на Python.

Пример 14-6. Вычисление расстояния L1 на Python

```
#!/usr/bin/env python
#
#
# calc_l1.py
#
# Принимает два файла с размерами и определением гистограммы
# (размер столбика, максимальный размер столбика) и вычисляет расстояние l1
# между гистограммами
#
# порядок использования:
# calc_l1 size min max file_a file_b
#
# size: размер столбика гистограммы
# min: минимальный размер столбика
# max: максимальный размер столбика
#
#

import sys

bin_size = int(sys.argv[1])
bin_min = int(sys.argv[2])
bin_max = int(sys.argv[3])
file_1 = sys.argv[4]
file_2 = sys.argv[5]

bin_count = 1 + ((bin_max - bin_min)/bin_size)
histograms = [[],[]]
totals = [0,0]

for i in range(0, bin_count):
    for j in range(0,2):
        histograms[j].append(0)

# Сгенерировать гистограммы
for h_index, file_name in ((0, file_1), (1,file_2)):
    fh = open(file_name, 'r')
    results = map(lambda x:int(x), fh.readlines())
    fh.close()
    for i in results:
        if i <= bin_max:
            index = (i - bin_min)/bin_size
            histograms[h_index][index] += 1
            totals[h_index] += 1
```



```
# Сравнить и вычислить расстояние l1
l1_d = 0.0
for i in range(0, bin_count):
    h0_pct = float(histograms[0][i])/float(totals[0])
    h1_pct = float(histograms[1][i])/float(totals[1])
    l1_d += abs(h0_pct - h1_pct)

print l1_d
```

Принадлежность трафика к категориям обмен сообщениями и передача файлов можно проверить определением размеров отдельных пакетов или, в случае передачи файлов, сравнением средних размеров пакетов. Если пакеты, посылаемые одной из сторон, имеют размер, близкий к MTU, высока вероятность, что это передача файла, а если трафик имеет явно выраженный асимметричный характер и размеры пакетов составляют больше 40 байт, скорее всего, это некоторая форма обмена сообщениями. Для иллюстрации рассмотрим графики на рис. 14-2 и рис. 14-3. Они показывают размеры пакетов для сеанса передачи файлов (HTTP) и чата (AIM) соответственно.

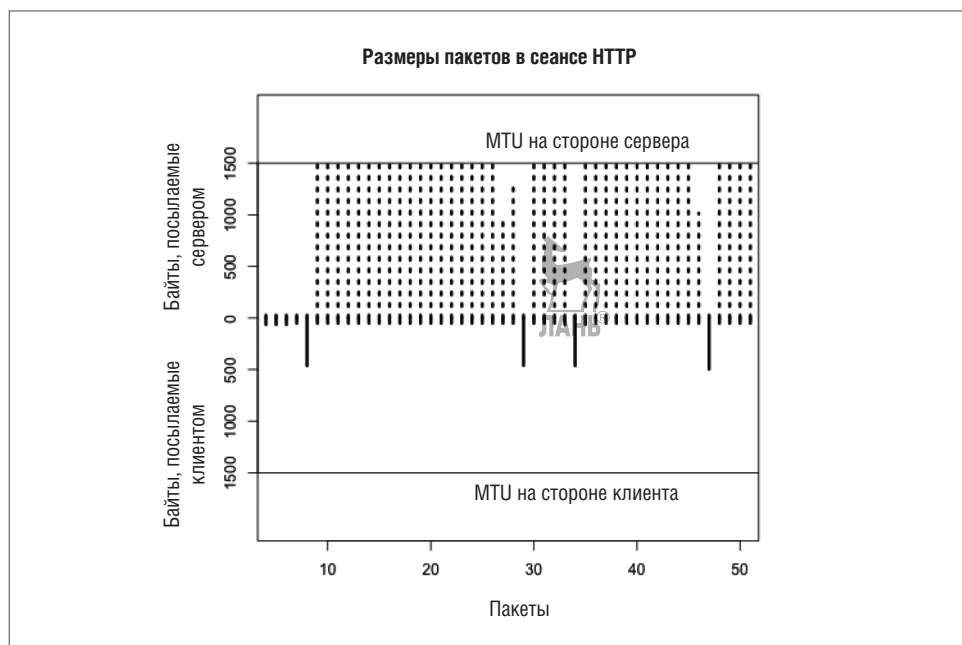


Рис. 14-2. Размеры пакетов в сеансе HTTP

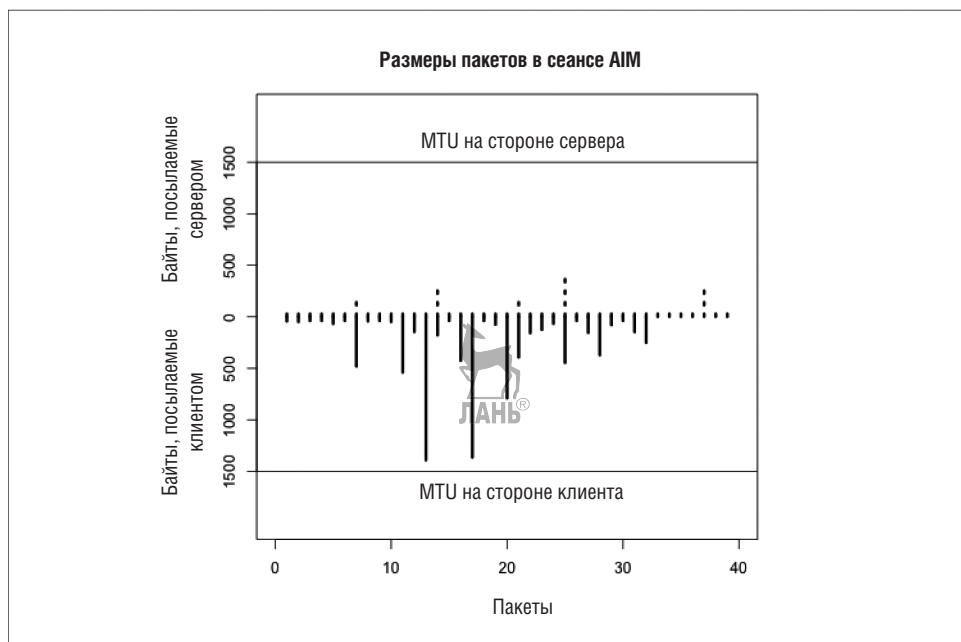


Рис. 14-3. Размеры пакетов в сеансе AIM

Идентификация приложений по обращениям к сайтам

Современные приложения редко обходятся без сетевых взаимодействий. Обновление программного обеспечения и баз данных, регистрация на сервере, доставка рекламы и наблюдение за действиями пользователей – все это примеры операций в сети, которые приложение может выполнять без ведома пользователя. В то же время пользователи могут посещать форумы поддержки, общаться на досках объявлений или обращаться к информации, необходимой для запуска приложения.

В качестве примера такого поведения рассмотрим два приложения: антивирус и BitTorrent. Любое антивирусное приложение должно регулярно связываться со своими домашними серверами для обновления базы данных о вирусах. Это действие настолько предсказуемо, что вредоносные программы нередко явно отключают адреса серверов обновлений на локальном хосте. Любой хост, на котором работает антивирус, должен регулярно связываться с этими адресами, и любой, кто это делает, вероятно, использует антивирус.

Теперь рассмотрим BitTorrent. В последние годы разработчики BitTorrent приложили много усилий по децентрализации протокола. В конце 2000-х можно было идентифицировать трекеры, а затем и пользователей, выяснив, кто обращался к трекерам. В настоящее время идентифицировать трекеры стало сложнее, но пользователям BitTorrent все еще необходимо как-то добраться до нужных им файлов, а соответствующие магнет-ссылки (magnet links) сосредоточены на таких сайтах, как Pirate Bay, KickAssTorrents, и других специализированных торрент-сайтах. Найдите пользователей, посещающих Pirate Bay, затем определите, кто из них загружает огромные файлы через необычные порты, и вы наверняка найдете

пользователя BitTorrent. Определив сервер или хост, на котором запущен определенный сервис, посмотрите, кто еще взаимодействует с ним.

БАННЕРЫ ПРИЛОЖЕНИЙ: ИДЕНТИФИКАЦИЯ И КЛАССИФИКАЦИЯ

Баннеры приложений могут многое сообщить о приложениях, серверах, операционных системах и их версиях. К сожалению, формат этих баннеров радикально отличается для разных сервисов. Однако баннеры большинства приложений, кроме веб-браузеров, относительно просты. С другой стороны, подавляющее большинство баннеров, которые вы будете видеть в трафике, принадлежат веб-браузерам.

Баннеры, не принадлежащие браузерам

В этом разделе мы рассмотрим баннеры серверов, не имеющих отношения к вебу. Они могут содержать информацию об операционной системе и протоколе или быть скрытыми, чтобы сканирующие не могли завладеть интеллектуальными данными.

Баннер SMTP определяется в RFC 5321. При попытке подключиться к серверу SMTP тот должен вернуть код состояния 220 (приветствие) и сообщить некоторую информацию. Учитывая, что серверы SMTP являются одной из излюбленных целей для занимающихся сканированием, нередко можно увидеть баннеры SMTP, урезанные до предела.

В Microsoft, например, для MS Exchange определили следующий баннер:

```
220 <Имя-сервера> Microsoft ESMTP MAIL service ready at
    <ДеньНедели-Дата-Время-в-24-часовом-формате> <ЧасовойПояс>
```

с некоторыми возможными дополнениями. Например, вот баннер для Exchange:

```
220 mailserver.bogodomain.com Microsoft ESMTP MAIL service ready at
    Sat, 16 Feb 2013 08:34:14 +0100
```

Баннер SSH определяется в RFC 4253. При попытке подключиться к серверу SSH тот должен вернуть краткое сообщение со строкой идентификации. Согласно описанию протокола, строка идентификации должна иметь вид:

```
SSH-ВерсияПротокола-ВерсияПО SP комментарий CR LF
```

где SP – это пробел, CR – символ возврата каретки, а LF – символ перевода строки. Все современные реализации SSH должны использовать версию протокола 2.0, но серверы, поддерживающие предыдущие версии SSH, должны идентифицировать свою версию как 1.99. Комментарий может отсутствовать.

Вот пример баннера сервера SSH, предшествовавшего версии 2.0:

```
SSH-1.99-OpenSSH_3.5p1
```

Для остальных серверов номера версий должны быть равны 2.0 или выше:

```
SSH-2.0-OpenSSH_4.3
```

Как показывают эти два примера, первый шаг к идентификации баннера обычно состоит в поиске соответствующей технической документации. Это может быть документ RFC, разработанный в IETF, с описанием протокола, такого как IMAP, POP3, SSH или SMTP. Для идентификации протоколов, в стандартизации ко-

торых организация IETF не принимала участия, может потребоваться выяснить, кто занимался их разработкой, и отыскать сайты поддержки. Например, описание протокола BitTorrent в настоящее время можно найти на вики-странице (<http://bit.ly/bt-spec>), на сайте *theory.org*.

Баннеры веб-клиентов: заголовок User-Agent

Веб-клиенты отправляют сложную конфигурационную строку, определяющую их возможности и предпочтения: платформу, на которой работает браузер, операционную систему и различные сведения о конфигурации. Эта строка, в заголовке User-Agent, определена в RFC 2616 и может быть феноменально сложной (а также информативной).

Некоторые примеры строк из заголовка User-Agent для разных браузеров показаны в примере 14-7.

Пример 14-7. Примеры строк из заголовка User-Agent для разных браузеров

Firefox:

```
Mozilla/5.0 (X11; U; Linux x86_64; en-US; rv:1.8.1.12) Gecko/20080214
  Firefox/2.0.0.12
Mozilla/5.0 (Windows; U; Windows NT 5.1; cs; rv:1.9.0.8) Gecko/2009032609
  Firefox/3.0.8
Mozilla/5.0 (X11; U; Linux i686; en-US; rv:1.8) Gecko/20051111 Firefox/1.5
```

Internet Explorer:

```
Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; WOW64; Trident/5.0; SLCC2;
  Media Center PC 6.0; InfoPath.3; MS-RTC LM 8; Zune 4.7)
Mozilla/5.0 (compatible; MSIE 10.0; Windows NT 6.1; Trident/6.0)
Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.1; Trident/5.0; Xbox)
```

Safari:

```
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_6_8) AppleWebKit/534.57.1
  (KHTML, like Gecko) Version/5.1.7 Safari/534.57.1
Mozilla/5.0 (iPad; CPU OS 6_0 like Mac OS X) AppleWebKit/536.26
  (KHTML, like Gecko) Version/6.0 Mobile/10A403 Safari/8536.25
```

Opera:

```
Opera/9.80 (Windows NT 6.0) Presto/2.12.388 Version/12.11
Opera/9.80 (Macintosh; Intel Mac OS X 10.8.2) Presto/2.12.388 Version/12.11
Opera/9.80 (X11; Linux i686; U; ru) Presto/2.8.131 Version/11.11
Mozilla/5.0 (Windows NT 6.1; rv:2.0) Gecko/20100101 Firefox/4.0 Opera 12.11
```

Chrome:

```
Mozilla/5.0 (Windows NT 6.2; WOW64) AppleWebKit/535.24
  (KHTML, like Gecko) Chrome/19.0.1055.1 Safari/535.24
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_7_3) AppleWebKit/535.19
  (KHTML, like Gecko) Chrome/18.0.1025.151 Safari/535.19
Mozilla/5.0 (Linux; Android 4.0.4; Galaxy Nexus Build/IMM76B)
  AppleWebKit/535.19 (KHTML, like Gecko) Chrome/18.0.1025.133
  Mobile Safari/535.19
Mozilla/5.0 (iPhone; U; CPU iPhone OS 5_1_1 like Mac OS X; en)
  AppleWebKit/534.46.0 (KHTML, like Gecko) CriOS/19.0.1084.60
  Mobile/9B206 Safari/7534.48.3
```

Googlebot:

Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)

Bingbot:

Mozilla/5.0 (compatible; bingbot/2.0; +http://www.bing.com/bingbot.htm)

Baiduspider:

Mozilla/5.0 (compatible; Baiduspider/2.0; +http://www.baidu.com/search/spider.html)

Строки в примере 14-7 следуют базовой структуре, которая определена в спецификации RFC 2616, и содержат разные артефакты, оставшиеся от «войны браузеров» в прошлом. Вот краткое описание этой структуры.

1. Начальный тег, обычно Mozilla/4.0 или выше. Использование тега Mozilla как строки по умолчанию – это пережиток «войны браузеров». Достаточно сказать, что почти все браузеры используют маскировку под Mozilla.
2. Набор значений в скобках подсказывает *истинную* природу браузера. Эти значения зависят от типа браузера и его конфигурации, но обычно содержат фактическое имя браузера, ОС и ряд необязательных параметров.
3. За скобками (обычно) следует название программного веб-движка, используемого браузером для отображения разметки HTML. Один и тот же движок может использоваться несколькими браузерами. К числу наиболее распространенных относятся такие движки, как Gecko (Firefox, Mozilla и SeaMonkey), WebKit (Safari и Chrome), Presto (Opera) и Trident (IE).

Как показывает пример 14-7, фактический состав строки во многом зависит от браузера, ОС и прихотей разработчика.

ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ ДЛЯ ЧТЕНИЯ

1. Майкл Коллинз (*Michael Collins*) и Майкл Рейтер (*Michael Reiter*). Finding Peer-to-Peer File Sharing Using Coarse Network Behaviors: материалы конференции ESORICS, прошедшей в 2007 году.
2. Хаджим Иноуэ (*Hajime Inoue*), Дана Янсенс (*Dana Jansens*), Абдулрахман Хиджази (*Abdulrahman Hijazi*) и Анил Сомаяджи (*Anil Somayaji*). NetADHICT: A Tool for Understanding Network Traffic: конференция по администрированию больших систем (LISA '07). Ноябрь, 2007.
3. Домашняя страница проекта NetADHICT (<http://bit.ly/netADHICT>).
4. Описание инструмента p0f (), созданного Майклом Залевски (*Michael Zalewski*).
5. UserAgentString.com (<http://bit.ly/browserlist>).



Глава 15

Сетевое картирование

В этой главе мы обсудим механизмы управления частотой ложных срабатываний систем обнаружения за счет упрощения. Рассмотрим следующий сценарий: я создал сигнатуру с целью идентификации «эксплоита недели» для IIS, а на следующий день она буквально «сошла с ума». Ух ты! Похоже, что кто-то начал использовать этотexploit! Я проверяю журналы и обнаруживаю, что никто меня не атакует, потому что *на самом деле в моей сети нет IIS*. Я не только потратил время аналитика на исследование причин тревоги, но и впустую потратил *свое* время, когда занимался разработкой оповещения, предупреждающего о событиях, которых в действительности не может быть.

Процесс учета имеющихся ресурсов, или инвентаризации, является необходимым условием для быстрой ориентации в ситуации. Учет позволяет перейти от простого реагирования на сигнатуры к постоянному аудиту и защите. Благодаря ему вы получаете базовые показатели и эффективную стратегию обнаружения аномалий, возможность выявлять критические активы и использовать контекстную информацию для ускорения процесса фильтрации предупреждений.

Создание начальной описи и карты сети

Сетевое картирование – это итеративный процесс, сочетающий технический анализ и общение с администраторами. В основе этого процесса лежит теория, суть которой заключается в том, что любая опись по определению является неточной до определенной степени, но достаточно точной, чтобы начать процесс контрольных измерений и анализа. Создание описи начинается с определения круга лиц, ответственных за управление сетью.

Процесс картирования, описанный в этой книге, состоит из четырех отдельных этапов, к числу которых относятся итеративный анализ трафика и получение ответов на вопросы от сетевых администраторов и персонала. Ответы помогают анализировать трафик, а анализ приводит к появлению новых вопросов. На рис. 15-1 показано, как протекает этот процесс: на этапе I выявляется пространство подконтрольных IP-адресов, и на каждом последующем этапе это пространство разбивается на разные категории.

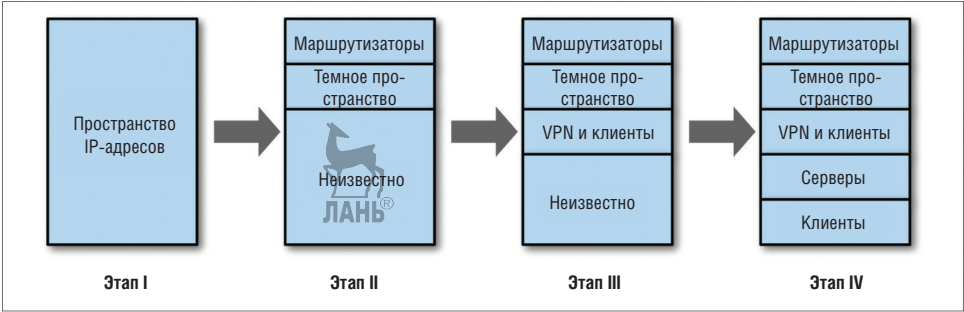


Рис. 15-1. Процесс картирования

Создание описи: данные, охват и файлы

В идеальном мире карта сети должна позволять на основе адресов и портов определять, какой трафик может наблюдаться на любом хосте в сети. Создать такую идеальную карту в корпоративной сети почти невозможно, потому что к тому моменту, когда вы закончите инвентаризацию, в сети *что-то* обязательно изменится. Карты являются динамическими и, следовательно, должны обновляться регулярно. Процесс обновления позволяет вам выполнять аудит сети практически непрерывно.

Опись системы безопасности должна включать все адресуемые ресурсы в сети (то есть все, до чего атакующий сможет добраться, имея доступ к сети, даже если ресурсы доступны только изнутри сети). Она должна включать описание сервисов, действующих на ресурсах, и того как осуществляется мониторинг. Пример подобной описи показан в табл. 15-1.

Таблица 15-1. Пример описи

Адрес	Имя	Протокол	Порт	Роль	Замечен в последний раз	Источник данных	Комментарий
128.2.1.4	www.server.com	tcp	80	HTTP-сервер	14.05.2013	Поток 1, журнал	Основной веб-сервер
128.2.1.4	www.server.com	tcp	22	SSH-сервер	14.05.2013	Поток 1, журнал	Только для администраторов
128.2.1.5-128.2.1.15	–	–	–	Клиент	14.05.2013	Поток 2	Рабочие станции
128.2.1.16-128.2.1.31	–	–	–	Свободные адреса	14.05.2013	Поток	Темное пространство

Для каждой уникальной наблюдаемой комбинации адрес/порт/протокол в табл. 15-1 имеется отдельная запись, в которой также указывается роль, дата, когда эта комбинация последний раз наблюдалась и где она наблюдалась. Это *минимальный* набор полей, которые должны присутствовать в описи. Также хорошо было бы включить в опись следующие пункты.

- Поле **Роль** в идеале должно быть перечислением, а не просто текстом. Это сделает поиск по роли намного менее болезненным. Вот какой набор категорий можно отразить в этом поле:
 - ◆ *Сервер* и его *тип*, например: «Сервер HTTP», «Сервер SSH» и т. д.;
 - ◆ *Рабочая станция* – для обозначения выделенного клиента;
 - ◆ *NAT* – для обозначения шлюза, осуществляющего преобразование сетевых адресов;
 - ◆ *Сервис прокси* – для обозначения любых прокси;
 - ◆ *Брандмауэр* – для обозначения брандмауэров;
 - ◆ *Датчик* – для обозначения точек сбора информации;
 - ◆ *Маршрутизатор* – для обозначения оборудования маршрутизации;
 - ◆ *VPN* – для обозначения концентраторов и другого оборудования VPN;
 - ◆ *DHCP* – для обозначения любого пространства с динамическим распределением адресов;
 - ◆ *Темный* – для обозначения любого адреса, принадлежащего сети, но не присвоенного никакому хосту.
- Идентификация VPN, NAT, DHCP и разнообразных прокси имеет большое значение, в чем вы убедитесь чуть ниже, потому что они искажают понятие «адрес» и усложняют анализ.
- Меры центральности и метрики, характеризующие объем трафика, также не будут лишними. Пятичисловая сводка с информацией об объеме трафика за месяц – неплохое подспорье в обнаружении аномалий.
- Белые списки для каждого хоста могут пригодиться для управления аномалиями (подробнее об этом рассказывается в главе 2). Опись – самое подходящее место для хранения белых списков и файлов правил.
- Владелец и его контактная информация имеют решающее значение. Часто выяснение, кому принадлежит хост, является одним из самых сложных шагов после обнаружения атаки.
- Описание конкретных сервисов на хостах и их версий помогает оценить, насколько конкретная система уязвима для текущих эксплоитов. Аналогичные выводы можно сделать, извлекая баннеры, но намного эффективнее просто сканировать сеть, используя опись в качестве руководства.

Опись можно хранить на бумаге или в электронной таблице, но лучше для этого использовать СУБД или другую похожую систему хранения. После создания описи она будет служить простой системой обнаружения аномалий и должна регулярно обновляться автоматизированными инструментами.

Этап I: три первых вопроса

Первый этап любого процесса инвентаризации включает выяснение вопроса: какие ресурсы уже имеются и доступны для мониторинга. Поэтому любая инвентаризация начинается со встречи с сетевыми администраторами¹. Цель этой встречи – определить охват мониторингом, а именно:

- Какие адреса охватывает сеть?
- Какие средства мониторинга имеются?
- Как эти средства влияют на трафик?

¹ Предпочтительно в пивном баре.

Начнем с адресов, потому что они служат основой описи. Более конкретно, вы должны получить ответы на следующие вопросы:

Какой тип адресации используется в сети, IPv4 или IPv6?

Если в сети используются адреса IPv6, значит, вы будете иметь дело с более обширным адресным пространством, что снижает потребность в DHCP и NAT. Однако пока чаще встречаются сети с адресами IPv4, а это означает, что в сети более или менее значительного размера почти всегда используется наложение имен, NAT и другие приемы преобразования адресов.

Сколько адресов доступно или скрыто за NAT?

В идеале ваша карта должна отражать особенности маршрутизации в сети, наличие особо защищенных зон DMZ и какая информация скрыта за NAT. Эти отдельные подсети являются будущими кандидатами для мониторинга.

Сколько хостов имеется в сети?

Выясните, сколько персональных компьютеров, клиентов, серверов и встроенных систем находится в сети. Эти системы – ваши подзащитные. Обратите особое внимание на встраиваемые системы, такие как принтеры и средства телеконференций, поскольку обычно они имеют сетевые серверы, их сложно обновлять и они нередко упускаются из виду при инвентаризации.

В итоге такой встречи вы должны получить список всех потенциальных IP-адресов. Этот список, вероятно, будет несколько раз включать одни и те же пространства эфемерных адресов. Например, если имеется шесть подсетей за межсетевыми экранами с NAT, можно ожидать, что диапазон 192.168.0.0/16 будет повторяться шесть раз. Вы также должны получить количество хостов в каждой подсети и в сети в целом.

Следующий набор вопросов, ответы на которые вы должны получить, касается текущего оборудования. Наличие инструментов мониторинга на конкретных хостах (например, механизмов журналирования на серверах и т. п., как описано в главе 3) не является основной целью на данный момент. Основная цель – определить, *доступны ли* такие инструменты на уровне сети. Если доступны, определите, какая информация собирается ими, а если недоступны, определите, можно ли их добавить. Более конкретно, вы должны получить ответы на следующие вопросы:

Какие сведения собираются в настоящее время?

Пригодиться могут не только источники, «представляющие интерес с точки зрения безопасности». Например, NetFlow в основном использовался в качестве биллинговой системы, но его также можно применять для мониторинга.

Имеются ли устройства с поддержкой NetFlow?

Например, если доступны маршрутизаторы Cisco со встроенной поддержкой NetFlow, используйте их в качестве источников информации.

Присутствуют ли какие-либо системы обнаружения вторжений?

Систему обнаружения вторжений (Intrusion Detection System, IDS), такую как Snort, можно настроить на сбор заголовков пакетов. В зависимости от местоположения IDS (например, на границе сети) ее можно настроить также на сбор потоков.

Получив ответы, вы должны составить схему сбора информации в сети. Цель этой схемы – извлечь выгоду из любых существующих систем мониторинга и получить возможность систематического мониторинга трансграничного трафика. Как правило, в большинстве корпоративных сетей проще активировать отключенные инструменты, такие как NetFlow, и, соответственно, труднее добавить новое программное и аппаратное обеспечение.

Стандартная сеть

В этой главе я буду делать небольшие отступления, чтобы обсудить конкретные методы, помогающие ответить на вопросы, задаваемые в тексте. В этих отступлениях будет приводиться большое количество запросов SiLK, для понимания которых достаточно иметь поверхностное представление, как SiLK разбивает данные.

Стандартная сеть, о которой пойдет речь в этом отступлении, показана на рис. 15-2. Как описывается в документации к набору инструментов SiLK, в этой сети два датчика (источника информации): R1 (Маршрутизатор 1) и R2 (Маршрутизатор 2). Они собирают данные трех типов: входящие (поступающие из облака в сеть), исходящие (исходящие из сети в облако) и внутренние (трафик, который не пересекает границы сети с облаком).

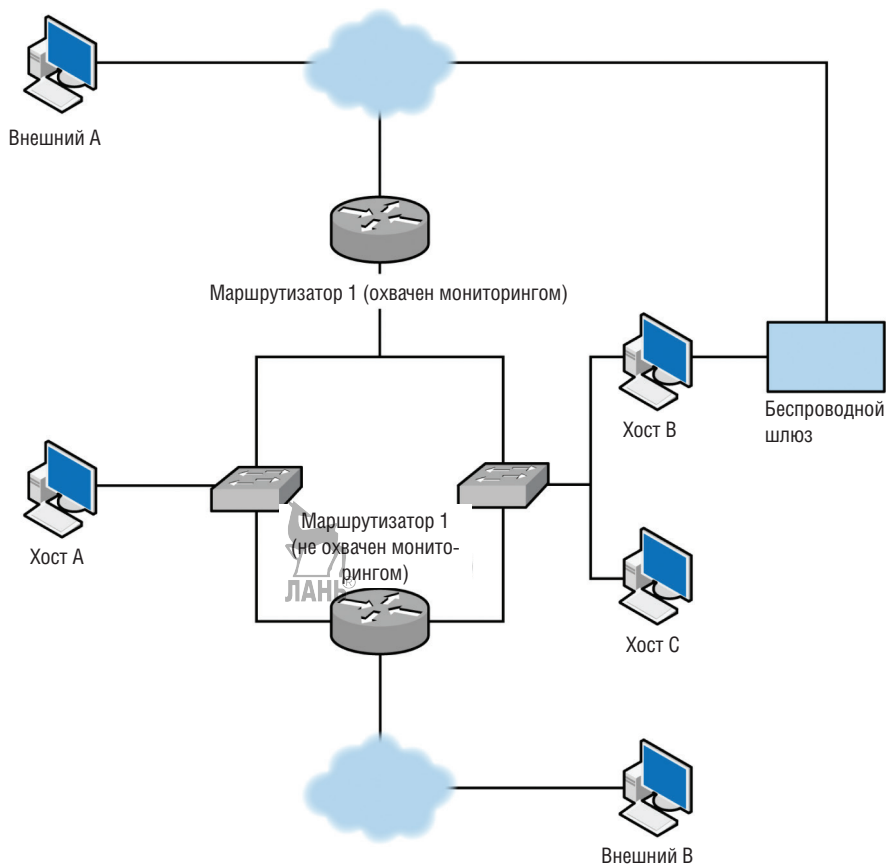


Рис. 15-2. Маршруты, не охваченные мониторингом

Кроме того, существует несколько списков IP-адресов. Список `initial.set` хостов в сети предоставлен администраторами на первой встрече. Он состоит из двух подсписков, `servers.set` и `clients.set`, с адресами серверов и клиентов соответственно. Список `servers.set` делится на подписки `webservers.set`, `dnsservers.set` и `sshservers.set`. Эти списки содержали точные сведения на момент встречи, но будут меняться с течением времени.

Этап II: исследование пространства IP-адресов

На этом этапе вы должны получить ответы на следующие вопросы:

- Существуют ли маршруты, не охваченные мониторингом?
- Какие IP-адреса принадлежат темному пространству?

Какие IP-адреса принадлежат сетевым устройствам?

По окончании этапа I у вас должна иметься примерная опись сети и средств мониторинга, как минимум контролирующего трансграничный трафик. Имея эту информацию, вы можете начать проверку описи, сверять получаемый трафик со списком IP-адресов, предоставленным администраторами. Обратите внимание на слова *проверка* – вы должны сопоставить наблюдаемый трафик с адресами, которые вам сообщили.

Ваша первая цель – оценить полноту охвата сети мониторингом, в частности есть ли у вас какие-либо неконтролируемые (не охваченные мониторингом) маршруты, то есть законные маршруты, где трафик не регистрируется. На рис. 15-2 показаны некоторые распространенные примеры темных маршрутов. Линии на этом рисунке соответствуют маршрутам между сущностями:

- первый неконтролируемый маршрут протекает через маршрутизатор 2, который не охвачен мониторингом. Например, если хост А подключится к внешнему адресу В через маршрутизатор 2, вы не увидите трафик от А к В или от В к А;
- более распространенной проблемой в современных сетях является наличие беспроводных шлюзов. Большинство современных хостов имеют доступ к нескольким беспроводным сетям, особенно в общественных местах. Хост В в этом примере может подключиться к интернету, полностью минуя маршрутизатор 1.

Ключом к выявлению неконтролируемых маршрутов является исследование *асимметричного* трафика. Протоколы маршрутизации пересылают трафик, не проявляя особого интереса к точке отправления, поэтому если имеется n точек доступа к вашей сети, вероятность, что какой-то конкретный сеанс будет иметь входящий и исходящий трафики, протекающие через одну и ту же точку, составляет около $1/n$. Можно ожидать, что в работе средств мониторинга сети иногда будут происходить сбои, поэтому информация о сеансах местами будет иметь пробелы, но если вы обнаружите *явную* асимметричность сеансов между парами адресов, это может служить доказательством, что в текущей конфигурации мониторинга чего-то не хватает.

Лучшим инструментом обнаружения асимметричных сеансов является трафик TCP, потому что протокол TCP наиболее распространен в стеке IP и гарантирует ответ. Чтобы определить допустимые сеансы TCP, воспользуйтесь подходом, противоположным описанному в главе 11: найдите сеансы с флагами SYN, ACK и FIN, в которых имеется несколько пакетов с полезной нагрузкой.

Выявление асимметричного трафика

Для выявления асимметричного трафика ищите сеансы TCP, несущие полезную нагрузку и не имеющие соответствующего исходящего сеанса. Это можно сделать с помощью `gwniq` и `rwfilter`:

```
$ rwfilter --start-date=2013/05/10:00 --end-date=2013/05/10:00 --proto=6 \
--type=out --packets=4- --flags-all=SAF/SAF --pass=stdout | \
gwniq --field=1,2 --no-title --sort | cut -d '|' -f 1,2 > outgoing.txt
# Обратите внимание на порядок следования значений 1,2 в команде gwniq выше
# и 2,1 в команде gwniq ниже.
# Это обеспечит перечисление полей в выводе в одном и том же порядке
# и упростит их сравнение.
$ rwfilter --start-date=2013/05/10:00 --end-date=2013/05/10:00 --proto=6 \
--type=in --packets=4- --flags-all=SAF/SAF --pass=stdout | gwniq \
--field=2,1 --no-title --sort | cut -d '|' -f 2,1 > incoming.txt
```

После выполнения этих команд появятся два файла с парами, включающими внутренние и внешние IP-адреса. Эти пары можно сравнить напрямую, используя ключ `-str` или собственную процедуру. В примере 15-1 показана такая процедура на Python, которая генерирует отчет об однонаправленных потоках:

Пример 15-1. Генерация отчета однонаправленных потоков

```
#!/usr/bin/env python
#
## compare_reports.py
#
# Порядок использования:
# compare_reports.py file1 file2
#
# Читает содержимое двух файлов и проверяет появление
# одинаковых пар IP-адресов.
#

import sys, os

def read_file(fn):
    ip_table = set()
    a = open(fn, 'r')
    for i in a.readlines():
        sip, dip = map(lambda x: x.strip(), i.split('|')[0:2])
        key = "%15s:%15s" % (sip, dip)
        ip_table.add(key)
    a.close()
    return ip_table

if __name__ == '__main__':
    incoming = read_file(sys.argv[1])
    outgoing = read_file(sys.argv[2])
    missing_pairs = set()
    total_pairs = set()
    # Проверим входящие и исходящие сеансы, чтобы выяснить,
    # какие пары присутствуют только в одном списке
    for i in incoming:
        total_pairs.add(i)
        if not i in outgoing:
            missing_pairs.add(i)
```

```

for i in outgoing:
    total_pairs.add(i)
    if not i in incoming:
        missing_pairs.add(i)
print missing_pairs, total_pairs
# Теперь выполним некоторые манипуляции с адресами
addrcount = {}
for i in missing_pairs:
    in_value, out_value = i.split(':')[0:2]
    if not addrcount.has_key(in_value):
        addrcount[in_value] = 0
    if not addrcount.has_key(out_value):
        addrcount[out_value] = 0
    addrcount[in_value] += 1
    addrcount[out_value] += 1

# Простой отчет, число пропущенных пар, список наиболее часто
# встречающихся адресов
print "%d missing pairs out of %d total" % (len(missing_pairs),
                                            len(total_pairs))

s = addrcount.items()
s.sort(lambda a,b:b[1] - a[1]) # lambda just guarantees order
print "Most common addresses:"
for i in s[0:10]:
    print "%15s %5d" % (i[0],addrcount[i[0]])

```

Этот подход лучше использовать с применением пассивных методов сбора информации, поскольку такой подход гарантирует получение трафика, приходящего из разных внешних адресов. Сканирование, напротив, лучше использовать для определения границ темного пространства и лазеек. Выполняя сканирование и проверяя пассивные инструменты сбора трафика, можно не только видеть результаты своего сканирования на рабочем столе, но и сравнивать трафик от сканирования с данными, предоставленными вашей системой сбора информации.

Вы можете сканировать сеть и проверять, насколько сеансы сканирования соответствуют ожиданиям (то есть видеть ответы от хостов и ничего из пустого пространства), но сканирование часто выполняется из одного места, тогда как в действительности желательно видеть трафик, происходящий из нескольких разных точек.

Обнаружив неконтролируемые маршруты, определите, можно ли добавить в них средства мониторинга и почему этого не было сделано раньше. Неконтролируемые маршруты представляют угрозу безопасности: их можно использовать для прощупывания и проникновения без опаски быть замеченным.

Неконтролируемые маршруты и темные пространства имеют сходные профили трафика; в обоих случаях отправленный пакет TCP не вызовет ответа. Но в неконтролируемом маршруте это происходит из-за неполного охвата мониторингом, тогда как в темном пространстве просто некому ответить на пакет. После выявления неконтролируемых маршрутов любые контролируемые адреса, которые ведут себя таким же образом, можно смело отнести к области темного пространства.

Выявление границ темного пространства

Темные пространства можно выявлять и пассивными, и активными методами.

Пассивные подходы основаны на сборе трафика в сети и постепенном устранении адресов, которые отвечают или не контролируются. Все адреса, оставшиеся после этого, относятся к темному пространству. Альтернативный подход – активная проверка адресов и перепись тех, которые не отвечают; эти адреса, скорее всего, относятся к темному пространству.

Пассивные подходы требуют сбора данных в течение длительного периода. Трафик должен собираться не менее недели, чтобы гарантировать более или менее полный охват динамических адресов и бизнес-процессов.

```
$ rfilter --type=out --start-date=2013/05/01:00 --end-date=2013/05/08:23 \
--proto=0-255 --pass=stdout | rset --sip-file=light.set
# Теперь уберем из общего списка ответившие адреса
$ rsettool --difference --output=dark.set initial.set light.set
```

Альтернативный подход – попробовать обратиться к каждому хосту в сети, чтобы определить его присутствие.

```
$ for i in `rsetcat initial.set`
do
# выполнить команду ping с 5-секундным тайм-аутом и 1 попыткой для каждой цели
ping -q -c 1 -t 5 ${i} | tail -2 >> pinglog.txt
done
```

Этот код запишет в файл *pinglog.txt* сводную информацию, возвращаемую командой *ping*, например:

```
--- 128.2.11.0 ping statistics ---
1 packets transmitted, 0 packets received, 100.0% packet loss
```

Его содержимое можно проанализировать и определить границы темного пространства.

Активный подход на основе сканирования позволяет получить результаты быстрее, но при этом вы должны гарантировать, что все узлы в сети возвращают ICMP-сообщения ECHO REPLY.

Еще один способ определения границ динамических пространств методом пассивного мониторинга – ежечасные проверки и сравнение списков темных и светлых адресов.

Под «сетевыми устройствами» в этом контексте подразумеваются интерфейсы маршрутизаторов. Интерфейсы маршрутизаторов идентифицируются присутствием в трафике протоколов маршрутизации, таких как BGP, RIP и OSPF. Еще один способ – проверка ICMP-сообщений «узел не найден» (также известных как сообщения о недоступности сети), которые генерируются только маршрутизаторами.

Поиск сетевых устройств

Выявить сетевые устройства можно с помощью *traceroute* или поиском определенных протоколов в трафике. Каждый хост, упомянутый в отчете *traceroute*, кроме конечной точки, является маршрутизатором. Если вы решите выполнить поиск по протоколам, тогда проверьте присутствие в трафике следующих протоколов:

BGP

Протокол BGP представляет типичный «язык», на котором между собой говорят маршрутизаторы в интернете. Он мало распространен в небольших и средних



корпоративных сетях. Протокол BGP использует TCP-порт с номером 179.

```
# Эта команда поможет выявить взаимодействия с внешним миром
# по протоколу BGP
$ rfilter --type=in --proto=6 --dport=179 --flags-all=SAF/SAF \
  --start-date=2013/05/01:00 --end-date=2013/05/01:00 --pass=bgp_speakers.rwf
```

OSPF и EIGRP

В небольших сетях маршрутизация часто осуществляется с применением протоколов OSPF и EIGRP. EIGRP использует порт с номером 88, а OSPF – с номером 89.

```
# Эта команда поможет выявить взаимодействия по протоколам OSPF и EIGRP,
# обратите внимание на ключ --type=internal, в данном случае
# предполагается исследовать только внутренний трафик
$ rfilter --type=internal --proto=88,89 --start-date=2013/05/01:00 \
  --end-date=2013/05/01:00 --pass=stdout | rfilter --proto=88 \
  --input-pipe=stdin --pass=eigrp.rwf --fail=ospf.rwf
```

RIP

Еще один протокол внутрисетевой маршрутизации. Протокол RIP основан на протоколе UDP и использует порт 520.

```
# Эта команда поможет выявить взаимодействия по протоколу RIP
$ rfilter --type=internal --proto=17 --aport=520 \
  --start-date=2013/05/01:00 --end-date=2013/05/01:00 --pass=rip_speakers.rwf
```

ICMP

Оба сообщения – «хост недостижим» (ICMP Type 3, Code 7) и «превышено время ожиданий» (ICMP Type 11) – генерируются маршрутизаторами.

```
# Отфильтровать icmp-сообщения, долгий период выбран потому,
# что эти ICMP-сообщения встречаются довольно редко
$ rfilter --type=out --proto=1 --icmp-type=3,11 --pass=stdout \
  --start-date=2013/05/01:00 \
  --end-date=2013/05/01:23 | rfilter --icmp-type=11 --input-pipe=stdin \
  --pass=ttl_exceeded.rwf --fail=stdout | rfilter --input-pipe=stdin \
  --icmp-code=7 --pass=not_found.rwf
$ rset --sip=routers_ttl.set ttl_exceeded.rwf
$ rset --sip=routers_nf.set not_found.rwf
$ rsettool --union --output-path=routers.set routers_nf.set routers_ttl.set
```

К концу этого этапа у вас должен появиться список адресов интерфейсов маршрутизаторов. Каждый маршрутизатор в сети управляет одним или несколькими такими интерфейсами. На этом этапе также рекомендуется обратиться к сетевым администраторам, чтобы связать эти интерфейсы с фактическим оборудованием.

Этап III: выявление слепого и странного трафика

На этом этапе вы должны найти ответы на следующие вопросы:

- Имеются ли шлюзы с NAT?
- Имеются ли прокси, обратные прокси или кеши?
- Присутствует ли трафик VPN?
- Имеются ли динамические адреса?

После завершения этапа II вы определите список активных адресов в вашей

сети. Следующий шаг – выявить проблематичные адреса. Жизнь была бы проще, если бы каждому хосту назначался статический IP-адрес и этот адрес использовался бы только одним хостом. В таком случае трафик легко можно было бы идентифицировать по порту и протоколу.

Очевидно, что так бывает не всегда, поэтому вы должны определить:

Шлюзы с NAT

Они доставляют больше всего неприятностей, потому что позволяют использовать ограниченные диапазоны IP-адресов для доступа к большому числу узлов.

Прокси, обратные прокси и кеши

Подобно шлюзам с NAT, прокси скрывают несколько IP-адресов за одним адресом самого прокси. Прокси обычно работают на более высоких уровнях в стеке OSI и часто используют определенные протоколы. Обратные прокси, как следует из названия, позволяют совмещать адреса для нескольких серверов и часто используются для балансировки нагрузки и кеширования. Кеши хранят часто используемые результаты (например, веб-страницы) для повышения производительности.

VPN

Трафик виртуальной частной сети (Virtual Private Network, VPN) скрывает содержимое протоколов, выполняемые операции и количество задействованных хостов. Трафик VPN включает протоколы передачи IPv6 через IPv4, такие как 6to4 и Teredo, и зашифрованные протоколы, такие как SSH и TOR. Все эти протоколы инкапсулируют трафик, а это означает, что адреса, видимые на уровне IP, являются адресами маршрутизаторов или концентраторов, а не фактических хостов.

Динамические адреса

Использование динамической адресации, например через DHCP, приводит к тому, что в разные моменты времени один и тот же хост может получать разные адреса. Динамическая адресация усложняет анализ, вводя понятие времени жизни адреса. Никогда нельзя быть уверенным, что IP-адреса будут принадлежать тем же хостам после истечения срока аренды DHCP.

Все перечисленные аспекты должны подробно документироваться сетевыми администраторами, но существует несколько разных подходов к их идентификации. Прокси и шлюзы с NAT можно идентифицировать, если найти доказательства, что один IP-адрес служит интерфейсом к нескольким адресам. В этом может помочь анализ полезной нагрузки пакетов или потока, хотя полезная нагрузка пакетов имеет более определенный характер.

Идентификация шлюзов с NAT

Выявить шлюз с NAT непросто, если нет доступа к данным полезной нагрузки. Лучше всего выявлять шлюзы с NAT, опрашивая сетевых администраторов. Иначе вам придется искать доказательства, что за одним и тем же адресом скрыто несколько адресов. Для этого можно использовать несколько разных индикаторов.

Строки User-Agent

Лучшее решение идентификации NAT, которое я видел, – анализ строк User-Agent веб-сеансов. С помощью сценария, такого как *bannergrab.py* из главы 14, можно извлечь и сохранить все экземпляры строки User-Agent, исходящие из предполагаемого шлюза с NAT. Если они будут представлять разные экземпляры одного и того же браузера или несколько браузеров, значит, это почти наверняка шлюз с NAT.

Однако этот способ допускает возможность ошибки. В настоящее время разные приложения (в том числе и почтовые клиенты) поддерживают возможность HTTP-взаимодействий. Поэтому, используя это решение, лучше ограничиться баннерами, явно принадлежащими браузерам, таким как Firefox, IE, Chrome и Opera.

Использование разных имен пользователей при взаимодействии с одними и теми же серверами

Определите основные внутренние и внешние сервисы, используемые вашей сетью. Примерами могут служить: сервер электронной почты компании, поисковая система Google и основные сайты новостей. Если адрес принадлежит шлюзу с NAT, можно ожидать большое число входов в систему с этого адреса. Журналы почтового сервера и внутреннего HTTP-сервера являются лучшими источниками информации для такого рода исследований.

Изменение TTL

Как вы знаете, стек IP назначает пакетам срок времени жизни (Time-To-Live, TTL), начальное значение которого зависит от ОС. Проверьте значения TTL в пакетах, поступающих от подозрительного адреса, и посмотрите, как они изменяются. Разнообразие предполагает наличие нескольких хостов за подозрительным адресом. Если значения одинаковы, *но ниже начального TTL для ОС*, это явное доказательство, что пакеты совершают несколько переходов, прежде чем достигнут этого адреса.

Идентификация прокси

Для идентификации прокси необходимо, чтобы мониторингом были охвачены маршруты по обе стороны прокси. На рис. 15-3 показан сетевой трафик между клиентами, прокси и серверами. Как видите, прокси принимают запросы от нескольких клиентов и пересылают их нескольким серверам. То есть прокси действует с одной стороны как сервер (для клиентов, которым оказывает посреднические услуги), а с другой как клиент (для серверов, которые он проксирует). Если средства мониторинга позволят вам увидеть связь между клиентом и прокси и между прокси и сервером, вы сможете идентифицировать прокси, отыскав этот шаблон в трафике. В отсутствие такой возможности можно использовать методы, описанные в предыдущем рецепте идентификации шлюза с NAT. Здесь действуют те же принципы, потому что, в конце концов, прокси является интерфейсом к внешнему миру для нескольких клиентов, подобно шлюзу с NAT.

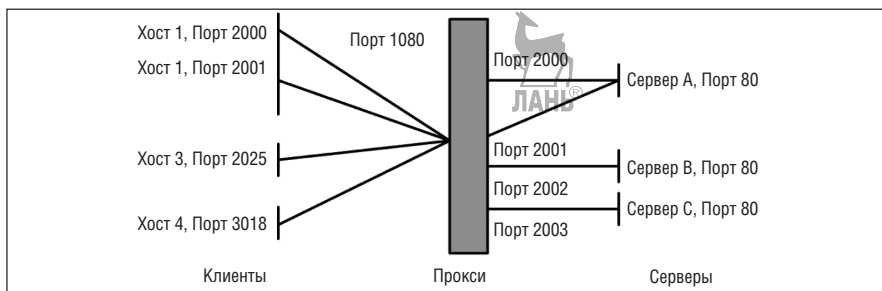


Рис. 15-3. Сетевые соединения с прокси

Для идентификации прокси в такой ситуации сначала найдите хосты, действующие как клиенты. Опознать клиента можно по использованию нескольких временных портов. Например, вот как можно идентифицировать клиентов в вашей сети с помощью *rwuniq*:

```
$ rwfilter --type=out --start-date=2013/05/10:00 --end-date=2013/05/10:01 \
--proto=6,17 --sport=1024-65535 --pass=stdout | rwuniq --field=1,3 \
--no-title | cut -d '|' -f 1 | sort | uniq -c | egrep -v '^[ ]+1' |\
cut -d ' ' -f 3 | rwsetbuild stdin clients.set
```

Эта команда выявляет все комбинации исходящего IP-адреса источника (*srcip*) и номера порта (*sport*) в данных и исключает любые ситуации, когда хост использовал только один порт. Оставшиеся после фильтрации хосты используют несколько портов. Возможно, что хосты, использующие сразу семь или восемь портов, работают с несколькими серверами, но по мере увеличения числа отдельных портов вероятность, что они работают сразу с несколькими сервисами, уменьшается.

Следующий шаг после определения клиентов – идентифицировать клиентов, которые ведут себя подобно серверам (см. раздел «Идентификация серверов» ниже).

Идентификация трафика VPN

VPN-трафик можно идентифицировать путем поиска характерных портов и протоколов, используемых VPN. VPN затрудняют анализ транспортируемого ими трафика, заключая его в другой протокол, такой как GRE. После определения конечных точек VPN используйте специализированные инструменты. После удаления обертки из VPN-трафика вы сможете различать потоки и сеансы.

В VPN используются в основном следующие протоколы и порты:

IPSec

Название IPSec относится к целому комплексу протоколов поддержки шифрованных соединений через VPN. Двумя ключевыми протоколами являются протокол AH (Authentication Header – заголовок аутентификации, порт 51) и ESP (Encapsulation Security Payload – инкапсуляция полезной нагрузки, порт 50):

```
$ rwfilter --start-date=2013/05/13:00 --end-date=2013/05/13:01 --proto=50,51 \
--pass=vpn.rwf
```

GRE

Протокол GRE (Generic Routing Encapsulation – универсальная инкапсуляция маршрутов) – это «рабочая лошадка» во множестве реализаций VPN. Его можно идентифицировать по использованию порта 47.


```
$ rfwfilter --start-date=2013/05/13:00 --end-date=2013/05/13:01 --proto=47 \
--pass=gre.rwf
```

Многие протоколы туннелирования тоже можно идентифицировать по номерам портов, хотя, в отличие от стандартных VPN, они, как правило, определяются программным обеспечением и не требуют специальных ресурсов для маршрутизации. К таким протоколам можно отнести SSH, Teredo, 6to4 и TOR.

Этап IV: идентификация клиентов и серверов

Следующим шагом после определения базовой структуры сети является профилирование и идентификация клиентов и серверов в ней. На этом этапе вы должны найти ответы на следующие вопросы:

- Каков список основных внутренних серверов?
- Используют ли серверы необычные порты?
- Присутствуют ли в сети серверы FTP, HTTP, SMTP и/или SSH, неизвестные системным администраторам?
- Какие серверы действуют как клиенты?
- Каков список основных клиентов?

Идентификация серверов

Серверы можно идентифицировать по портам, которые принимают сеансы, и путем анализа распределения взаимодействий по портам.

Чтобы определить порты, принимающие сеансы, нужно иметь доступ к данным *pcap* или провести мониторинг потока на наличие отличительных начальных флагов в пакетах (что можно сделать с помощью YAF, как описано в разделе «YAF» главы 5). Задача сводится к определению хостов, которые отвечают пакетами с флагами SYN и ACK:

```
$ rfwfilter --proto=6 --flags-init=SA|SA --pass=server_traffic.rwf \
--start-date=2013/05/13:00 --end-date=2013/05/13:00 --type=in
```

Этот подход непригоден для протокола UDP, потому что хост может посылать трафик UDP в любой порт по своему выбору и не получать в ответ ничего. Альтернативный подход, который может применяться и к UDP, и к TCP, заключается в анализе *распространения* комбинации порт/протокол. Я кратко коснулся этого вопроса в разделе «Идентификация прокси» выше, и сейчас мы обсудим его более подробно.

Сервер – это публичный ресурс. Это означает, что его адрес должен сообщаться клиентам, и со временем можно ожидать, что несколько клиентов подключатся к серверу. Следовательно, со временем вы увидите несколько потоков с разными комбинациями исходящих IP-адресов / портов, и все они будут взаимодействовать с одной и той же комбинацией IP-адреса/порта назначения. Это поведение отличается от поведения клиента, который создает несколько сеансов с несколькими разными хостами, как показано на рис. 15-4.

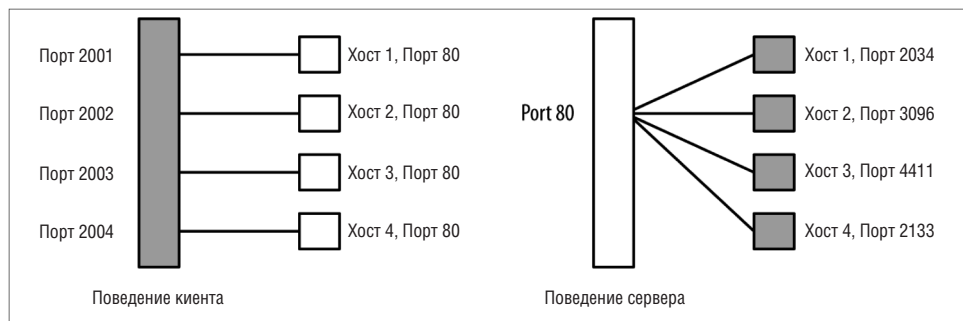


Рис. 15-4. Разное поведение клиента и сервера

Распространенность легко рассчитать с помощью команды `gwnipq`. При наличии подозрения, что с какого-то IP-адреса исходит трафик, обусловленный передачей файлов, выполните следующую команду:

```
$rwunipq --field=1,2 --dip-distinct candidate_file | sort -t '|' -k3 -nr |\
head -15
```

Чем больше разных IP-адресов взаимодействует с одной и той же комбинацией хост/порт, тем выше вероятность, что эта комбинация принадлежит серверу. Эта команда поместит серверы в верхнюю часть списка.

Анализируя распространенность адреса и сами пакеты, можно узнать большинство комбинаций IP-адрес/порт, на которых работают серверы. Эти комбинации желательно сканировать периодически, чтобы проверить, что на самом деле работает в сети: в частности, чтобы найти серверы, которые используют нетипичные порты. Серверы – это публичные ресурсы (до определенной степени), и когда они используют нетипичные порты, это может указывать на то, что у пользователя не хватает привилегий для запуска сервера в нормальной конфигурации (что очень подозрительно) или он пытается скрыть это (тоже очень подозрительно, особенно если вы прочитали главу 11).

Определив, какие серверы присутствуют в сети, выясните, какие из них особенно важны. Оценить важность можно по нескольким разным показателям, в том числе:

Общий объем трафика

Самый простой и самый распространенный показатель.

Объем внутреннего и внешнего трафиков

Этот параметр позволяет отличить серверы, доступные только вашим пользователям, от серверов, доступных для внешнего мира.

Центральность в графе

Центральность по пути и степени часто помогает выявить важные хосты, которые можно пропустить, если использовать чистую статистику степени (количество контактов). За дополнительными подробностями обращайтесь к главе 13.

Цель в данном случае – составить список серверов, упорядоченных по приоритету: от тех, которые требуют особенно пристального внимания, до маловажных и даже тех, которые можно безболезненно удалить.

После идентификации всех серверов в сети самое время вновь встретиться с сетевыми администраторами¹. Это необходимо, потому что вы почти наверняка найдете в сети серверы, о которых никто не знал, например:

- системы, управляемые опытными пользователями;
- встроенные веб-серверы;
- захваченные хосты.

Идентификация инфраструктуры контроля и блокирования

Далее нужно получить ответы на следующие вопросы:

- Имеются ли какие-либо системы обнаружения или предотвращения вторжений (IDS или IPS)? Можно ли изменить их настройки?
- К журналам каких систем имеется доступ?
- Существуют ли в сети какие-либо брандмауэры?
- Существуют ли в сети какие-либо маршрутизаторы, ограничивающие доступ?
- Существуют ли на границе сети системы для борьбы со спамом? А может быть, спам фильтруется почтовым сервером? Или и там, и там?
- Используются ли антивирусы?

Завершающим этапом любого нового проекта мониторинга является выяснение, какое программное обеспечение поддержки безопасности в настоящее время установлено и действует. Обычно эти системы идентифицируются по *отсутствию* трафика. Например, если ни один из хостов в сети не порождает трафика BitTorrent (порты 6881–6889), вероятно, маршрутизатор блокирует трафик BitTorrent.

ОБНОВЛЕНИЕ ОПИСИ: К НЕПРЕРЫВНОМУ АУДИТУ

После создания начальной описи организуйте регулярный запуск всех сценариев анализа, которые вы написали. Цель этого действия состоит в том, чтобы организовать регулярное выявление изменений в сети.

Наличие актуальной описи упрощает обнаружение аномалий. Первый и наиболее очевидный подход – проверять наличие изменений в описи. Вот примеры вопросов, на которые вы должны регулярно отвечать:

- Появились ли новые клиенты и/или серверы?
- Ушли ли какие-либо адреса в темную область?
- Появились ли новые сервисы на клиентах?

Изменения в описи можно использовать для определения необходимости проведения других видов анализа. Например, когда в сети появляется новый клиент или сервер, можно проанализировать его трафик, чтобы увидеть, с кем он общается, выполнить процедуру сканирования или как-то иначе поэкспериментировать с хостом, чтобы заполнить опись информацией о новом узле сети.

В долгосрочной перспективе мониторинг известных адресов является залогом надежной защиты сети. Невозможно сказать, что «хост X более защищен, чем хост Y»; у нас просто нет возможности количественно оценить защищенность X.

¹ Теперь желательно там, где подают крепкие напитки.

Работая с картой сети, можно оценить охват мониторингом либо в абсолютном выражении (из X адресов в сети Y охвачены мониторингом), либо в процентном.

ДОПОЛНИТЕЛЬНЫЕ МАТЕРИАЛЫ ДЛЯ ЧТЕНИЯ

1. *Умеш Шанкар и Верн Паксон (Umesh Shankar and Vern Paxson)*. Активное отображение: сопротивление уклонению NIDS без изменения трафика: продолжения симпозиума IEEE 2003 года по безопасности и конфиденциальности.
2. *Остин Уиснант и Сид Фабер (Austin Whisnant and Sid Faber)*. Программирование сети с использованием потока. CMU/SEI-2012-TR-006, Институт программной инженерии.





Предметный указатель

А

Адреса IPv4 [154](#)
Адреса MAC [151](#)
Адреса шлюза [154](#)
Алгоритм журналирования [56](#)
Альтернативная гипотеза (H1) [130](#)
Антивирусные системы [138](#)
Асимметричный поток трафика [293](#)
Атрибуты центрированности [263](#)

Б

Бесклассовая адресация [46](#)
Библиотеку GeoIP [186](#)
Блок Classless Internet Domain Routing (CIDR) [154](#)

В

Вектор R [113](#)
Виды адресов [40](#)
Виды атак [29](#)
Волоконно-оптический канал [46](#)
Время жизни пакета данных [39](#)

Г

Гистограмма [197](#)
Граф [257](#)

Д

Датчики
 сервиса [51](#)
 хоста [51](#)
Действие сенсора [30](#)
Домен коллизий [36](#)

Ж

Журнал безопасности [54](#)
Журналы
 Windows [53](#)
Журналы хоста [36](#)

З

Записи NS [168](#)
Записи SOA [169](#)
Записи почтового обмена (MX) [167](#)

И

Иерархия устройства хранения
 данных [77](#)
Инструмент gwsut [83](#)
Инструмент преобразования пакета
 данных в поток [108](#)
Инструменты распределенного
 запроса [69](#)
Инструменты форматирования [85](#)

К

Кадры данных [122](#)
Каноническое имя (CNAME) [167](#)
Карта префикса SiLK (PMAP) [104](#)
Карта сети [225](#)
Квартет Энскомба [192](#)
Консоль R [111](#)
Комитет по цифровым адресам
 в интернете (IANA) [156](#)
Коэффициент кластеризации [267](#)
Кратчайшие пути [261](#)

Л

Логарифмическое
 масштабирование [212](#)
Ложноотрицательный уровень [138](#)
Ложноположительный уровень [138](#)
Локальность [242](#)

М

Маршрутизация [38](#)
Маршрутизируемый трафик в IPv6 [156](#)
Метод EDA [195](#)
Модели нападения [218](#)
Модель Pareto [244](#)
Модель TCP/IP [34](#)

Н

Национальная база данных
 уязвимости (NVD) [186](#)
Нулевая гипотеза [130](#)

О

Область обзора
 сети [33](#)

Область обзора сенсора 24
 Обратное рассеяние 226
 Обратный поиск 169
 Отличие между IPv4 и IPv6 155
 Отражение DNS 251
 Отчет однонаправленных потоков 294
 Ошибка тарифной ставки 142

П

Пакетный фильтр Беркли (BPF) 43
 Пакеты TCP SYN 226
 Парадигма CRUD 69
 Парадигма MapReduce 72
 Протокол определения адресов (ARP) 153
 Пятичисловая сводка 202

Р

Рабочий набор 243
 Разрушительность 148
 Ротация файла журнала 65

С

Связанный компонент 266
 Сервис UDP 228
 Сетевая модель OSI 34
 Сетевой сенсор 33
 Сетевые журналы 51
 Системный журнал 66
 Системы обнаружения проникновения (IDS) 135
 Ситуационная осведомленность 11
 Сообщения ICMP 226
 Сообщения журнала SMTP
 Unix, Windows 62
 Специфичность 140

Т

Тест Kolmogorov-Smirnov 133
 Тест Shapiro-Wilk 132

У

Уровень сенсора 27
 Устройства хранения данных 73

Ф

Файл рабочей области 122
 Файлы данных LBNL 82
 Файлы журнала
 Unix 52

Фильтрация
 по времени жизни 45
 по протоколам 45
 Фильтрация по адресу 90
 Формат журнала HTTP
 Формат журнала
 HTML 58
 Функции визуализации R 125

Х

Хранение данных 68

Ц

Циклический буфер 42

Ч

Черный список DNS (DNSBL) 173
 Чувствительность 140

Ш

Широковещательный адрес сети 154

А

ATM 46

Д

DDos-атаки 246

Е

Exploratory Data Analysis—EDA 15

Ф

function 118

Г

GeoIP MaxMind 185
 Graphviz 176

Н

HTTP-заголовки 59

І

IP Sets (наборы IP) 98
 IP-адреса, 44

Н

netcat 179
 NetFlow 47

Р

plot 126
 p-value (p-значения) 131



Rrwbag [103](#)rwnfilter-опции [88](#)rwsettool [100](#)R-функции [118](#)**S**Scapy [182](#)scatterplot [206](#)

SiLK (System for Internet-Level

Knowledge) [14](#)switch [120](#)**W**Wireshark [185](#)YAF [106](#)

Книги издательства «ДМК Пресс» можно заказать
в торгово-издательском холдинге «Планета Альянс» наложенным платежом,
выслав открытку или письмо по почтовому адресу:

115487, г. Москва, 2-й Нагатинский пр-д, д. 6А.

При оформлении заказа следует указать адрес (полностью),
по которому должны быть высланы книги;
фамилию, имя и отчество получателя.

Желательно также указать свой телефон и электронный адрес.

Эти книги вы можете заказать и в интернет-магазине: **www.a-planet.ru**.

Оптовые закупки: тел. **(499) 782-38-89**.

Электронный адрес: **books@alians-kniga.ru**.



Майкл Коллинз

Защита сетей.

Подход на основе анализа данных

Главный редактор *Мовчан Д. А.*
dmkpress@gmail.com

Перевод *Добровольская А.В.*

Корректор *Синяева Г. И.*

Верстка *Луценко С. В.*

Дизайн обложки *Мовчан А. Г.*

Формат 70×100 1/16.

Гарнитура «PT Serif». Печать цифровая.

Усл. печ. л. 22,03. Тираж 200 экз.

Веб-сайт издательства: **www.dmkpress.com**